

Part III

Graphics, Motion and Sound



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

The Rendering Engine

When most people think about computer and video games, the first thing that comes to mind is the stunning three-dimensional graphics. Real-time 3D rendering is an exceptionally broad and profound topic, so there's simply no way to cover all of the details in a single chapter. Thankfully there are a great many excellent books and other resources available on this topic. In fact, real-time 3D graphics is perhaps one of the best covered of all the technologies that make up a game engine. The goal of this chapter, then, is to provide you with a broad understanding of real-time rendering technology and to serve as a jumping-off point for further learning. After you've read through these pages, you should find that reading other books on 3D graphics seems like a journey through familiar territory. You might even be able to impress your friends at parties (...or alienate them...).

We'll begin by laying a solid foundation in the concepts, theory and mathematics that underlie any real-time 3D rendering engine. Next, we'll have a look at the software and hardware pipelines used to turn this theoretical framework into reality. We'll discuss some of the most common optimization techniques and see how they drive the structure of the tools pipeline and the runtime rendering API in most engines. We'll end with a survey of some of the advanced rendering techniques and lighting models in use by game engines today. Throughout this chapter, I'll point you to some of my favorite books and

other resources that should help you to gain an even deeper understanding of the topics we'll cover here.

11.1 Foundations of Depth-Buffered Triangle Rasterization

When you boil it down to its essence, rendering a three-dimensional scene involves the following basic steps:

- A *virtual scene* is described, usually in terms of 3D surfaces represented in some mathematical form.
- A *virtual camera* is positioned and oriented to produce the desired view of the scene. Typically the camera is modeled as an idealized focal point, with an imaging surface hovering some small distance in front of it, composed of *virtual light sensors* corresponding to the picture elements (*pixels*) of the target display device.
- Various *light sources* are defined. These sources provide all the light rays that will interact with and reflect off the objects in the environment and eventually find their way onto the image-sensing surface of the virtual camera.
- The *visual properties* of the surfaces in the scene are described. This defines how light should interact with each surface.
- For each pixel within the imaging rectangle, the rendering engine calculates the color and intensity of the light ray(s) converging on the virtual camera's focal point through that pixel. This is known as *solving the rendering equation* (also called the *shading equation*).

This high-level rendering process is depicted in Figure 11.1.

Many different technologies can be used to perform the basic rendering steps described above. The primary goal is usually *photorealism*, although some games aim for a more stylized look (e.g., cartoon, charcoal sketch, watercolor and so on). As such, rendering engineers and artists usually attempt to describe the properties of their scenes as realistically as possible and to use light transport models that match physical reality as closely as possible. Within this context, the gamut of rendering technologies ranges from techniques designed for real-time performance at the expense of visual fidelity, to those designed for photorealism but which are not intended to operate in real time.

Real-time rendering engines perform the steps listed above repeatedly, displaying rendered images at a rate of 30, 50 or 60 frames per second to provide

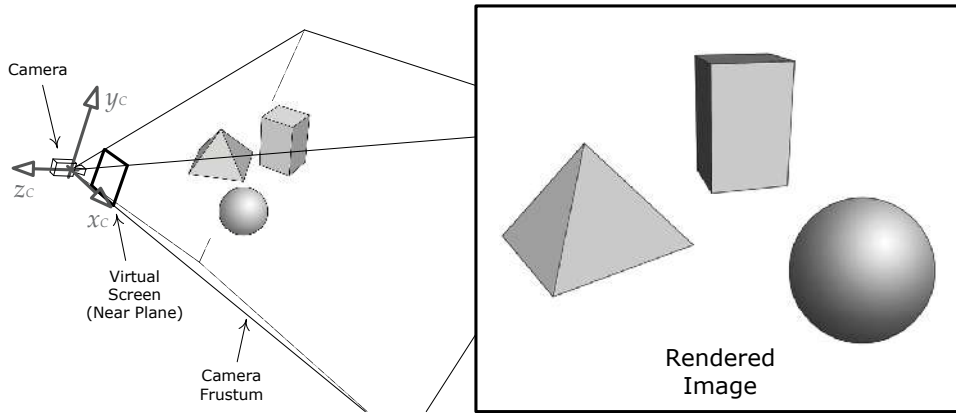


Figure II.1. The high-level rendering approach used by virtually all 3D computer graphics technologies.

the illusion of motion. This means a real-time rendering engine has at most 33.3 ms to generate each image (to achieve a frame rate of 30 FPS). Usually much less time is available, because bandwidth is also consumed by other engine systems like animation, AI, collision detection, physics simulation, audio, player mechanics and other gameplay logic. Considering that film rendering engines often take anywhere from many minutes to many hours to render a single frame, the quality of real-time computer graphics these days is truly astounding.

II.1.1 Describing a Scene

A real-world scene is composed of objects. Some objects are solid, like a brick, and some are amorphous, like a cloud of smoke, but every object occupies a volume of 3D space. An object might be *opaque* (in which case light cannot pass through its volume), *transparent* (in which case light passes through it without being scattered, so that we can see a reasonably clear image of whatever is behind the object), or *translucent* (meaning that light can pass through the object but is scattered in all directions in the process, yielding only a blur of colors that hint at the objects behind it).

Opaque objects can be rendered by considering only their *surfaces*. We don't need to know what's inside an opaque object in order to render it, because light cannot penetrate its surface. When rendering a transparent or translucent object, we really should model how light is reflected, refracted, scattered and absorbed as it passes through the object's volume. This requires knowledge of the interior structure and properties of the object. However,

most game engines don't go to all that trouble. They just render the surfaces of transparent and translucent objects in almost the same way opaque objects are rendered. A simple numeric opacity measure known as *alpha* is used to describe how opaque or transparent a surface is. This approach can lead to various visual anomalies (for example, surface features on the far side of the object may be rendered incorrectly), but the approximation can be made to look reasonably realistic in many cases. Even amorphous objects like clouds of smoke are often represented using particle effects, which are typically composed of large numbers of semitransparent rectangular cards. Therefore, it's safe to say that most game rendering engines are primarily concerned with rendering *surfaces*.

II.I.I.I Representations Used by High-End Rendering Packages

Theoretically, a surface is a two-dimensional sheet comprised of an infinite number of points in three-dimensional space. However, such a description is clearly not practical. In order for a computer to process and render arbitrary surfaces, we need a compact way to represent them numerically.

Some surfaces can be described exactly in analytical form using a *parametric surface equation*. For example, a sphere centered at the origin can be represented by the equation $x^2 + y^2 + z^2 = r^2$. However, parametric equations aren't particularly useful for modeling arbitrary shapes.

In the film industry, surfaces are often represented by a collection of rectangular *patches* each formed from a two-dimensional spline defined by a small number of control points. Various kinds of splines are used, including Bézier surfaces (e.g., bicubic patches, which are third-order Béziers—see http://en.wikipedia.org/wiki/Bezier_surface for more information), nonuniform rational B-splines (NURBS—see <http://en.wikipedia.org/wiki/Nurbs>), Bézier triangles and *N-patches* (also known as *normal patches*—see <http://ubm.io/1iGnvJ5> for more details). Modeling with patches is a bit like covering a statue with little rectangles of cloth or paper maché.

High-end film rendering engines like Pixar's RenderMan use *subdivision surfaces* to define geometric shapes. Each surface is represented by a mesh of control polygons (much like a spline), but the polygons can be subdivided into smaller and smaller polygons using the Catmull-Clark algorithm. This subdivision typically proceeds until the individual polygons are smaller than a single pixel in size. The biggest benefit of this approach is that no matter how close the camera gets to the surface, it can always be subdivided further so that its silhouette edges won't look faceted. To learn more about subdivision surfaces, check out the following great article: <http://ubm.io/1lx6th5>.

11.1.1.2 Triangle Meshes

Game developers have traditionally modeled their surfaces using triangle meshes. Triangles serve as a piecewise linear approximation to a surface, much as a chain of connected line segments acts as a piecewise approximation to a function or curve (see Figure 11.2).

Triangles are the polygon of choice for real-time rendering because they have the following desirable properties:

- *The triangle is the simplest type of polygon.* Any fewer than three vertices, and we wouldn't have a surface at all.
- *A triangle is always planar.* Any polygon with four or more vertices need not have this property because, while the first three vertices define a plane, the fourth vertex might lie above or below that plane.
- *Triangles remain triangles under most kinds of transformations, including affine transforms and perspective projections.* At worst, a triangle viewed edge-on will degenerate into a line segment. At every other orientation, it remains triangular.
- *Virtually all commercial graphics-acceleration hardware is designed around triangle rasterization.* Starting with the earliest 3D graphics accelerators for the PC, rendering hardware has been designed almost exclusively around triangle rasterization. This decision can be traced all the way back to the first software rasterizers used in the earliest 3D games like *Castle Wolfenstein 3D* and *Doom*. Like it or not, triangle-based technologies are entrenched in our industry and probably will be for years to come.

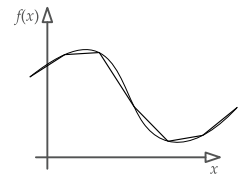


Figure 11.2. A mesh of triangles is a linear approximation to a surface, just as a series of connected line segments can serve as a linear approximation to a function or curve.

Tessellation

The term *tessellation* describes a process of dividing a surface up into a collection of discrete polygons (which are usually either quadrilaterals, also known as *quads*, or triangles). *Triangulation* is tessellation of a surface into triangles.

One problem with the kind of triangle mesh used in games is that its level of tessellation is fixed by the artist when he or she creates it. Fixed tessellation can cause an object's *silhouette edges* to look blocky, as shown in Figure 11.3; this is especially noticeable when the object is close to the camera.

Ideally, we'd like a solution that can arbitrarily increase tessellation as an object gets closer to the virtual camera. In other words, we'd like to have a uniform triangle-to-pixel density, no matter how close or far away the object is. Subdivision surfaces can achieve this ideal—surfaces can be tessellated based on distance from the camera, so that every triangle is less than one pixel in size.

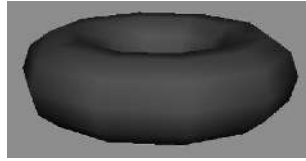


Figure II.3. Fixed tessellation can cause an object's silhouette edges to look blocky, especially when the object is close to the camera.

Game developers often attempt to approximate this ideal of uniform triangle-to-pixel density by creating a chain of alternate versions of each triangle mesh, each known as a *level of detail* (LOD). The first LOD, often called LOD 0, represents the highest level of tessellation; it is used when the object is very close to the camera. Subsequent LODs are tessellated at lower and lower resolutions (see Figure 11.4). As the object moves farther away from the camera, the engine switches from LOD 0 to LOD 1 to LOD 2 and so on. This allows the rendering engine to spend the majority of its time transforming and lighting the vertices of the objects that are closest to the camera (and therefore occupy the largest number of pixels on-screen).

Some game engines apply *dynamic tessellation* techniques to expansive meshes like water or terrain. In this technique, the mesh is usually represented by a height field defined on some kind of regular grid pattern. The region of the mesh that is closest to the camera is tessellated to the full resolution of the grid. Regions that are farther away from the camera are tessellated using fewer and fewer grid points.

Progressive meshes are another technique for dynamic tessellation and LOD-ing. With this technique, a single high-resolution mesh is created for display when the object is very close to the camera. (This is essentially the

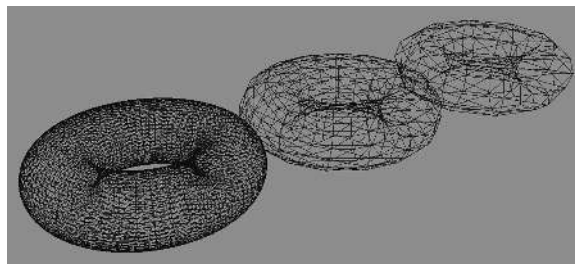


Figure II.4. A chain of LOD meshes, each with a fixed level of tessellation, can be used to approximate uniform triangle-to-pixel density. The leftmost torus is constructed from 5000 triangles, the center torus from 450 triangles and the rightmost torus from 200 triangles.

LOD 0 mesh.) This mesh is automatically detessellated as the object gets farther away by collapsing certain edges. In effect, this process automatically generates a semi-continuous chain of LODs. See <http://research.microsoft.com/en-us/um/people/hoppe/pm.pdf> for a detailed discussion of progressive mesh technology.

11.1.1.3 Constructing a Triangle Mesh

Now that we understand what triangle meshes are and why they're used, let's take a brief look at how they're constructed.

Winding Order

A triangle is defined by the position vectors of its three vertices, which we can denote $\mathbf{p}_1$, $\mathbf{p}_2$ and $\mathbf{p}_3$. The edges of a triangle can be found by simply subtracting the position vectors of adjacent vertices. For example,

$$\begin{aligned}\mathbf{e}_{12} &= \mathbf{p}_2 - \mathbf{p}_1, \\ \mathbf{e}_{13} &= \mathbf{p}_3 - \mathbf{p}_1, \\ \mathbf{e}_{23} &= \mathbf{p}_3 - \mathbf{p}_2.\end{aligned}$$

The normalized cross product of any two edges defines a unit *face normal* $\mathbf{N}$:

$$\mathbf{N} = \frac{\mathbf{e}_{12} \times \mathbf{e}_{13}}{|\mathbf{e}_{12} \times \mathbf{e}_{13}|}.$$

These derivations are illustrated in Figure 11.5. To know the *direction* of the face normal (i.e., the sense of the edge cross product), we need to define which side of the triangle should be considered the front (i.e., the outside surface of an object) and which should be the back (i.e., its inside surface). This can be defined easily by specifying a *winding order*—clockwise (CW) or counterclockwise (CCW).

Most low-level graphics APIs give us a way to *cull* back-facing triangles based on winding order. For example, if we set the cull mode parameter in

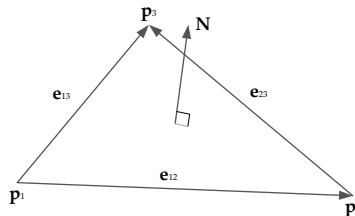


Figure 11.5. Deriving the edges and plane of a triangle from its vertices.

Direct3D (D3DRS_CULL) to D3DCULLMODE_CW, then any triangle whose vertices wind in a clockwise fashion in screen space will be treated as a back-facing triangle and will not be drawn.

Back-face culling is important because we generally don't want to waste time drawing triangles that aren't going to be visible anyway. Also, rendering the back faces of transparent objects can actually cause visual anomalies. The choice of winding order is an arbitrary one, but of course it must be consistent across all assets in the entire game. Inconsistent winding order is a common error among junior 3D modelers.

Triangle Lists

The easiest way to define a mesh is simply to list the vertices in groups of three, each triple corresponding to a single triangle. This data structure is known as a *triangle list*; it is illustrated in Figure 11.6.

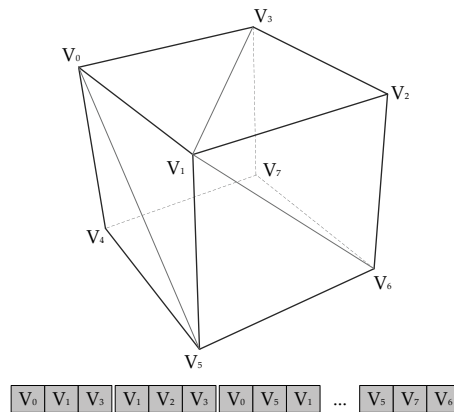


Figure 11.6. A triangle list.

Indexed Triangle Lists

You probably noticed that many of the vertices in the triangle list shown in Figure 11.6 were duplicated, often multiple times. As we'll see in Section 11.1.2.1, we often store quite a lot of metadata with each vertex, so repeating this data in a triangle list wastes memory. It also wastes GPU bandwidth, because a duplicated vertex will be transformed and lit multiple times.

For these reasons, most rendering engines make use of a more efficient data structure known as an *indexed triangle list*. The basic idea is to list the vertices once with no duplication and then to use lightweight vertex *indices* (usually occupying only 16 bits each) to define the triples of vertices that constitute the

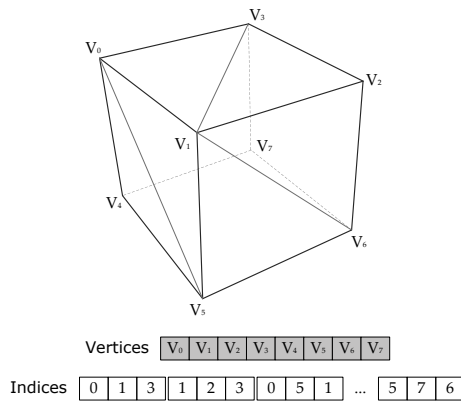


Figure 11.7. An indexed triangle list.

triangles. The vertices are stored in an array known as a *vertex buffer* (DirectX) or *vertex array* (OpenGL). The indices are stored in a separate buffer known as an *index buffer* or *index array*. This technique is shown in Figure 11.7.

Strips and Fans

Specialized mesh data structures known as *triangle strips* and *triangle fans* are sometimes used for game rendering. Both of these data structures eliminate the need for an index buffer, while still reducing vertex duplication to some degree. They accomplish this by predefining the order in which vertices must appear and how they are combined to form triangles.

In a strip, the first three vertices define the first triangle. Each subsequent vertex forms an entirely new triangle, along with its previous two neighbors. To keep the winding order of a triangle strip consistent, the previous two neighbor vertices swap places after each new triangle. A triangle strip is shown in Figure 11.8.

In a fan, the first three vertices define the first triangle and each subsequent vertex defines a new triangle with the previous vertex and the first vertex in the fan. This is illustrated in Figure 11.9.

Vertex Cache Optimization

When a GPU processes an indexed triangle list, each triangle can refer to any vertex within the vertex buffer. The vertices must be processed in the order they appear within the triangles, because the integrity of each triangle must be maintained for the rasterization stage. As vertices are processed by the vertex shader, they are cached for reuse. If a subsequent primitive refers to a vertex

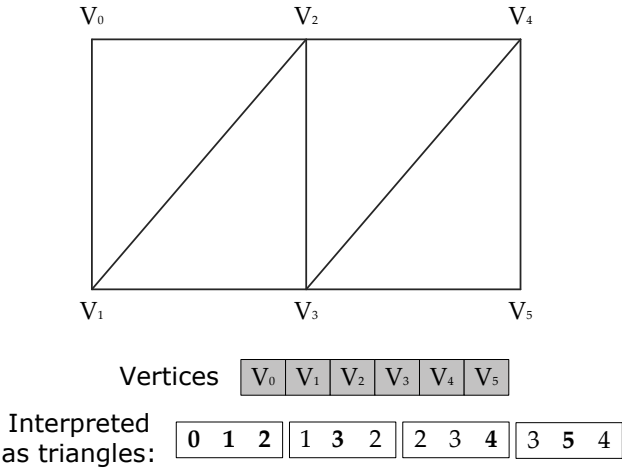


Figure II.8. A triangle strip.

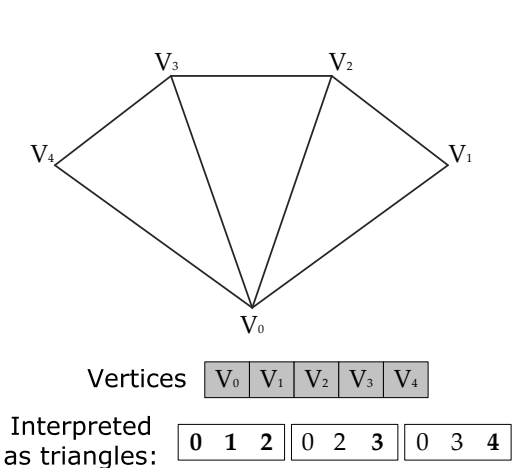


Figure II.9. A triangle fan.

that already resides in the cache, its processed attributes are used instead of reprocessing the vertex.

Strips and fans are used in part because they can potentially save memory (no index buffer required) and in part because they tend to improve the cache coherency of the memory accesses made by the GPU to video RAM. Even better, we can use an *indexed strip* or *indexed fan* to virtually eliminate vertex duplication (which can often save more memory than eliminating the index buffer), while still reaping the cache coherency benefits of the strip or fan vertex ordering.

Indexed triangle lists can also be cache-optimized without restricting ourselves to strip or fan vertex ordering. A *vertex cache optimizer* is an offline geometry processing tool that attempts to list the triangles in an order that optimizes vertex reuse within the cache. It generally takes into account factors such as the size of the vertex cache(s) present on a particular type of GPU and the algorithms used by the GPU to decide when to cache vertices and when to discard them. For example, the vertex cache optimizer included in Sony’s *Edge* geometry processing library can achieve rendering throughput that is up to 4% better than what is possible with triangle stripping.

II.I.I.4 Model Space

The position vectors of a triangle mesh’s vertices are usually specified relative to a convenient local coordinate system called *model space*, *local space*, or *object space*. The origin of model space is usually either in the center of the object or at

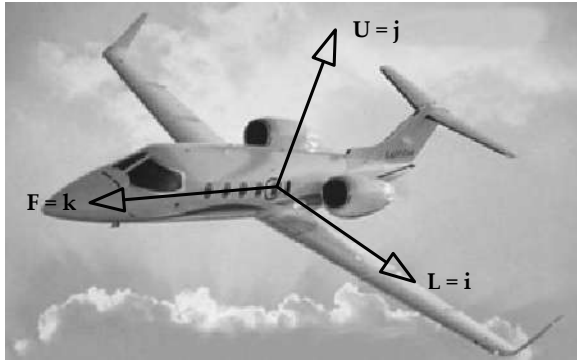


Figure 11.10. One possible mapping of the model-space axes.

some other convenient location, like on the floor between the feet of a character or on the ground at the horizontal centroid of the wheels of a vehicle.

As we learned in Section 5.3.9.1, the sense of the model-space axes is arbitrary, but the axes typically align with the natural “front,” “left,” “right” and “up” directions on the model. For a little mathematical rigor, we can define three unit vectors F , L (or R) and U and map them as desired onto the unit basis vectors i , j and k (and hence to the x -, y - and z -axes, respectively) in model space. For example, a common mapping is $L = i$, $U = j$ and $F = k$. The mapping is completely arbitrary, but it’s important to be consistent for all models across the entire engine. Figure 11.10 shows one possible mapping of the model-space axes for an aircraft model.

11.1.1.5 World Space and Mesh Instancing

Many individual meshes are composed into a complete scene by positioning and orienting them within a common coordinate system known as *world space*. Any one mesh might appear many times in a scene—examples include a street lined with identical lamp posts, a faceless mob of soldiers or a swarm of spiders attacking the player. We call each such object a *mesh instance*.

A mesh instance contains a reference to its shared mesh data and also includes a transformation matrix that converts the mesh’s vertices from model space to world space, within the context of that particular instance. This matrix is called the *model-to-world* matrix, or sometimes just the *world matrix*. Using the notation from Section 5.3.10.2, this matrix can be written as follows:

$$\mathbf{M}_{M \rightarrow W} = \begin{bmatrix} (\mathbf{RS})_{M \rightarrow W} & 0 \\ \mathbf{t}_M & 1 \end{bmatrix},$$

where the upper 3×3 matrix $(\mathbf{RS})_{M \rightarrow W}$ rotates and scales model-space vertices into world space, and $\mathbf{t}_M$ is the translation of the model-space axes expressed in world space. If we have the unit model-space basis vectors $\mathbf{i}_M$, $\mathbf{j}_M$ and $\mathbf{k}_M$, expressed in world-space coordinates, this matrix can also be written as follows:

$$\mathbf{M}_{M \rightarrow W} = \left[\begin{array}{c|c} \mathbf{i}_M & 0 \\ \mathbf{j}_M & 0 \\ \mathbf{k}_M & 0 \\ \hline \mathbf{t}_M & 1 \end{array} \right].$$

Given a vertex expressed in model-space coordinates, the rendering engine calculates its world-space equivalent as follows:

$$\mathbf{v}_W = \mathbf{v}_M \mathbf{M}_{M \rightarrow W}.$$

We can think of the matrix $\mathbf{M}_{M \rightarrow W}$ as a description of the position and orientation of the model-space axes themselves, expressed in world-space coordinates. Or we can think of it as a matrix that transforms vertices from model space to world space.

When rendering a mesh, the model-to-world matrix is also applied to the surface normals of the mesh (see Section 11.1.2.1). Recall from Section 5.3.11, that in order to transform normal vectors properly, we must multiply them by the inverse transpose of the model-to-world matrix. If our matrix does not contain any scale or shear, we can transform our normal vectors correctly by simply setting their w components to zero prior to multiplication by the model-to-world matrix, as described in Section 5.3.6.1.

Some meshes like buildings, terrain and other background elements are entirely static and unique. The vertices of these meshes are often expressed in world space, so their model-to-world matrices are identity and can be ignored.

11.1.2 Describing the Visual Properties of a Surface

In order to properly render and light a surface, we need a description of its *visual properties*. Surface properties include geometric information, such as the direction of the surface normal at various points on the surface. They also encompass a description of how light should interact with the surface. This includes diffuse color, shininess/reflectivity, roughness or texture, degree of opacity or transparency, index of refraction and other optical properties. Surface properties might also include a specification of how the surface should change over time (e.g., how an animated character's skin should track the joints of its skeleton or how the surface of a body of water should move).

The key to rendering photorealistic images is properly accounting for light's behavior as it interacts with the objects in the scene. Hence rendering engineers need to have a good understanding of how light works, how it is transported through an environment and how the virtual camera "senses" it and translates it into the colors stored in the pixels on-screen.

11.1.2.1 Introduction to Light and Color

Light is electromagnetic radiation; it acts like both a wave and a particle in different situations. The color of light is determined by its *intensity* I and its *wavelength* λ (or its frequency f , where $f = 1/\lambda$). The visible gamut ranges from a wavelength of 740 nm (or a frequency of 430 THz) to a wavelength of 380 nm (750 THz). A beam of light may contain a single pure wavelength (i.e., the colors of the rainbow, also known as the *spectral colors*), or it may contain a mixture of various wavelengths. We can draw a graph showing how much of each frequency a given beam of light contains, called a *spectral plot*. White light contains a little bit of all wavelengths, so its spectral plot would look roughly like a box extending across the entire visible band. Pure green light contains only one wavelength, so its spectral plot would look like a single infinitesimally narrow spike at about 570 THz.

Light-Object Interactions

Light can have many complex interactions with matter. Its behavior is governed in part by the *medium* through which it is traveling and in part by the shape and properties of the *interfaces* between different types of media (air-solid, air-water, water-glass, etc.). Technically speaking, a surface is really just an interface between two different types of media.

Despite all of its complexity, light can really only do four things:

- It can be *absorbed*.
- It can be *reflected*.
- It can be *transmitted* through an object, usually being *refracted* (bent) in the process.
- It can be *diffracted* when passing through very narrow openings.

Most photorealistic rendering engines account for the first three of these behaviors; diffraction is not usually taken into account because its effects are rarely noticeable in most scenes.

Only certain wavelengths may be absorbed by a surface, while others are reflected. This is what gives rise to our perception of the color of an object. For example, when white light falls on a red object, all wavelengths except

red are absorbed, hence the object appears red. The same perceptual effect is achieved when red light is cast onto a white object—our eyes don't know the difference.

Reflections can be *diffuse*, meaning that an incoming ray is scattered equally in all directions. Reflections can also be *specular*, meaning that an incident light ray will reflect directly or be spread only into a narrow cone. Reflections can also be *anisotropic*, meaning that the way in which light reflects from a surface changes depending on the angle at which the surface is viewed.

When light is transmitted through a volume, it can be *scattered* (as is the case for translucent objects), partially *absorbed* (as with colored glass), or *refracted* (as happens when light travels through a prism). The refraction angles can be different for different wavelengths, leading to spectral spreading. This is why we see rainbows when light passes through raindrops and glass prisms. Light can also enter a semi-solid surface, bounce around and then exit the surface at a different point from the one at which it entered the surface. We call this *subsurface scattering*, and it is one of the effects that gives skin, wax and marble their characteristic warm appearance.

Color Spaces and Color Models

A *color model* is a three-dimensional coordinate system that measures colors. A *color space* is a specific standard for how numerical colors in a particular color model should be mapped onto the colors perceived by human beings in the real world. Color models are typically three-dimensional because of the three types of color sensors (cones) in our eyes, which are sensitive to different wavelengths of light.

The most commonly used color model in computer graphics is the RGB model. In this model, color space is represented by a unit cube, with the relative intensities of red, green and blue light measured along its axes. The red, green and blue components are called *color channels*. In the canonical RGB color model, each channel ranges from zero to one. So the color (0, 0, 0) represents black, while (1, 1, 1) represents white.

When colors are stored in a bitmapped image, various color formats can be employed. A color format is defined in part by the number of *bits per pixel* it occupies and, more specifically, the number of bits used to represent each color channel. The RGB888 format uses eight bits per channel, for a total of 24 bits per pixel. In this format, each channel ranges from 0 to 255 rather than from zero to one. RGB565 uses five bits for red and blue and six for green, for a total of 16 bits per pixel. A paletted format might use eight bits per pixel to store indices into a 256-element color palette, each entry of which might be stored in RGB888 or some other suitable format.

A number of other color models are also used in 3D rendering. We'll see how the log-LUV color model is used for *high dynamic range* (HDR) lighting in Section 11.3.1.5.

Opacity and the Alpha Channel

A fourth channel called *alpha* is often tacked on to RGB color vectors. As mentioned in Section 11.1.1, alpha measures the opacity of an object. When stored in an image pixel, alpha represents the opacity of the pixel.

RGB color formats can be extended to include an alpha channel, in which case they are referred to as RGBA or ARGB color formats. For example, RGBA8888 is a 32 bit-per-pixel format with eight bits each for red, green, blue and alpha. RGBA5551 is a 16 bit-per-pixel format with one-bit alpha; in this format, colors can either be fully opaque or fully transparent.

11.1.2.2 Vertex Attributes

The simplest way to describe the visual properties of a surface is to specify them at discrete points on the surface. The vertices of a mesh are a convenient place to store surface properties, in which case they are called *vertex attributes*.

A typical triangle mesh includes some or all of the following attributes at each vertex. As rendering engineers, we are of course free to define any additional attributes that may be required in order to achieve a desired visual effect on-screen.

- *Position vector* ($\mathbf{p}_i = [p_{ix} \ p_{iy} \ p_{iz}]$). This is the 3D position of the i th vertex in the mesh. It is usually specified in a coordinate space local to the object, known as *model space*.
- *Vertex normal* ($\mathbf{n}_i = [n_{ix} \ n_{iy} \ n_{iz}]$). This vector defines the unit surface normal at the position of vertex i . It is used in per-vertex dynamic lighting calculations.
- *Vertex tangent* ($\mathbf{t}_i = [t_{ix} \ t_{iy} \ t_{iz}]$) and *bitangent* ($\mathbf{b}_i = [b_{ix} \ b_{iy} \ b_{iz}]$). These two unit vectors lie perpendicular to one another and to the vertex normal $\mathbf{n}_i$. Together, the three vectors $\mathbf{n}_i$, $\mathbf{t}_i$ and $\mathbf{b}_i$ define a set of coordinate axes known as *tangent space*. This space is used for various per-pixel lighting calculations, such as normal mapping and environment mapping. (The bitangent $\mathbf{b}_i$ is sometimes confusingly called the *binormal*, even though it is *not* normal to the surface.)
- *Diffuse color* ($\mathbf{d}_i = [d_{iR} \ d_{iG} \ d_{iB} \ d_{iA}]$). This four-element vector describes the diffuse color of the surface, expressed in the RGB color space. It typically also includes a specification of the opacity or *alpha* (A) of the

surface at the position of the vertex. This color may be calculated offline (static lighting) or at runtime (dynamic lighting).

- *Specular color* ($\mathbf{s}_i = [s_{iR} \ s_{iG} \ s_{iB} \ s_{iA}]$). This quantity describes the color of the specular highlight that should appear when light reflects directly from a shiny surface onto the virtual camera's imaging plane.
- *Texture coordinates* ($\mathbf{u}_{ij} = [u_{ij} \ v_{ij}]$). Texture coordinates allow a two- (or sometimes three-) dimensional bitmap to be “shrink wrapped” onto the surface of a mesh—a process known as *texture mapping*. A texture coordinate (u, v) describes the location of a particular vertex within the two-dimensional normalized coordinate space of the texture. A triangle can be mapped with more than one texture; hence it can have more than one set of texture coordinates. We've denoted the distinct sets of texture coordinates via the subscript j above.
- *Skinning weights* ($\mathbf{k}_{ij} = [k_{ij} \ w_{ij}]$). In skeletal animation, the vertices of a mesh are attached to individual joints in an articulated skeleton. In this case, each vertex must specify to which joint it is attached via an index, k . A vertex can be influenced by multiple joints, in which case the final vertex position becomes a *weighted average* of these influences. Thus, the weight of each joint's influence is denoted by a weighting factor w . In general, a vertex i can have multiple joint influences j , each denoted by the pair of numbers (k_{ij}, w_{ij}) .

II.1.2.3 Vertex Formats

Vertex attributes are typically stored within a data structure such as a C struct or a C++ class. The layout of such a data structure is known as a *vertex format*. Different meshes require different combinations of attributes and hence need different vertex formats. The following are some examples of common vertex formats:

```
// Simplest possible vertex -- position only (useful for
// shadow volume extrusion, silhouette edge detection
// for cartoon rendering, z-prepass, etc.)
struct Vertex1P
{
    Vector3 m_p; // position
};

// A typical vertex format with position, vertex normal
// and one set of texture coordinates.
struct Vertex1P1N1UV
{
    Vector3 m_p; // position
```

```

    Vector3 m_n;        // vertex normal
    F32      m_uv[2];   // (u, v) texture coordinate
};

// A skinned vertex with position, diffuse and specular
// colors and four weighted joint influences.
struct Vertex1P1D1S2UV4J
{
    Vector3 m_p;        // position
    Color4  m_d;        // diffuse color and translucency
    Color4  m_s;        // specular color
    F32      m_uv0[2];  // first set of tex coords
    F32      m_uv1[2];  // second set of tex coords
    U8       m_k[4];    // four joint indices, and...
    F32      m_w[3];    // three joint weights, for
                        // skinning (fourth is calc'd
                        // from the first three)
};

```

Clearly the number of possible permutations of vertex attributes—and hence the number of distinct vertex formats—can grow to be extremely large. (In fact the number of formats is theoretically unbounded, if one were to permit any number of texture coordinates and/or joint weights.) Management of all these vertex formats is a common source of headaches for any graphics programmer.

Some steps can be taken to reduce the number of vertex formats that an engine has to support. In practical graphics applications, many of the theoretically possible vertex formats are simply not useful, or they cannot be handled by the graphics hardware or the game’s shaders. Some game teams also limit themselves to a subset of the useful/feasible vertex formats in order to keep things more manageable. For example, they might only allow zero, two or four joint weights per vertex, or they might decide to support no more than two sets of texture coordinates per vertex. Some GPUs are capable of extracting a subset of attributes from a vertex data structure, so game teams can also choose to use a single “überformat” for all meshes and let the hardware select the relevant attributes based on the requirements of the shader.

11.1.2.4 Attribute Interpolation

The attributes at a triangle’s vertices are just a coarse, discretized approximation to the visual properties of the surface as a whole. When rendering a triangle, what really matters are the visual properties at the interior points of the triangle as “seen” through each pixel on-screen. In other words, we need to know the values of the attributes on a *per-pixel* basis, not a per-vertex basis.

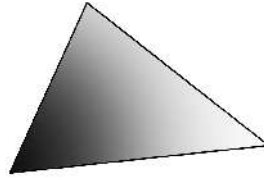


Figure II.11. A Gouraud-shaded triangle with different shades of gray at the vertices.

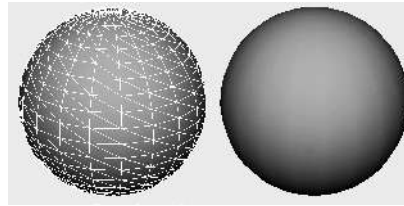


Figure II.12. Gouraud shading can make faceted objects appear to be smooth.

One simple way to determine the per-pixel values of a mesh's surface attributes is to *linearly interpolate* the per-vertex attribute data. When applied to vertex colors, attribute interpolation is known as *Gouraud shading*. An example of Gouraud shading applied to a triangle is shown in Figure 11.11, and its effects on a simple triangle mesh are illustrated in Figure 11.12. Interpolation is routinely applied to other kinds of vertex attribute information as well, such as vertex normals, texture coordinates and depth.

Vertex Normals and Smoothing

As we'll see in Section 11.1.3, *lighting* is the process of calculating the color of an object at various points on its surface, based on the visual properties of the surface and the properties of the light impinging upon it. The simplest way to light a mesh is to calculate the color of the surface on a *per-vertex* basis. In other words, we use the properties of the surface and the incoming light to calculate the diffuse color of each vertex ($\mathbf{d}_i$). These vertex colors are then interpolated across the triangles of the mesh via Gouraud shading.

In order to determine how a ray of light will reflect from a point on a surface, most lighting models make use of a vector that is *normal* to the surface at the point of the light ray's impact. Since we're performing lighting calculations on a per-vertex basis, we can use the vertex normal $\mathbf{n}_i$ for this purpose. Therefore, the directions of a mesh's vertex normals can have a significant impact on the final appearance of a mesh.

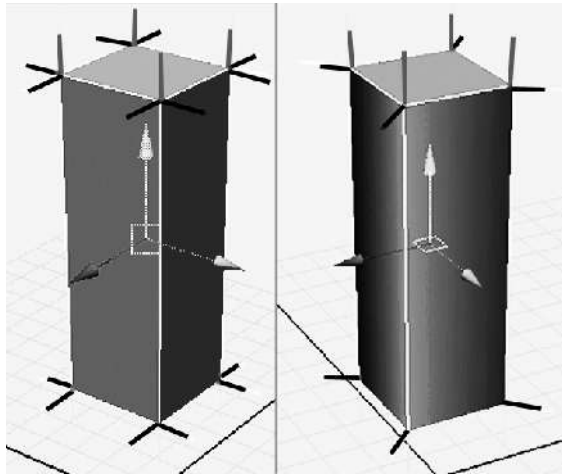


Figure 11.13. The directions of a mesh’s vertex normals can have a profound effect on the colors calculated during per-vertex lighting calculations.

As an example, consider a tall, thin, four-sided box. If we want the box to appear to be sharp-edged, we can specify the vertex normals to be perpendicular to the faces of the box. As we light each triangle, we will encounter the same normal vector at all three vertices, so the resulting lighting will appear flat, and it will abruptly change at the corners of the box just as the vertex normals do.

We can also make the same box mesh look a bit like a smooth cylinder by specifying vertex normals that point radially outward from the box’s center line. In this case, the vertices of each triangle will have different vertex normals, causing us to calculate different colors at each vertex. Gouraud shading will smoothly interpolate these vertex colors, resulting in lighting that appears to vary smoothly across the surface. This effect is illustrated in Figure 11.13.

11.1.2.5 Textures

When triangles are relatively large, specifying surface properties on a per-vertex basis can be too coarse-grained. Linear attribute interpolation isn’t always what we want, and it can lead to undesirable visual anomalies.

As an example, consider the problem of rendering the bright *specular highlight* that can occur when light shines on a glossy object. If the mesh is highly tessellated, per-vertex lighting combined with Gouraud shading can yield reasonably good results. However, when the triangles are too large, the errors that arise from linearly interpolating the specular highlight can become jarringly obvious, as shown in Figure 11.14.

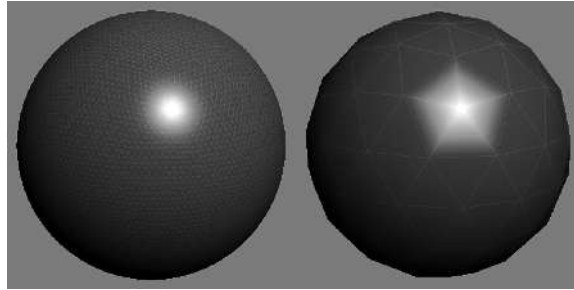


Figure II.14. Linear interpolation of vertex attributes does not always yield an adequate description of the visual properties of a surface, especially when tessellation is low.

To overcome the limitations of per-vertex surface attributes, rendering engineers use bitmapped images known as *texture maps*. A texture often contains color information and is usually projected onto the triangles of a mesh. In this case, it acts a bit like those silly fake tattoos we used to apply to our arms when we were kids. But a texture can contain other kinds of visual surface properties as well as colors. And a texture needn't be projected onto a mesh—for example, a texture might be used as a stand-alone data table. The individual picture elements of a texture are called *texels* to differentiate them from the pixels on the screen.

The dimensions of a texture bitmap are constrained to be powers of two on some graphics hardware. Typical texture dimensions include 256×256 , 512×512 , 1024×1024 and 2048×2048 , although textures can be any size on most hardware, provided the texture fits into video memory. Some graphics hardware imposes additional restrictions, such as requiring textures to be square, or lifts some restrictions, such as not constraining texture dimensions to be powers of two.

Types of Textures

The most common type of texture is known as a *diffuse map*, or *albedo map*. It describes the diffuse surface color at each texel on a surface and acts like a decal or paint job on the surface.

Other types of textures are used in computer graphics as well, including *normal maps* (which store unit normal vectors at each texel, encoded as RGB values), *gloss maps* (which encode how shiny a surface should be at each texel), *environment maps* (which contain a picture of the surrounding environment for rendering reflections) and many others. See Section 11.3.1 for a discussion of how various types of textures can be used for image-based lighting and other effects.

We can actually use texture maps to store any information that we happen to need in our lighting calculations. For example, a one-dimensional texture could be used to store sampled values of a complex math function, a color-to-color mapping table, or any other kind of look-up table (LUT).

Texture Coordinates

Let's consider how to project a two-dimensional texture onto a mesh. To do this, we define a two-dimensional coordinate system known as *texture space*. A texture coordinate is usually represented by a normalized pair of numbers denoted (u, v) . These coordinates always range from $(0, 0)$ at the bottom left corner of the texture to $(1, 1)$ at the top right. Using normalized coordinates like this allows the same coordinate system to be used regardless of the dimensions of the texture.

To map a triangle onto a 2D texture, we simply specify a pair of texture coordinates (u_i, v_i) at each vertex i . This effectively maps the triangle onto the image plane in texture space. An example of texture mapping is depicted in Figure 11.15.

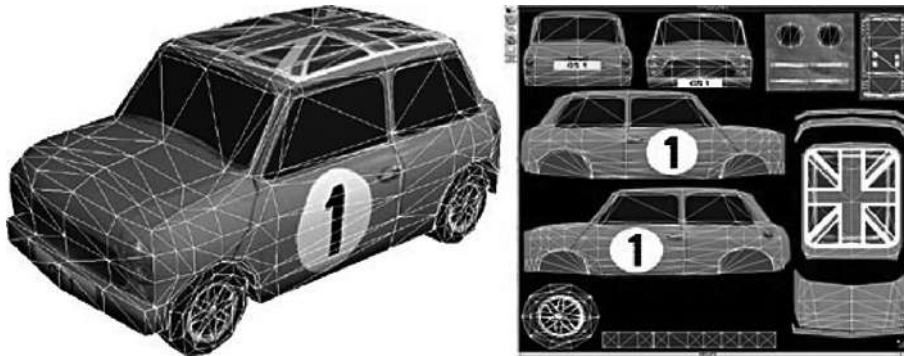


Figure 11.15. An example of texture mapping. The triangles are shown both in three-dimensional space and in texture space.

Texture Addressing Modes

Texture coordinates are permitted to extend beyond the $[0, 1]$ range. The graphics hardware can handle out-of-range texture coordinates in any one of the following ways. These are known as *texture addressing modes*; which mode is used is under the control of the user.

- *Wrap*. In this mode, the texture is repeated over and over in every direction. All texture coordinates of the form (ju, kv) are equivalent to the coordinate (u, v) , where j and k are arbitrary integers.

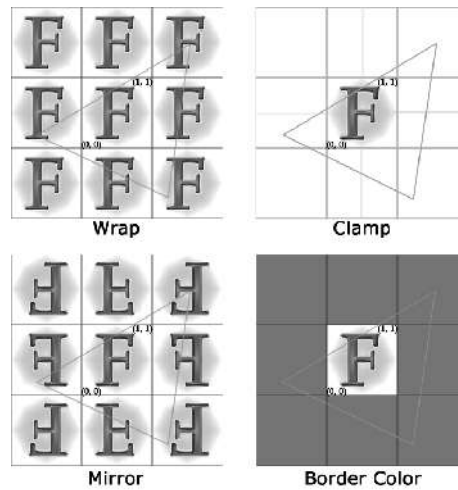


Figure 11.16. Texture addressing modes.

- *Mirror*. This mode acts like wrap mode, except that the texture is mirrored about the v -axis for odd integer multiples of u , and about the u -axis for odd integer multiples of v .
- *Clamp*. In this mode, the colors of the texels around the outer edge of the texture are simply extended when texture coordinates fall outside the normal range.
- *Border color*. In this mode, an arbitrary user-specified color is used for the region outside the $[0, 1]$ texture coordinate range.

These texture addressing modes are depicted in Figure 11.16.

Texture Formats

Texture bitmaps can be stored on disk in virtually any image format, provided your game engine includes the code necessary to read it into memory. Common formats include Targa (.tga), Portable Network Graphics (.png), Windows Bitmap (.bmp) and Tagged Image File Format (.tif). In memory, textures are usually represented as two-dimensional (strided) arrays of pixels using various color formats, including RGB888, RGBA8888, RGB565, RGBA5551 and so on.

Most modern graphics cards and graphics APIs support *compressed textures*. DirectX supports a family of compressed formats known as DXT or S3 Texture Compression (S3TC). We won't cover the details here, but the basic idea is to break the texture into 4×4 blocks of pixels and use a small

color palette to store the colors for each block. You can read more about S3 compressed texture formats at http://en.wikipedia.org/wiki/S3_Texture_Compression.

Compressed textures have the obvious benefit of using less memory than their uncompressed counterparts. An additional unexpected plus is that they are faster to render with as well. S3 compressed textures achieve this speed-up because of more cache-friendly memory access patterns— 4×4 blocks of adjacent pixels are stored in a single 64- or 128-bit machine word—and because more of the texture can fit into the cache at once. Compressed textures do suffer from compression artifacts. While the anomalies are usually not noticeable, there are situations in which uncompressed textures must be used.

Texel Density and Mipmapping

Imagine rendering a full-screen quad (a rectangle composed of two triangles) that has been mapped with a texture whose resolution exactly matches that of the screen. In this case, each texel maps exactly to a single pixel on-screen, and we say that the *texel density* (ratio of texels to pixels) is one. When this same quad is viewed at a distance, its on-screen area becomes smaller. The resolution of the texture hasn't changed, so the quad's texel density is now greater than one (meaning that more than one texel is contributing to each pixel).

Clearly texel density is not a fixed quantity—it changes as a texture-mapped object moves relative to the camera. Texel density affects the memory consumption and the visual quality of a three-dimensional scene. When the texel density is much less than one, the texels become significantly larger than a pixel on-screen, and you can start to see the edges of the texels. This destroys the illusion. When texel density is much greater than one, many texels contribute to a single pixel on-screen. This can cause a *moiré banding pattern*, as shown in Figure 11.17. Worse, a pixel's color can appear to swim and flicker as different texels within the boundaries of the pixel dominate its color depending on subtle changes in camera angle or position. Rendering a distant object with a very high texel density can also be a waste of memory if the player can *never* get close to it. After all, why keep such a high-res texture in memory if no one will ever see all that detail?

Ideally we'd like to maintain a texel density that is close to one at all times, for both nearby and distant objects. This is impossible to achieve exactly, but it can be approximated via a technique called *mipmapping*. For each texture, we create a sequence of lower-resolution bitmaps, each of which is one-half the width and one-half the height of its predecessor. We call each of these images a *mipmap*, or *mip level*. For example, a 64×64 texture would have the

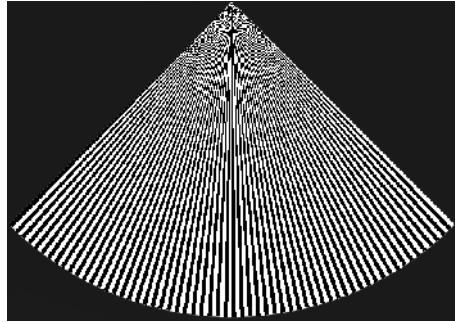


Figure II.17. A texel density greater than one can lead to a moiré pattern.

following mip levels: 64×64 , 32×32 , 16×16 , 8×8 , 4×4 , 2×2 and 1×1 , as shown in Figure 11.18. Once we have mipmapped our textures, the graphics hardware selects the appropriate mip level based on a triangle's distance away from the camera, in an attempt to maintain a texel density that is close to one. For example, if a texture takes up an area of 40×40 on-screen, the 64×64 mip level might be selected; if that same texture takes up only a 10×10 area, the 16×16 mip level might be used. As we'll see below, *trilinear filtering* allows the hardware to sample two adjacent mip levels and blend the results. In this case, a 10×10 area might be mapped by blending the 16×16 and 8×8 mip levels together.

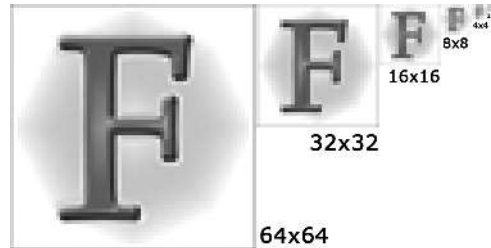


Figure II.18. Mip levels for a 64×64 texture.

World-Space Texel Density

The term “texel density” can also be used to describe the ratio of texels to world-space area on a textured surface. For example, a 2 m cube mapped with a 256×256 texture would have a texel density of $256^2 / 2^2 = 16,384$. I will call this *world-space texel density* to differentiate it from the screen-space texel density we’ve been discussing thus far.

World-space texel density need not be close to one, and in fact the specific value will usually be much greater than one and depends entirely upon your choice of world units. Nonetheless, it is important for objects to be texture mapped with a reasonably consistent world-space texel density. For example, we would expect all six sides of a cube to occupy the same texture area. If this were not the case, the texture on one side of the cube would have a lower-resolution appearance than another side, which can be noticeable to the player. Many game studios provide their art teams with guidelines and in-engine texel density visualization tools in an effort to ensure that all objects in the game have a reasonably consistent world-space texel density.

Texture Filtering

When rendering a pixel of a textured triangle, the graphics hardware samples the texture map by considering where the pixel center falls in texture space. There is usually not a clean one-to-one mapping between texels and pixels, and pixel centers can fall at any place in texture space, including directly on the boundary between two or more texels. Therefore, the graphics hardware usually has to sample more than one texel and blend the resulting colors to arrive at the actual sampled texel color. We call this *texture filtering*.

Most graphics cards support the following kinds of texture filtering:

- *Nearest neighbor*. In this crude approach, the texel whose center is closest to the pixel center is selected. When mipmapping is enabled, the mip level is selected whose resolution is nearest to but greater than the ideal theoretical resolution needed to achieve a screen-space texel density of one.
- *Bilinear*. In this approach, the four texels surrounding the pixel center are sampled, and the resulting color is a weighted average of their colors (where the weights are based on the distances of the texel centers from the pixel center). When mipmapping is enabled, the nearest mip level is selected.
- *Trilinear*. In this approach, bilinear filtering is used on each of the two nearest mip levels (one higher-res than the ideal and the other lower-res), and these results are then linearly interpolated. This eliminates abrupt visual boundaries between mip levels on-screen.
- *Anisotropic*. Both bilinear and trilinear filtering sample 2×2 square blocks of texels. This is the right thing to do when the textured surface is being viewed head-on, but it's incorrect when the surface is at an oblique angle relative to the virtual screen plane. Anisotropic filtering samples texels within a trapezoidal region corresponding to the view

angle, thereby increasing the quality of textured surfaces when viewed at an angle.

II.1.2.6 Materials

A *material* is a complete description of the visual properties of a mesh. This includes a specification of the textures that are mapped to its surface and also various higher-level properties, such as which shader programs to use when rendering the mesh, the input parameters to those shaders and other parameters that control the functionality of the graphics acceleration hardware itself.

While technically part of the surface properties description, vertex attributes are not considered to be part of the material. However, they come along for the ride with the mesh, so a mesh-material pair contains all the information we need to render the object. Mesh-material pairs are sometimes called *render packets*, and the term “geometric primitive” is sometimes extended to encompass mesh-material pairs as well.

A 3D model typically uses more than one material. For example, a model of a human would have separate materials for the hair, skin, eyes, teeth and various kinds of clothing. For this reason, a mesh is usually divided into *sub-meshes*, each mapped to a single material. The OGRE rendering engine implements this design via its `Ogre::SubMesh` class.



Figure II.19. A variation on the classic “Cornell box” scene illustrating how realistic lighting can make even the simplest scene appear photorealistic.



Figure 11.20. A scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) rendered without textures. (See Color Plate XV.)

11.1.3 Lighting Basics

Lighting is at the heart of all CG rendering. Without good lighting, an otherwise beautifully modeled scene will look flat and artificial. Likewise, even the simplest of scenes can be made to look extremely realistic when it is lit accurately. The classic “Cornell box” scene, shown in Figure 11.19, is an excellent example of this.

The sequence of screenshots from Naughty Dog’s *The Last of Us: Remastered* is another good illustration of the importance of lighting. In Figure 11.20, the scene is rendered without textures. Figure 11.21 shows the same scene with diffuse textures applied. The fully lit scene is shown in Figure 11.22. Notice the marked jump in realism when lighting is applied to the scene.

The term *shading* is often used as a loose generalization of lighting plus other visual effects. As such, “shading” encompasses procedural deformation of vertices to simulate the motion of a water surface, generation of hair curves or fur shells, tessellation of high-order surfaces, and pretty much any other calculation that’s required to render a scene.

In the following sections, we’ll lay the foundations of lighting that we’ll need in order to understand graphics hardware and the rendering pipeline. We’ll return to the topic of lighting in Section 11.3, where we’ll survey some advanced lighting and shading techniques.

11.1.3.1 Local and Global Illumination Models

Rendering engines use various mathematical models of light-surface and light-volume interactions called *light transport models*. The simplest models only account for *direct lighting* in which light is emitted, bounces off a single



Figure II.21. The same scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) with only diffuse textures applied. (See Color Plate XVI.)



Figure II.22. Scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) with full lighting. (See Color Plate XVII.)

object in the scene, and then proceeds directly to the imaging plane of the virtual camera. Such simple models are called *local illumination models*, because only the local effects of light on a single object are considered; objects do not affect one another's appearance in a local lighting model. Not surprisingly, local models were the first to be used in games, and they are still in use today—local lighting can produce surprisingly realistic results in some circumstances.

True photorealism can only be achieved by accounting for *indirect lighting*, where light bounces multiple times off many surfaces before reaching the virtual camera. Lighting models that account for indirect lighting are called *global illumination models*. Some global illumination models are targeted at simulating one specific visual phenomenon, such as producing realistic shadows, modeling reflective surfaces, accounting for interreflection between objects (where the color of one object affects the colors of surrounding objects), and modeling caustic effects (the intense reflections from water or a shiny metal surface). Other global illumination models attempt to provide a holistic account of a wide range of optical phenomena. Ray tracing and radiosity methods are examples of such technologies.

Global illumination is described completely by a mathematical formulation known as the *rendering equation* or *shading equation*. It was introduced in 1986 by J. T. Kajiya as part of a seminal SIGGRAPH paper. In a sense, every rendering technique can be thought of as a full or partial solution to the rendering equation, although they differ in their fundamental approach to solving it and in the assumptions, simplifications and approximations they make. See http://en.wikipedia.org/wiki/Rendering_equation, [10], [2] and virtually any other text on advanced rendering and lighting for more details on the rendering equation.

11.1.3.2 The Phong Lighting Model

The most common local lighting model employed by game rendering engines is the *Phong* reflection model. It models the light reflected from a surface as a sum of three distinct terms:

- The *ambient* term models the overall lighting level of the scene. It is a gross approximation of the amount of indirect bounced light present in the scene. Indirect bounces are what cause regions in shadow not to appear totally black.
- The *diffuse* term accounts for light that is reflected uniformly in all directions from each direct light source. This is a good approximation to the way in which real light bounces off a matte surface, such as a block of wood or a piece of cloth.
- The *specular* term models the bright highlights we sometimes see when viewing a glossy surface. Specular highlights occur when the viewing angle is closely aligned with a path of direct reflection from a light source.

Figure 11.23 shows how the ambient, diffuse and specular terms add together to produce the final intensity and color of a surface.

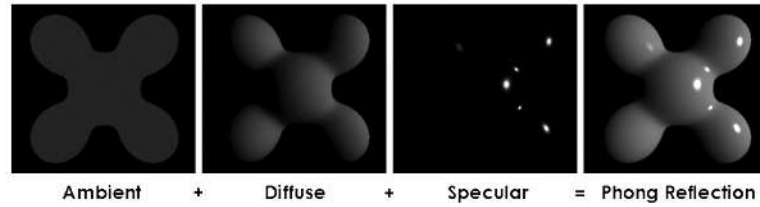


Figure 11.23. Ambient, diffuse and specular terms are summed to calculate Phong reflection.

To calculate Phong reflection at a specific point on a surface, we require a number of input parameters. The Phong model is normally applied to all three color channels (R, G and B) independently, so all of the color parameters in the following discussion are three-element vectors. The inputs to the Phong model are:

- the viewing direction vector $\mathbf{V} = [V_x \ V_y \ V_z]$, which extends from the reflection point to the virtual camera's focal point (i.e., the negation of the camera's world-space "front" vector);
- the ambient light intensity for the three color channels, $\mathbf{A} = [A_R \ A_G \ A_B]$;
- the surface normal $\mathbf{N} = [N_x \ N_y \ N_z]$ at the point the light ray impinges on the surface;
- the surface reflectance properties, which are
 - the ambient reflectivity $\mathbf{k}_A = [k_{AR} \ k_{AG} \ k_{AB}]$,
 - the diffuse reflectivity $\mathbf{k}_D = [k_{DR} \ k_{DG} \ k_{DB}]$,
 - the specular reflectivity $\mathbf{k}_S = [k_{SR} \ k_{SG} \ k_{SB}]$,
 - a specular "glossiness" exponent α ;
- and, for each light source i ,
 - the light's color and intensity $\mathbf{C}_i = [C_{iR} \ C_{iG} \ C_{iB}]$,
 - the direction vector $\mathbf{L}_i$ from the reflection point to the light source.

In the Phong model, the intensity $\mathbf{I}$ of light reflected from a point can be expressed with the following vector equation:

$$\mathbf{I} = (\mathbf{k}_A \otimes \mathbf{A}) + \sum_i [\mathbf{k}_D(\mathbf{N} \cdot \mathbf{L}_i) + \mathbf{k}_S(\mathbf{R}_i \cdot \mathbf{V})^\alpha] \otimes \mathbf{C}_i,$$

where the sum is taken over all lights i affecting the point in question. Recall that the operator $\otimes$ represents the *component-wise* multiplication of two vectors

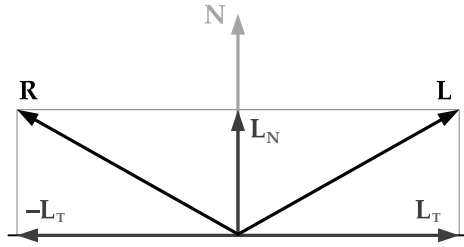


Figure II.24. Calculation of the reflected lighting vector $\mathbf{R}$ from the original lighting vector $\mathbf{L}$ and the surface normal $\mathbf{N}$.

(the so-called Hadamard product). This expression can be broken into three scalar equations, one for each color channel, as follows:

$$\begin{aligned} I_R &= k_{AR}A_R + \sum_i [k_{DR}(\mathbf{N} \cdot \mathbf{L}_i) + k_{SR}(\mathbf{R}_i \cdot \mathbf{V})^\alpha] C_{iR}, \\ I_G &= k_{AG}A_G + \sum_i [k_{DG}(\mathbf{N} \cdot \mathbf{L}_i) + k_{SG}(\mathbf{R}_i \cdot \mathbf{V})^\alpha] C_{iG}, \\ I_B &= k_{AB}A_B + \sum_i [k_{DB}(\mathbf{N} \cdot \mathbf{L}_i) + k_{SB}(\mathbf{R}_i \cdot \mathbf{V})^\alpha] C_{iB}. \end{aligned}$$

In these equations, the vector $\mathbf{R}_i = [R_{ix} \ R_{iy} \ R_{iz}]$ is the *reflection* of the light ray's direction vector $\mathbf{L}_i$ about the surface normal $\mathbf{N}$.

The vector $\mathbf{R}_i$ can be easily calculated via a bit of vector math (see Figure 11.24). Any vector can be expressed as a sum of its normal and tangential components. For example, we can break up the light direction vector $\mathbf{L}$ as follows:

$$\mathbf{L} = \mathbf{L}_N + \mathbf{L}_T.$$

We know that the dot product $(\mathbf{N} \cdot \mathbf{L})$ represents the projection of $\mathbf{L}$ normal to the surface (a scalar quantity). So the normal component $\mathbf{L}_N$ is just the unit normal vector $\mathbf{N}$ scaled by this dot product:

$$\mathbf{L}_N = (\mathbf{N} \cdot \mathbf{L})\mathbf{N}.$$

The reflected vector $\mathbf{R}$ has the same normal component as $\mathbf{L}$ but the *opposite* tangential component ($-\mathbf{L}_T$). So we can find $\mathbf{R}$ as follows:

$$\begin{aligned} \mathbf{R} &= \mathbf{L}_N - \mathbf{L}_T \\ &= \mathbf{L}_N - (\mathbf{L} - \mathbf{L}_N) \\ &= 2\mathbf{L}_N - \mathbf{L}; \\ \mathbf{R} &= 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L} \end{aligned}$$

This equation can be used to find all of the $\mathbf{R}_i$ values corresponding to the light directions $\mathbf{L}_i$.

Blinn-Phong

The *Blinn-Phong* lighting model is a variation on Phong shading that calculates specular reflection in a slightly different way. We define the vector $\mathbf{H}$ to be the vector that lies halfway between the view vector $\mathbf{V}$ and the light direction vector $\mathbf{L}$. The Blinn-Phong specular component is then $(\mathbf{N} \cdot \mathbf{H})^a$, as opposed to Phong's $(\mathbf{R} \cdot \mathbf{V})^\alpha$. The exponent a is slightly different than the Phong exponent α , but its value is chosen in order to closely match the equivalent Phong specular term.

The Blinn-Phong model offers increased runtime efficiency at the cost of some accuracy, although it actually matches empirical results more closely than Phong for some kinds of surfaces. The Blinn-Phong model was used almost exclusively in early computer games and was hard-wired into the fixed-function pipelines of early GPUs. See http://en.wikipedia.org/wiki/Blinn%E2%80%93Phong_shading_model for more details.

BRDF Plots

The three terms in the Phong lighting model are special cases of a general local reflection model known as a *bidirectional reflection distribution function* (BRDF). A BRDF calculates the ratio of the outgoing (reflected) radiance along a given viewing direction $\mathbf{V}$ to the incoming irradiance along the incident ray $\mathbf{L}$.

A BRDF can be visualized as a hemispherical plot, where the radial distance from the origin represents the intensity of the light that would be seen if the reflection point were viewed from that direction. The *diffuse* Phong reflection term is $k_D(\mathbf{N} \cdot \mathbf{L})$. This term only accounts for the incoming illumination ray $\mathbf{L}$, not the viewing angle $\mathbf{V}$. Hence the value of this term is the same for all viewing angles. If we were to plot this term as a function of the viewing angle in three dimensions, it would look like a hemisphere centered on the point at which we are calculating the Phong reflection. This is shown in two dimensions in Figure 11.25.

The specular term of the Phong model is $k_D(\mathbf{R} \cdot \mathbf{V})^\alpha$. This term is dependent on both the illumination direction $\mathbf{L}$ and the viewing direction $\mathbf{V}$. It produces a specular “hot spot” when the viewing angle aligns closely with the reflection $\mathbf{R}$ of the illumination direction $\mathbf{L}$ about the surface normal. However, its contribution falls off very quickly as the viewing angle diverges from the reflected illumination direction. This is shown in two dimensions in Figure 11.26.

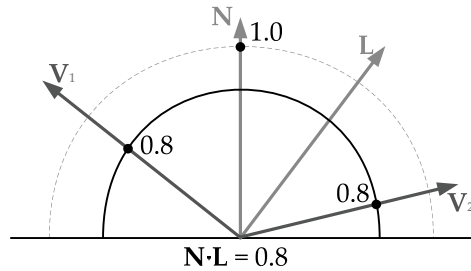


Figure II.25. The diffuse term of the Phong reflection model is dependent upon $\mathbf{N} \cdot \mathbf{L}$ but is independent of the viewing angle $\mathbf{V}$.

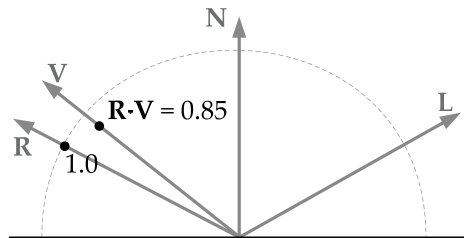


Figure II.26. The specular term of the Phong reflection model is at its maximum when the viewing angle $\mathbf{V}$ coincides with the reflected light direction $\mathbf{R}$ and drops off quickly as $\mathbf{V}$ diverges from $\mathbf{R}$.

II.1.3.3 Modeling Light Sources

In addition to modeling the light's interactions with surfaces, we need to describe the sources of light in the scene. As with all things in real-time rendering, we approximate real-world light sources using various simplified models.

Static Lighting

The fastest lighting calculation is the one you don't do at all. Lighting is therefore performed offline whenever possible. We can precalculate Phong reflection at the vertices of a mesh and store the results as diffuse vertex color attributes. We can also precalculate lighting on a per-pixel basis and store the results in a kind of texture map known as a *light map*. At runtime, the light map texture is projected onto the objects in the scene in order to determine the light's effects on them.

You might wonder why we don't just bake lighting information directly into the diffuse textures in the scene. There are a few reasons for this. For one thing, diffuse texture maps are often tiled and/or repeated throughout a scene,

so baking lighting into them wouldn't be practical. Instead, a single light map is usually generated per light source and applied to any objects that fall within that light's area of influence. This approach permits dynamic objects to move past a light source and be properly illuminated by it. It also means that our light maps can be of a different (often lower) resolution than our diffuse texture maps. Finally, a "pure" light map usually compresses better than one that includes diffuse color information.

Ambient Lights

An *ambient light* corresponds to the ambient term in the Phong lighting model. This term is independent of the viewing angle and has no specific direction. An ambient light is therefore represented by a single color, corresponding to the **A** color term in the Phong equation (which is scaled by the surface's ambient reflectivity k_A at runtime). The intensity and color of ambient light may vary from region to region within the game world.

Directional Lights

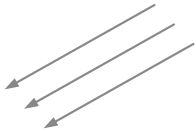


Figure 11.27. Model of a directional light source.

A *directional light* models a light source that is effectively an infinite distance away from the surface being illuminated—like the sun. The rays emanating from a directional light are parallel, and the light itself does not have any particular location in the game world. A directional light is therefore modeled as a light color **C** and a direction vector **L**. A directional light is depicted in Figure 11.27.

Point (Omnidirectional) Lights

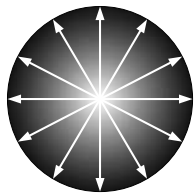


Figure 11.28. Model of a point light source.

A *point light (omnidirectional light)* has a distinct position in the game world and radiates uniformly in all directions. The intensity of the light is usually considered to fall off with the square of the distance from the light source, and beyond a predefined maximum radius its effects are simply clamped to zero. A point light is modeled as a light position **P**, a source color/intensity **C** and a maximum radius $r_{\max}$. The rendering engine only applies the effects of a point light to those surfaces that fall within its sphere of influence (a significant optimization). Figure 11.28 illustrates a point light.

Spot Lights

A *spot light* acts like a point light whose rays are restricted to a cone-shaped region, like a flashlight. Usually two cones are specified with an inner and an outer angle. Within the inner cone, the light is considered to be at full intensity.

The light intensity falls off as the angle increases from the inner to the outer angle, and beyond the outer cone it is considered to be zero. Within both cones, the light intensity also falls off with radial distance. A spot light is modeled as a position $\mathbf{P}$, a source color $\mathbf{C}$, a central direction vector $\mathbf{L}$, a maximum radius $r_{\max}$ and inner and outer cone angles $\theta_{\min}$ and $\theta_{\max}$. Figure 11.29 illustrates a spot light source.

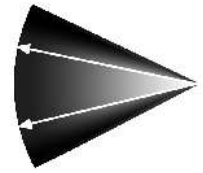


Figure 11.29. Model of a spot light source.

Area Lights

All of the light sources we've discussed thus far radiate from an idealized point, either at infinity or locally. A real light source almost always has a nonzero area—this is what gives rise to the umbra and penumbra in the shadows it casts.

Rather than trying to model area lights explicitly, CG engineers often use various “tricks” to account for their behavior. For example to simulate a penumbra, we might cast multiple shadows and blend the results, or we might blur the edges of a sharp shadow in some manner.

Emissive Objects

Some surfaces in a scene are themselves light sources. Examples include flashlights, glowing crystal balls, flames from a rocket engine and so on. Glowing surfaces can be modeled using an *emissive texture map*—a texture whose colors are always at full intensity, independent of the surrounding lighting environment. Such a texture could be used to define a neon sign, a car's headlights and so on.

Some kinds of emissive objects are rendered by combining multiple techniques. For example, a flashlight might be rendered using an emissive texture for when you're looking head-on into the beam, a colocated spot light that casts light into the scene, a yellow translucent mesh to simulate the light cone, some camera-facing transparent cards to simulate lens flare (or a *bloom* effect if high dynamic range lighting is supported by the engine), and a projected texture to produce the caustic effect that a flashlight has on the surfaces it illuminates. The flashlight in *Luigi's Mansion* is a great example of this kind of effect combination, as shown in Figure 11.30.

11.1.4 The Virtual Camera

In computer graphics, the virtual camera is much simpler than a real camera or the human eye. We treat the camera as an ideal focal point with a rectangular virtual sensing surface called the *imaging rectangle* floating some small distance



Figure II.30. The flashlight in *Luigi's Mansion* by Nintendo (Wii) is composed of numerous visual effects, including a cone of translucent geometry for the beam, a dynamic spot light to cast light into the scene, an emissive texture on the lens and camera-facing cards for the lens flare. (See Color Plate XVIII.)

in front of it. The imaging rectangle consists of a grid of square or rectangular virtual light sensors, each corresponding to a single pixel on-screen. Rendering can be thought of as the process of determining what color and intensity of light would be recorded by each of these virtual sensors.

II.I.4.1 View Space

The focal point of the virtual camera is the origin of a 3D coordinate system known as *view space* or *camera space*. The camera usually “looks” down the positive or negative z -axis in view space, with y up and x to the left or right. Typical left- and right-handed view-space axes are illustrated in Figure 11.31.

The camera’s position and orientation can be specified using a view-to-world matrix, just as a mesh instance is located in the scene with its model-to-world matrix. If we know the position vector and three unit basis vectors of camera space, expressed in world-space coordinates, the view-to-world matrix can be written as follows, in a manner analogous to that used to construct a model-to-world matrix:

$$\mathbf{M}_{V \rightarrow W} = \left[\begin{array}{c|c} \mathbf{i}_V & 0 \\ \mathbf{j}_V & 0 \\ \mathbf{k}_V & 0 \\ \hline \mathbf{t}_V & 1 \end{array} \right].$$

When rendering a triangle mesh, its vertices are transformed first from model space to world space, and then from world space to view space. To perform this latter transformation, we need the world-to-view matrix, which is the inverse of the view-to-world matrix. This matrix is sometimes called the *view matrix*:

$$\mathbf{M}_{W \rightarrow V} = \mathbf{M}_{V \rightarrow W}^{-1} = \mathbf{M}_{\text{view}}.$$

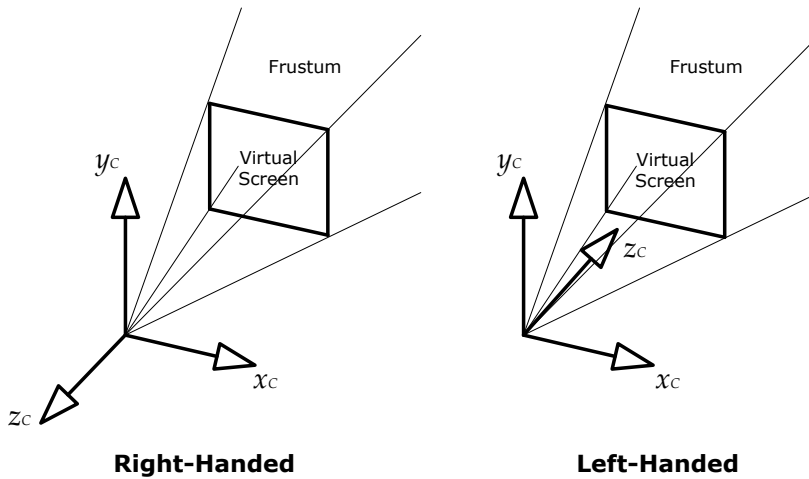


Figure 11.31. Left- and right-handed camera-space axes.

Be careful here. The fact that the camera's matrix is inverted relative to the matrices of the objects in the scene is a common point of confusion and bugs among new game developers.

The world-to-view matrix is often concatenated to the model-to-world matrix prior to rendering a particular mesh instance. This combined matrix is called the *model-view matrix* in OpenGL. We precalculate this matrix so that the rendering engine only needs to do a single matrix multiply when transforming vertices from model space into view space:

$$\mathbf{M}_{M \rightarrow V} = \mathbf{M}_{M \rightarrow W} \mathbf{M}_{W \rightarrow V} = \mathbf{M}_{\text{modelview}}.$$

11.1.4.2 Projections

In order to render a 3D scene onto a 2D image plane, we use a special kind of transformation known as a *projection*. The *perspective* projection is the most common projection in computer graphics, because it mimics the kinds of images produced by a typical camera. With this projection, objects appear smaller the farther away they are from the camera—an effect known as *perspective foreshortening*.

The length-preserving *orthographic* projection is also used by some games, primarily for rendering *plan views* (e.g., front, side and top) of 3D models or game levels for editing purposes, and for overlaying 2D graphics onto the screen for heads-up displays and the like. Figure 11.32 illustrates how a cube would look when rendered with these two types of projections.

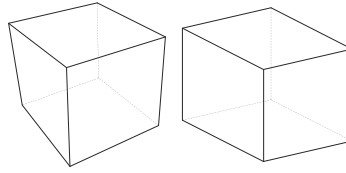


Figure 11.32. A cube rendered using a perspective projection (on the left) and an orthographic projection (on the right).

11.1.4.3 The View Volume and the Frustum

The region of space that the camera can “see” is known as the view volume. A view volume is defined by six planes. The *near plane* corresponds to the virtual image-sensing surface. The four side planes correspond to the edges of the virtual screen. The *far plane* is used as a rendering optimization to ensure that extremely distant objects are not drawn. It also provides an upper limit for the depths that will be stored in the depth buffer (see Section 11.1.4.8).

When rendering the scene with a *perspective* projection, the shape of the view volume is a truncated pyramid known as a *frustum*. When using an orthographic projection, the view volume is a rectangular prism. Perspective and orthographic view volumes are illustrated in Figure 11.33 and Figure 11.34, respectively.

The six planes of the view volume can be represented compactly using six four-element vectors $(n_{ix}, n_{iy}, n_{iz}, d_i)$, where $\mathbf{n} = (n_x, n_y, n_z)$ is the plane normal and d is its perpendicular distance from the origin. If we prefer the

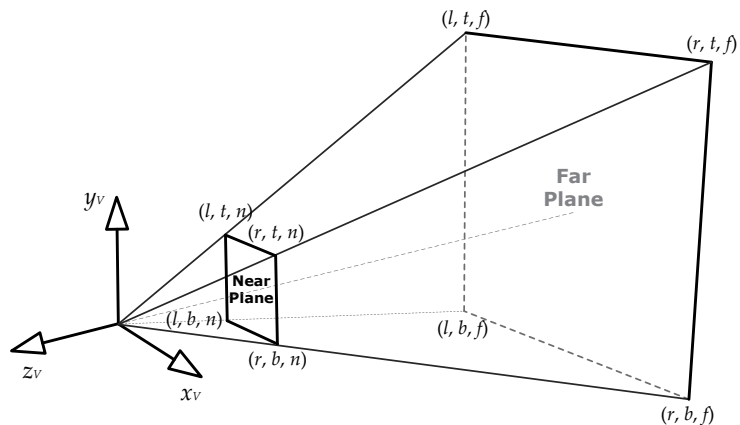


Figure 11.33. A perspective view volume (frustum).

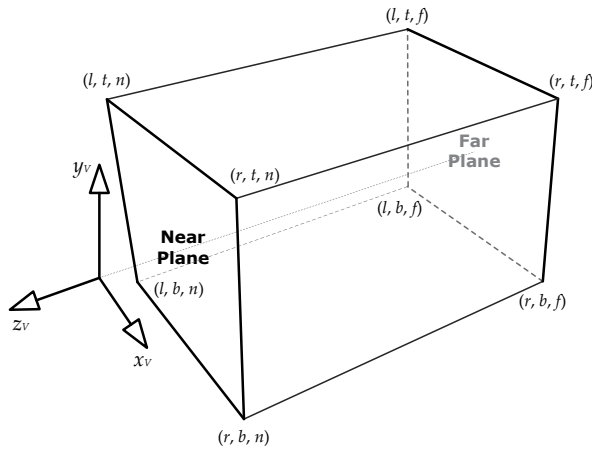


Figure 11.34. An orthographic view volume.

point-normal plane representation, we can also describe the planes with six pairs of vectors $(\mathbf{Q}_i, \mathbf{n}_i)$, where $\mathbf{Q}$ is the arbitrary point on the plane and $\mathbf{n}$ is the plane normal. (In both cases, i is an index representing the six planes.)

11.1.4.4 Projection and Homogeneous Clip Space

Both perspective and orthographic projections transform points in view space into a coordinate space called *homogeneous clip space*. This three-dimensional space is really just a warped version of view space. The purpose of clip space is to convert the camera-space view volume into a canonical view volume that is independent both of the kind of *projection* used to convert the 3D scene into 2D screen space, and of the *resolution* and *aspect ratio* of the screen onto which the scene is going to be rendered.

In clip space, the canonical view volume is a rectangular prism extending from -1 to $+1$ along the x - and y -axes. Along the z -axis, the view volume extends either from -1 to $+1$ (OpenGL) or from 0 to 1 (DirectX). We call this coordinate system “clip space” because the view volume planes are axis-aligned, making it convenient to *clip* triangles to the view volume in this space (even when a perspective projection is being used). The canonical clip-space view volume for OpenGL is depicted in Figure 11.35. Notice that the z -axis of clip space goes into the screen, with y up and x to the right. In other words, homogeneous clip space is usually *left-handed*. A left-handed convention is used here because it causes increasing z values to correspond to increasing *depth* into the screen, with y increasing up and x increasing to the right as usual.

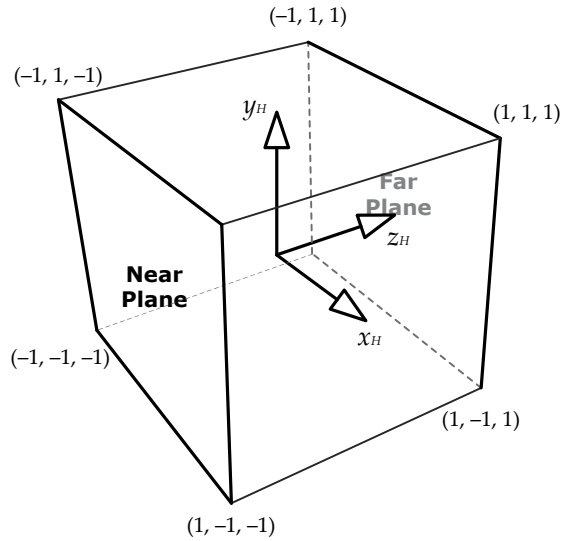


Figure II.35. The canonical view volume in homogeneous clip space.

Perspective Projection

An excellent explanation of perspective projection is given in Section 4.5.1 of [32], so we won't repeat it here. Instead, we'll simply present the perspective projection matrix $\mathbf{M}_{V \rightarrow H}$ below. (The subscript $V \rightarrow H$ indicates that this matrix transforms vertices from view space into homogeneous clip space.) If we take view space to be right-handed, then the near plane intersects the z -axis at $z = -n$, and the far plane intersects it at $z = -f$. The virtual screen's left, right, bottom, and top edges lie at $x = l$, $x = r$, $y = b$ and $y = t$ on the near plane, respectively. (Typically the virtual screen is centered on the camera-space z -axis, in which case $l = -r$ and $b = -t$, but this isn't always the case.) Using these definitions, the perspective projection matrix for OpenGL is as follows:

$$\mathbf{M}_{V \rightarrow H} = \begin{bmatrix} \left(\frac{2n}{r-l} \right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b} \right) & 0 & 0 \\ \left(\frac{r+l}{r-l} \right) & \left(\frac{t+b}{t-b} \right) & \left(-\frac{f+n}{f-n} \right) & -1 \\ 0 & 0 & \left(-\frac{2nf}{f-n} \right) & 0 \end{bmatrix}.$$

DirectX defines the z -axis extents of the clip-space view volume to lie in

the range $[0, 1]$ rather than in the range $[-1, 1]$ as OpenGL does. We can easily adjust the perspective projection matrix to account for DirectX's conventions as follows:

$$(\mathbf{M}_{V \rightarrow H})_{\text{DirectX}} = \begin{bmatrix} \left(\frac{2n}{r-l}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b}\right) & 0 & 0 \\ \left(\frac{r+l}{r-l}\right) & \left(\frac{t+b}{t-b}\right) & \left(-\frac{f}{f-n}\right) & -1 \\ 0 & 0 & \left(-\frac{nf}{f-n}\right) & 0 \end{bmatrix}.$$

Division by z

Perspective projection results in each vertex's x - and y -coordinates being divided by its z -coordinate. This is what produces perspective foreshortening. To understand why this happens, consider multiplying a view-space point $\mathbf{p}_V$ expressed in four-element homogeneous coordinates by the OpenGL perspective projection matrix:

$$\begin{aligned} \mathbf{p}_H &= \mathbf{p}_V \mathbf{M}_{V \rightarrow H} \\ &= \begin{bmatrix} p_{Vx} & p_{Vy} & p_{Vz} & 1 \end{bmatrix} \begin{bmatrix} \left(\frac{2n}{r-l}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b}\right) & 0 & 0 \\ \left(\frac{r+l}{r-l}\right) & \left(\frac{t+b}{t-b}\right) & \left(-\frac{f+n}{f-n}\right) & -1 \\ 0 & 0 & \left(-\frac{2nf}{f-n}\right) & 0 \end{bmatrix}. \end{aligned}$$

The result of this multiplication takes the form

$$\mathbf{p}_H = \begin{bmatrix} a & b & c & -p_{Vz} \end{bmatrix}. \quad (11.1)$$

When we convert any homogeneous vector into three-dimensional coordinates, the x -, y - and z -components are divided by the w -component:

$$\begin{bmatrix} x & y & z & w \end{bmatrix} \equiv \begin{bmatrix} \frac{x}{w} & \frac{y}{w} & \frac{z}{w} \end{bmatrix}.$$

So, after dividing Equation (11.1) by the homogeneous w -component, which is really just the negative view-space z -coordinate $-p_{Vz}$, we have:

$$\begin{aligned} \mathbf{p}_H &= \begin{bmatrix} \frac{a}{-p_{Vz}} & \frac{b}{-p_{Vz}} & \frac{c}{-p_{Vz}} \end{bmatrix} \\ &= \begin{bmatrix} p_{Hx} & p_{Hy} & p_{Hz} \end{bmatrix}. \end{aligned}$$

Thus, the homogeneous clip-space coordinates have been divided by the view-space z -coordinate, which is what causes perspective foreshortening.

Perspective-Correct Vertex Attribute Interpolation

In Section 11.1.2.4, we learned that vertex attributes are interpolated in order to determine appropriate values for them within the interior of a triangle. Attribute interpolation is performed in *screen space*. We iterate over each pixel of the screen and attempt to determine the value of each attribute at the corresponding location *on the surface of the triangle*. When rendering a scene with a perspective projection, we must do this very carefully so as to account for perspective foreshortening. This is known as *perspective-correct* attribute interpolation.

A derivation of perspective-correct interpolation is beyond our scope, but suffice it to say that we must divide our interpolated attribute values by the corresponding z -coordinates (depths) at each vertex. For any pair of vertex attributes A_1 and A_2 , we can write the interpolated attribute at a percentage t of the distance between them as follows:

$$\frac{A}{p_z} = (1 - t) \left(\frac{A_1}{p_{1z}} \right) + t \left(\frac{A_2}{p_{2z}} \right) = \text{LERP} \left(\frac{A_1}{p_{1z}}, \frac{A_2}{p_{2z}}, t \right).$$

Refer to [32] for an excellent derivation of the math behind perspective-correct attribute interpolation.

Orthographic Projection

An orthographic projection is performed by the following matrix:

$$(\mathbf{M}_{V \rightarrow H})_{\text{ortho}} = \begin{bmatrix} \left(\frac{2}{r-l} \right) & 0 & 0 & 0 \\ 0 & \left(\frac{2}{t-b} \right) & 0 & 0 \\ 0 & 0 & \left(-\frac{2}{f-n} \right) & 0 \\ \left(-\frac{r+l}{r-l} \right) & \left(-\frac{t+b}{t-b} \right) & \left(-\frac{f+n}{f-n} \right) & 1 \end{bmatrix}.$$

This is just an everyday scale-and-translate matrix. (The upper-left 3×3 contains a diagonal nonuniform scaling matrix, and the lower row contains the translation.) Since the view volume is a rectangular prism in both view space and clip space, we need only scale and translate our vertices to convert from one space to the other.

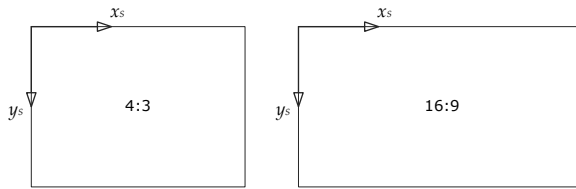


Figure 11.36. The two most prevalent screen-space aspect ratios are 4:3 and 16:9.

11.1.4.5 Screen Space and Aspect Ratios

Screen space is a two-dimensional coordinate system whose axes are measured in terms of screen pixels. The x -axis typically points to the right, with the origin at the top-left corner of the screen and y pointing down. (The reason for the inverted y -axis is that CRT monitors scan the screen from top to bottom.) The ratio of screen width to screen height is known as the *aspect ratio*. The most common aspect ratios are 4:3 (the aspect ratio of a traditional television screen) and 16:9 (the aspect ratio of a movie screen or HDTV). These aspect ratios are illustrated in Figure 11.36.

We can render triangles expressed in homogeneous clip space by simply drawing their (x, y) coordinates and ignoring z . But before we do, we scale and shift the clip-space coordinates so that they lie in screen space rather than within the normalized unit square. This scale-and-shift operation is known as *screen mapping*.

11.1.4.6 The Frame Buffer

The final rendered image is stored in a bitmapped color buffer known as the *frame buffer*. Pixel colors are usually stored in RGBA8888 format, although other frame buffer formats are supported by most graphics cards as well. Some common formats include RGB565, RGB5551, and one or more paletted modes.

The display hardware (CRT, flat-screen monitor, HDTV, etc.) reads the contents of the frame buffer at a periodic rate of 60 Hz for NTSC televisions used in North America and Japan, or 50 Hz for PAL/SECAM televisions used in Europe and many other places in the world. Rendering engines typically maintain at least two frame buffers. While one is being scanned by the display hardware, the other one can be updated by the rendering engine. This is known as *double buffering*. By swapping or “flipping” the two buffers during the *vertical blanking interval* (the period during which the CRT’s electron gun is being reset to the top-left corner of the screen), double buffering ensures that the display hardware always scans the complete frame buffer. This avoids a jarring effect

known as *tearing*, in which the upper portion of the screen displays the newly rendered image while the bottom shows the remnants of the previous frame's image.

Some engines make use of three frame buffers—a technique aptly known as *triple buffering*. This is done so that the rendering engine can start work on the next frame, even while the previous frame is still being scanned by the display hardware. For example, the hardware might still be scanning buffer A when the engine finishes drawing buffer B. With triple buffering, it can proceed to render a new frame into buffer C, rather than idling while it waits for the display hardware to finish scanning buffer A.

Render Targets

Any buffer into which the rendering engine draws graphics is known as a *render target*. As we'll see later in this chapter, rendering engines make use of all sorts of other off-screen render targets in addition to the frame buffers. These include the depth buffer, the stencil buffer and various other buffers used for storing intermediate rendering results.

II.1.4.7 Triangle Rasterization and Fragments

To produce an image of a triangle on-screen, we need to fill in the pixels it overlaps. This process is known as rasterization. During rasterization, the triangle's surface is broken into pieces called *fragments*, each one representing a small region of the triangle's surface that corresponds to a single pixel on the screen. (In the case of multisample antialiasing, a fragment corresponds to a *portion* of a pixel—see below.)

A fragment is like a pixel in training. Before it is written into the frame buffer, it must pass a number of tests (described in more depth below). If it fails any of these tests, it will be discarded. Fragments that pass the tests are shaded (i.e., their colors are determined), and the fragment color is either written into the frame buffer or blended with the pixel color that's already there. Figure 11.37 illustrates how a fragment becomes a pixel.

II.1.4.8 Occlusion and the Depth Buffer

When rendering two triangles that overlap each other in screen space, we need some way of ensuring that the triangle that is closer to the camera will appear on top. We could accomplish this by always rendering our triangles in back-to-front order (the so-called painter's algorithm). However, as shown in Figure 11.38, this doesn't work if the triangles are intersecting one another.

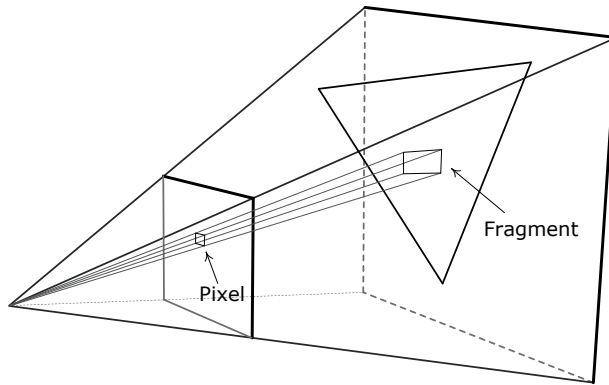


Figure II.37. A fragment is a small region of a triangle corresponding to a pixel on the screen. It passes through the rendering pipeline and is either discarded or its color is written into the frame buffer.

To implement triangle occlusion properly, independent of the order in which the triangles are rendered, rendering engines use a technique known as *depth buffering* or *z-buffering*. The depth buffer is a full-screen buffer that typically contains 24-bit integer or (more rarely) floating-point depth information for each pixel in the frame buffer. (The depth buffer is usually stored in a 32-bits-per-pixel format, with a 24-bit depth value and an 8-bit stencil value packed into each pixel's 32-bit quadword.) Every fragment has a z-coordinate that measures its depth "into" the screen. (The depth of a fragment is found by interpolating the depths of the triangle's vertices.) When a fragment's color is written into the frame buffer, its depth is stored into the corresponding pixel of the depth buffer. When another fragment (from another triangle) is drawn into the same pixel, the engine compares the new fragment's depth to the depth already present in the depth buffer. If the fragment is closer to the camera (i.e.,

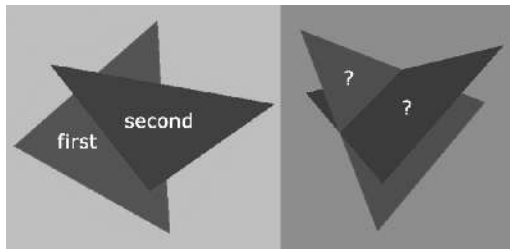


Figure II.38. The painter's algorithm renders triangles in a back-to-front order to produce proper triangle occlusion. However, the algorithm breaks down when triangles intersect one another.

if it has a smaller depth), it overwrites the pixel in the frame buffer. Otherwise the fragment is discarded.

z-Fighting and the w-Buffer

When rendering parallel surfaces that are very close to one another, it's important that the rendering engine can distinguish between the depths of the two planes. If our depth buffer had infinite precision, this would never be a problem. Unfortunately, a real depth buffer only has limited precision, so the depth values of two planes can collapse into a single discrete value when the planes are close enough together. When this happens, the more-distant plane's pixels start to "poke through" the nearer plane, resulting in a noisy effect known as *z-fighting*.

To reduce *z-fighting* to a minimum across the entire scene, we would like to have equal precision whether we're rendering surfaces that are close to the camera or far away. However, with *z-buffering* this is not the case. The precision of clip-space *z*-depths (p_{H_z}) are not evenly distributed across the entire range from the near plane to the far plane, because of the division by the view-space *z*-coordinate. Because of the shape of the $1/z$ curve, most of the depth buffer's precision is concentrated near the camera.

The plot of the function $p_{H_z} = 1/p_{V_z}$ shown in Figure 11.39 demonstrates this effect. Near the camera, the distance between two planes in view space Δp_{V_z} gets transformed into a reasonably large delta in clip space Δp_{H_z} . But far from the camera, this same separation gets transformed into a tiny delta in clip space. The result is *z-fighting*, and it becomes rapidly more prevalent as objects get farther away from the camera.

To circumvent this problem, we would like to store *view-space* *z*-coordinates (p_{V_z}) in the depth buffer instead of clip-space *z*-coordinates (p_{H_z}). View-space *z*-coordinates vary linearly with the distance from the camera, so using them as our depth measure achieves uniform precision across the entire depth range.

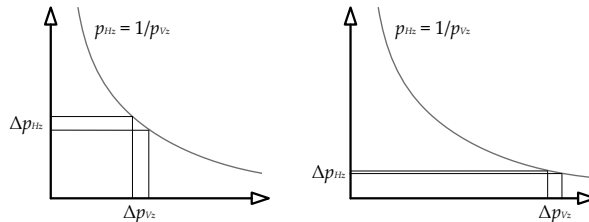


Figure 11.39. A plot of the function $1/p_{V_z}$, showing how most of the precision lies close to the camera.

This technique is called *w-buffering*, because the view-space z -coordinate conveniently appears in the w -component of our homogeneous clip-space coordinates. (Recall from Equation (11.1) that $p_{H_w} = -p_{V_z}$.)

The terminology can be very confusing here. The z - and w -buffers store coordinates that are expressed in *clip space*. But in terms of *view-space* coordinates, the z -buffer stores $1/z$ (i.e., $1/p_{V_z}$) while the w -buffer stores z (i.e., p_{V_z})!

We should note here that the *w-buffering* approach is a bit more expensive than its z -based counterpart. This is because with *w-buffering*, we cannot linearly interpolate depths directly. Depths must be inverted prior to interpolation and then re-inverted prior to being stored in the w -buffer.

11.2 The Rendering Pipeline

Now that we've completed our whirlwind tour of the major theoretical and practical underpinnings of triangle rasterization, let's turn our attention to how it is typically implemented. In real-time game rendering engines, the high-level rendering steps described in Section 11.1 are implemented using a software/hardware architecture known as a *pipeline*. A pipeline is just an ordered chain of computational stages, each with a specific purpose, operating on a stream of input data items and producing a stream of output data.

Each stage of a pipeline can typically operate independently of the other stages. Hence, one of the biggest advantages of a pipelined architecture is that it lends itself extremely well to parallelization. While the first stage is chewing on one data element, the second stage can be processing the results previously produced by the first stage, and so on down the chain.

Parallelization can also be achieved within an individual stage of the pipeline. For example, if the computing hardware for a particular stage is duplicated N times on the die, N data elements can be processed in parallel by that stage. A parallelized pipeline is shown in Figure 11.40. Ideally the stages operate in parallel (most of the time), and certain stages are capable of operating on multiple data items simultaneously as well.

The *throughput* of a pipeline measures how many data items are processed per second overall. The pipeline's *latency* measures the amount of time it takes for a single data element to make it through the entire pipeline. The latency of an individual stage measures how long that stage takes to process a single item. The slowest stage of a pipeline dictates the throughput of the entire pipeline. It also has an impact on the average latency of the pipeline as a whole. Therefore, when designing a rendering pipeline, we attempt to minimize and balance latency across the entire pipeline and eliminate bottlenecks. In a well-

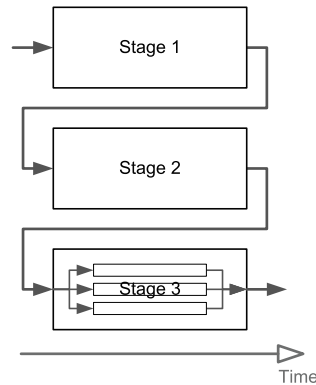


Figure II.40. A parallelized pipeline. The stages all operate in parallel, and some stages are capable of operating on multiple data items simultaneously as well.

designed pipeline, all the stages operate simultaneously, and no stage is ever idle for very long waiting for another stage to become free.

II.2.1 Overview of the Rendering Pipeline

Some graphics texts divide the rendering pipeline into three coarse-grained stages. In this book, we'll extend this pipeline back even further, to encompass the offline tools used to create the scenes that are ultimately rendered by the game engine. The high-level stages in our pipeline are:

- *Tools stage (offline)*. Geometry and surface properties (materials) are defined.
- *Asset conditioning stage (offline)*. The geometry and material data are processed by the asset conditioning pipeline (ACP) into an engine-ready format.
- *Application stage (CPU)*. Potentially visible mesh instances are identified and submitted to the graphics hardware along with their materials for rendering.
- *Geometry processing stage (GPU)*. Vertices are transformed and lit and projected into homogeneous clip space. Triangles are processed by the optional geometry shader and then clipped to the frustum.
- *Rasterization stage (GPU)*. Triangles are converted into fragments that are shaded, passed through various tests (z-test, alpha test, stencil test, etc.) and finally blended into the frame buffer.

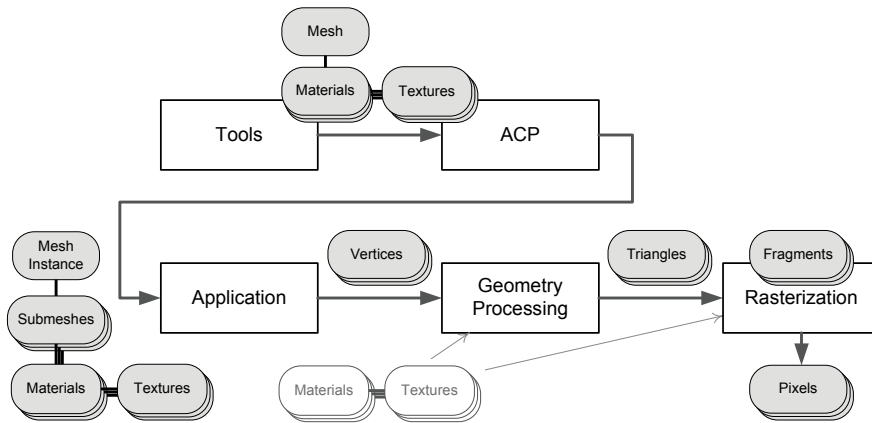


Figure 11.41. The format of geometric data changes radically as it passes through the various stages of the rendering pipeline.

11.2.1.1 How the Rendering Pipeline Transforms Data

It's interesting to note how the format of geometry data changes as it passes through the rendering pipeline. The tools and asset conditioning stages deal with meshes and materials. The application stage deals in terms of mesh instances and submeshes, each of which is associated with a single material. During the geometry stage, each submesh is broken down into individual vertices, which are processed largely in parallel. At the conclusion of this stage, the triangles are reconstructed from the fully transformed and shaded vertices. In the rasterization stage, each triangle is broken into fragments, and these fragments are either discarded, or they are eventually written into the frame buffer as colors. This process is illustrated in Figure 11.41.

11.2.1.2 Implementation of the Pipeline

The first two stages of the rendering pipeline are implemented offline, usually executed by a Windows or Linux machine. The application stage is typically run on one or more CPU cores, whereas the geometry and rasterization stages are usually executed by the graphics processing unit (GPU). In the following sections, we'll explore some of the details of how each of these stages is implemented.

11.2.2 The Tools Stage

In the tools stage, meshes are authored by 3D modelers in a *digital content creation* (DCC) application like Maya, 3ds Max, Lightwave, Softimage/XSI,

SketchUp, etc. The models may be defined using any convenient surface description—NURBS, quads, triangles, etc. However, they are invariably tessellated into triangles prior to rendering by the runtime portion of the pipeline.

The vertices of a mesh may also be skinned. This involves associating each vertex with one or more joints in an articulated skeletal structure, along with weights describing each joint's relative influence over the vertex. Skinning information and the skeleton are used by the animation system to drive the movements of a model—see Chapter 12 for more details.

Materials are also defined by the artists during the tools stage. This involves selecting a shader for each material, selecting textures as required by the shader, and specifying the configuration parameters and options of each shader. Textures are mapped onto the surfaces, and other vertex attributes are also defined, often by “painting” them with some kind of intuitive tool within the DCC application.

Materials are usually authored using a commercial or custom in-house material editor. The material editor is sometimes integrated directly into the DCC application as a plug-in, or it may be a stand-alone program. Some material editors are live-linked to the game, so that material authors can see what the materials will look like in the real game. Other editors provide an offline 3D visualization view. Some editors even allow shader programs to be written and debugged by the artist or a shader engineer. Such tools allow rapid prototyping of visual effects by connecting various kinds of nodes together with a mouse. These tools generally provide a WYSIWYG display of the resulting material. NVIDIA's Fx Composer is an example of such a tool. Sadly, NVIDIA is no longer updating Fx Composer, and it only supports shader models up to DirectX 10. But they do offer a new Visual Studio plugin called NVIDIA® Nsight™ Visual Studio Edition. Depicted in Figure 11.42, Nsight provides powerful shader authoring and debugging facilities. The Unreal Engine also provides a graphical shader editor called Material Editor; it is shown in Figure 11.43.

Materials may be stored and managed with the individual meshes. However, this can lead to duplication of data—and effort. In many games, a relatively small number of materials can be used to define a wide range of objects in the game. For example, we might define some standard, reusable materials like wood, rock, metal, plastic, cloth, skin and so on. There's no reason to duplicate these materials inside every mesh. Instead, many game teams build up a library of materials from which to choose, and the individual meshes refer to the materials in a loosely coupled manner.

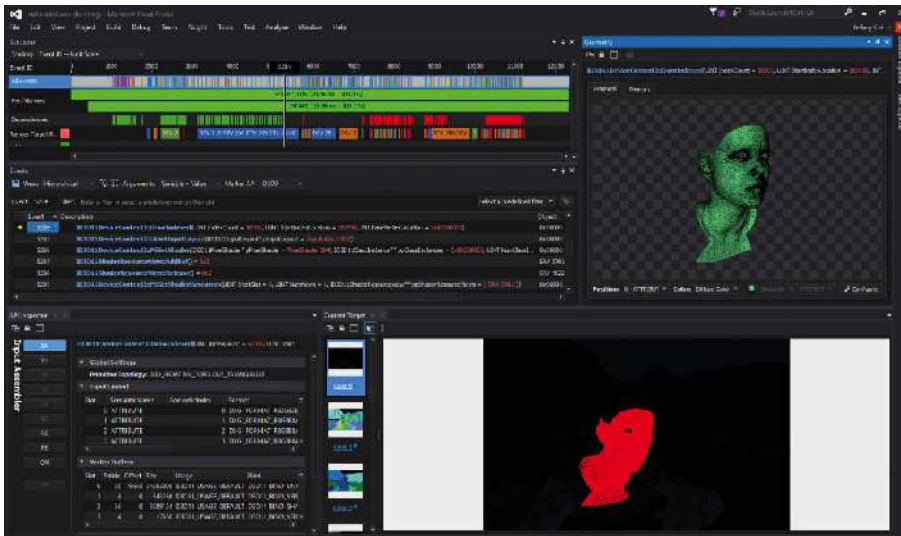


Figure 11.42. NVIDIA® Nsight™ Visual Studio Edition allows shader programs to be written, previalized and debugged easily.

11.2.3 The Asset Conditioning Stage

The asset conditioning stage is itself a pipeline, sometimes called the *asset conditioning pipeline* (ACP) or the *tools pipeline*. As we saw in Section 7.2.1.4, its job is to export, process and link together multiple types of assets into a cohesive whole. For example, a 3D model is comprised of geometry (vertex and index buffers), materials, textures and an optional skeleton. The ACP ensures that all of the individual assets referenced by a 3D model are available and ready to be loaded by the engine.

Geometric and material data is extracted from the DCC application and is usually stored in a platform-independent intermediate format. The data is then further processed into one or more platform-specific formats, depending on how many target platforms the engine supports. Ideally the platform-specific assets produced by this stage are ready to load into memory and use with little or no postprocessing at runtime. For example, mesh data targeted for the Xbox One or PS4 might be output as index and vertex buffers that are ready to be consumed by the GPU; on the PS3, geometry might be produced in compressed data streams that are ready to be DMA'd to the SPUs for decompression. The ACP often takes the needs of the material/shader into account

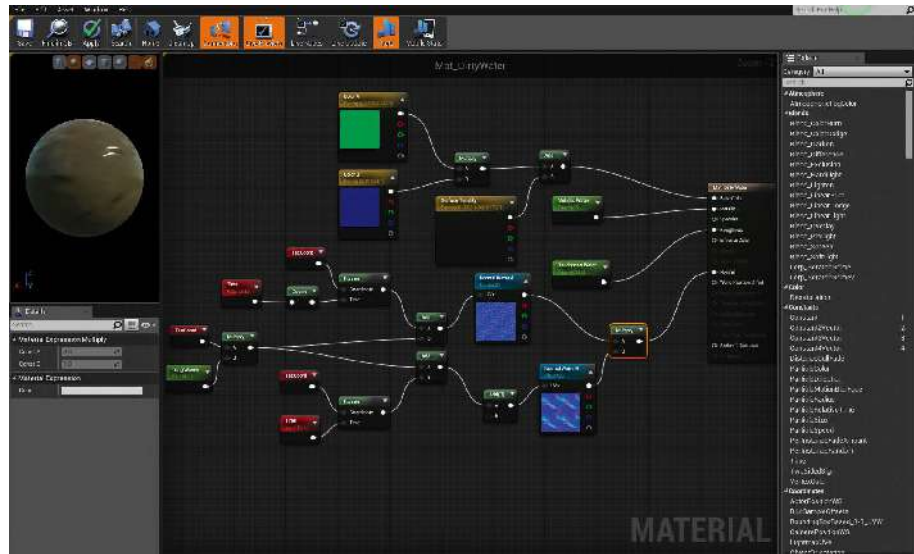


Figure 11.43. The Unreal Engine 4 Material Editor.

when building assets. For example, a particular shader might require tangent and bitangent vectors as well as a vertex normal; the ACP could generate these vectors automatically.

High-level *scene graph* data structures may also be computed during the asset conditioning stage. For example, static-level geometry may be processed in order to build a BSP tree. (As we'll investigate in Section 11.2.7.4, scene graph data structures help the rendering engine to very quickly determine which objects should be rendered, given a particular camera position and orientation.)

Expensive lighting calculations are often done offline as part of the asset conditioning stage. This is called *static lighting*; it may include calculation of light colors at the vertices of a mesh (this is called “baked” vertex lighting), construction of texture maps that encode per-pixel lighting information known as *light maps*, calculation of *precomputed radiance transfer* (PRT) coefficients (usually represented by spherical harmonic functions) and so on.

11.2.4 The GPU Pipeline

Graphics hardware has evolved around a specialized type of microprocessor known as a *graphics processing unit* or GPU. As we discussed in Section 4.11,

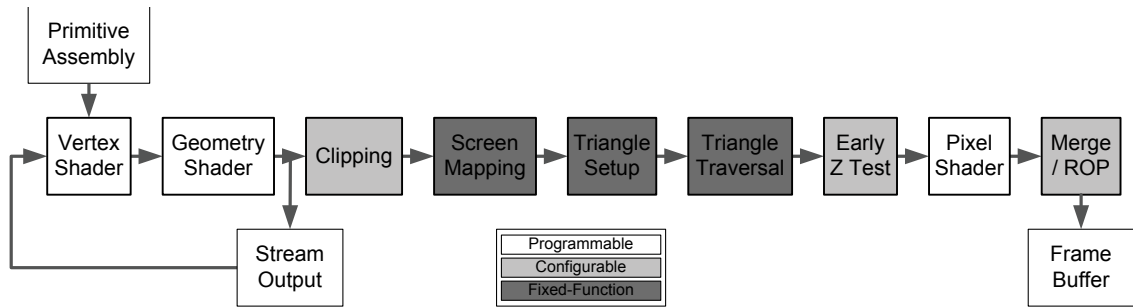


Figure 11.44. The geometry processing and rasterization stages of the rendering pipeline, as implemented by a typical GPU. The white stages are programmable, the light grey stages are configurable, and the dark grey boxes are fixed-function.

a GPU is designed to maximize throughput of the graphics pipeline, which it achieves through massive parallelization of tasks like vertex processing and per-pixel shading calculations. For example, a modern GPU like the AMD Radeon™ 7970 can achieve a peak performance of 4 TFLOPS, which it does by executing workloads in parallel across 32 compute units, each of which contains four 16-lane SIMD VPUs, which in turn execute pipelined wavefronts consisting of 64 threads each. A GPU can be used to render graphics, but today's GPUs are also fully programmable, allowing programmers to leverage the awesome computing power of a GPU to execute *compute shaders*. This is known as *general-purpose GPU* computing (GPGPU).

Virtually all GPUs break the graphics pipeline into the substages described below and depicted in Figure 11.44. Each stage is shaded to indicate whether its functionality is programmable, fixed but configurable, or fixed and non-configurable.

11.2.4.1 Vertex Shader

This stage is fully programmable. It is responsible for transformation and shading/lighting of individual vertices. The input to this stage is a single vertex (although in practice many vertices are processed in parallel). Its position and normal are typically expressed in model space or world space. The vertex shader handles transformation from model space to view space via the model-view transform. Perspective projection is also applied, as well as per-vertex lighting and texturing calculations, and skinning for animated characters. The vertex shader can also perform procedural animation by modifying the position of the vertex. Examples of this include foliage that sways in the breeze or an undulating water surface. The output of this stage is a fully transformed and lit vertex, whose position and normal are expressed in homogeneous clip

space (see Section 11.1.4.4).

On modern GPUs, the vertex shader has full access to texture data—a capability that used to be available only to the pixel shader. This is particularly useful when textures are used as stand-alone data structures like heightmaps or look-up tables.

11.2.4.2 Geometry Shader

This optional stage is also fully programmable. The geometry shader operates on entire primitives (triangles, lines and points) in homogeneous clip space. It is capable of culling or modifying input primitives, and it can also generate new primitives. Typical uses include shadow volume extrusion (see Section 11.3.3.1), rendering the six faces of a cube map (see Section 11.3.1.4), fur fin extrusion around silhouette edges of meshes, creation of particle quads from point data (see Section 11.4.1), dynamic tessellation, fractal subdivision of line segments for lightning effects, cloth simulations, and the list goes on.

11.2.4.3 Stream Output

Some GPUs permit the data that has been processed up to this point in the pipeline to be written back to memory. From there, it can then be looped back to the top of the pipeline for further processing. This feature is called *stream output*.

Stream output permits a number of intriguing visual effects to be achieved without the aid of the CPU. An excellent example is hair rendering. Hair is often represented as a collection of cubic spline curves. It used to be that hair physics simulation would be done on the CPU. The CPU would also tessellate the splines into line segments. Finally the GPU would render the segments.

With stream output, the GPU can do the physics simulation on the control points of the hair splines within the vertex shader. The geometry shader tessellates the splines, and the stream output feature is used to write the tessellated vertex data to memory. The line segments are then piped back into the top of the pipeline so they can be rendered.

11.2.4.4 Clipping

The clipping stage chops off those portions of the triangles that straddle the frustum. Clipping is done by identifying vertices that lie outside the frustum and then finding the intersection of the triangle's edges with the planes of the frustum. These intersection points become new vertices that define one or more clipped triangles.

This stage is fixed in function, but it is somewhat configurable. For example, user-defined clipping planes can be added in addition to the frustum

planes. This stage can also be configured to cull triangles that lie entirely outside the frustum.

11.2.4.5 Screen Mapping

Screen mapping simply scales and shifts the vertices from homogeneous clip space into screen space. This stage is entirely fixed and non-configurable.

11.2.4.6 Triangle Set-up

During triangle set-up, the rasterization hardware is initialized for efficient conversion of the triangle into fragments. This stage is not configurable.

11.2.4.7 Triangle Traversal

Each triangle is broken into fragments (i.e., rasterized) by the triangle traversal stage. Usually one fragment is generated for each pixel, although with certain antialiasing techniques, multiple fragments may be created per pixel (see Section 11.1.4.7). The triangle traversal stage also interpolates vertex attributes in order to generate per-fragment attributes for processing by the pixel shader. Perspective-correct interpolation is used where appropriate. This stage's functionality is fixed and not configurable.

11.2.4.8 Early z-Test

Many graphics cards are capable of checking the depth of the fragment at this point in the pipeline, discarding it if it is being occluded by the pixel already in the frame buffer. This allows the (potentially very expensive) pixel shader stage to be skipped entirely for occluded fragments.

Surprisingly, not all graphics hardware supports depth testing at this stage of the pipeline. In older GPU designs, the z-test was done along with alpha testing, after the pixel shader had run. For this reason, this stage is called the *early z-test* or *early depth test* stage.

11.2.4.9 Pixel Shader

This stage is fully programmable. Its job is to shade (i.e., light and otherwise process) each fragment. The pixel shader can also discard fragments, for example because they are deemed to be entirely transparent. The pixel shader can address one or more texture maps, run per-pixel lighting calculations, and do whatever else is necessary to determine the fragment's color.

The input to this stage is a collection of per-fragment attributes (which have been interpolated from the vertex attributes by the triangle traversal stage). The output is a single color vector describing the desired color of the fragment.

II.2.4.10 Merging / Raster Operations Stage

The final stage of the pipeline is known as the *merging stage* or *blending stage*, also known as the *raster operations* stage or ROP in NVIDIA parlance. This stage is not programmable, but it is highly configurable. It is responsible for running various fragment tests including the depth test (see Section 11.1.4.8), alpha test (in which the values of the fragment's and pixel's alpha channels can be used to reject certain fragments) and stencil test (see Section 11.3.3.1).

If the fragment passes all of the tests, its color is blended (merged) with the color that is already present in the frame buffer. The way in which blending occurs is controlled by the *alpha blending function*—a function whose basic structure is hard-wired, but whose operators and parameters can be configured in order to produce a wide variety of blending operations.

Alpha blending is most commonly used to render semitransparent geometry. In this case, the following blending function is used:

$$\mathbf{C}'_D = A_S \mathbf{C}_S + (1 - A_S) \mathbf{C}_D.$$

The subscripts *S* and *D* stand for “source” (the incoming fragment) and “destination” (the pixel in the frame buffer), respectively. Therefore, the color that is written into the frame buffer ($\mathbf{C}'_D$) is a *weighted average* of the existing frame buffer contents ($\mathbf{C}_D$) and the color of the fragment being drawn ($\mathbf{C}_S$). The blend weight (A_S) is just the source alpha of the incoming fragment.

For alpha blending to look right, the semitransparent and translucent surfaces in the scene must be sorted and rendered in back-to-front order, *after* the opaque geometry has been rendered to the frame buffer. This is because after alpha blending has been performed, the depth of the new fragment *overwrites* the depth of the pixel with which it was blended. In other words, the depth buffer ignores transparency (unless depth writes have been turned off, of course). If we are rendering a stack of translucent objects on top of an opaque backdrop, the resulting pixel color should ideally be a blend between the opaque surface's color and the colors of *all* of the translucent surfaces in the stack. If we try to render the stack in any order other than back-to-front, depth-test failures will cause some of the translucent fragments to be discarded, resulting in an incomplete blend (and a rather odd-looking image).

Other alpha blending functions can be defined as well, for purposes other than transparency blending. The general blending equation takes the form $\mathbf{C}'_D = (\mathbf{w}_S \otimes \mathbf{C}_S) + (\mathbf{w}_D \otimes \mathbf{C}_D)$, where the weighting factors $\mathbf{w}_S$ and $\mathbf{w}_D$ can be selected by the programmer from a predefined set of values including zero,

one, source or destination color, source or destination alpha and one minus the source or destination color or alpha. The operator $\otimes$ is either a regular scalar-vector multiplication or a component-wise vector-vector multiplication (a Hadamard product—see Section 5.2.4.1) depending on the data types of $\mathbf{w}_S$ and $\mathbf{w}_D$.

11.2.5 Programmable Shaders

Now that we have an end-to-end picture of the GPU pipeline in mind, let's take a deeper look at the most interesting part of the pipeline—the programmable shaders. Shader architectures have evolved significantly since their introduction with DirectX 8. Early *shader models* supported only low-level assembly language programming, and the instruction set and register set of the pixel shader differed significantly from those of the vertex shader. DirectX 9 brought with it support for high-level C-like shader languages such as Cg (C for graphics), HLSL (High-Level Shading Language—Microsoft's implementation of the Cg language) and GLSL (OpenGL shading language). With DirectX 10, the geometry shader was introduced, and with it came a unified shader architecture called *shader model 4.0* in DirectX parlance. In the unified shader model, all three types of shaders support roughly the same instruction set and have roughly the same set of capabilities, including the ability to read texture memory.

A shader takes a single element of input data and transforms it into zero or more elements of output data.

- In the case of the vertex shader, the input is a vertex whose position and normal are expressed in model space or world space. The output of the vertex shader is a fully transformed and lit vertex, expressed in homogeneous clip space.
- The input to the geometry shader is a single n -vertex primitive—a point ($n = 1$), line segment ($n = 2$) or triangle ($n = 3$)—with up to n additional vertices that act as control points. The output is zero or more primitives, possibly of a different type than the input. For example, the geometry shader could convert points into two-triangle quads, or it could transform triangles into triangles but optionally discard some triangles and so on.
- The pixel shader's input is a fragment whose attributes have been interpolated from the three vertices of the triangle from which it came. The output of the pixel shader is the color that will be written into the frame buffer (presuming the fragment passes the depth test and other optional

tests). The pixel shader is also capable of discarding fragments explicitly, in which case it produces no output.

II.2.5.1 Accessing Memory

Because the GPU implements a data processing pipeline, access to RAM is carefully controlled. A shader program usually cannot read from or write to memory directly. Instead, its memory accesses are limited to two methods: registers and texture maps.

However, we should note that these restrictions are lifted on systems in which the GPU and CPU share memory directly. For example, the AMD Jaguar system on a chip (SoC) that sits at the heart of the PlayStation 4 is an example of a *heterogeneous system architecture* (HSA). On a non-HSA system, the CPU and GPU are typically separate devices, each with its own private memory, and each usually residing on a separate circuit board. Transferring data between the two processors requires cumbersome, high-latency communication over a specialized bus such as AGP or PCIe. With HSA, the CPU and GPU share a single unified memory store called a *heterogeneous unified memory architecture* (hUMA). Shaders running on a system with hUMA, like the PS4, can therefore be passed a *shader resource table* (SRT) as input. This is just a pointer to a C/C++ struct in memory that can be read from or written to by both the CPU and the shader running on the GPU. On the PS4, SRTs take the place of the constant registers described in the following sections.

Shader Registers

A shader can access RAM indirectly via *registers*. All GPU registers are in 128-bit SIMD format. Each register is capable of holding four 32-bit floating-point or integer values (represented by the `float4` data type in the Cg language). Such a register can contain a four-element vector in homogeneous coordinates or a color in RGBA format, with each component in 32-bit floating-point format. Matrices can be represented by groups of three or four registers (represented by built-in matrix types like `float4x4` in Cg). A GPU register can also be used to hold a single 32-bit scalar, in which case the value is usually replicated across all four 32-bit fields. Some GPUs can operate on 16-bit fields, known as *halfs*. (Cg provides various built-in types like `half4` and `half4x4` for this purpose.)

Registers come in four flavors, as follows:

- *Input registers.* These registers are the shader's primary source of input data. In a vertex shader, the input registers contain attribute data

obtained directly from the vertices. In a pixel shader, the input registers contain interpolated vertex attribute data corresponding to a single fragment. The values of all input registers are set automatically by the GPU prior to invoking the shader.

- *Constant registers.* The values of constant registers are set by the application and can change from primitive to primitive. Their values are constant only from the point of view of the shader program. They provide a secondary form of input to the shader. Typical contents include the model-view matrix, the projection matrix, light parameters and any other parameters required by the shader that are not available as vertex attributes.
- *Temporary registers.* These registers are for use by the shader program internally and are typically used to store intermediate results of calculations.
- *Output registers.* The contents of these registers are filled in by the shader and serve as its only form of output. In a vertex shader, the output registers contain vertex attributes such as the transformed position and normal vectors in homogeneous clip space, optional vertex colors, texture coordinates and so on. In a pixel shader, the output register contains the final color of the fragment being shaded.

The application provides the values of the constant registers when it submits primitives for rendering. The GPU automatically copies vertex or fragment attribute data from video RAM into the appropriate input registers prior to calling the shader program, and it also writes the contents of the output registers back into RAM at the conclusion of the program's execution so that the data can be passed to the next stage of the pipeline.

GPUs typically cache output data so that it can be reused without being recalculated. For example, the *post-transform vertex cache* stores the most-recently processed vertices emitted by the vertex shader. If a triangle is encountered that refers to a previously processed vertex, it will be read from the post-transform vertex cache if possible—the vertex shader need only be called again if the vertex in question has since been ejected from the cache to make room for newly processed vertices.

Textures

A shader also has direct read-only access to *texture maps*. Texture data is addressed via texture coordinates, rather than via absolute memory addresses. The GPU's texture samplers automatically *filter* the texture data, blending values between adjacent texels or adjacent mipmap levels as appropriate. Texture

filtering can be disabled in order to gain direct access to the values of particular texels. This can be useful when a texture map is used as a data table, for example.

Shaders can only *write* to texture maps in an indirect manner—by rendering the scene to an off-screen frame buffer that is interpreted as a texture map by subsequent rendering passes. This feature is known as *render to texture*.

II.2.5.2 Introduction to High-Level Shader Language Syntax

High-level shader languages like Cg and GLSL are modeled after the C programming language. The programmer can declare functions, define a simple struct, and perform arithmetic. However, as we said above, a shader program only has access to registers and textures. As such, the struct and variable we declare in Cg or GLSL is mapped directly onto registers by the shader compiler. We define these mappings in the following ways:

- *Semantics.* Variables and struct members can be suffixed with a colon followed by a keyword known as a *semantic*. The semantic tells the shader compiler to bind the variable or data member to a particular vertex or fragment attribute. For example, in a vertex shader we might declare an input struct whose members map to the *position* and *color* attributes of a vertex as follows:

```
struct VtxOut
{
    float4 pos : POSITION; // map to position attribute
    float4 color : COLOR; // map to color attribute
};
```

- *Input versus output.* The compiler determines whether a particular variable or struct should map to input or output registers from the context in which it is used. If a variable is passed as an argument to the shader program's main function, it is assumed to be an input; if it is the return value of the main function, it is taken to be an output.

```
VtxOut vshaderMain(VtxIn in) // maps to input registers
{
    VtxOut out;
    // ...
    return out; // maps to output registers
}
```

- *Uniform declaration.* To gain access to the data supplied by the application via the constant registers, we can declare a variable with the keyword

uniform. For example, the model-view matrix could be passed to a vertex shader as follows:

```
VtxOut vshaderMain(
    VtxIn in,
    uniform float4x4 modelViewMatrix)
{
    VtxOut out;
    // ...
    return out;
}
```

Arithmetic operations can be performed by invoking C-style operators, or by calling intrinsic functions as appropriate. For example, to multiply the input vertex position by the model-view matrix, we could write:

```
VtxOut vshaderMain(VtxIn in,
    uniform float4x4 modelViewMatrix)
{
    VtxOut out;

    out.pos = mul(modelViewMatrix, in.pos);
    out.color = float4(0, 1, 0, 1); // RGBA green

    return out;
}
```

Data is obtained from textures by calling special intrinsic functions that read the value of the texels at a specified texture coordinate. A number of variants are available for reading one-, two- and three-dimensional textures in various formats, with and without filtering. Special texture addressing modes are also available for accessing cube maps and shadow maps. References to the texture maps themselves are declared using a special data type known as a *texture sampler* declaration. For example, the data type `sampler2D` represents a reference to a typical two-dimensional texture. The following simple Cg pixel shader applies a diffuse texture to a triangle:

```
struct FragmentOut
{
    float4 color : COLOR;
};

FragmentOut pshaderMain(float2 uv : TEXCOORD0,
    uniform sampler2D texture)
{
    FragmentOut out;
```

```

        // look up texel at (u,v)
        out.color = tex2D(texture, uv);

    return out;
}

```

II.2.5.3 Effect Files

By itself, a shader program isn't particularly useful. Additional information is required by the GPU pipeline in order to call the shader program with meaningful inputs. For example, we need to specify how the application-specified parameters, like the model-view matrix, light parameters and so on, map to the uniform variables declared in the shader program. In addition, some visual effects require two or more rendering passes, but a shader program only describes the operations to be applied during a single rendering pass. If we are writing a game for the PC platform, we will need to define "fallback" versions of some of our more-advanced rendering effects, so that they will work even on older graphics cards. To tie our shader program(s) together into a complete visual effect, we turn to a file format known as an *effect file*.

Different rendering engines implement effects in slightly different ways. In Cg, the effect file format is known as CgFX. OGRE uses a file format very similar to CgFX known as a *material file*. GLSL effects can be described using the COLLADA format, which is based on XML. Despite the differences, effects generally take on the following hierarchical format:

- At global scope, structs, shader programs (implemented as various "main" functions) and global variables (which map to application-specified constant parameters) are defined.
- One or more *techniques* are defined. A technique represents one way to render a particular visual effect. An effect typically provides a primary technique for its highest-quality implementation and possibly a number of fallback techniques for use on lower-powered graphics hardware.
- Within each technique, one or more *passes* are defined. A pass describes how a single full-frame image should be rendered. It typically includes a reference to a vertex, geometry and/or pixel shader program's "main" function, various parameter bindings and optional render state settings.

II.2.5.4 Further Reading

In this section, we've only had a small taste of what high-level shader programming is like—a complete tutorial is beyond our scope here. For a much

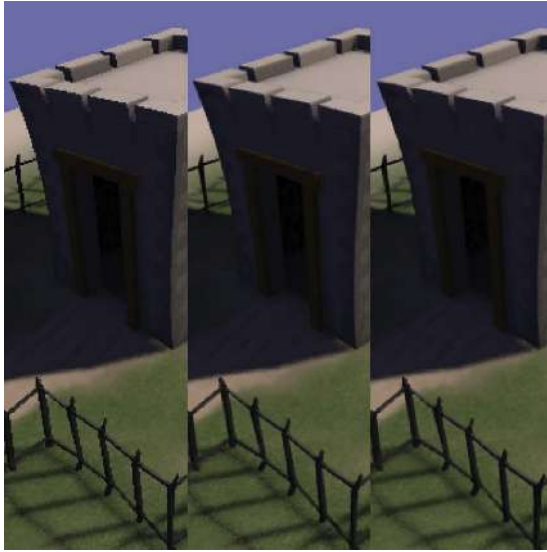


Figure 11.45. No antialiasing (left), 4× MSAA (center) and Nvidia's FXAA, preset 3 (right). Image from Nvidia's FXAA white paper by Timothy Lottes (<http://bit.ly/1mlzCTv>). (See Color Plate XIX.)

more detailed introduction to Cg shader programming, refer to the Cg tutorial available on NVIDIA's website at <https://developer.nvidia.com/content/hello-cg-introductory-tutorial>.

11.2.6 Antialiasing

When a triangle is rasterized, its edges can look jagged—the familiar “stair step” effect we have all come to know and love (or hate). Technically speaking, aliasing arises because we are using a discrete set of pixels to *sample* an image that is really a smooth, continuous two-dimensional signal. (See Section 14.3.2.1 for a detailed discussion of sampling and aliasing.)

The term *antialiasing* describes any technique that reduces the visual artifacts caused by aliasing. There are many different ways to antialias a rendered scene. The net effect of pretty much all of them is to “soften” the edges of rendered triangles by blending them with surrounding pixels. Each technique has unique performance, memory-usage and quality characteristics. Figure 11.45 shows a scene rendered first without antialiasing, then with 4× MSAA and finally with Nvidia's FXAA technique.

II.2.6.1 Full-Screen Antialiasing (FSAA)

In this technique, also known as *super-sampled antialiasing* (SSAA), the scene is rendered into a frame buffer that is larger than the actual screen. Once rendering of the frame is complete, the resulting oversized image is *downsampled* to the desired resolution. In $4\times$ supersampling, the rendered image is twice as wide and twice as tall as the screen, resulting in a frame buffer that occupies four times the memory. It also requires four times the GPU processing power because the pixel shader must be run four times for each screen pixel. As you can see, FSAA is an incredibly expensive technique both in terms of memory consumption and GPU cycles. As such, it is rarely used in practice.

II.2.6.2 Multisampled Antialiasing (MSAA)

Multisampled antialiasing is a technique that provides visual quality comparable to that of FSAA, while consuming a great deal less GPU bandwidth (and the same amount of video RAM). The MSAA approach is based on the observation that, thanks to the natural antialiasing effect of texture *mipmapping*, aliasing tends to be a problem primarily at the *edges* of triangles, not in their interiors.

To understand how MSAA works, recall that the process of rasterizing a triangle really boils down to three distinct operations: (1) Determining which pixels the triangle overlaps (coverage), (2) determining whether or not each pixel is occluded by some other triangle (depth testing) and (3) determining the color of each pixel, presuming that the coverage and depth tests tell us that the pixel should in fact be drawn (pixel shading).

When rasterizing a triangle without antialiasing, the coverage test, depth test and pixel shading operations are all run at a single idealized point within each screen pixel, usually located at its center. In MSAA, the coverage and depth tests are run for N points known as *subsamples* within each screen pixel. N is typically chosen to be 2, 4, 5, 8 or 16. However, the pixel shader is only run *once* per screen pixel, no matter how many subsamples we use. This gives MSAA a big advantage over FSAA in terms of GPU bandwidth, because shading is typically a great deal more expensive than coverage and depth testing.

In $N\times$ MSAA, the depth, stencil and color buffers are each allocated to be N times as large as they would otherwise be. For each screen pixel, these buffers contain N “slots,” one slot for each subsample. When rasterizing a triangle, the coverage and depth tests are run N times for the N subsamples within each fragment of the triangle. If at least one of the N tests indicates that the fragment should be drawn, the pixel shader is run once. The color obtained from the pixel shader is then stored *only* into those slots that correspond to

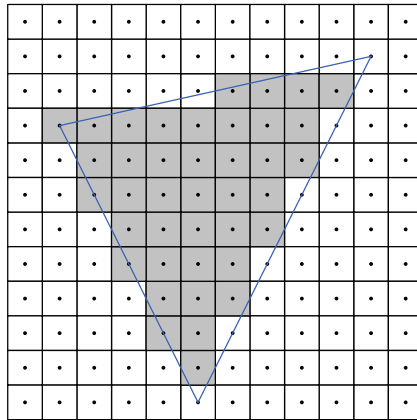


Figure 11.46. Rasterizing a triangle without antialiasing.

the subsamples that fell *inside* the triangle. Once the entire scene has been rendered, the oversized color buffer is downsampled to yield the final screen-resolution image. This process involves averaging the color values found in the N subsample slots for each screen pixel. The net result is an antialiased image with a shading cost equal to that of a non-antialiased image.

In Figure 11.46 we see a triangle that has been rasterized without antialiasing. Figure 11.47 illustrates the $4\times$ MSAA technique. For more information on MSAA, see <http://mynameismjp.wordpress.com/2012/10/24/msaa-overview>.

11.2.6.3 Coverage Sample Antialiasing (CSAA)

This technique is an optimization of the MSAA technique pioneered by Nvidia. For $4\times$ CSAA, the pixel shader is run once, the depth test and color storage is done for four subsample points per fragment, but the pixel coverage test is performed for 16 “coverage subsamples” per fragment. This produces finer-grained color blending at the edges of triangles, similar to what you’d see with $8\times$ or $16\times$ MSAA, but at the memory and GPU cost of $4\times$ MSAA.

11.2.6.4 Morphological Antialiasing (MLAA)

Morphological antialiasing focuses its efforts on correcting only those regions of a scene that suffer the most from the effects of aliasing. In MLAA, the scene is rendered at normal size, and then scanned in order to identify stair-stepped patterns. When these patterns are found, they are blurred to reduce the effects of aliasing. Fast approximate antialiasing (FXAA) is an optimized technique

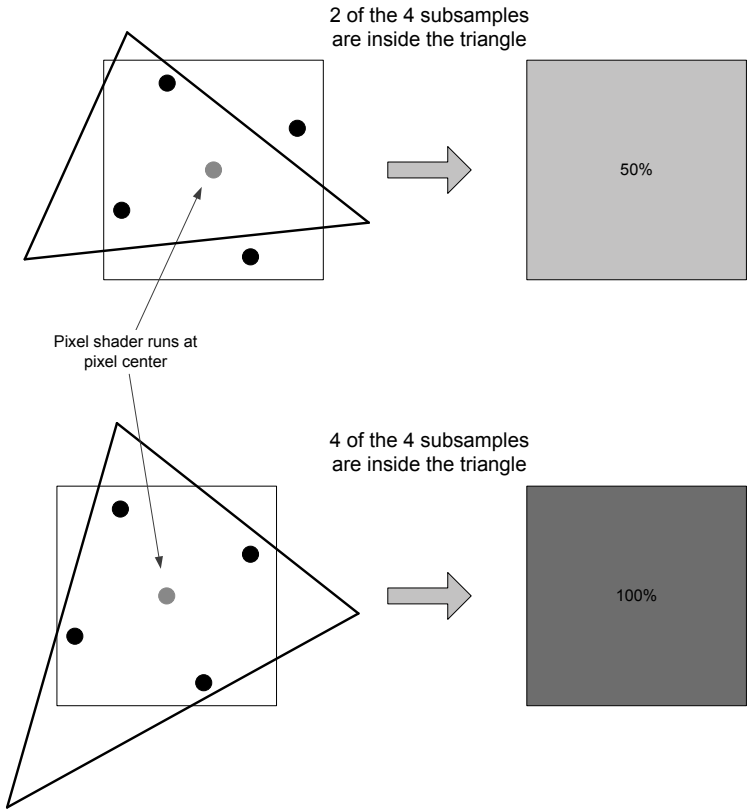


Figure II.47. Multisampled antialiasing (MSAA).

developed by Nvidia that is similar to MLAA in its approach.

For a detailed discussion of MLAA, see <https://intel.ly/2HhrQWX>. FXAA is described in detail here: <https://bit.ly/1mIzCTv>.

11.2.6.5 Subpixel Morphological Antialiasing (SMAA)

Subpixel Morphological Antialiasing (SMAA) combines morphological antialiasing (MLAA and FXAA) techniques with multisampling/supersampling strategies (MSAA, SSAA) to produce more accurate subpixel features. Like FXAA, it's an inexpensive technique, but it blurs the final image less than FXAA. For these reasons, it's arguably the best AA solution available today. A detailed coverage of this topic is beyond our scope in this book, but you can read more about SMAA at <http://www.iryoku.com/smaa/>.

11.2.7 The Application Stage

Now that we understand how the GPU works, we can discuss the pipeline stage that is responsible for driving it—the application stage. This stage has three roles:

1. *Visibility determination.* Only objects that are visible (or at least *potentially* visible) should be submitted to the GPU, lest we waste valuable resources processing triangles that will never be seen.
2. *Submitting geometry to the GPU for rendering.* Submesh-material pairs are sent to the GPU via a rendering call like `DrawIndexedPrimitive()` (DirectX) or `glDrawArrays()` (OpenGL), or via direct construction of the GPU command list. The geometry may be sorted for optimal rendering performance. Geometry might be submitted more than once if the scene needs to be rendered in multiple passes.
3. *Controlling shader parameters and render state.* The uniform parameters passed to the shader via constant registers are configured by the application stage on a per-primitive basis. In addition, the application stage must set all of the configurable parameters of the non-programmable pipeline stages to ensure that each primitive is rendered appropriately.

In the following sections, we'll briefly explore how the application stage performs these tasks.

11.2.7.1 Visibility Determination

The cheapest triangles are the ones you never draw. So it's incredibly important to *cull* objects from the scene that do not contribute to the final rendered

image prior to submitting them to the GPU. The process of constructing the list of visible mesh instances is known as *visibility determination*.

Frustum Culling

In frustum culling, all objects that lie entirely outside the frustum are excluded from our render list. Given a candidate mesh instance, we can determine whether or not it lies inside the frustum by performing some simple tests between the object's *bounding volume* and the six frustum planes. The bounding volume is usually a sphere, because spheres are particularly easy to cull. For each frustum plane, we move the plane outward a distance equal to the radius of the sphere, then we determine on which side of each modified plane the center point of the sphere lies. If the sphere is found to be on the front side of all six modified planes, the sphere is inside the frustum.

In practice, we don't need to actually move the frustum planes. Recall from Equation (5.13) that the perpendicular distance h from a point to a plane can be calculated by plugging the point directly into the plane equation as follows: $h = ax + by + cz + d = \mathbf{n} \cdot \mathbf{P} - \mathbf{n} \cdot \mathbf{P}_0$ (see Section 5.6.3). So all we need to do is plug the center point of our bounding sphere into the plane equations for each frustum plane, giving us a value of h_i for each plane i , and then we can compare the h_i values to the radius of the bounding sphere to determine whether or not it lies inside each plane.

A scene graph data structure, described in Section 11.2.7.4, can help optimize frustum culling by allowing us to ignore objects whose bounding spheres are nowhere close to being inside the frustum.

Occlusion and Potentially Visible Sets

Even when objects lie entirely within the frustum, they may occlude one another. Removing objects from the visible list that are entirely occluded by other objects is called *occlusion culling*. In crowded environments viewed from ground level, there can be a great deal of inter-object occlusion, making occlusion culling extremely important. In less crowded scenes, or when scenes are viewed from above, much less occlusion may be present, and the cost of occlusion culling may outweigh its benefits.

Gross occlusion culling of a large-scale environment can be done by precalculating a *potentially visible set* (PVS). For any given camera vantage point, a PVS lists those scene objects that might be visible. A PVS errs on the side of including objects that aren't actually visible, rather than excluding objects that actually would have contributed to the rendered scene.

One way to implement a PVS system is to chop the level up into regions of some kind. Each region can be provided with a list of the other regions

that can be seen when the camera is inside it. These PVSs might be manually specified by the artists or game designers. More commonly, an automated offline tool generates the PVS based on user-specified regions. Such a tool usually operates by rendering the scene from various randomly distributed vantage points within a region. Every region's geometry is color coded, so the list of visible regions can be found by scanning the resulting frame buffer and tabulating the region colors that are found. Because automated PVS tools are imperfect, they typically provide the user with a mechanism for tweaking the results, either by manually placing vantage points for testing, or by manually specifying a list of regions that should be explicitly included or excluded from a particular region's PVS.

Portals

Another way to determine what portions of a scene are visible is to use *portals*. In portal rendering, the game world is divided up into semiclosed regions that are connected to one another via holes, such as windows and doorways. These holes are called portals. They are usually represented by polygons that describe their boundaries.

To render a scene with portals, we start by rendering the region that contains the camera. Then, for each portal in the region, we extend a frustum-like volume consisting of planes extending from the camera's focal point through each edge of the portal's bounding polygon. The contents of the neighboring region can be culled to this portal volume in exactly the same way geometry is culled against the camera frustum. This ensures that only the visible geometry in the adjacent regions will be rendered. Figure 11.48 provides an illustration of this technique.

Occlusion Volumes (Antiportals)

If we flip the portal concept on its head, pyramidal volumes can also be used to describe regions of the scene that *cannot* be seen because they are being occluded by an object. These volumes are known as *occlusion volumes* or *antiportals*. To construct an occlusion volume, we find the silhouette edges of each occluding object and extend planes outward from the camera's focal point through each of these edges. We test more-distant objects against these occlusion volumes and cull them if they lie entirely within the occlusion region. This is illustrated in Figure 11.49.

Portals are best used when rendering enclosed indoor environments with a relatively small number of windows and doorways between "rooms." In this kind of scene, the portals occupy a relatively small percentage of the total

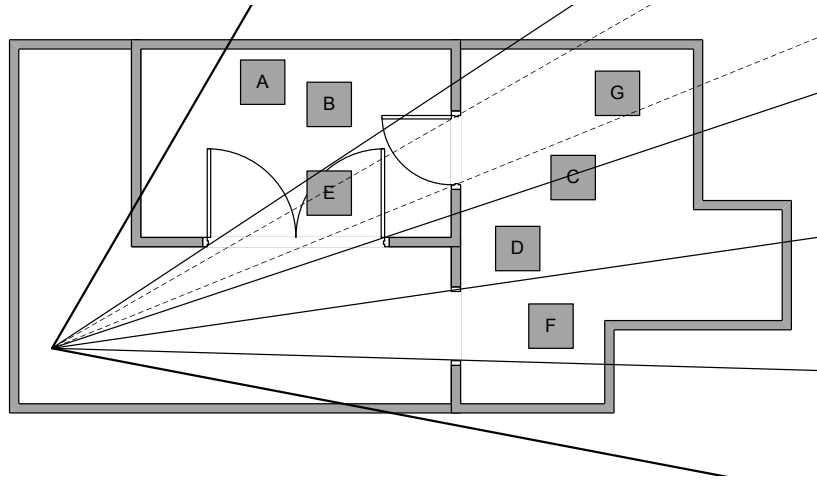


Figure 11.48. Portals are used to define frustum-like volumes, which are used to cull the contents of neighboring regions. In this example, objects A, B and D will be culled because they lie outside one of the portals; the other objects will be visible.

volume of the camera frustum, resulting in a large number of objects outside the portals that can be culled. Antiportals are best applied to large outdoor environments, in which nearby objects often occlude large swaths of the camera frustum. In this case, the antiportals occupy a relatively large percentage of the total camera frustum volume, resulting in large numbers of culled objects.

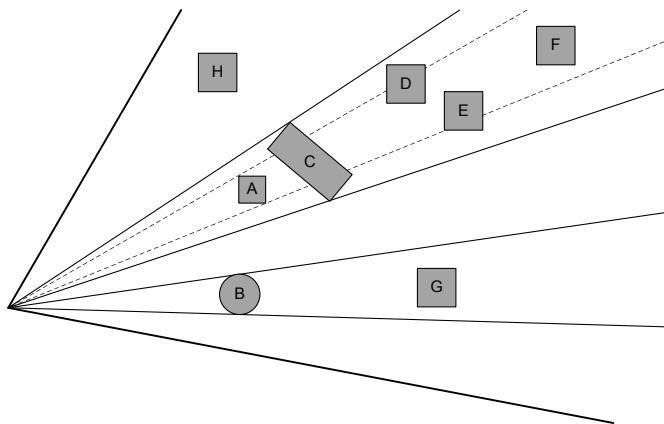


Figure 11.49. As a result of the antiportals corresponding to objects A, B and C, objects D, E, F and G are culled. Therefore, only A, B, C and H are visible.

11.2.7.2 Primitive Submission

Once a list of visible geometric primitives has been generated, the individual primitives must be *submitted* to the GPU pipeline for rendering. This can be accomplished by making calls to `DrawIndexedPrimitive()` in DirectX or `glDrawArrays()` in OpenGL.

Render State

As we learned in Section 11.2.4, the functionality of many of the GPU pipeline's stages is fixed but configurable. And even programmable stages are driven in part by configurable parameters. Some examples of these configurable parameters are listed below (although this is by no means a complete list):

- world-view matrix;
- light direction vectors;
- texture bindings (i.e., which textures to use for a given material/shader);
- texture addressing and filtering modes;
- time base for scrolling textures and other animated effects;
- z-test (enabled or disabled); and
- alpha blending options.

The set of all configurable parameters within the GPU pipeline is known as the *hardware state* or *render state*. It is the application stage's responsibility to ensure that the hardware state is configured properly and completely for each submitted primitive. Ideally these state settings are described completely by the material associated with each submesh. So the application stage's job boils down to iterating through the list of visible mesh instances, iterating over each submesh-material pair, setting the render state based on the material's specifications and then calling the low-level primitive submission functions (`DrawIndexedPrimitive()`, `glDrawArrays()`, or similar).

State Leaks

If we forget to set some aspect of the render state between submitted primitives, the settings used on the previous primitive will “leak” over onto the new primitive. A *render state leak* might manifest itself as an object with the wrong texture or an incorrect lighting effect, for example. Clearly it's important that the application stage never allow state leaks to occur.

The GPU Command List

The application stage actually communicates with the GPU via a command list. These commands interleave render state settings with references to the geometry that should be drawn. For example, to render objects A and B with material 1, followed by objects C, D and E using material 2, the command list might look like this:

- Set render state for material 1 (multiple commands, one per render state setting).
- Submit primitive A.
- Submit primitive B.
- Set render state for material 2 (multiple commands).
- Submit primitive C.
- Submit primitive D.
- Submit primitive E.

Under the hood, API functions like `DrawIndexedPrimitive()` actually just construct and submit GPU command lists. The cost of these API calls can themselves be too high for some applications. To maximize performance, some game engines build GPU command lists manually or by calling a low-level rendering API like Vulkan (<https://www.khronos.org/vulkan/>).

II.2.7.3 Geometry Sorting

Render state settings are global—they apply to the entire GPU as a whole. So in order to change render state settings, the entire GPU pipeline must be flushed before the new settings can be applied. This can cause massive performance degradation if not managed carefully.

Clearly we'd like to change render settings as infrequently as possible. The best way to accomplish this is to sort our geometry by material. That way, we can install material A's settings, render all geometry associated with material A and then move on to material B.

Unfortunately, sorting geometry by material can have a detrimental effect on rendering performance because it increases *overdraw*—a situation in which the same pixel is filled multiple times by multiple overlapping triangles. Certainly some overdraw is necessary and desirable, as it is the only way to properly alpha-blend transparent and translucent surfaces into a scene. However, overdraw of *opaque* pixels is always a waste of GPU bandwidth.

The early z-test is designed to discard occluded fragments before the expensive pixel shader has a chance to execute. But to take maximum advantage of early z, we need to draw the triangles in front-to-back order. That way, the

closest triangles will fill the z-buffer right off the bat, and all of the fragments coming from more-distant triangles behind them can be quickly discarded, with little or no overdraw.

z-Prepass to the Rescue

How can we reconcile the need to sort geometry by material with the conflicting need to render opaque geometry in a front-to-back order? The answer lies in a GPU feature known as *z-prepass*.

The idea behind *z-prepass* is to render the scene twice: the first time to generate the contents of the z-buffer as efficiently as possible and the second time to populate the frame buffer with full color information (but this time with no overdraw, thanks to the contents of the z-buffer). The GPU provides a special double-speed rendering mode in which the pixel shaders are disabled, and only the z-buffer is updated. Opaque geometry can be rendered in front-to-back order during this phase, to minimize the time required to generate the z-buffer contents. Then the geometry can be resorted into material order and rendered in full color with minimal state changes for maximum pipeline throughput.

Once the opaque geometry has been rendered, transparent surfaces can be drawn in back-to-front order. This brute-force method allows us to achieve the proper alpha-blended result. *Order-independent transparency* (OIT) is a technique that permits transparent geometry to be rendered in an arbitrary order. It works by storing multiple fragments per pixel, sorting each pixel's fragments and blending them only after the entire scene has been rendered. This technique produces correct results without the need for pre-sorting the geometry, but it comes at a high memory cost because the frame buffer must be large enough to store all of the translucent fragments for each pixel.

11.2.7.4 Scene Graphs

Modern game worlds can be very large. The majority of the geometry in most scenes does not lie within the camera frustum, so frustum culling all of these objects explicitly is usually incredibly wasteful. Instead, we would like to devise a data structure that manages all of the geometry in the scene and allows us to quickly discard large swaths of the world that are nowhere near the camera frustum prior to performing detailed frustum culling. Ideally, this data structure should also help us to sort the geometry in the scene, either in front-to-back order for the *z-prepass* or in material order for full-color rendering.

Such a data structure is often called a *scene graph*, in reference to the graph-like data structures often used by film rendering engines and DCC tools like Maya. However, a game's scene graph needn't actually be a graph, and in fact the data structure of choice is usually some kind of tree (which is, of course,

a special case of a graph). The basic idea behind most of these data structures is to partition three-dimensional space in a way that makes it easy to discard regions that do not intersect the frustum, without having to frustum cull all of the individual objects within them. Examples include quadtrees and octrees, BSP trees, *kd*-trees and spatial hashing techniques.

Quadtrees and Octrees

A *quadtree* divides space into quadrants recursively. Each level of recursion is represented by a node in the quadtree with four children, one for each quadrant. The quadrants are typically separated by vertically oriented, axis-aligned planes, so that the quadrants are square or rectangular. However, some quadtrees subdivide space using arbitrarily shaped regions.

Quadrees can be used to store and organize virtually any kind of spatially distributed data. In the context of rendering engines, quadrees are often used to store renderable primitives such as mesh instances, subregions of terrain geometry or individual triangles of a large static mesh, for the purposes of efficient frustum culling. The renderable primitives are stored at the leaves of the tree, and we usually aim to achieve a roughly uniform number of primitives within each leaf region. This can be achieved by deciding whether to continue or terminate the subdivision based on the number of primitives within a region.

To determine which primitives are visible within the camera frustum, we walk the tree from the root to the leaves, checking each region for intersection with the frustum. If a given quadrant does not intersect the frustum, then we know that none of its child regions will do so either, and we can stop traversing that branch of the tree. This allows us to search for potentially visible primitives much more quickly than would be possible with a linear search (usually in $O(\log n)$ time). An example of a quadtree subdivision of space is shown in Figure 11.50.

An *octree* is the three-dimensional equivalent of a quadtree, dividing space into eight subregions at each level of the recursive subdivision. The regions of an octree are often cubes or rectangular prisms but can be arbitrarily shaped three-dimensional regions in general.

Bounding Sphere Trees

In the same way that a quadtree or octree subdivides space into (usually) rectangular regions, a *bounding sphere tree* divides space into spherical regions hierarchically. The leaves of the tree contain the bounding spheres of the renderable primitives in the scene. We collect these primitives into small logical

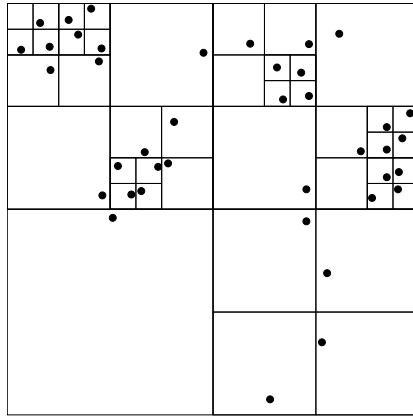


Figure 11.50. A top-down view of a space divided recursively into quadrants for storage in a quadtree, based on the criterion of one point per region.

groups and calculate the net bounding sphere of each group. The groups are themselves collected into larger groups, and this process continues until we have a single group with a bounding sphere that encompasses the entire virtual world. To generate a list of potentially visible primitives, we walk the tree from the root to the leaves, testing each bounding sphere against the frustum, and only recursing down branches that intersect it.

BSP Trees

A binary space partitioning (BSP) tree divides space in half recursively until the objects within each half-space meet some predefined criteria (much as a quadtree divides space into quadrants). BSP trees have numerous uses, including collision detection and constructive solid geometry, as well as their most well-known application as a method for increasing the performance of frustum culling and geometry sorting for 3D graphics. A *kd-tree* is a generalization of the BSP tree concept to k dimensions.

In the context of rendering, a BSP tree divides space with a single plane at each level of the recursion. The dividing planes can be axis-aligned, but more commonly each subdivision corresponds to the plane of a single triangle in the scene. All of the other triangles are then categorized as being either on the front side or the back side of the plane. Any triangles that intersect the dividing plane are themselves divided into three new triangles, so that every triangle lies either entirely in front of or entirely behind the plane, or is coplanar with it. The result is a binary tree with a dividing plane and one or more triangles at each interior node and triangles at the leaves.

A BSP tree can be used for frustum culling in much the same way a quad-tree, octree or bounding sphere tree can. However, when generated with individual triangles as described above, a BSP tree can also be used to sort triangles into a strictly back-to-front or front-to-back order. This was particularly important for early 3D games like *Doom*, which did not have the benefit of a z-buffer and so were forced to use the painter's algorithm (i.e., to render the scene from back to front) to ensure proper inter-triangle occlusion.

Given a camera view point in 3D space, a back-to-front sorting algorithm walks the tree from the root. At each node, we check whether the view point is in front of or behind that node's dividing plane. If the camera is in front of a node's plane, we visit the node's back children first, then draw any triangles that are coplanar with its dividing plane, and finally we visit its front children. Likewise, when the camera's view point is found to be behind a node's dividing plane, we visit the node's front children first, then draw the triangles coplanar with the node's plane and finally we visit its back children. This traversal scheme ensures that the triangles farthest from the camera will be visited before those that are closer to it, and hence it yields a back-to-front ordering. Because this algorithm traverses *all* of the triangles in the scene, the order of the traversal is independent of the direction the camera is looking. A secondary frustum culling step would be required in order to traverse only visible triangles. A simple BSP tree is shown in Figure 11.51, along with the tree traversal that would be done for the camera position shown.

Full coverage of BSP tree generation and usage algorithms is beyond our scope here. See <http://www.gamedev.net/reference/articles/article657.asp>

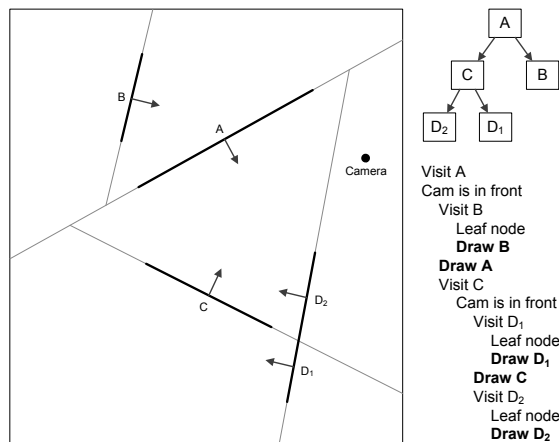


Figure 11.51. An example of back-to-front traversal of the triangles in a BSP tree. The triangles are shown edge-on in two dimensions for simplicity, but in a real BSP tree the triangles and dividing planes would be arbitrarily oriented in space.

for more details on BSP trees.

11.2.7.5 Choosing a Scene Graph

Clearly there are many different kinds of scene graphs. Which data structure to select for your game will depend upon the nature of the scenes you expect to be rendering. To make the choice wisely, you must have a clear understanding of what is required—and more importantly what is *not* required—when rendering scenes for your particular game.

For example, if you're implementing a fighting game, in which two characters battle it out in a ring surrounded by a mostly static environment, you may not need much of a scene graph at all. If your game takes place primarily in enclosed indoor environments, a BSP tree or portal system may serve you well. If the action takes place outdoors on relatively flat terrain, and the scene is viewed primarily from above (as might be the case in a strategy game or god game), a simple quadtree might be all that's required to achieve high rendering speeds. On the other hand, if an outdoor scene is viewed primarily from the point of view of someone on the ground, we may need additional culling mechanisms. Densely populated scenes can benefit from an occlusion volume (antiportal) system, because there will be plenty of occluders. On the other hand, if your outdoor scene is very sparse, adding an antiportal system probably won't pay dividends (and might even hurt your frame rate).

Ultimately, your choice of scene graph should be based on hard data obtained by actually measuring the performance of your rendering engine. You may be surprised to learn where all your cycles are actually going! But once you know, you can select scene graph data structures and/or other optimizations to target the specific problems at hand.

11.3 Advanced Lighting and Global Illumination

In order to render photorealistic scenes, we need physically accurate global illumination algorithms. A complete coverage of these techniques is beyond our scope. In the following sections, we will briefly outline the most prevalent techniques in use within the game industry today. Our goal here is to provide you with an awareness of these techniques and a jumping-off point for further investigation. For an excellent in-depth coverage of this topic, see [10].

11.3.1 Image-Based Lighting

A number of advanced lighting and shading techniques make heavy use of image data, usually in the form of two-dimensional texture maps. These are

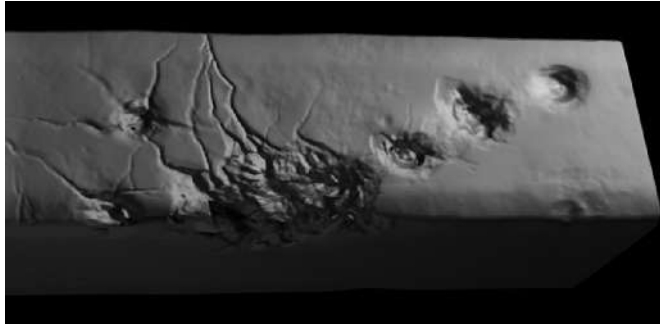


Figure II.52. An example of a normal-mapped surface.

called *image-based lighting* algorithms.

II.3.1.1 Normal Mapping

A *normal map* specifies a surface normal direction vector at each texel. This allows a 3D modeler to provide the rendering engine with a highly detailed description of a surface's shape, without having to tessellate the model to a high degree (as would be required if this same information were to be provided via vertex normals). Using a normal map, a single flat triangle can be made to look as though it were constructed from millions of tiny triangles. An example of normal mapping is shown in Figure 11.52.

The normal vectors are typically encoded in the RGB color channels of the texture, with a suitable bias to overcome the fact that RGB channels are strictly positive while normal vector components can be negative. Sometimes only two coordinates are stored in the texture; the third can be easily calculated at runtime, given the assumption that the surface normals are unit vectors.

II.3.1.2 Heightmaps: Bump, Parallax and Displacement Mapping

As its name implies, a *heightmap* encodes the height of the ideal surface above or below the surface of the triangle. Heightmaps are typically encoded as grayscale images, since we only need a single height value per texel. Heightmaps can be used for *bump mapping*, *parallax occlusion mapping* and *displacement mapping*—three techniques that can make a planar surface appear to have height variation.

In bump mapping, a heightmap is used as a cheap way to generate surface normals. This technique was primarily used in the early days of 3D graphics—nowadays, most game engines store surface normal information



Figure 11.53. Comparison of bump mapping (left), parallax occlusion mapping (center) and displacement mapping (right).

explicitly in a *normal map*, rather than calculating the normals from a heightmap.

Parallax occlusion mapping uses the information in a heightmap to artificially adjust the texture coordinates used when rendering a flat surface, in such a way as to make the surface appear to contain surface details that move semi-correctly as the camera moves. (This technique was used to produce the bullet impact decals in the *Uncharted* series of games by Naughty Dog.)

Displacement mapping (also known as relief mapping) produces real surface details by actually tessellating and then extruding surface polygons, again using a heightmap to determine how much to displace each vertex. This produces the most convincing effect—one that properly self-occludes and self-shadows—because real geometry is being generated. Figure 11.53 compares bump mapping, parallax mapping and displacement mapping. Figure 11.54 shows an example of displacement mapping implemented in DirectX 9.

11.3.1.3 Specular/Gloss Maps

When light reflects directly off a shiny surface, we call this *specular* reflection. The intensity of a specular reflection depends on the relative angles of the viewer, the light source and the surface normal. As we saw in Section 11.1.3.2, the specular intensity takes the form $k_s(\mathbf{R} \cdot \mathbf{V})^\alpha$, where $\mathbf{R}$ is the reflection of the light's direction vector about the surface normal, $\mathbf{V}$ is the direction to the viewer, k_s is the overall specular reflectivity of the surface and α is called the specular power.

Many surfaces aren't uniformly glossy. For example, when a person's face is sweaty and dirty, wet regions appear shiny, while dry or dirty areas appear dull. We can encode high-detail specular information in a special texture map known as a *specular map*.

If we store the value of k_s in the texels of a specular map, we can control how much specular reflection should be applied at each texel. This kind of specular map is sometimes called a *gloss map*. It is also called a *specular mask*, because zero-valued texels can be used to “mask off” regions of the surface where we do not want specular reflection applied. If we store the value of



Figure II.54. DirectX 9 displacement mapping. Simple source geometry is tessellated at runtime to produce the surface details.

α in our specular map, we can control the amount of “focus” our specular highlights will have at each texel. This kind of texture is called a *specular power map*. An example of a gloss map is shown in Figure 11.55.

II.3.1.4 Environment Mapping

An environment map looks like a panoramic photograph of the environment taken from the point of view of an object in the scene, covering a full 360 degrees horizontally and either 180 degrees or 360 degrees vertically. An environment map acts like a description of the general lighting environment surrounding an object. It is generally used to inexpensively render reflections.

The two most common formats are *spherical environment maps* and *cubic environment maps*. A spherical map looks like a photograph taken through a fisheye lens, and it is treated as though it were mapped onto the inside of a sphere whose radius is infinite, centered about the object being rendered. The problem with sphere maps is that they are addressed using spherical coordinates. Around the equator, there is plenty of resolution both horizontally and vertically. However, as the vertical (azimuthal) angle approaches vertical, the resolution of the texture along the horizontal (zenith) axis decreases to a single



Figure 11.55. This screenshot from EA's *Fight Night Round 3* shows how a gloss map can be used to control the degree of specular reflection that should be applied to each texel of a surface. (See Color Plate XX.)

texel. Cube maps were devised to avoid this problem.

A cube map looks like a composite photograph pieced together from photos taken in the six primary directions (up, down, left, right, front and back). During rendering, a cube map is treated as though it were mapped onto the six inner surfaces of a box at infinity, centered on the object being rendered.

To read the environment map texel corresponding to a point P on the surface of an object, we take the ray from the camera to the point P and reflect it about the surface normal at P . The reflected ray is followed until it intersects the sphere or cube of the environment map. The value of the texel at this intersection point is used when shading the point P .

11.3.1.5 Three-Dimensional Textures

Modern graphics hardware also includes support for three-dimensional textures. A 3D texture can be thought of as a stack of 2D textures. The GPU knows how to address and filter a 3D texture, given a three-dimensional texture coordinate (u, v, w) .

Three-dimensional textures can be useful for describing the appearance or volumetric properties of an object. For example, we could render a marble sphere and allow it to be cut by an arbitrary plane. The texture would look continuous and correct across the cut no matter where it was made, because the texture is well-defined and continuous throughout the entire volume of the sphere.

11.3.2 High Dynamic Range Lighting

A display device like a television set or CRT monitor can only produce a limited range of intensities. This is why the color channels in the frame buffer are limited to a zero to one range. But in the real world, light intensities can grow arbitrarily large. *High dynamic range* (HDR) lighting attempts to capture this wide range of light intensities.

HDR lighting performs lighting calculations without clamping the resulting intensities arbitrarily. The resulting image is stored in a format that permits intensities to grow beyond one. The net effect is an image in which extreme dark and light regions can be represented without loss of detail within either type of region.

Prior to display on-screen, a process called *tone mapping* is used to shift and scale the image's intensity range into the range supported by the display device. Doing this permits the rendering engine to reproduce many real-world visual effects, like the temporary blindness that occurs when you walk from a dark room into a brightly lit area, or the way light seems to bleed out from behind a brightly back-lit object (an effect known as *bloom*).

One way to represent an HDR image is to store the R, G and B channels using 32-bit floating-point numbers, instead of 8-bit integers. Another alternative is to employ an entirely different color model altogether. The log-LUV color model is a popular choice for HDR lighting. In this model, color is represented as an intensity channel (L) and two chromaticity channels (U and V). Because the human eye is more sensitive to changes in intensity than it is to changes in chromaticity, the L channel is stored in 16 bits while U and V are given only eight bits each. In addition, L is represented using a logarithmic scale (base two) in order to capture a very wide range of light intensities.

11.3.3 Global Illumination

As we noted in Section 11.1.3.1, global illumination refers to a class of lighting algorithms that account for light's interactions with multiple objects in the scene, on its way from the light source to the virtual camera. Global illumination accounts for effects like the shadows that arise when one surface occludes another, reflections, caustics and the way the color of one object can "bleed" onto the objects around it. In the following sections, we'll take a brief look at some of the most common global illumination techniques. Some of these methods aim to reproduce a single isolated effect, like shadows or reflections. Others like radiosity and ray tracing methods aim to provide a holistic model of global light transport.

11.3.3.1 Shadow Rendering

Shadows are created when a surface blocks light's path. The shadows caused by an ideal point light source would be sharp, but in the real world shadows have blurry edges; this is called the penumbra. A penumbra arises because real-world light sources cover some area and so produce light rays that graze the edges of an object at different angles.

The two most prevalent shadow rendering techniques are *shadow volumes* and *shadow maps*. We'll briefly describe each in the sections below. In both techniques, objects in the scene are generally divided into three categories: objects that cast shadows, objects that are to receive shadows and objects that are entirely excluded from consideration when rendering shadows. Likewise, the lights are tagged to indicate whether or not they should generate shadows. This important optimization limits the number of light-object combinations that need to be processed in order to produce the shadows in a scene.

Shadow Volumes

In the shadow volume technique, each shadow caster is viewed from the vantage point of a shadow-generating light source, and the shadow caster's silhouette edges are identified. These edges are extruded in the direction of the light rays emanating from the light source. The result is a new piece of geometry that describes the volume of space in which the light is occluded by the shadow caster in question. This is shown in Figure 11.56.



Figure 11.56. A shadow volume generated by extruding the silhouette edges of a shadow casting object as seen from the point of view of the light source.

A shadow volume is used to generate a shadow by making use of a special full-screen buffer known as the stencil buffer. This buffer stores a single integer value corresponding to each pixel of the screen. Rendering can be masked by the values in the stencil buffer—for example, we could configure the GPU to only render fragments whose corresponding stencil values are nonzero. In addition, the GPU can be configured so that rendered geometry updates the values in the stencil buffer in various useful ways.

To render shadows, the scene is first drawn to generate an unshadowed image in the frame buffer, along with an accurate z-buffer. The stencil buffer is cleared so that it contains zeros at every pixel. Each shadow volume is then rendered from the point of view of the camera in such a way that front-facing triangles increase the values in the stencil buffer by one, while back-facing triangles decrease them by one. In areas of the screen where the shadow volume does not appear at all, of course the stencil buffer's pixels will be left containing zero. The stencil buffer will also contain zeros where both the front and back faces of the shadow volume are visible, because the front face will increase the stencil value but the back face will decrease it again. In areas where the back face of the shadow volume has been occluded by “real” scene geometry, the stencil value will be one. This tells us which pixels of the screen are in shadow. So we can render shadows in a third pass, by simply darkening those regions of the screen that contain a nonzero stencil buffer value.

Shadow Maps

The shadow mapping technique is effectively a per-fragment depth test performed from the point of view of the light instead of from the point of view of the camera. The scene is rendered in two steps: First, a *shadow map* texture is generated by rendering the scene from the point of view of the light source and saving off the contents of the depth buffer. Second, the scene is rendered as usual, and the shadow map is used to determine whether or not each fragment is in shadow. At each fragment in the scene, the shadow map tells us whether or not the light is being occluded by some geometry that is closer to the light source, in just the same way that the z-buffer tells us whether a fragment is being occluded by a triangle that is closer to the camera.

A shadow map contains only depth information—each texel records how far away it is from the light source. Shadow maps are therefore typically rendered using the hardware's double-speed z-only mode (since all we care about is the depth information). For a point light source, a perspective projection is used when rendering the shadow map; for a directional light source, an orthographic projection is used instead.

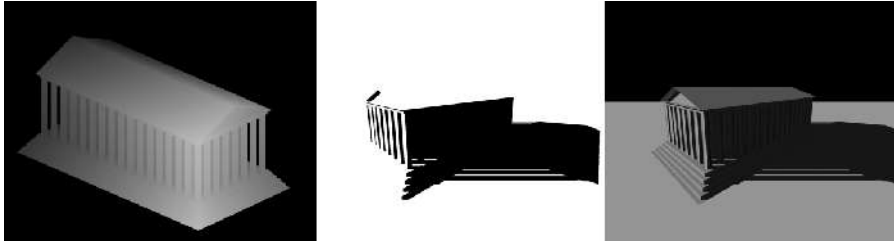


Figure 11.57. The far left image is a shadow map—the contents of the z-buffer as rendered from the point of view of a particular light source. The pixels of the center image are black where the light-space depth test failed (fragment in shadow) and white where it succeeded (fragment not in shadow). The far right image shows the final scene rendered with shadows.

To render a scene using a shadow map, we draw the scene as usual from the point of view of the camera. For each vertex of every triangle, we calculate its position in *light space*—i.e., in the same “view space” that was used when generating the shadow map in the first place. These light-space coordinates can be interpolated across the triangle, just like any other vertex attribute. This gives us the position of each fragment *in light space*. To determine whether a given fragment is in shadow or not, we convert the fragment’s light-space (x, y) -coordinates into texture coordinates (u, v) within the shadow map. We then compare the fragment’s light-space z -coordinate with the depth stored at the corresponding texel in the shadow depth map. If the fragment’s light-space z is farther away from the light than the texel in the shadow map, then it must be occluded by some other piece of geometry that is closer to the light source—hence it is in shadow. Likewise, if the fragment’s light-space z is closer to the light source than the texel in the shadow map, then it is not occluded and is not in shadow. Based on this information, the fragment’s color can be adjusted accordingly. The shadow mapping process is illustrated in Figure 11.57.

11.3.3.2 Ambient Occlusion

Ambient occlusion is a technique for modeling *contact shadows*—the soft shadows that arise when a scene is illuminated by only ambient light. In effect, ambient occlusion describes how “accessible” each point on a surface is to light in general. For example, the interior of a section of pipe is less accessible to ambient light than its exterior. If the pipe were placed outside on an overcast day, its interior would generally appear darker than its exterior.

Figure 11.58 shows how ambient occlusion produces shadows underneath a car and in its wheel wells, as well as within the seams between body panels. Ambient occlusion is measured at a point on a surface by constructing



Figure 11.58. A car rendered with ambient occlusion. Notice the darkened areas underneath the vehicle and in the wheel wells.

a hemisphere with a very large radius centered on that point and determining what percentage of that hemisphere's area is visible from the point in question. It can be precomputed offline for static objects, because ambient occlusion is independent of view direction and the direction of incident light. It is typically stored in a texture map that records the level of ambient occlusion at each texel across the surface.

11.3.3.3 Reflections

Reflections occur when light bounces off a highly specular (shiny) surface producing an image of another portion of the scene in the surface. Reflections can be implemented in a number of ways. Environment maps are used to produce general reflections of the surrounding environment on the surfaces of shiny objects. Direct reflections in flat surfaces like mirrors can be produced by reflecting the camera's position about the plane of the reflective surface and then rendering the scene from that reflected point of view into a texture. The texture is then applied to the reflective surface in a second pass (see Figure 11.59).

11.3.3.4 Caustics

Caustics are the bright specular highlights arising from intense reflections or refractions from very shiny surfaces like water or polished metal. When the reflective surface moves, as is the case for water, the caustic effects glimmer and "swim" across the surfaces on which they fall. Caustic effects can be produced by projecting a (possibly animated) texture containing semi-random bright highlights onto the affected surfaces. An example of this technique is shown in Figure 11.60.



Figure 11.59. Mirror reflections in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog. PlayStation 4) implemented by rendering the scene to a texture that is subsequently applied to the mirror's surface. (See Color Plate XXI.)

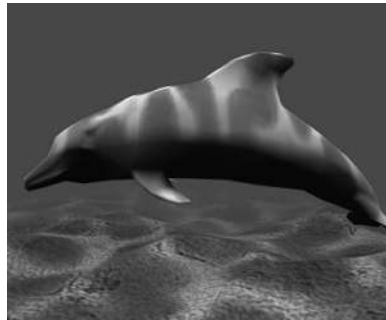


Figure 11.60. Water caustics produced by projecting an animated texture onto the affected surfaces.

11.3.3.5 Subsurface Scattering

When light enters a surface at one point, is scattered beneath the surface, and then reemerges at a different point on the surface, we call this *subsurface scattering*. This phenomenon is responsible for the “warm glow” of human skin, wax and marble statues (e.g., Figure 11.61). Subsurface scattering is described by a more-advanced variant of the BRDF (see Section 11.1.3.2) known as the BSSRDF (*bidirectional surface scattering reflectance distribution function*).

Subsurface scattering can be simulated in a number of ways. Depth-map-based subsurface scattering renders a shadow map (see Section 11.3.3.1), but instead of using it to determine which pixels are in shadow, it is used to measure how far a beam of light would have to travel in order to pass all the



Figure 11.61. On the left, a dragon rendered without subsurface scattering (i.e., using a BRDF lighting model). On the right, the same dragon rendered with subsurface scattering (i.e., using a BSSRDF model). Images rendered by Rui Wang at the University of Virginia.

way through the occluding object. The shadowed side of the object is then given an artificial diffuse lighting term whose intensity is inversely proportional to the distance the light had to travel in order to emerge on the opposite side of the object. This causes objects to appear to be glowing slightly on the side opposite to the light source but only where the object is relatively thin. For more information on subsurface scattering techniques, see http://http.developer.nvidia.com/GPUGems/gpugems_ch16.html.

11.3.3.6 Precomputed Radiance Transfer (PRT)

Precomputed radiance transfer (PRT) is a popular technique that attempts to simulate the effects of radiosity-based rendering methods in real time. It does so by precomputing and storing a complete description of how an incident light ray would interact with a surface (reflect, refract, scatter, etc.) when approaching from every possible direction. At runtime, the response to a particular incident light ray can be looked up and quickly converted into very accurate lighting results.

In general the light's response at a point on the surface is a complex function defined on a hemisphere centered about the point. A compact representation of this function is required to make the PRT technique practical. A common approach is to approximate the function as a linear combination of spherical harmonic basis functions. This is essentially the three-dimensional equivalent of encoding a simple scalar function $f(x)$ as a linear combination of shifted and scaled sine waves.

The details of PRT are far beyond our scope. For more information, see <http://web4.cs.ucl.ac.uk/staff/j.kautz/publications/prtSIG02.pdf>. PRT lighting techniques are demonstrated in a DirectX sample program available in the DirectX SDK—see <http://msdn.microsoft.com/en-us/library/bb147287.aspx> for more details.

11.3.4 Deferred Rendering

In traditional triangle-rasterization-based rendering, all lighting and shading calculations are performed on the triangle fragments in world space, view space or tangent space. The problem with this technique is that it is inherently inefficient. For one thing, we potentially do work that we don't need to do. We shade the vertices of triangles, only to discover during the rasterization stage that the entire triangle is being depth-culled by the z-test. Early z-tests help eliminate unnecessary pixel shader evaluations, but even this isn't perfect. What's more, in order to handle a complex scene with lots of lights, we end up with a proliferation of different versions of our vertex and pixel shaders—versions that handle different numbers of lights, different types of lights, different numbers of skinning weights, etc.

Deferred rendering is an alternative way to shade a scene that addresses many of these problems. In deferred rendering, the majority of the lighting calculations are done in screen space, not view space. We efficiently render the scene without worrying about lighting. During this phase, we store all the information we're going to need to light the pixels in a “deep” frame buffer known as the *G-buffer*. Once the scene has been fully rendered, we use the information in the G-buffer to perform our lighting and shading calculations. This is usually much more efficient than view-space lighting, avoids the proliferation of shader variants and permits some very pleasing effects to be rendered relatively easily.

The G-buffer may be physically implemented as a collection of buffers, but conceptually it is a single frame buffer containing a rich set of information about the lighting and surface properties of the objects in the scene at every pixel on the screen. A typical G-buffer might contain the following per-pixel attributes: depth, surface normal in view space or world space, diffuse color, specular power, even precomputed radiance transfer (PRT) coefficients. The following sequence of screenshots from Guerrilla Games' *Killzone 2* (Figure 11.62) shows some of the typical components of the G-buffer.

An in-depth discussion of deferred rendering is beyond our scope, but the folks at Guerrilla Games have prepared an excellent presentation on the topic, which is available at <http://www.slideshare.net/guerrillagames/deferred-rendering-in-killzone-2-9691589>.

11.3.5 Physically Based Shading

Traditional game lighting engines have required artists and lighters to tweak a wide variety of sometimes non-intuitive parameters, across numerous disparate rendering engine systems, in order to achieve a desired “look” in-game.

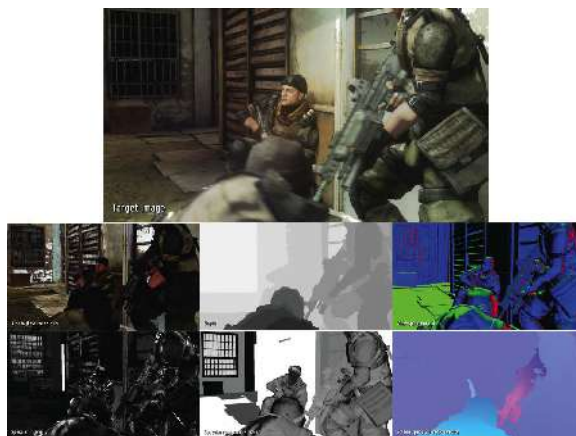


Figure 11.62. Screenshots from *Killzone 2* by Guerrilla Games, showing some of the typical components of the G-buffer used in deferred rendering. The upper image shows the final rendered image. Below it, clockwise from the upper left, are the albedo (diffuse) color, depth, view-space normal, screen-space 2D motion vector (for motion blurring), specular power and specular intensity. (See Color Plate XXII.)

This can be an arduous and time-consuming process. What's worse, parameter settings that work well under one set of lighting conditions might not work well under other lighting scenarios. To address these problems, rendering programmers are turning toward *physically based shading* models.

A physically based shading model attempts to approximate the ways in which light travels and interacts with materials in the real world, allowing artists and lighters to tweak shader parameters using intuitive, real-world quantities measured in real-world units. A complete discussion of physically based shading is beyond the scope of this book, but you can start to learn more about it here: <https://www.marmoset.co/toolbag/learn/pbr-theory>.

11.4 Visual Effects and Overlays

The rendering pipeline we've discussed to this point is responsible primarily for rendering three-dimensional solid objects. A number of specialized rendering systems are typically layered on top of this pipeline, responsible for rendering visual elements like particle effects, decals (small geometry overlays that represent bullet holes, cracks, scratches and other surface details), hair and fur, rain or falling snow, water and other specialized visual effects. Full-screen post effects may be applied, including vignette (a reduction of brightness and saturation around the edges of the screen), motion blur, depth

of field blurring, artificial/enhanced colorization, and the list goes on. Finally, the game's menu system and heads-up display (HUD) are typically realized by rendering text and other two- or three-dimensional graphics in screen space overlaid on top of the three-dimensional scene.

An in-depth coverage of these engine systems is beyond our scope. In the following sections, we'll provide a brief overview of these rendering systems, and point you in the direction of additional information.

11.4.1 Particle Effects

A particle rendering system is concerned with rendering amorphous objects like clouds of smoke, sparks, flame and so on. These are called *particle effects*. The key features that differentiate a particle effect from other kinds of renderable geometry are as follows:

- It is composed of a very *large number of relatively simple* pieces of geometry—most often simple cards called *quads*, composed of two triangles each.
- The geometry is often *camera-facing* (i.e., billboarded), meaning that the engine must take steps to ensure that the face normals of each quad always point directly at the camera's focal point.
- Its materials are almost always *semitransparent* or *translucent*. As such, particle effects have some stringent *rendering order* constraints that do not apply to the majority of opaque objects in a scene.
- Particles *animate* in a rich variety of ways. Their positions, orientations, sizes (scales), texture coordinates and many of their shader parameters vary from frame to frame. These changes are defined either by hand-authored animation curves or via procedural methods.
- Particles are typically *spawned and killed* continually. A particle emitter is a logical entity in the world that creates particles at some user-specified rate; particles are killed when they hit a predefined death plane, or when they have lived for a user-defined length of time, or as decided by some other user-specified criteria.

Particle effects could be rendered using regular triangle mesh geometry with appropriate shaders. However, because of the unique characteristics listed above, a specialized particle effect animation and rendering system is always used to implement them in a real production game engine. A few example particle effects are shown in Figure 11.63.



Figure 11.63. Flame, smoke and bullet tracer particle effects in *Uncharted 3: Drake's Deception* (© 2011/™ SIE. Created and developed by Naughty Dog, PlayStation 3). (See Color Plate XXIII.)

Particle system design and implementation is a rich topic that could occupy many chapters all on its own. For more information on particle systems, see [2, Section 10.7], [16, Section 20.5], [11, Section 13.7] and [12, Section 4.1.2].

11.4.2 Decals

A *decal* is a relatively small piece of geometry that is overlaid on top of the regular geometry in the scene, allowing the visual appearance of the surface to be modified dynamically. Examples include bullet holes, foot prints, scratches, cracks, etc.

The approach most often used by modern engines is to model a decal as a rectangular area that is to be projected along a ray into the scene. This gives rise to a rectangular prism in 3D space. Whatever surface the prism intersects first becomes the surface of the decal. The triangles of the intersected geometry are extracted and clipped against the four bounding planes of the decal's projected prism. The resulting triangles are texture-mapped with a desired decal texture by generating appropriate texture coordinates for each vertex. These texture-mapped triangles are then rendered over the top of the regular scene, often using parallax mapping to give them the illusion of depth and with a slight z-bias (usually implemented by shifting the near plane slightly) so they don't experience z-fighting with the geometry on which they are overlaid. The result is the appearance of a bullet hole, scratch or other kind of surface modification. Some bullet-hole decals are depicted in Figure 11.64.

For more information on creating and rendering decals, see [9, Section 4.8] and [32, Section 9.2].

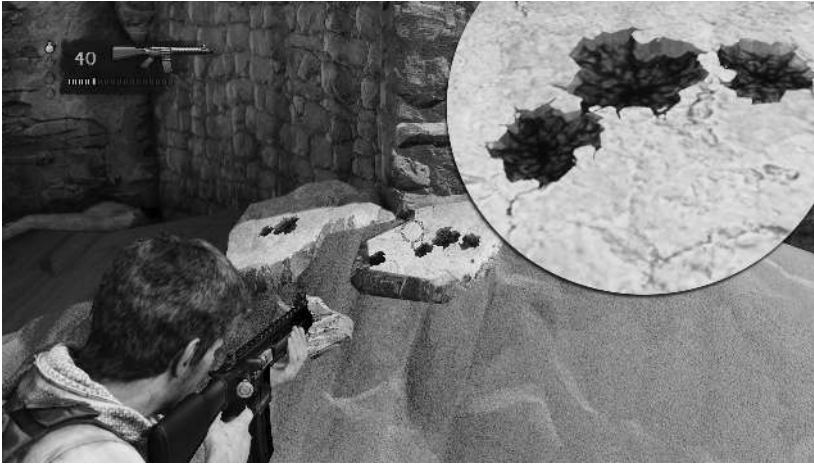


Figure 11.64. Parallax-mapped decals from *Uncharted 3: Drake's Deception* (© 2011/™ SIE. Created and developed by Naughty Dog, PlayStation 3). (See Color Plate XXIV.)

11.4.3 Environmental Effects

Any game that takes place in a somewhat natural or realistic environment requires some kind of environmental rendering effects. These effects are usually implemented via specialized rendering systems. We'll take a brief look at a few of the more common of these systems in the following sections.

11.4.3.1 Skies

The sky in a game world needs to contain vivid detail, yet technically speaking it lies an extremely long distance away from the camera. Therefore, we cannot model it as it really is and must turn instead to various specialized rendering techniques.

One simple approach is to fill the frame buffer with the sky texture prior to rendering any 3D geometry. The sky texture should be rendered at an approximate 1:1 texel-to-pixel ratio, so that the texture is roughly or exactly the resolution of the screen. The sky texture can be rotated and scrolled to correspond to the motions of the camera in-game. During rendering of the sky, we make sure to set the depth of all pixels in the frame buffer to the maximum possible depth value. This ensures that the 3D scene elements will always sort on top of the sky. The arcade hit *Hydro Thunder* rendered its skies in exactly this manner.

On modern game platforms, where pixel shading costs can be high, sky rendering is often done *after* the rest of the scene has been rendered. First the z-buffer is cleared to the maximum z-value. Then the scene is rendered.

Finally the sky is rendered, with z-testing enabled, z writing turned off, and using a z-test value that is one less than the maximum. This causes the sky to be drawn only where it is not occluded by closer objects like terrain, buildings and trees. Drawing the sky last ensures that its pixel shader is run for the minimum possible number of screen pixels.

For games in which the player can look in any direction, we can use a *sky dome* or *sky box*. The dome or box is rendered with its center always at the camera's current location, so that it appears to lie at infinity, no matter where the camera moves in the game world. As with the sky texture approach, the sky box or dome is rendered before any other 3D geometry, and all of the pixels in the frame buffer are set to the maximum z-value when the sky is rendered. This means that the dome or box can actually be tiny, relative to other objects in the scene. Its size is irrelevant, as long as it fills the entire frame buffer when it is drawn. For more information on sky rendering, see [2, Section 10.3] and [44, page 253].

Clouds are often implemented with a specialized rendering and animation system as well. In early games like *Doom* and *Quake*, the clouds were just planes with scrolling semitransparent cloud textures on them. More-recent cloud techniques include camera-facing cards (billboards), particle-effect based clouds and volumetric cloud effects.

II.4.3.2 Terrain

The goal of a terrain system is to model the surface of the earth and provide a canvas of sorts upon which other static and dynamic elements can be laid out. Terrain is sometimes modeled explicitly in a package like Maya. But if the player can see far into the distance, we usually want some kind of dynamic tessellation or other level of detail (LOD) system. We may also need to limit the amount of data required to represent very large outdoor areas.

Height field terrain is one popular choice for modeling large terrain areas. The data size can be kept relatively small because a height field is typically stored in a grayscale texture map. In most height-field-based terrain systems, the horizontal ($y = 0$) plane is tessellated in a regular grid pattern, and the heights of the terrain vertices are determined by sampling the height field texture. The number of triangles per unit area can be varied based on distance from the camera, thereby allowing large-scale features to be seen in the distance, while still permitting a good deal of detail to be represented for nearby terrain. An example of a terrain defined via a height field bitmap is shown in Figure 11.65.

Terrain systems usually provide specialized tools for “painting” the height field itself, carving out terrain features like roads, rivers and so on. Texture

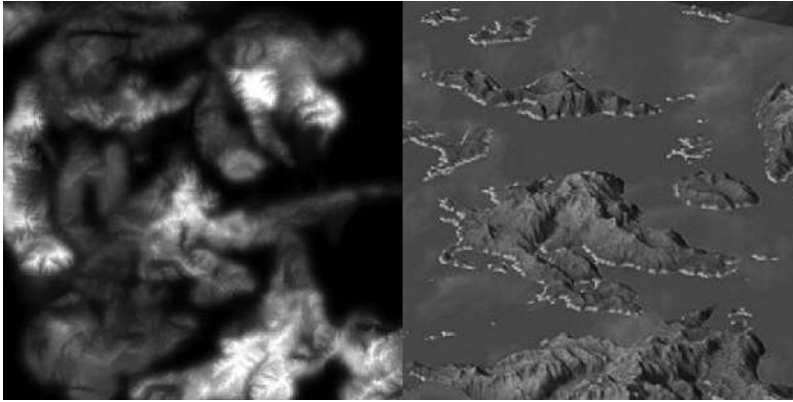


Figure 11.65. A grayscale height field bitmap (left) can be used to control the vertical positions of the vertices in a terrain grid mesh (right). In this example, a water plane intersects the terrain mesh to create islands.

mapping in a terrain system is often a blend between four or more textures. This allows artists to “paint” in grass, dirt, gravel and other terrain features by simply exposing one of the texture layers. The layers can be cross-blended from one to another to provide smooth textural transitions. Some terrain tools also permit sections of the terrain to be cut out to permit buildings, trenches and other specialized terrain features to be inserted in the form of regular mesh geometry. Terrain authoring tools are sometimes integrated directly into the game world editor, while in other engines they may be stand-alone tools.

Of course, height field terrain is just one of many options for modeling the surface of the Earth in a game. For more information on terrain rendering, see [8, Sections 4.16 through 4.19] and [9, Section 4.2].

11.4.3.3 Water

Water renderers are commonplace in games nowadays. There are lots of different kinds of water, including oceans, pools, rivers, waterfalls, fountains, jets, puddles and damp solid surfaces. Each type of water generally requires some specialized rendering technology. Some also require dynamic motion simulations. Large bodies of water may require dynamic tessellation or other LOD methodologies similar to those employed in a terrain system.

Water systems sometimes interact with a game’s rigid body dynamics system (flotation, force from water jets, etc.) and with gameplay (slippery surfaces, swimming mechanics, diving mechanics, riding vertical jets of water and so on). Water effects are often created by combining disparate rendering technologies and subsystems. For example, a waterfall might make use

of specialized water shaders, scrolling textures, particle effects for mist at the base, a decal-like overlay for foam, and the list goes on. Today's games offer some pretty amazing water effects, and active research into technologies like real-time fluid dynamics promises to make water simulations even richer and more realistic in the years ahead. For more information on water rendering and simulation techniques, see [2, Sections 9.3, 9.5 and 9.6], [15] and [8, Sections 2.6 and 5.11].

II.4.4 Overlays

Most games have heads-up displays, in-game graphical user interfaces and menu systems. These *overlays* are typically comprised of two- and three-dimensional graphics rendered directly in view space or screen space.

Overlays are generally rendered after the primary scene, with z-testing disabled to ensure that they appear on top of the three-dimensional scene. Two-dimensional overlays are typically implemented by rendering quads (triangle pairs) in screen space using an orthographic projection. Three-dimensional overlays may be rendered using an orthographic projection or via the regular perspective projection with the geometry positioned in view space so that it follows the camera around.

II.4.4.1 Text and Fonts

A game engine's text/font system is typically implemented as a special kind of two-dimensional (or sometimes three-dimensional) overlay. At its core, a text rendering system needs to be capable of displaying a sequence of character glyphs corresponding to a text string, arranged in various orientations on the screen.

A font is often implemented via a texture map known as a *glyph atlas*, which contains the various required glyphs. This texture typically consists of a single alpha channel—the value at each pixel representing the percentage of that pixel that is covered by the interior of a glyph. A font description file provides information such as the bounding boxes of each glyph within the texture, and font layout information such as kerning, baseline offsets and so on. A glyph is rendered by drawing a quad whose (u, v) coordinates correspond to the bounding box of the desired glyph within the atlas texture map. The texture map provides the alpha value, while the color is specified separately, allowing glyphs of any color to be rendered from the same atlas.

Another option for font rendering is to make use of a font library like FreeType (<https://www.freetype.org/>). The FreeType library enables a game or other application to read fonts in a wide variety of formats, including

TrueType (TTF) and OpenType (OTF), and to render glyphs into in-memory pixmaps at any desired point size. FreeType renders each glyph using its Bezier curve outlines, so it produces very accurate results.

Typically a real-time application like a game will use FreeType to prerender the necessary glyphs into an atlas, which is in turn used as a texture map to render glyphs as simple quads every frame. However, by embedding FreeType or a similar library in your engine, it's possible to render some glyphs into the atlas on the fly, on an as-needed basis. This can be useful when rendering text in a language with a very large number of possible glyphs, like Chinese or Korean.

Yet another way to render high-quality character glyphs is to use *signed distance fields* to describe the glyphs. In this approach, glyphs are rendered to pixmaps (as they would be with a library like FreeType), but the value at each pixel is no longer an alpha "coverage" value. Instead, each pixel contains a signed distance from that pixel center to the nearest edge of the glyph. Inside the glyph, the distances are negative; outside the glyph's outlines, they are positive. When rendering a glyph from a signed distance field texture atlas, the distances are used in the pixel shader to calculate highly accurate alpha values. The net result is text that looks smooth at any distance or viewing angle. You can read more about signed distance field text rendering by searching online for Konstantin Käfer's article entitled "Drawing Text with Signed Distance Fields in Mapbox GL," or the article written by Chris Green of Valve entitled "Improved Alpha-Tested Magnification for Vector Textures and Special Effects."

Glyphs can also be rendered directly from the Bézier curve outlines that define them. The Slug font rendering library by Terathon Software LLC performs its outline-based glyph rendering on the GPU, thereby making this technique practical for use in a real-time game application.

A good text/font system must account for the differences in character sets and reading directions inherent in various languages. Laying out the characters in a text string is a process known as *shaping* the string. The characters are laid out from left to right or right to left, depending on the language, with each character aligned to a common *baseline*. The spacing between characters is determined in part by metrics provided by the creator of the font (and stored in the font file), and partly by *Kerning* rules that dictate contextual intercharacter spacing adjustments.

Some text systems also provide various fun features like the ability to animate characters across the screen in various ways, the ability to animate individual characters and so on. However, it's important to remember when implementing a game font system that only those features that are *actually re-*

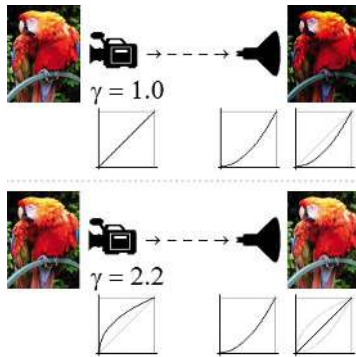


Figure 11.66. The effect of a CRT's gamma response on image quality and how the effect can be corrected for. Image courtesy of www.wikipedia.org.

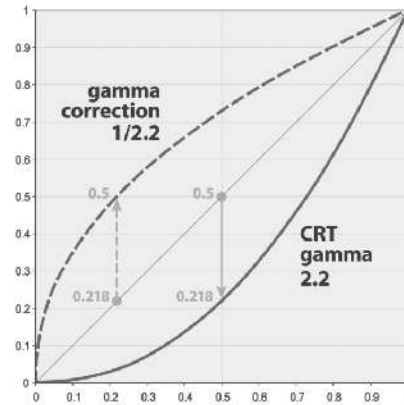


Figure 11.67. Gamma encoding and decoding curves. Image courtesy of www.wikipedia.org.

quired by the game should be implemented. There's no point in furnishing your engine with an advanced text animation if your game never needs to display animated text, for example.

11.4.5 Gamma Correction

CRT monitors tend to have a nonlinear response to luminance values. That is, if a linearly increasing ramp of R, G or B values were to be sent to a CRT, the image that would result on-screen would be perceptually nonlinear to the human eye. Visually, the dark regions of the image would look darker than they should. This is illustrated in Figure 11.66.

The gamma response curve of a typical CRT display can be modeled quite simply by the formula

$$V_{\text{out}} = V_{\text{in}}^{\gamma}$$

where $\gamma_{\text{CRT}} > 1$. To correct for this effect, the colors sent to the CRT display are usually passed through an inverse transformation (i.e., using a gamma value $\gamma_{\text{corr}} < 1$). The value of γ_{CRT} for a typical CRT monitor is 2.2, so the correction value is usually $\gamma_{\text{corr}} \approx 1/2.2 = 0.455$. These gamma encoding and decoding curves are shown in Figure 11.67.

Gamma encoding can be performed by the 3D rendering engine to ensure that the values in the final image are properly gamma-corrected. One problem that is encountered, however, is that the bitmap images used to represent texture maps are often gamma-corrected themselves. A high-quality rendering engine takes this fact into account, by gamma-decoding the textures prior to

rendering and then re-encoding the gamma of the final rendered scene so that its colors can be reproduced properly on-screen.

11.4.6 Full-Screen Post Effects

Full-screen *post effects* are effects applied to a rendered three-dimensional scene that provide additional realism or a stylized look. These effects are often implemented by passing the entire contents of the screen through a pixel shader that applies the desired effect(s). This can be accomplished by rendering a full-screen quad that has been mapped with a texture containing the unfiltered scene. A few examples of full-screen post effects are given below:

- *Motion blur*. This is typically implemented by rendering a buffer of screen-space velocity vectors and using this vector field to selectively blur the rendered image. Blurring is accomplished by passing a *convolution kernel* over the image (see “Image Smoothing and Sharpening by Discrete Convolution” by Dale A. Schumacher, published in [5], for details).
- *Depth of field blur*. This blur effect can be produced by using the contents of the depth buffer to adjust the degree of blur applied at each pixel.
- *Vignette*. In this filmic effect, the brightness or saturation of the image is reduced at the corners of the screen for dramatic effect. It is sometimes implemented by literally rendering a texture overlay on top of the screen. A variation on this effect is used to produce the classic circular effect used to indicate that the player is looking through a pair of binoculars or a weapon scope.
- *Colorization*. The colors of screen pixels can be altered in arbitrary ways as a post-processing effect. For example, all colors except red could be desaturated to grey to produce a striking effect similar to the famous scene of the little girl in the red coat from *Schindler’s List*.

11.5 Further Reading

We’ve covered a lot of material in a very short space in this chapter, but we’ve only just scratched the surface. No doubt you’ll want to explore many of these topics in much greater detail. For an excellent overview of the entire process of creating three-dimensional computer graphics and animation for games and film, I highly recommend [27]. The technology that underlies modern real-time rendering is covered in excellent depth in [2], while [16] is well known as the definitive reference guide to all things related to computer graphics.

Other great books on 3D rendering include [49], [11] and [12]. The mathematics of 3D rendering is covered very well in [32]. No graphics programmer's library would be complete without one or more books from the *Graphics Gems* series ([20], [5], [28], [22] and [42]) and/or the *GPU Gems* series ([15], [44] and [40]). Of course, this short reference list is only the beginning—you will undoubtedly encounter a great many more excellent books on rendering and shaders over the course of your career as a game programmer.

12

Animation Systems

The majority of modern 3D games revolve around *characters*—often human or humanoid, sometimes animal or alien. Characters are unique because they need to move in a fluid, organic way. This poses a host of new technical challenges, over and above what is required to simulate and animate rigid objects like vehicles, projectiles, soccer balls and Tetris pieces. The task of imbuing characters with natural-looking motion is handled by an engine component known as the *character animation system*.

As we'll see, an animation system gives game designers a powerful suite of tools that can be applied to non-characters as well as characters. Any game object that is not 100% rigid can take advantage of the animation system. So whenever you see a vehicle with moving parts, a piece of articulated machinery, trees waving gently in the breeze or even an exploding building in a game, chances are good that the object makes at least partial use of the game engine's animation system.

12.1 Types of Character Animation

Character animation technology has come a long way since *Donkey Kong*. At first, games employed very simple techniques to provide the illusion of lifelike movement. As game hardware improved, more-advanced techniques became

feasible in real time. Today, game designers have a host of powerful animation methods at their disposal. In this section, we'll take a brief look at the evolution of character animation and outline the three most-common techniques used in modern game engines.

12.1.1 Cel Animation

The precursor to all game animation techniques is known as *traditional animation*, or *hand-drawn animation*. This is the technique used in the earliest animated cartoons. The illusion of motion is produced by displaying a sequence of still pictures known as *frames* in rapid succession. Real-time 3D rendering can be thought of as an electronic form of traditional animation, in that a sequence of still full-screen images is presented to the viewer over and over to produce the illusion of motion.

Cel animation is a specific type of traditional animation. A *cel* is a transparent sheet of plastic on which images can be painted or drawn. An animated sequence of cels can be placed on top of a fixed background painting or drawing to produce the illusion of motion without having to redraw the static background over and over.

The electronic equivalent to cel animation is a technology known as *sprite animation*. A *sprite* is a small bitmap that can be overlaid on top of a full-screen background image without disrupting it, often drawn with the aid of specialized graphics hardware. Hence, a sprite is to 2D game animation what a cel was to traditional animation. This technique was a staple during the 2D game era. Figure 12.1 shows the famous sequence of sprite bitmaps that were used to produce the illusion of a running humanoid character in almost every Mattel Intellivision game ever made. The sequence of frames was designed so that it animates smoothly even when it is repeated indefinitely—this is known as a *looping animation*. This particular animation would be called a *run cycle* in modern parlance, because it makes the character appear to be running. Characters typically have a number of looping animation cycles, including various idle cycles, a walk cycle and a run cycle.



Figure 12.1. The sequence of sprite bitmaps used in most Intellivision games.

12.1.2 Rigid Hierarchical Animation

Early 3D games like *Doom* continued to make use of a sprite-like animation system: Its monsters were nothing more than camera-facing quads, each of which displayed a sequence of texture bitmaps (known as an *animated texture*) to produce the illusion of motion. And this technique is still used today for low-resolution and/or distant objects—for example crowds in a stadium, or hordes of soldiers fighting a distant battle in the background. But for high-quality foreground characters, 3D graphics brought with it the need for improved character animation methods.

The earliest approach to 3D character animation is a technique known as *rigid hierarchical animation*. In this approach, a character is modeled as a collection of rigid pieces. A typical breakdown for a humanoid character might be pelvis, torso, upper arms, lower arms, upper legs, lower legs, hands, feet and head. The rigid pieces are constrained to one another in a hierarchical fashion, analogous to the manner in which a mammal's bones are connected at the joints. This allows the character to move naturally. For example, when the upper arm is moved, the lower arm and hand will automatically follow it. A typical hierarchy has the pelvis at the root, with the torso and upper legs as its immediate children and so on as shown below:

```

Pelvis
  Torso
    UpperRightArm
      LowerRightArm
        RightHand
    UpperLeftArm
      UpperLeftArm
        LeftHand
  Head
  UpperRightLeg
    LowerRightLeg
      RightFoot
  UpperLeftLeg
    UpperLeftLeg
      LeftFoot

```

The big problem with the rigid hierarchy technique is that the behavior of the character's body is often not very pleasing due to "cracking" at the joints. This is illustrated in Figure 12.2. Rigid hierarchical animation works well for robots and machinery that really are constructed of rigid parts, but it breaks down under scrutiny when applied to "fleshy" characters.

12.1.3 Per-Vertex Animation and Morph Targets

Rigid hierarchical animation tends to look unnatural because it is rigid. What we really want is a way to move individual vertices so that triangles can stretch to produce more natural-looking motion.

One way to achieve this is to apply a brute-force technique known as *per-vertex animation*. In this approach, the vertices of the mesh are animated by an artist, and motion data is exported, which tells the game engine how to move each vertex at runtime. This technique can produce any mesh deformation imaginable (limited only by the tessellation of the surface). However, it is a data-intensive technique, since time-varying motion information must be stored for each vertex of the mesh. For this reason, it has little application to real-time games.

A variation on this technique known as *morph target animation* is used in some real-time games. In this approach, the vertices of a mesh are moved by an animator to create a relatively small set of fixed, extreme poses. Animations are produced by *blending* between two or more of these fixed poses at runtime. The position of each vertex is calculated using a simple linear interpolation (LERP) between the vertex's positions in each of the extreme poses.

The morph target technique is often used for facial animation, because the human face is an extremely complex piece of anatomy, driven by roughly 50 muscles. Morph target animation gives an animator full control over every vertex of a facial mesh, allowing him or her to produce both subtle and extreme movements that approximate the musculature of the face well. Figure 12.3 shows a set of facial morph targets.

As computing power continues to increase, some studios are using jointed facial rigs containing hundreds of joints as an alternative to morph targets. Other studios combine the two techniques, using jointed rigs to achieve the primary pose of the face and then applying small tweaks via morph targets.

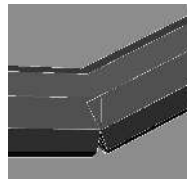


Figure 12.2. Cracking at the joints is a big problem in rigid hierarchical animation.

12.1.4 Skinned Animation

As the capabilities of game hardware improved further, an animation technology known as *skinned animation* was developed. This technique has many of the benefits of per-vertex and morph target animation—permitting the triangles of an animated mesh to deform. But it also enjoys the much more efficient performance and memory usage characteristics of rigid hierarchical animation. It is capable of producing reasonably realistic approximations to the movement of skin and clothing.

Skinned animation was first used by games like *Super Mario 64*, and it is still the most prevalent technique in use today, both by the game industry and the feature film industry. A host of famous modern game and movie characters, including the dinosaurs from *Jurrassic Park*, Solid Snake (*Metal Gear Solid 4*), Gollum (*Lord of the Rings*), Nathan Drake (*Uncharted*), Buzz Lightyear (*Toy Story*), Marcus Fenix (*Gears of War*) and Joel (*The Last of Us*) were all animated, in whole or in part, using skinned animation techniques. The remainder of this chapter will be devoted primarily to the study of skinned/skeletal animation.

In skinned animation, a *skeleton* is constructed from rigid “bones,” just as in rigid hierarchical animation. However, instead of rendering the rigid pieces on-screen, they remain hidden. A smooth continuous triangle mesh called a *skin* is bound to the joints of the skeleton; its vertices track the movements of the joints. Each vertex of the skin mesh can be weighted to multiple joints, so the skin can stretch in a natural way as the joints move.

In Figure 12.4, we see Crank the Weasel, a game character designed by Eric Browning for Midway Home Entertainment in 2001. Crank’s outer skin is composed of a mesh of triangles, just like any other 3D model. However, inside him we can see the rigid bones and joints that make his skin move.



Figure 12.3. A set of facial morph targets for the Ellie character in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

12.1.5 Animation Methods as Data Compression Techniques

The most flexible animation system conceivable would give the animator control over literally every infinitesimal point on an object's surface. Of course, animating like this would result in an animation that contains a potentially infinite amount of data! Animating the vertices of a triangle mesh is a simplification of this ideal—in effect, we are *compressing* the amount of information needed to describe an animation by restricting ourselves to moving only the vertices. (Animating a set of control points is the analog of vertex animation for models constructed out of higher-order patches.) Morph targets can be thought of as an additional level of compression, achieved by imposing additional constraints on the system—vertices are constrained to move only along linear paths between a fixed number of predefined vertex positions. Skeletal animation is just another way to compress vertex animation data by imposing constraints. In this case, the motions of a relatively large number of vertices are constrained to follow the motions of a relatively small number of skeletal joints.

When considering the trade-offs between various animation techniques, it can be helpful to think of them as compression methods, analogous in many respects to video compression techniques. We should generally aim to select the animation method that provides the best compression without producing unacceptable visual artifacts. Skeletal animation provides the best compression when the motion of a single joint is magnified into the motions of many vertices. A character's limbs act like rigid bodies for the most part, so they can

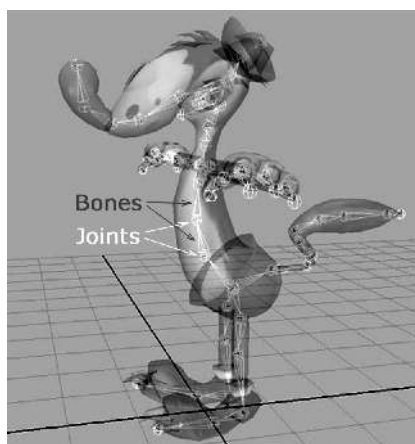


Figure 12.4. Eric Browning's Crank the Weasel character, with internal skeletal structure.

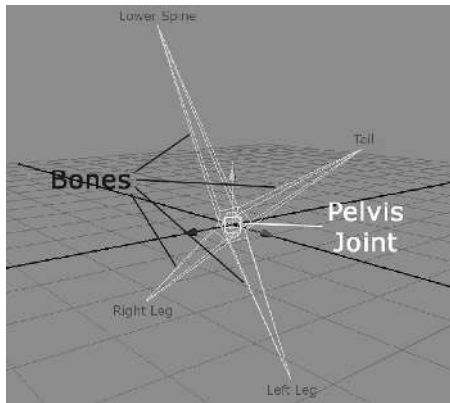


Figure 12.5. The pelvis joint of this character connects to four other joints (tail, spine and two legs), and so it produces four bones.

be moved very efficiently with a skeleton. However, the motion of a face tends to be much more complex, with the motions of individual vertices being more independent. To convincingly animate a face using the skeletal approach, the required number of joints approaches the number of vertices in the mesh, thus diminishing its effectiveness as a compression technique. This is one reason why morph target techniques are often favored over the skeletal approach for facial animation. (Another common reason is that morph targets tend to be a more natural way for animators to work.)

12.2 Skeletons

A skeleton is comprised of a *hierarchy* of rigid pieces known as *joints*. In the game industry, we often use the terms “joint” and “bone” interchangeably, but the term *bone* is actually a misnomer. Technically speaking, the joints are the objects that are directly manipulated by the animator, while the bones are simply the empty spaces between the joints. As an example, consider the pelvis joint in the Crank the Weasel character model. It is a single joint, but because it connects to four other joints (the tail, the spine and the left and right hip joints), this one joint appears to have four bones sticking out of it. This is shown in more detail in Figure 12.5. Game engines don’t care a whip about bones—only the joints matter. So whenever you hear the term “bone” being used in the industry, remember that 99% of the time we are actually speaking about joints.

12.2.1 The Skeleal Hierarchy

As we've mentioned, the joints in a skeleton form a hierarchy or tree structure. One joint is selected as the root, and all other joints are its children, grandchildren and so on. A typical joint hierarchy for skinned animation looks almost identical to a typical rigid hierarchy. For example, a humanoid character's joint hierarchy might look something like the one depicted in Figure 12.6.

We usually assign each joint an index from 0 to $N - 1$. Because each joint has one and only one parent, the hierarchical structure of a skeleton can be fully described by storing the index of its parent with each joint. The root joint has no parent, so its parent index is usually set to an invalid value such as -1 .

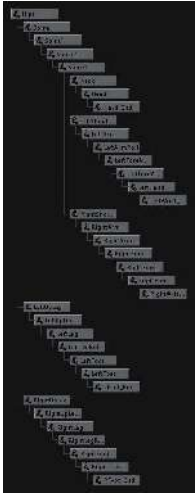


Figure 12.6. Example of a skeletal hierarchy, as it would appear in Maya's Hypergraph Hierarchy view.

12.2.2 Representing a Skeleton in Memory

A skeleton is usually represented by a small top-level data structure that contains an array of data structures for the individual joints. The joints are usually listed in an order that ensures a child joint will always appear after its parent in the array. This implies that joint zero is always the root of the skeleton.

Joint indices are usually used to refer to joints within animation data structures. For example, a child joint typically refers to its parent joint by specifying its index. Likewise, in a skinned triangle mesh, a vertex refers to the joint or joints to which it is bound by index. This is much more efficient than referring to joints by name, both in terms of the amount of storage required (a joint index can be 8 bits wide, as long as we are willing to accept a maximum of 256 joints per skeleton) and in terms of the amount of time it takes to look up a referenced joint (we can use the joint index to jump immediately to a desired joint in the array).

Each joint data structure typically contains the following information:

- The *name* of the joint, either as a string or a hashed 32-bit string id.
- The *index* of the joint's *parent* within the skeleton.
- The *inverse bind pose transform* of the joint. The bind pose of a joint is the position, orientation and scale of that joint at the time it was bound to the vertices of the skin mesh. We usually store the *inverse* of this transformation for reasons we'll explore in more depth in the following sections.

A typical skeleton data structure might look something like this:

```
struct Joint
{
    Matrix4x3    m_invBindPose; // inverse bind pose
```

```

        // transform
        const char* m_name;           // human-readable joint
        // name
        U8          m_iParent;        // parent index or 0xFF
        // if root
    };

    struct Skeleton
    {
        U32          m_jointCount;     // number of joints
        Joint*      m_aJoint;         // array of joints
    };

```

12.3 Poses

No matter what technique is used to produce an animation, be it cel-based, rigid hierarchical or skinned/skeletal, every animation takes place over time. A character is imbued with the illusion of motion by arranging the character's body into a sequence of discrete, still *poses* and then displaying those poses in rapid succession, usually at a rate of 30 or 60 *poses per second*. (Actually, as we'll see in Section 12.4.1.1, we often *interpolate* between adjacent poses rather than displaying a single pose verbatim.) In skeletal animation, the pose of the skeleton directly controls the vertices of the mesh, and posing is the animator's primary tool for breathing life into her characters. So clearly, before we can animate a skeleton, we must first understand how to *pose* it.

A skeleton is posed by rotating, translating and possibly scaling its joints in arbitrary ways. The *pose* of a joint is defined as the joint's position, orientation and scale, relative to some frame of reference. A joint pose is usually represented by a 4×4 or 4×3 matrix, or by an SRT data structure (scale, quaternion rotation and vector translation). The pose of a skeleton is just the set of all of its joints' poses and is normally represented as a simple array of matrices or SRTs.

12.3.1 Bind Pose

Two different poses of the same skeleton are shown in Figure 12.7. The pose on the left is a special pose known as the *bind pose*, also sometimes called the *reference pose* or the *rest pose*. This is the pose of the 3D mesh prior to being bound to the skeleton (hence the name). In other words, it is the pose that the mesh would assume if it were rendered as a regular, unskinned triangle mesh, without any skeleton at all. The bind pose is also called the *T-pose* because

the character is usually standing with his feet slightly apart and his arms outstretched in the shape of the letter T. This particular stance is chosen because it keeps the limbs away from the body and each other, making the process of binding the vertices to the joints easier.

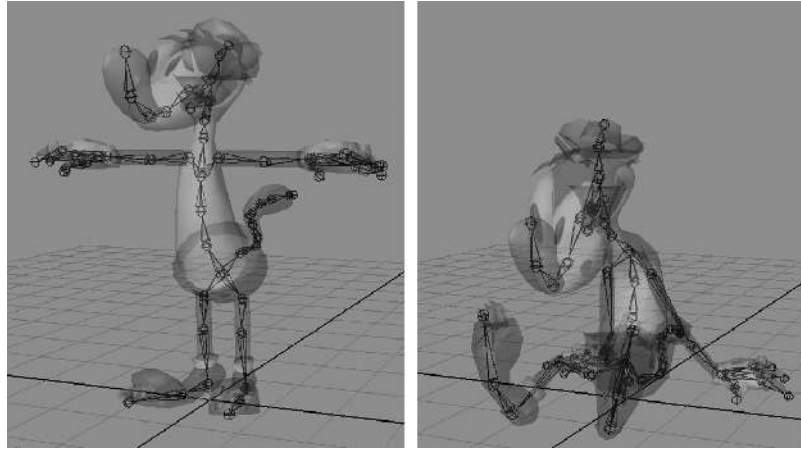


Figure 12.7. Two different poses of the same skeleton. The pose on the left is the special pose known as *bind pose*.

12.3.2 Local Poses

A joint's pose is most often specified relative to its *parent* joint. A parent-relative pose allows a joint to move naturally. For example, if we rotate the shoulder joint, but leave the parent-relative poses of the elbow, wrist and fingers unchanged, the entire arm will rotate about the shoulder in a rigid manner, as we'd expect. We sometimes use the term *local pose* to describe a parent-relative pose. Local poses are almost always stored in SRT format, for reasons we'll explore when we discuss animation blending.

Graphically, many 3D authoring packages like Maya represent joints as small spheres. However, a joint has a rotation and a scale, not just a translation, so this visualization can be a bit misleading. In fact, a joint actually defines a coordinate space no different in principle from the other spaces we've encountered (like model space, world space or view space). So it is best to picture a joint as a set of Cartesian coordinate axes. Maya gives the user the option of displaying a joint's local coordinate axes—this is shown in Figure 12.8.

Mathematically, a joint pose is nothing more than an *affine transformation*. The pose of joint j can be written as the 4×4 affine transformation matrix $\mathbf{P}_j$, which is comprised of a translation vector $\mathbf{T}_j$, a 3×3 diagonal scale matrix

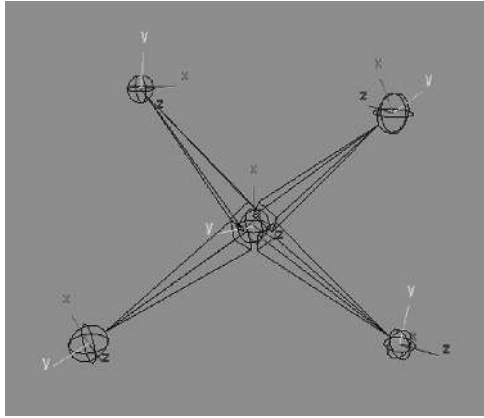


Figure 12.8. Every joint in a skeletal hierarchy defines a set of local coordinate space axes, known as joint space.

$\mathbf{S}_j$ and a 3×3 rotation matrix $\mathbf{R}_j$. The pose of an entire skeleton $\mathbf{P}^{\text{skel}}$ can be written as the set of all poses $\mathbf{P}_j$, where j ranges from 0 to $N - 1$:

$$\mathbf{P}_j = \begin{bmatrix} \mathbf{S}_j \mathbf{R}_j & 0 \\ \mathbf{T}_j & 1 \end{bmatrix},$$

$$\mathbf{P}^{\text{skel}} = \left\{ \mathbf{P}_j \right\}_{j=0}^{N-1}.$$

12.3.2.1 Joint Scale

Some game engines assume that joints will never be scaled, in which case $\mathbf{S}_j$ is simply omitted and assumed to be the identity matrix. Other engines make the assumption that scale will be *uniform* if present, meaning it is the same in all three dimensions. In this case, scale can be represented using a single scalar value s_j . Some engines even permit *nonuniform* scale, in which case scale can be compactly represented by the three-element vector $\mathbf{s}_j = [s_{jx} \ s_{jy} \ s_{jz}]$. The elements of the vector $\mathbf{s}_j$ correspond to the three diagonal elements of the 3×3 scaling matrix $\mathbf{S}_j$, so it is not really a vector *per se*. Game engines almost never permit shear, so $\mathbf{S}_j$ is almost never represented by a full 3×3 scale/shear matrix, although it certainly *could* be.

There are a number of benefits to omitting or constraining scale in a pose or animation. Clearly using a lower-dimensional scale representation can save memory. (Uniform scale requires a single floating-point scalar per joint per animation frame, while nonuniform scale requires three floats, and a full 3×3 scale-shear matrix requires nine.) Restricting our engine to uniform scale has the added benefit of ensuring that the bounding sphere of a joint will never

be transformed into an ellipsoid, as it could be when scaled in a nonuniform manner. This greatly simplifies the mathematics of frustum and collision tests in engines that perform such tests on a per-joint basis.

12.3.2.2 Representing a Joint Pose in Memory

As we mentioned above, joint poses are usually stored in SRT format. In C++, such a data structure might look like this, where *Q* is first to ensure proper alignment and optimal structure packing. (Can you see why?)

```
struct JointPose
{
    Quaternion m_rot;    // R
    Vector3    m_trans;  // T
    F32        m_scale;  // S (uniform scale only)
};
```

If nonuniform scale is permitted, we might define a joint pose like this instead:

```
struct JointPose
{
    Quaternion m_rot;    // R
    Vector4    m_trans;  // T
    Vector4    m_scale;  // S
};
```

The local pose of an entire skeleton can be represented as follows, where it is understood that the array *m_aLocalPose* is dynamically allocated to contain just enough occurrences of *JointPose* to match the number of joints in the skeleton.

```
struct SkeletonPose
{
    Skeleton* m_pSkeleton; // skeleton + num joints
    JointPose* m_aLocalPose; // local joint poses
};
```

12.3.2.3 The Joint Pose as a Change of Basis

It's important to remember that a *local* joint pose is specified relative to the joint's immediate parent. Any affine transformation can be thought of as transforming points and vectors from one coordinate space to another. So when the joint pose transform P_j is applied to a point or vector that is expressed in the coordinate system of the joint *j*, the result is that same point or vector expressed in the space of the parent joint.

As we’ve done in earlier chapters, we’ll adopt the convention of using subscripts to denote the direction of a transformation. Since a joint pose takes points and vectors from the *child* joint’s space (C) to that of its *parent* joint (P), we can write it $(\mathbf{P}_{C \rightarrow P})_j$. Alternatively, we can introduce the function $p(j)$, which returns the parent index of joint j , and write the local pose of joint j as $\mathbf{P}_{j \rightarrow p(j)}$.

On occasion we will need to transform points and vectors in the opposite direction—from *parent space* into the space of the *child* joint. This transformation is just the inverse of the local joint pose. Mathematically, $\mathbf{P}_{p(j) \rightarrow j} = (\mathbf{P}_{j \rightarrow p(j)})^{-1}$.

12.3.3 Global Poses

Sometimes it is convenient to express a joint’s pose in model space or world space. This is called a *global pose*. Some engines express global poses in matrix form, while others use the SRT format.

Mathematically, the model-space pose of a joint ($j \rightarrow M$) can be found by walking the skeletal hierarchy from the joint in question all the way to the root, multiplying the local poses ($j \rightarrow p(j)$) as we go. Consider the hierarchy shown in Figure 12.9. The parent space of the root joint is defined to be model space, so $p(0) \equiv M$. The model-space pose of joint J_2 can therefore be written as follows:

$$\mathbf{P}_{2 \rightarrow M} = \mathbf{P}_{2 \rightarrow 1} \mathbf{P}_{1 \rightarrow 0} \mathbf{P}_{0 \rightarrow M}.$$

Likewise, the model-space pose of joint J_5 is just

$$\mathbf{P}_{5 \rightarrow M} = \mathbf{P}_{5 \rightarrow 4} \mathbf{P}_{4 \rightarrow 3} \mathbf{P}_{3 \rightarrow 0} \mathbf{P}_{0 \rightarrow M}.$$

In general, the global pose (joint-to-model transform) of any joint j can be written as follows:

$$\mathbf{P}_{j \rightarrow M} = \prod_{i=j}^0 \mathbf{P}_{i \rightarrow p(i)}, \quad (12.1)$$

where it is understood that i becomes $p(i)$ (the parent of joint i) after each iteration in the product, and $p(0) \equiv M$.

12.3.3.1 Representing a Global Pose in Memory

We can extend our `SkeletonPose` data structure to include the global pose as follows, where again we dynamically allocate the `m_aGlobalPose` array based on the number of joints in the skeleton:

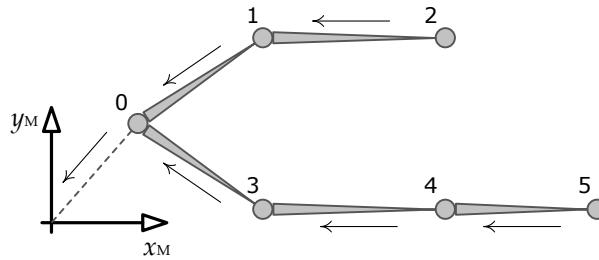


Figure 12.9. A global pose can be calculated by walking the hierarchy from the joint in question towards the root and model-space origin, concatenating the child-to-parent (local) transforms of each joint as we go.

```
struct SkeletonPose
{
    Skeleton* m_pSkeleton;    // skeleton + num joints
    JointPose* m_aLocalPose; // local joint poses
    Matrix44* m_aGlobalPose; // global joint poses
};
```

12.4 Clips

In a film, every aspect of each scene is carefully planned out before any animations are created. This includes the movements of every character and prop in the scene, and even the movements of the camera. This means that an entire scene can be animated as one long, contiguous sequence of frames. And characters need not be animated at all whenever they are off-camera.

Game animation is different. A game is an interactive experience, so one cannot predict beforehand how the characters are going to move and behave. The player has full control over his or her character and usually has partial control over the camera as well. Even the decisions of the computer-driven non-player characters are strongly influenced by the unpredictable actions of the human player. As such, game animations are almost never created as long, contiguous sequences of frames. Instead, a game character's movement must be broken down into a large number of fine-grained motions. We call these individual motions *animation clips*, or sometimes just *animations*.

Each clip causes the character to perform a single well-defined action. Some clips are designed to be looped—for example, a walk cycle or run cycle. Others are designed to be played once—for example, throwing an object or tripping and falling to the ground. Some clips affect the entire body of the character—the character jumping into the air for instance. Other clips affect

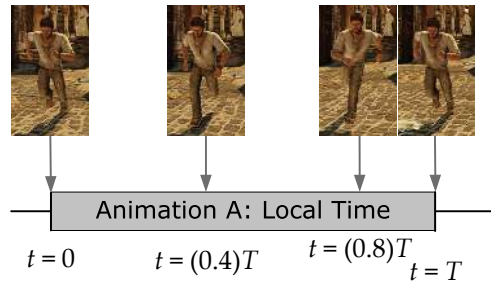


Figure 12.10. The local timeline of an animation showing poses at selected time indices. Images courtesy of Naughty Dog, Inc., © 2014/™ SIE.

only a part of the body—perhaps the character waving his right arm. The movements of any one game character are typically broken down into literally thousands of clips.

The only exception to this rule is when game characters are involved in a noninteractive portion of the game, known as an *in-game cinematic* (IGC), *non-interactive sequence* (NIS) or *full-motion video* (FMV). Noninteractive sequences are typically used to communicate story elements that do not lend themselves well to interactive gameplay, and they are created in much the same way computer-generated films are made (although they often make use of in-game assets like character meshes, skeletons and textures). The terms IGC and NIS typically refer to noninteractive sequences that are rendered in real time by the game engine itself. The term FMV applies to sequences that have been prerendered to an MP4, WMV or other type of movie file and are played back at runtime by the engine’s full-screen movie player.

A variation on this style of animation is a semi-interactive sequence known as a *quick time event* (QTE). In a QTE, the player must hit a button at the right moment during an otherwise noninteractive sequence in order to see the success animation and proceed; otherwise, a failure animation is played, and the player must try again, possibly losing a life or suffering some other consequence as a result.

12.4.1 The Local Timeline

We can think of every animation clip as having a local timeline, usually denoted by the independent variable t . At the start of a clip, $t = 0$, and at the end, $t = T$, where T is the duration of the clip. Each unique value of the variable t is called a *time index*. An example of this is shown in Figure 12.10.

12.4.1.1 Pose Interpolation and Continuous Time

It's important to realize that the rate at which frames are displayed to the viewer is not necessarily the same as the rate at which poses are created by the animator. In both film and game animation, the animator almost never poses the character every $1/30$ or $1/60$ of a second. Instead, the animator generates important poses known as *key poses* or *key frames* at specific times within the clip, and the computer calculates the poses in between via linear or curve-based interpolation. This is illustrated in Figure 12.11.

Because of the animation engine's ability to *interpolate* poses (which we'll explore in depth later in this chapter), we can actually sample the pose of the character at *any time* during the clip—not just on integer frame indices. In other words, an animation clip's timeline is *continuous*. In computer animation, the time variable t is a *real* (floating-point) number, not an *integer*.

Film animation doesn't take full advantage of the continuous nature of the animation timeline, because its frame rate is locked at exactly 24, 30 or 60 frames per second. In film, the viewer sees the characters' poses at frames 1, 2, 3 and so on—there's never any need to find a character's pose on frame 3.7, for example. So in film animation, the animator doesn't pay much (if any) attention to how the character looks in between the integral frame indices.

In contrast, a real-time game's frame rate always varies a little, depending on how much load is currently being placed on the CPU and GPU. Also, game animations are sometimes *time-scaled* in order to make the character appear to move faster or slower than originally animated. So in a real-time game, an animation clip is almost *never* sampled on integer frame numbers. In theory, with a time scale of 1.0, a clip should be sampled at frames 1, 2, 3 and so on. But in practice, the player might actually see frames 1.1, 1.9, 3.2 and so on. And if the time scale is 0.5, then the player might actually see frames

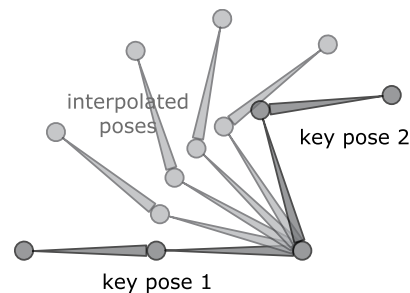


Figure 12.11. An animator creates a relatively small number of key poses, and the engine fills in the rest of the poses via interpolation.

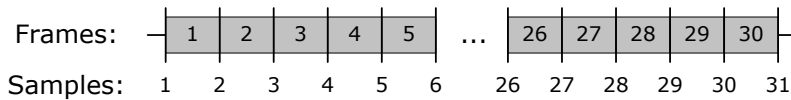


Figure 12.12. A one-second animation sampled at 30 frames per second is 30 frames in duration and consists of 31 samples.

1.1, 1.4, 1.9, 2.6, 3.2 and so on. A negative time scale can even be used to play an animation in reverse. So in game animation, time is both *continuous* and *scalable*.

12.4.1.2 Time Units

Because an animation's timeline is continuous, time is best measured in units of seconds. Time can also be measured in units of *frames*, presuming we define the duration of a frame beforehand. Typical frame durations are $1/30$ or $1/60$ of a second for game animation. However, it's important not to make the mistake of defining your time variable t as an integer that counts whole frames. No matter which time units are selected, t should be a real (floating-point) quantity, a fixed-point number or an integer that measures very small subframe time intervals. The goal is to have sufficient resolution in your time measurements for doing things like "tweening" between frames or scaling an animation's playback speed.

12.4.1.3 Frame versus Sample

Unfortunately, the term *frame* has more than one common meaning in the game industry. This can lead to a great deal of confusion. Sometimes a frame is taken to be a *period of time* that is $1/30$ or $1/60$ of a second in duration. But in other contexts, the term frame is applied to a *single point in time* (e.g., we might speak of the pose of the character "at frame 42").

I personally prefer to use the term *sample* to refer to a single point in time, and I reserve the word *frame* to describe a time period that is $1/30$ or $1/60$ of a second in duration. So for example, a one-second animation created at a rate of 30 frames per second would consist of 31 *samples* and would be 30 *frames* in duration, as shown in Figure 12.12. The term "sample" comes from the field of signal processing. A continuous-time signal (i.e., a function $f(t)$) can be converted into a set of discrete data points by sampling that signal at uniformly spaced time intervals. See Section 14.3.2.1 for more information on sampling.

12.4.1.4 Frames, Samples and Looping Clips

When a clip is designed to be played over and over repeatedly, we say it is *looped*. If we imagine two copies of a 1 s (30-frame/31-sample) clip laid back-to-front, then sample 31 of the first clip will coincide exactly in time with sample 1 of the second clip, as shown in Figure 12.13. For a clip to loop properly, then, we can see that the pose of the character at the end of the clip must exactly match the pose at the beginning. This, in turn, implies that the last sample of a looping clip (in our example, sample 31) is redundant. Many game engines therefore omit the last sample of a looping clip.

This leads us to the following rules governing the number of samples and frames in any animation clip:

- If a clip is *non-looping*, an N -frame animation will have $N + 1$ unique samples.
- If a clip is *looping*, then the last sample is redundant, so an N -frame animation will have N unique samples.

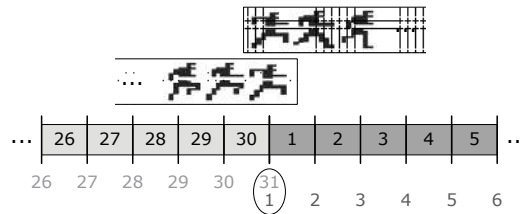


Figure 12.13. The last sample of a looping clip coincides in time with its first sample and is, therefore, redundant.

12.4.1.5 Normalized Time (Phase)

It is sometimes convenient to employ a normalized time unit u , such that $u = 0$ at the start of the animation, and $u = 1$ at the end, no matter what its duration T may be. We sometimes refer to normalized time as the *phase* of the animation clip, because u acts like the phase of a sine wave when the animation is looped. This is illustrated in Figure 12.14.

Normalized time is useful when synchronizing two or more animation clips that are not necessarily of the same absolute duration. For example, we might want to smoothly cross-fade from a 2-second (60-frame) run cycle into a 3-second (90-frame) walk cycle. To make the cross-fade look good, we want to ensure that the two animations remain synchronized at all times, so that the

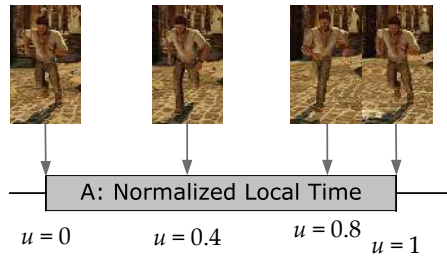


Figure 12.14. An animation clip, showing normalized time units. Images courtesy of Naughty Dog, Inc., © 2014/™ SIE.

feet line up properly in both clips. We can accomplish this by simply setting the normalized start time of the walk clip, u_{walk} , to match the normalized time index of the run clip, u_{run} . We then advance both clips at the same normalized rate so that they remain in sync. This is quite a bit easier and less error-prone than doing the synchronization using the absolute time indices t_{walk} and t_{run} .

12.4.2 The Global Timeline

Just as every animation clip has a local timeline (whose clock starts at 0 at the beginning of the clip), every character in a game has a global timeline (whose clock starts when the character is first spawned into the game world, or perhaps at the start of the level or the entire game). In this book, we'll use the time variable τ to measure global time, so as not to confuse it with the local time variable t .

We can think of *playing* an animation as simply *mapping* that clip's local timeline onto the character's global timeline. For example, Figure 12.15 illustrates playing animation clip A starting at a global time of $\tau_{\text{start}} = 102$ seconds.

As we saw above, playing a looping animation is like laying down an infinite number of back-to-front copies of the clip onto the global timeline. We can also imagine looping an animation a finite number of times, which corresponds to laying down a finite number of copies of the clip. This is illustrated in Figure 12.16.

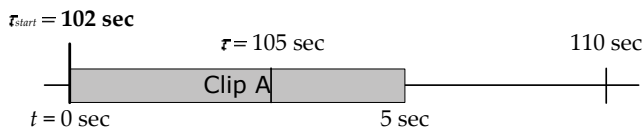


Figure 12.15. Playing animation clip A starting at a global time of 102 seconds.



Figure 12.16. Playing a looping animation corresponds to laying down multiple back-to-back copies of the clip.

Time-scaling a clip makes it appear to play back more quickly or more slowly than originally animated. To accomplish this, we simply scale the image of the clip when it is laid down onto the global timeline. Time-scaling is most naturally expressed as a *playback rate*, which we'll denote R . For example, if an animation is to play back at twice the speed ($R = 2$), then we would scale the clip's local timeline to one-half ($1/R = 0.5$) of its normal length when mapping it onto the global timeline. This is shown in Figure 12.17.

Playing a clip in reverse corresponds to using a time scale of -1 , as shown in Figure 12.18.

In order to map an animation clip onto a global timeline, we need the following pieces of information about the clip:

- its global start time τ_{start} ,

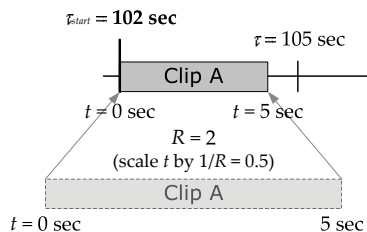


Figure 12.17. Playing an animation at twice the speed corresponds to scaling its local timeline by a factor of $1/2$.

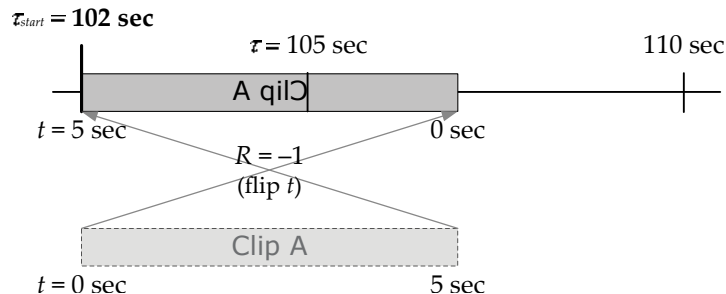


Figure 12.18. Playing a clip in reverse corresponds to a time scale of -1 .

- its playback rate R ,
- its duration T , and
- the number of times it should loop, which we'll denote N .

Given this information, we can map from any global time τ to the corresponding local time t , and vice versa, using the following two relations:

$$\begin{aligned} t &= (\tau - \tau_{\text{start}})R, \\ \tau &= \tau_{\text{start}} + \frac{1}{R}t. \end{aligned} \tag{12.2}$$

If the animation doesn't loop ($N = 1$), then we should clamp t into the valid range $[0, T]$ before using it to sample a pose from the clip:

$$t = \text{clamp} \left[(\tau - \tau_{\text{start}})R \right] \Big|_0^T.$$

If the animation loops forever ($N = \infty$), then we bring t into the valid range by taking the *remainder* of the result after dividing by the duration T . This is accomplished via the *modulo* operator (`mod`, or `%` in C/C++), as shown below:

$$t = ((\tau - \tau_{\text{start}})R) \bmod T.$$

If the clip loops a *finite* number of times ($1 < N < \infty$), we must first clamp t into the range $[0, NT]$ and then modulo *that* result by T in order to bring t into a valid range for sampling the clip:

$$t = \left(\text{clamp} \left[(\tau - \tau_{\text{start}})R \right] \Big|_0^{NT} \right) \bmod T.$$

Most game engines work directly with local animation timelines and don't use the global timeline directly. However, working directly in terms of global times can have some incredibly useful benefits. For one thing, it makes synchronizing animations trivial.

12.4.3 Comparison of Local and Global Clocks

The animation system must keep track of the time indices of every animation that is currently playing. To do so, we have two choices:

- *Local clock.* In this approach, each clip has its own local clock, usually represented by a floating-point time index stored in units of seconds or frames, or in normalized time units (in which case it is often called the *phase* of the animation). At the moment the clip begins to play, the local

time index t is usually taken to be zero. To advance the animations forward in time, we advance the local clocks of each clip individually. If a clip has a non-unit playback rate R , the amount by which its local clock advances must be scaled by R .

- *Global clock.* In this approach, the character has a global clock, usually measured in seconds, and each clip simply records the global time at which it started playing, τ_{start} . The clips' local clocks are *calculated* from this information using Equation (12.2).

The local clock approach has the benefit of being simple, and it is the most obvious choice when designing an animation system. However, the global clock approach has some distinct advantages, especially when it comes to synchronizing animations, either within the context of a single character or across multiple characters in a scene.

12.4.3.1 Synchronizing Animations with a Local Clock

With a local clock approach, we said that the origin of a clip's local timeline ($t = 0$) is usually defined to coincide with the moment at which the clip starts playing. Thus, to synchronize two or more clips, they must be played at exactly the same moment in game time. This seems simple enough, but it can become quite tricky when the commands used to play the animations are coming from disparate engine subsystems.

For example, let's say we want to synchronize the player character's punch animation with a non-player character's corresponding hit reaction animation. The problem is that the player's punch is initiated by the player subsystem in response to detecting that a button was hit on the joy pad. Meanwhile, the non-player character's (NPC) hit reaction animation is played by the artificial intelligence (AI) subsystem. If the AI code runs *before* the player code in the game loop, there will be a one-frame delay between the start of the player's punch and the start of the NPC's reaction. And if the player code runs before the AI code, then the opposite problem occurs when an NPC tries to punch the player. If a message-passing (event) system is used to communicate between the two subsystems, additional delays might be incurred (see Section 16.8 for more details). This problem is illustrated in Figure 12.19.

```
void GameLoop()
{
    while (!quit)
    {
        // preliminary updates...
```



```

    UpdateAllNpcs(); // react to punch event
                      // from last frame

    // more updates...

    UpdatePlayer(); // punch button hit - start punch
                   // anim, and send event to NPC to
                   // react

    // still more updates...
  }
}

```

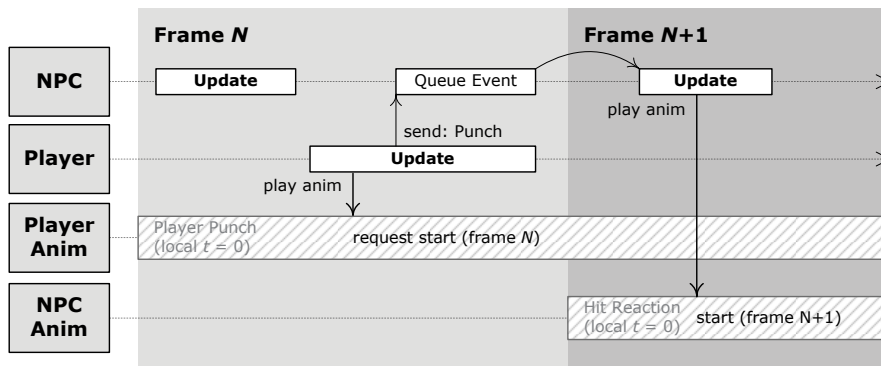


Figure 12.19. The order of execution of disparate gameplay systems can introduce animation synchronization problems when local clocks are used.

12.4.3.2 Synchronizing Animations with a Global Clock

A global clock approach helps to alleviate many of these synchronization problems, because the origin of the timeline ($\tau = 0$) is common across all clips by definition. If two or more animations' global start times are numerically equal, the clips will start in perfect synchronization. If their playback rates are also equal, then they will remain in sync with no drift. It no longer matters *when* the code that plays each animation executes. Even if the AI code that plays the hit reaction ends up running a frame later than the player's punch code, it is still trivial to keep the two clips in sync by simply noting the global start time of the punch and setting the global start time of the reaction animation to match it. This is shown in Figure 12.20.

Of course, we do need to ensure that the two characters' global clocks match, but this is trivial to do. We can either adjust the global start times to

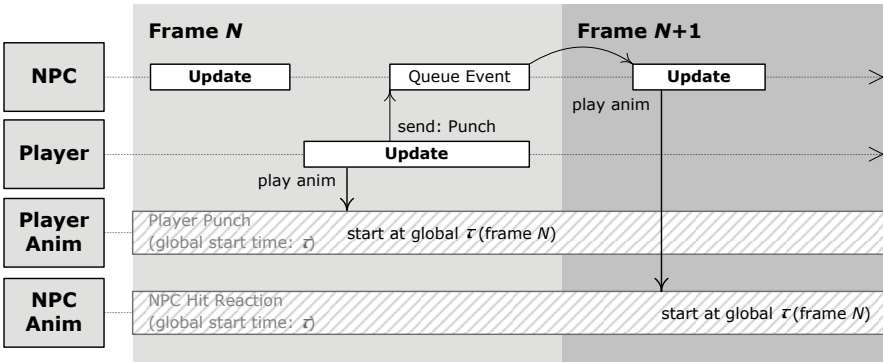


Figure 12.20. A global clock approach can alleviate animation synchronization problems.

take account of any differences in the characters' clocks, or we can simply have all characters in the game share a single master clock.

12.4.4 A Simple Animation Data Format

Typically, animation data is extracted from a Maya scene file by sampling the pose of the skeleton discretely at a rate of 30 or 60 samples per second. A sample comprises a full pose for each joint in the skeleton. The poses are usually stored in SRT format: For each joint j , the scale component is either a single floating-point scalar S_j or a three-element vector $\mathbf{S}_j = [S_{jx} \ S_{jy} \ S_{jz}]$. The rotational component is of course a four-element quaternion $\mathbf{Q}_j = [Q_{jx} \ Q_{jy} \ Q_{jz} \ Q_{jw}]$. And the translational component is a three-element vector $\mathbf{T}_j =$

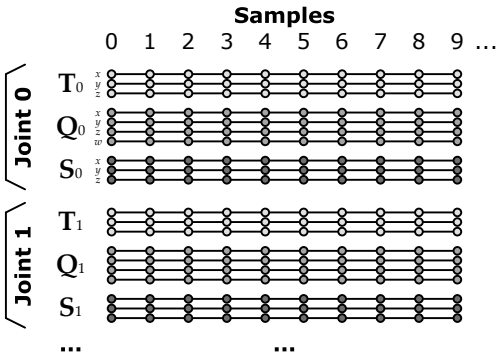


Figure 12.21. An uncompressed animation clip contains 10 channels of floating-point data per sample, per joint.

$[T_{jx} \ T_{jy} \ T_{jz}]$. We sometimes say that an animation consists of up to 10 *channels* per joint, in reference to the 10 components of $\mathbf{S}_j$, $\mathbf{Q}_j$, and $\mathbf{T}_j$. This is illustrated in Figure 12.21.

In C++, an animation clip can be represented in many different ways. Here is one possibility:

```
struct JointPose { ... }; // SRT, defined as above

struct AnimationSample
{
    JointPose*      m_aJointPose; // array of joint
                                // poses
};

struct AnimationClip
{
    Skeleton*       m_pSkeleton;
    F32             m_framesPerSecond;
    U32             m_frameCount;
    AnimationSample* m_aSamples; // array of samples
    bool            m_isLooping;
};
```

An animation clip is authored for a specific skeleton and generally won't work on any other skeleton. As such, our example AnimationClip data structure contains a reference to its skeleton, `m_pSkeleton`. (In a real engine, this might be a unique skeleton id rather than a `Skeleton*` pointer. In this case, the engine would presumably provide a way to quickly and conveniently look up a skeleton by its unique id.)

The number of JointPoses in the `m_aJointPose` array within each sample is presumed to match the number of joints in the skeleton. The number of samples in the `m_aSamples` array is dictated by the frame count and by whether or not the clip is intended to loop. For a non-looping animation, the number of samples is $(m_frameCount + 1)$. However, if the animation loops, then the last sample is identical to the first sample and is usually omitted. In this case, the sample count is equal to `m_frameCount`.

It's important to realize that in a real game engine, animation data isn't actually stored in this simplistic format. As we'll see in Section 12.8, the data is usually *compressed* in various ways to save memory.

12.4.5 Continuous Channel Functions

The samples of an animation clip are really just definitions of continuous functions over time. You can think of these as 10 scalar-valued functions of time

per joint, or as two vector-valued functions and one quaternion-valued function per joint. Theoretically, these *channel functions* are smooth and continuous across the entire clip's local timeline, as shown in Figure 12.22 (with the exception of explicitly authored discontinuities like camera cuts). In practice, however, many game engines interpolate *linearly* between the samples, in which case the functions actually used are *piecewise linear approximations* to the underlying continuous functions. This is depicted in Figure 12.23.

12.4.6 Metachannels

Many games permit additional “metachannels” of data to be defined for an animation. These channels can encode game-specific information that doesn't have to do directly with posing the skeleton but which needs to be synchronized with the animation.

It is quite common to define a special channel that contains *event triggers* at various time indices, as shown in Figure 12.24. Whenever the animation's local time index passes one of these triggers, an *event* is sent to the game engine, which can respond as it sees fit. (We'll discuss events in detail in Chapter 16.) One common use of event triggers is to denote at which points during the

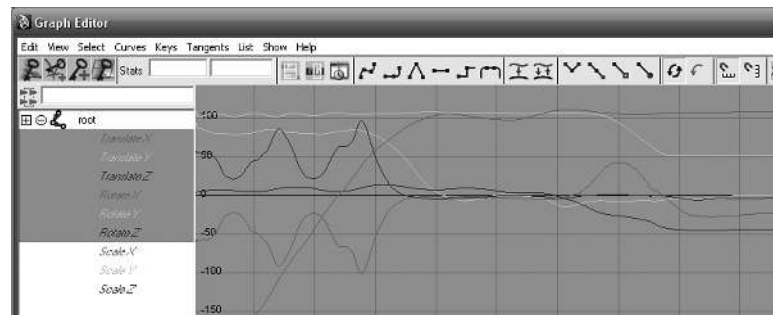


Figure 12.22. The animation samples in a clip define continuous functions over time.

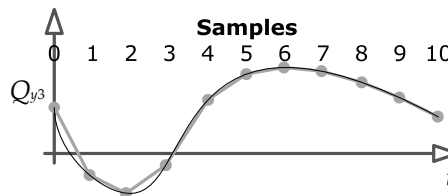


Figure 12.23. Many game engines use a piecewise linear approximation when interpolating channel functions.

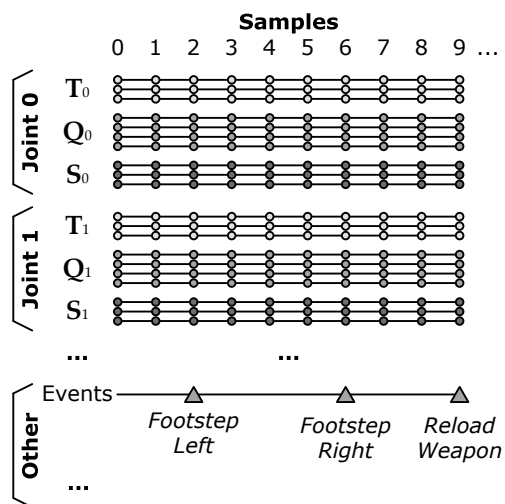


Figure 12.24. A special event trigger channel can be added to an animation clip in order to synchronize sound effects, particle effects and other game events with an animation.

animation certain sound or particle effects should be played. For example, when the left or right foot touches the ground, a footstep sound and a “cloud of dust” particle effect could be initiated.

Another common practice is to permit special joints, known in Maya as *locators*, to be animated along with the joints of the skeleton itself. Because a joint or locator is just an affine transform, these special joints can be used to encode the position and orientation of virtually any object in the game.

A typical application of animated locators is to specify how the game’s camera should be positioned and oriented during an animation. In Maya, a locator is constrained to a camera, and the camera is then animated along with the joints of the character(s) in the scene. The camera’s locator is exported and used in-game to move the game’s camera around during the animation. The field of view (focal length) of the camera, and possibly other camera attributes, can also be animated by placing the relevant data into one or more additional *floating-point channels*.

Other examples of non-joint animation channels include:

- texture coordinate scrolling,
- texture animation (a special case of texture coordinate scrolling in which frames are arranged linearly within a texture, and the texture is scrolled by one complete frame at each iteration),

- animated material parameters (color, specular, transparency, etc.),
- animated lighting parameters (radius, cone angle, intensity, color, etc.), and
- any other parameters that need to change over time and are in some way synchronized with an animation.

12.4.7 Relationship between Meshes, Skeletons and Clips

The UML diagram in Figure 12.25 shows how animation clip data interfaces with the skeletons, poses, meshes and other data in a game engine. Pay particular attention to the *cardinality* and *direction* of the relationships between these classes. The cardinality is shown just beside the tip or tail of the relationship arrow between classes—a one represents a single instance of the class, while an asterisk indicates many instances. For any one type of character, there will be one skeleton, one or more meshes and one or more animation clips. The skeleton is the central unifying element—the skins are attached to the skeleton but don't have any relationship with the animation clips. Likewise, the clips are targeted at a particular skeleton, but they have no “knowledge” of the skin meshes. Figure 12.26 illustrates these relationships.

Game designers often try to reduce the number of unique skeletons in the game to a minimum, because each new skeleton generally requires a whole new set of animation clips. To provide the illusion of many different types of characters, it is usually better to create multiple meshes skinned to the same skeleton when possible, so that all of the characters can share a single set of animations.

12.4.7.1 Animation Retargeting

We said above that an animation is typically only compatible with a single skeleton. This limitation can be overcome via *animation retargeting* techniques.

Retargeting means using an animation authored for one skeleton to animate a different skeleton. If the two skeletons are morphologically identical, retargeting may boil down to a simple matter of joint index remapping. But when the two skeletons don't match exactly, the retargeting problem becomes more complex. At Naughty Dog, the animators define a special pose known as the *retarget pose*. This pose captures the essential differences between the bind poses of the source and target skeletons, allowing the runtime retargeting system to adjust source poses so they will work more naturally on the target character.

Other more-advanced techniques exist for retargeting animations authored for one skeleton so that they work on a different skeleton. For more infor-

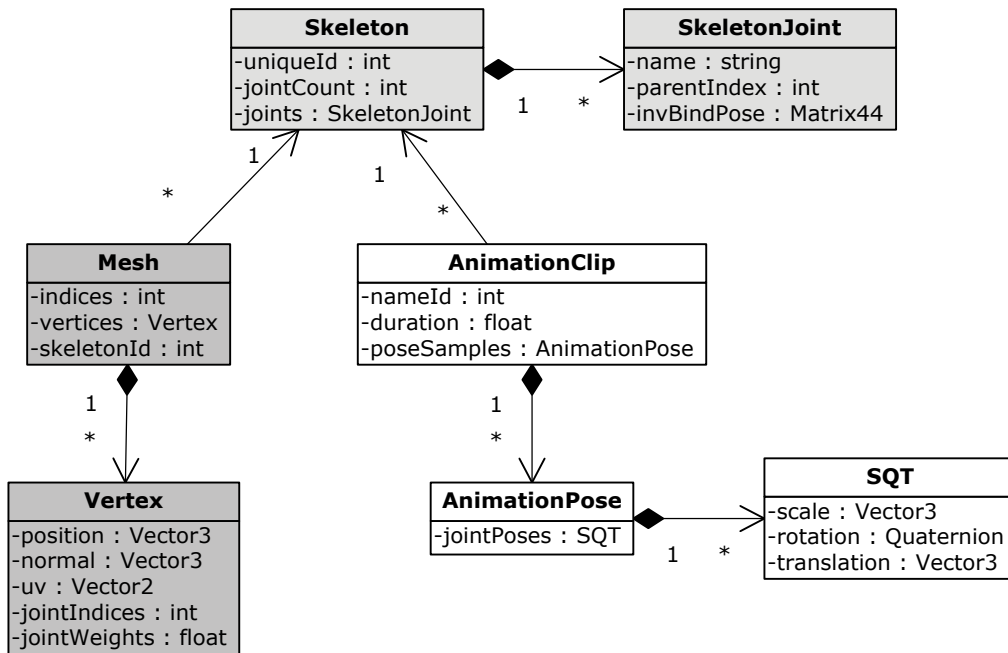


Figure 12.25. UML diagram of shared animation resources.

mation, see “Feature Points Based Facial Animation Retargeting” by Ludovic Dutreuve et al. (<https://bit.ly/2HL9Cdr>) and “Real-time Motion Retargeting to Highly Varied User-Created Morphologies” by Chris Hecker et al. (<https://bit.ly/2vviG3x>).

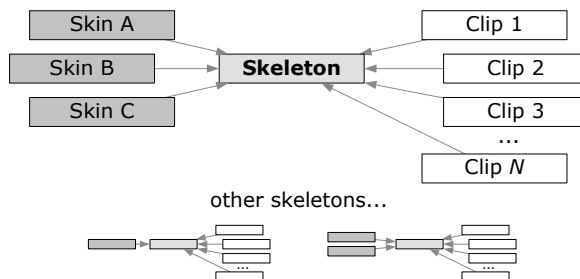


Figure 12.26. Many animation clips and one or more meshes target a single skeleton.

12.5 Skinning and Matrix Palette Generation

We've seen how to pose a skeleton by rotating, translating and possibly scaling its joints. And we know that any skeletal pose can be represented mathematically as a set of local ($\mathbf{P}_{j \rightarrow p(j)}$) or global ($\mathbf{P}_{j \rightarrow M}$) joint pose transformations, one for each joint j . Next, we will explore the process of attaching the vertices of a 3D mesh to a posed skeleton. This process is known as *skinning*.

12.5.1 Per-Vertex Skinning Information

A skinned mesh is attached to a skeleton by means of its vertices. Each vertex can be *bound* to one or more joints. If bound to a single joint, the vertex tracks that joint's movement exactly. If bound to two or more joints, the vertex's position becomes a *weighted average* of the positions it would have assumed had it been bound to each joint independently.

To skin a mesh to a skeleton, a 3D artist must supply the following additional information at each vertex:

- the *index* or *indices* of the joint(s) to which it is bound, and
- for each joint, a *weighting factor* describing how much influence that joint should have on the final vertex position.

The weighting factors are assumed to add to one, as is customary when calculating any weighted average.

Usually a game engine imposes an upper limit on the number of joints to which a single vertex can be bound. A four-joint limit is typical for a number of reasons. First, four 8-bit joint indices can be packed into a 32-bit word, which is convenient. Also, while it's pretty easy to see a difference in quality between a two-, three- and even a four-joint-per-vertex model, most people cannot see a quality difference as the number of joints per vertex is increased beyond four.

Because the joint weights must sum to one, the last weight can be omitted and often is. (It can be calculated at runtime as $w_3 = 1 - (w_0 + w_1 + w_2)$.) As such, a typical skinned vertex data structure might look as follows:

```
struct SkinnedVertex
{
    float  m_position[3];    // (Px, Py, Pz)
    float  m_normal[3];     // (Nx, Ny, Nz)
    float  m_u, m_v;        // texture coords (u,v)
    U8      m_jointIndex[4]; // joint indices
    float  m_jointWeight[3]; // joint weights (last
                           // weight omitted)
};
```


12.5.2 The Mathematics of Skinning

The vertices of a skinned mesh track the movements of the joint(s) to which they are bound. To make this happen mathematically, we would like to find a matrix that can transform the vertices of the mesh from their original positions (in bind pose) into new positions that correspond to the current pose of the skeleton. We shall call such a matrix a *skinning matrix*.

Like all mesh vertices, the position of a skinned vertex is specified in model space. This is true whether its skeleton is in bind pose or in any other pose. So the matrix we seek will transform vertices from model space (bind pose) to model space (current pose). Unlike the other transforms we've seen thus far, such as the model-to-world transform or the world-to-view transform, a skinning matrix is *not* a change of basis transform. It morphs vertices into new positions, but the vertices are in model space both before and after the transformation.

12.5.2.1 Simple Example: One-Jointed Skeleton

Let us derive the basic equation for a skinning matrix. To keep things simple at first, we'll work with a skeleton consisting of a single joint. We therefore have two coordinate spaces to work with: model space, which we'll denote with the subscript M, and the joint space of our one and only joint, which will be indicated by the subscript J. The joint's coordinate axes start out in bind pose, which we'll denote with the superscript B. At any given moment during an animation, the joint's axes move to a new position and orientation in model space—we'll indicate this *current pose* with the superscript C.

Now consider a single vertex that is skinned to our joint. In bind pose, its model-space position is $\mathbf{v}_M^B$. The skinning process calculates the vertex's new model-space position in the current pose, $\mathbf{v}_M^C$. This is illustrated in Figure 12.27.

The “trick” to finding the skinning matrix for a given joint is to realize that the position of a vertex bound to a joint is *constant* when expressed in *that joint's coordinate space*. So we take the bind-pose position of the vertex in model space, convert it into joint space, move the joint into its current pose, and finally convert the vertex back into model space. The net effect of this round trip from model space to joint space and back again is to “morph” the vertex from bind pose into the current pose.

Referring to the illustration in Figure 12.28, let's assume that the coordinates of the vertex $\mathbf{v}_M^B$ are (4, 6) in model space (when the skeleton is in bind pose). We convert this vertex into its equivalent joint-space coordinates $\mathbf{v}_J$, which are roughly (1, 3) as shown in the diagram. Because the vertex is bound

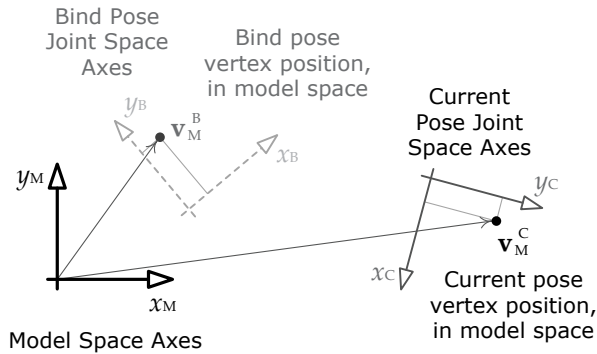


Figure 12.27. Bind pose and current pose of a simple, one-joint skeleton and a single vertex bound to that joint.

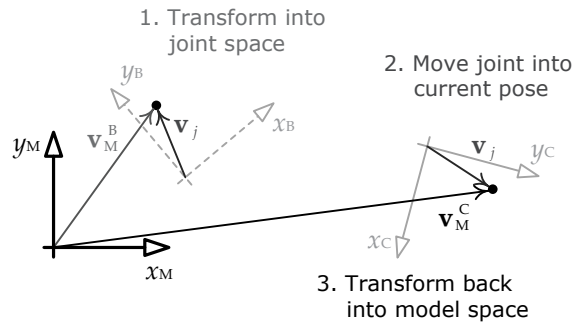


Figure 12.28. By transforming a vertex's position into joint space, it can be made to "track" the joint's movements.

to the joint, its joint-space coordinates will *always* be (1, 3) no matter how the joint may move. Once we have the joint in the desired current pose, we convert the vertex's coordinates back into model space, which we'll denote with the symbol v_M^C . In our diagram, these coordinates are roughly (18, 2). So the skinning transformation has morphed our vertex from (4, 6) to (18, 2) in model space, due entirely to the motion of the joint from its bind pose to the current pose shown in the diagram.

Looking at the problem mathematically, we can denote the *bind pose* of the joint j in model space by the matrix $B_{j \rightarrow M}$. This matrix transforms a point or vector whose coordinates are expressed in joint j 's space into an equivalent set of model-space coordinates. Now, consider a vertex whose coordinates are expressed in model space with the skeleton in bind pose. To convert these vertex coordinates into the space of joint j , we simply multiply it by the *inverse*

bind pose matrix, $\mathbf{B}_{M \rightarrow j} = (\mathbf{B}_{j \rightarrow M})^{-1}$:

$$\mathbf{v}_j = \mathbf{v}_M^B \mathbf{B}_{M \rightarrow j} = \mathbf{v}_M^B (\mathbf{B}_{j \rightarrow M})^{-1}. \quad (12.3)$$

Likewise, we can denote the joint's *current pose* (i.e., any pose that is *not* bind pose) by the matrix $\mathbf{C}_{j \rightarrow M}$. To convert $\mathbf{v}_j$ from joint space back into model space, we simply multiply it by the current pose matrix as follows:

$$\mathbf{v}_M^C = \mathbf{v}_j \mathbf{C}_{j \rightarrow M}.$$

If we expand $\mathbf{v}_j$ using Equation (12.3), we obtain an equation that takes our vertex directly from its position in bind pose to its position in the current pose:

$$\begin{aligned} \mathbf{v}_M^C &= \mathbf{v}_j \mathbf{C}_{j \rightarrow M} \\ &= \mathbf{v}_M^B (\mathbf{B}_{j \rightarrow M})^{-1} \mathbf{C}_{j \rightarrow M} \\ &= \mathbf{v}_M^B \mathbf{K}_j. \end{aligned} \quad (12.4)$$

The combined matrix $\mathbf{K}_j = (\mathbf{B}_{j \rightarrow M})^{-1} \mathbf{C}_{j \rightarrow M}$ is known as a *skinning matrix*.

12.5.2.2 Extension to Multijointed Skeletons

In the example above, we considered only a single joint. However, the math we derived above actually applies to any joint in any skeleton imaginable, because we formulated everything in terms of global poses (i.e., joint space to model space transforms). To extend the above formulation to a skeleton containing multiple joints, we therefore need to make only two minor adjustments:

1. We must make sure that our $\mathbf{B}_{j \rightarrow M}$ and $\mathbf{C}_{j \rightarrow M}$ matrices are calculated properly for the joint in question, using Equation (12.1). $\mathbf{B}_{j \rightarrow M}$ and $\mathbf{C}_{j \rightarrow M}$ are just the bind pose and current pose equivalents, respectively, of the matrix $\mathbf{P}_{j \rightarrow M}$ used in that equation.
2. We must calculate an array of skinning matrices $\mathbf{K}_j$, one for each joint j . This array is known as a *matrix palette*. The matrix palette is passed to the rendering engine when rendering a skinned mesh. For each vertex, the renderer looks up the appropriate joint's skinning matrix in the palette and uses it to transform the vertex from bind pose into current pose.

We should note here that the current pose matrix $\mathbf{C}_{j \rightarrow M}$ changes every frame as the character assumes different poses over time. However, the inverse bind-pose matrix is constant throughout the entire game, because the bind pose of the skeleton is fixed when the model is created. Therefore, the

matrix $(\mathbf{B}_{j \rightarrow M})^{-1}$ is generally cached with the skeleton, and needn't be calculated at runtime. Animation engines generally calculate local poses for each joint $(\mathbf{C}_{j \rightarrow p(j)})$, then use Equation (12.1) to convert these into global poses $(\mathbf{C}_{j \rightarrow M})$, and finally multiply each global pose by the corresponding cached inverse bind pose matrix $(\mathbf{B}_{j \rightarrow M})^{-1}$ in order to generate a skinning matrix $(\mathbf{K}_j)$ for each joint.

12.5.2.3 Incorporating the Model-to-World Transform

Every vertex must eventually be transformed from model space into world space. Some engines therefore premultiply the palette of skinning matrices by the object's model-to-world transform. This can be a useful optimization, as it saves the rendering engine one matrix multiply per vertex when rendering skinned geometry. (With hundreds of thousands of vertices to process, these savings can really add up!)

To incorporate the model-to-world transform into our skinning matrices, we simply concatenate it to the regular skinning matrix equation, as follows:

$$(\mathbf{K}_j)_W = (\mathbf{B}_{j \rightarrow M})^{-1} \mathbf{C}_{j \rightarrow M} \mathbf{M}_{M \rightarrow W}.$$

Some engines bake the model-to-world transform into the skinning matrices like this, while others don't. The choice is entirely up to the engineering team and is driven by all sorts of factors. For example, one situation in which we would definitely *not* want to do this is when a single animation is being applied to multiple characters simultaneously—a technique known as *animation instancing* that is sometimes used for animating large crowds of characters. In this case we need to keep the model-to-world transforms separate so that we can share a single matrix palette across all characters in the crowd.

12.5.2.4 Skinning a Vertex to Multiple Joints

When a vertex is skinned to more than one joint, we calculate its final position by assuming it is skinned to each joint individually, calculating a model-space position for each joint and then taking a *weighted average* of the resulting positions. The weights are provided by the character rigging artist, and they must always sum to one. (If they do not sum to one, they should be renormalized by the tools pipeline.)

The general formula for a weighted average of N quantities a_0 through a_{N-1} , with weights w_0 through w_{N-1} and with $\sum w_i = 1$ is:

$$a = \sum_{i=0}^{N-1} w_i a_i.$$

This works equally well for vector quantities $\mathbf{a}_i$. So, for a vertex skinned to N joints with indices j_0 through j_{N-1} and weights w_0 through w_{N-1} , we can extend Equation (12.4) as follows:

$$\mathbf{v}_M^C = \sum_{i=0}^{N-1} w_i \mathbf{v}_M^B \mathbf{K}_{j_i},$$

where $\mathbf{K}_{j_i}$ is the skinning matrix for the joint j_i .

12.6 Animation Blending

The term *animation blending* refers to any technique that allows more than one animation clip to contribute to the final pose of the character. To be more precise, blending combines two or more *input poses* to produce an *output pose* for the skeleton.

Blending usually combines two or more poses at a single point in time, and generates an output at that same moment in time. In this context, blending is used to combine two or more animations into a host of new animations, without having to create them manually. For example, by blending an injured walk animation with an uninjured walk, we can generate various intermediate levels of apparent injury for our character while he is walking. As another example, we can blend between an animation in which the character is aiming to the left and one in which he's aiming to the right, in order to make the character aim along any desired angle between the two extremes. Blending can be used to interpolate between extreme facial expressions, body stances, locomotion modes and so on.

Blending can also be used to find an intermediate pose between two known poses at *different* points in time. This is used when we want to find the pose of a character at a point in time that does not correspond exactly to one of the sampled frames available in the animation data. We can also use temporal animation blending to smoothly transition from one animation to another, by gradually blending from the source animation to the destination over a short period of time.

12.6.1 LERP Blending

Given a skeleton with N joints, and two skeletal poses $\mathbf{P}_A^{\text{skel}} = \{(\mathbf{P}_A)_j\}_{j=0}^{N-1}$ and $\mathbf{P}_B^{\text{skel}} = \{(\mathbf{P}_B)_j\}_{j=0}^{N-1}$, we wish to find an intermediate pose $\mathbf{P}_{\text{LERP}}^{\text{skel}}$ between these two extremes. This can be done by performing a *linear interpolation* (LERP) between the local poses of each individual joint in each of the two

source poses. This can be written as follows:

$$\begin{aligned} (\mathbf{P}_{\text{LERP}})_j &= \text{LERP}((\mathbf{P}_A)_j, (\mathbf{P}_B)_j, \beta) \\ &= (1 - \beta)(\mathbf{P}_A)_j + \beta(\mathbf{P}_B)_j. \end{aligned} \quad (12.5)$$

The interpolated pose of the whole skeleton is simply the set of interpolated poses for all of the joints:

$$\mathbf{P}_{\text{LERP}}^{\text{skel}} = \{(\mathbf{P}_{\text{LERP}})_j\}_{j=0}^{N-1}. \quad (12.6)$$

In these equations, β is called the *blend percentage* or *blend factor*. When $\beta = 0$, the final pose of the skeleton will exactly match $\mathbf{P}_A^{\text{skel}}$; when $\beta = 1$, the final pose will match $\mathbf{P}_B^{\text{skel}}$. When β is between zero and one, the final pose is an intermediate between the two extremes. This effect is illustrated in Figure 12.11.

We’ve glossed over one small detail here: We are linearly interpolating *joint poses*, which means interpolating 4×4 *transformation matrices*. But, as we saw in Chapter 5, interpolating matrices directly is not practical. This is one of the reasons why local poses are usually expressed in SRT format—doing so allows us to apply the LERP operation defined in Section 5.2.5 to each component of the SRT individually. The linear interpolation of the translation component $\mathbf{T}$ of an SRT is just a straightforward vector LERP:

$$\begin{aligned} (\mathbf{T}_{\text{LERP}})_j &= \text{LERP}((\mathbf{T}_A)_j, (\mathbf{T}_B)_j, \beta) \\ &= (1 - \beta)(\mathbf{T}_A)_j + \beta(\mathbf{T}_B)_j. \end{aligned} \quad (12.7)$$

The linear interpolation of the rotation component is a quaternion LERP or SLERP (spherical linear interpolation):

$$\begin{aligned} (\mathbf{Q}_{\text{LERP}})_j &= \text{normalize}(\text{LERP}((\mathbf{Q}_A)_j, (\mathbf{Q}_B)_j, \beta)) \\ &= \text{normalize}((1 - \beta)(\mathbf{Q}_A)_j + \beta(\mathbf{Q}_B)_j). \end{aligned} \quad (12.8)$$

or

$$\begin{aligned} (\mathbf{Q}_{\text{SLERP}})_j &= \text{SLERP}((\mathbf{Q}_A)_j, (\mathbf{Q}_B)_j, \beta) \\ &= \frac{\sin((1 - \beta)\theta)}{\sin(\theta)}(\mathbf{Q}_A)_j + \frac{\sin(\beta\theta)}{\sin(\theta)}(\mathbf{Q}_B)_j. \end{aligned} \quad (12.9)$$

Finally, the linear interpolation of the scale component is either a scalar or vector LERP, depending on the type of scale (uniform or nonuniform scale) supported by the engine:

$$\begin{aligned} (\mathbf{S}_{\text{LERP}})_j &= \text{LERP}((\mathbf{S}_A)_j, (\mathbf{S}_B)_j, \beta) \\ &= (1 - \beta)(\mathbf{S}_A)_j + \beta(\mathbf{S}_B)_j. \end{aligned} \quad (12.10)$$

or

$$\begin{aligned}(S_{\text{LERP}})_j &= \text{LERP}((S_A)_j, (S_B)_j, \beta) \\ &= (1 - \beta)(S_A)_j + \beta(S_B)_j.\end{aligned}\tag{12.11}$$

When linearly interpolating between two skeletal poses, the most natural-looking intermediate pose is generally one in which each joint pose is interpolated independently of the others, in the space of that joint's immediate parent. In other words, pose blending is generally performed on *local poses*. If we were to blend global poses directly in model space, the results would tend to look biomechanically implausible.

Because pose blending is done on local poses, the linear interpolation of any one joint's pose is totally independent of the interpolations of the other joints in the skeleton. This means that linear pose interpolation can be performed entirely in parallel on multiprocessor architectures.

12.6.2 Applications of LERP Blending

Now that we understand the basics of LERP blending, let's have a look at some typical gaming applications.

12.6.2.1 Temporal Interpolation

As we mentioned in Section 12.4.1.1, game animations are almost never sampled exactly on integer frame indices. Because of variable frame rate, the player might actually see frames 0.9, 1.85 and 3.02, rather than frames 1, 2 and 3 as one might expect. In addition, some animation compression techniques involve storing only disparate key frames, spaced at uneven intervals across the clip's local timeline. In either case, we need a mechanism for finding intermediate poses between the sampled poses that are actually present in the animation clip.

LERP blending is typically used to find these intermediate poses. As an example, let's imagine that our animation clip contains evenly spaced pose samples at times $0, \Delta t, 2\Delta t, 3\Delta t$ and so on. To find a pose at time $t = 2.18\Delta t$, we simply find the linear interpolation between the poses at times $2\Delta t$ and $3\Delta t$, using a blend percentage of $\beta = 0.18$.

In general, we can find the pose at time t given pose samples at any two times t_1 and t_2 that bracket t , as follows:

$$\mathbf{P}_j(t) = \text{LERP}(\mathbf{P}_j(t_1), \mathbf{P}_j(t_2), \beta(t))\tag{12.12}$$

$$= (1 - \beta(t)) \mathbf{P}_j(t_1) + \beta(t) \mathbf{P}_j(t_2),\tag{12.13}$$

where the blend factor $\beta(t)$ can be determined by the ratio

$$\beta(t) = \frac{t - t_1}{t_2 - t_1}.\tag{12.14}$$

12.6.2.2 Motion Continuity: Cross-Fading

Game characters are animated by piecing together a large number of fine-grained animation clips. If your animators are any good, the character will appear to move in a natural and physically plausible way *within* each individual clip. However, it is notoriously difficult to achieve the same level of quality when transitioning from one clip to the next. The vast majority of the “pops” we see in game animations occur when the character transitions from one clip to the next.

Ideally, we would like the movements of each part of a character’s body to be perfectly smooth, even during transitions. In other words, the three-dimensional paths traced out by each joint in the skeleton as it moves should contain no sudden “jumps.” We call this *C0 continuity*; it is illustrated in Figure 12.29.

Not only should the paths themselves be continuous, but their first derivatives (velocity) should be continuous as well. This is called *C1 continuity* (or continuity of velocity and momentum). The perceived quality and realism of an animated character’s movement improves as we move to higher- and higher-order continuity. For example, we might want to achieve *C2 continuity*, in which the second derivatives of the motion paths (acceleration curves) are also continuous.

Strict mathematical continuity up to C1 or higher is often infeasible to achieve. However, LERP-based animation blending can be applied to achieve a reasonably pleasing form of C0 motion continuity. It usually also does a pretty good job of approximating C1 continuity. When applied to transitions between clips in this manner, LERP blending is sometimes called *cross-fading*. LERP blending can introduce unwanted artifacts, such as the dreaded “sliding feet” problem, so it must be applied judiciously.

To cross-fade between two animations, we overlap the timelines of the two clips by some reasonable amount, and then blend the two clips together. The blend percentage β starts at zero at time t_{start} , meaning that we see only clip A

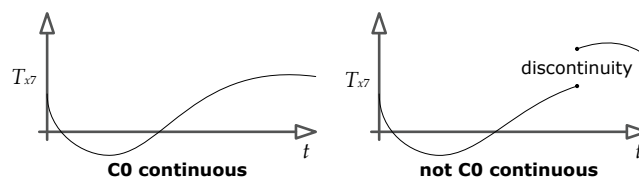


Figure 12.29. The channel function on the left has C0 continuity, while the path on the right does not.

when the cross-fade begins. We gradually increase β until it reaches a value of one at time t_{end} . At this point only clip B will be visible, and we can retire clip A altogether. The time interval over which the cross-fade occurs ($\Delta t_{\text{blend}} = t_{\text{end}} - t_{\text{start}}$) is sometimes called the *blend time*.

Types of Cross-Fades

There are two common ways to perform a cross-blended transition:

- *Smooth transition.* Clips A and B both play simultaneously as β increases from zero to one. For this to work well, the two clips must be looping animations, and their timelines must be synchronized so that the positions of the legs and arms in one clip match up roughly with their positions in the other clip. (If this is not done, the cross-fade will often look totally unnatural.) This technique is illustrated in Figure 12.30.
- *Frozen transition.* The local clock of clip A is stopped at the moment clip B starts playing. Thus, the pose of the skeleton from clip A is frozen while clip B gradually takes over the movement. This kind of transitional blend works well when the two clips are unrelated and cannot be time-synchronized, as they must be when performing a *smooth* transition. This approach is depicted in Figure 12.31.

We can also control how the blend factor β varies during the transition. In Figure 12.30 and Figure 12.31, the blend factor varied linearly with time. To achieve an even smoother transition, we could vary β according to a cubic function of time, such as a one-dimensional Bézier. When such a curve is applied to a currently running clip that is being blended out, it is known as an *ease-out curve*; when it is applied to a new clip that is being blended in, it is known as an *ease-in curve*. This is shown in Figure 12.32.

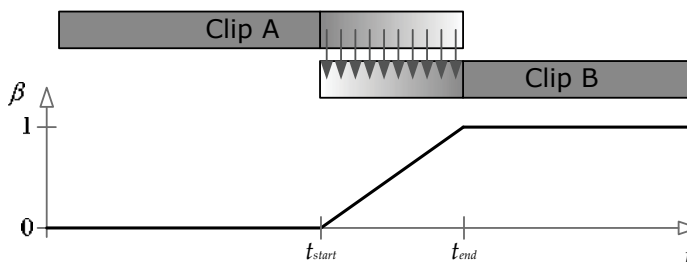


Figure 12.30. A smooth transition, in which the local clocks of both clips keep running during the transition.

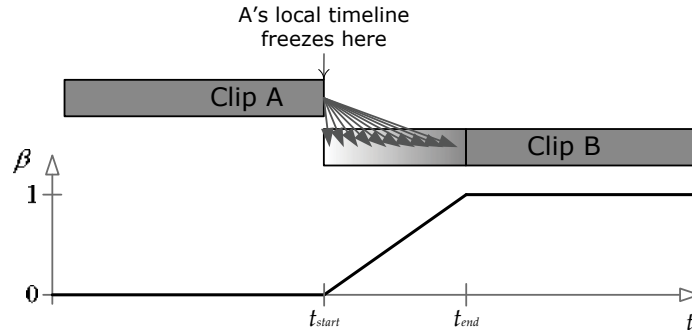


Figure 12.31. A frozen transition, in which clip A's local clock is stopped during the transition.

The equation for a Bézier ease-in/ease-out curve is given below. It returns the value of β at any time t within the blend interval. β_{start} is the blend factor at the start of the blend interval t_{start} , and β_{end} is the final blend factor at time t_{end} . The parameter u is the *normalized time* between t_{start} and t_{end} , and for convenience we'll also define $v = 1 - u$ (the *inverse normalized time*). Note that the Bézier tangents T_{start} and T_{end} are taken to be equal to the corresponding blend factors β_{start} and β_{end} , because this yields a well-behaved curve for our purposes:

$$\begin{aligned} \text{let } u &= \left(\frac{t - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}} \right) \\ \text{and } v &= 1 - u. \\ \beta(t) &= (v^3)\beta_{\text{start}} + (3v^2u)T_{\text{start}} + (3vu^2)T_{\text{end}} + (u^3)\beta_{\text{end}} \\ &= (v^3 + 3v^2u)\beta_{\text{start}} + (3vu^2 + u^3)\beta_{\text{end}}. \end{aligned}$$

Core Poses

This is an appropriate time to mention that motion continuity can actually be

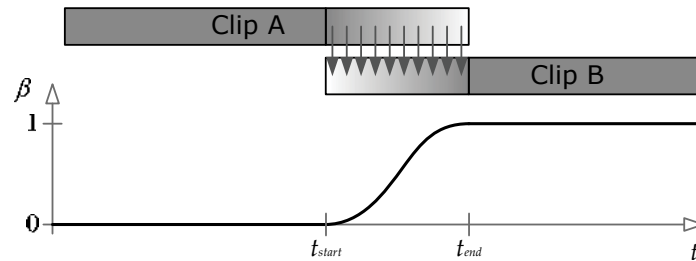


Figure 12.32. A smooth transition, with a cubic ease-in/ease-out curve applied to the blend factor.

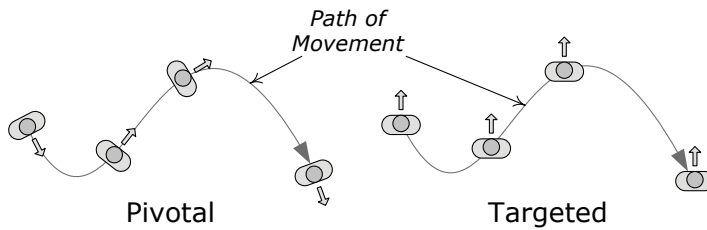


Figure 12.33. In pivotal movement, the character faces the direction she is moving and pivots about her vertical axis to turn. In targeted movement, the movement direction need not match the facing direction.

achieved *without* blending if the animator ensures that the last pose in any given clip matches the first pose of the clip that follows it. In practice, animators often decide upon a set of *core poses*—for example, we might have a core pose for standing upright, one for crouching, one for lying prone and so on. By making sure that the character starts in one of these core poses at the beginning of every clip and returns to a core pose at the end, C0 continuity can be achieved by simply ensuring that the core poses match when animations are spliced together. C1 or higher-order motion continuity can also be achieved by ensuring that the character’s movement at the end of one clip smoothly transitions into the motion at the start of the next clip. This can be achieved by authoring a single smooth animation and then breaking it into two or more clips.

12.6.2.3 Directional Locomotion

LERP-based animation blending is often applied to character locomotion. When a real human being walks or runs, he can change the direction in which he is moving in two basic ways: First, he can turn his entire body to change direction, in which case he always faces in the direction he’s moving. I’ll call this *pivotal movement*, because the person pivots about his vertical axis when he turns. Second, he can keep facing in one direction while walking forward, backward or sideways (known as *strafing* in the gaming world) in order to move in a direction that is independent of his facing direction. I’ll call this *targeted movement*, because it is often used in order to keep one’s eye—or one’s weapon—trained on a target while moving. These two movement styles are illustrated in Figure 12.33.

Targeted Movement

To implement *targeted movement*, the animator authors three separate looping

animation clips—one moving forward, one strafing to the left, and one strafing to the right. I'll call these *directional locomotion clips*. The three directional clips are arranged around the circumference of a semicircle, with forward at 0 degrees, left at 90 degrees and right at -90 degrees. With the character's facing direction fixed at 0 degrees, we find the desired movement direction on the semicircle, select the two adjacent movement animations and blend them together via LERP-based blending. The blend percentage β is determined by how close the angle of movement is to the angles of two adjacent clips. This is illustrated in Figure 12.34.

Note that we did not include backward movement in our blend, for a full circular blend. This is because blending between a sideways strafe and a backward run cannot be made to look natural in general. The problem is that when strafing to the left, the character usually crosses its right foot in front of its left so that the blend into the pure forward run animation looks correct. Likewise, the right strafe is usually authored with the left foot crossing in front of the right. When we try to blend such strafe animations directly into a backward run, one leg will start to pass through the other, which looks extremely awkward and unnatural. There are a number of ways to solve this problem. One feasible approach is to define two hemispherical blends, one for forward motion and one for backward motion, each with strafe animations that have been crafted to work properly when blended with the corresponding straight run. When passing from one hemisphere to the other, we can play some kind of explicit transition animation so that the character has a chance to adjust its gait and leg crossing appropriately.

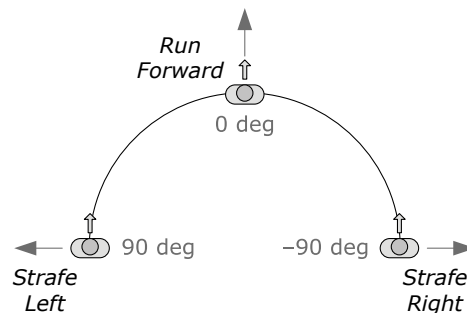


Figure 12.34. Targeted movement can be implemented by blending together looping locomotion clips that move in each of the four principal directions.

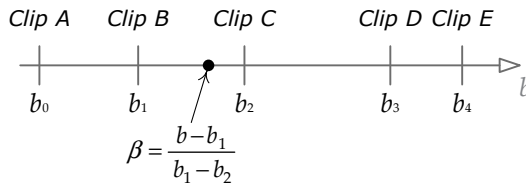


Figure 12.35. A generalized linear blend between N animation clips.

Pivotal Movement

To implement *pivotal movement*, we can simply play the forward locomotion loop while rotating the entire character about its vertical axis to make it turn. Pivotal movement looks more natural if the character’s body doesn’t remain bolt upright when it is turning—real humans tend to lean into their turns a little bit. We could try slightly tilting the vertical axis of the character as a whole, but that would cause problems with the inner foot sinking into the ground while the outer foot comes off the ground. A more natural-looking result can be achieved by animating three variations on the basic forward walk or run—one going perfectly straight, one making an extreme left turn and one making an extreme right turn. We can then LERP-blend between the straight clip and the extreme left turn clip to implement any desired lean angle.

12.6.3 Complex LERP Blends

In a real game engine, characters make use of a wide range of complex blends for various purposes. It can be convenient to “prepackage” certain commonly used types of complex blends for ease of use. In the following sections, we’ll investigate a few popular types of prepackaged complex blends.

12.6.3.1 Generalized One-Dimensional LERP Blending

LERP blending can be easily extended to more than two animation clips, using a technique I call *one-dimensional LERP blending*. We define a new blend parameter b that lies in any linear range desired (e.g., from -1 to $+1$, or from 0 to 1 , or even from 27 to 136). Any number of clips can be positioned at arbitrary points along this range, as shown in Figure 12.35. For any given value of b , we select the two clips immediately adjacent to it and blend them together using Equation (12.5). If the two adjacent clips lie at points b_1 and b_2 , then the blend percentage β can be determined using a technique analogous to that used in Equation (12.14), as follows:

$$\beta(t) = \frac{b - b_1}{b_2 - b_1}. \quad (12.15)$$

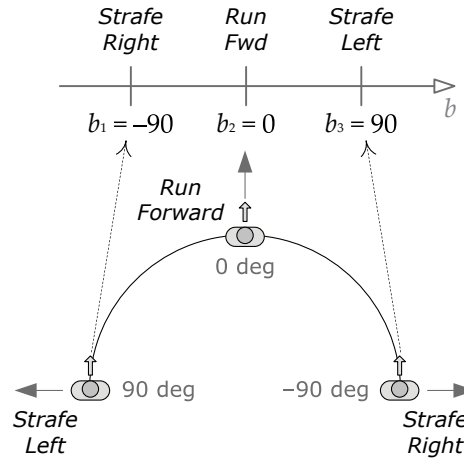


Figure 12.36. The directional clips used in targeted movement can be thought of as a special case of one-dimensional LERP blending.

Targeted movement is just a special case of one-dimensional LERP blending. We simply straighten out the circle on which the directional animation clips were placed and use the movement direction angle θ as the parameter b (with a range of -90 to 90 degrees). Any number of animation clips can be placed onto this blend range at arbitrary angles. This is shown in Figure 12.36.

12.6.3.2 Simple Two-Dimensional LERP Blending

Sometimes we would like to smoothly vary *two* aspects of a character's motion simultaneously. For example, we might want the character to be capable of aiming his weapon vertically and horizontally. Or we might want to allow our character to vary her pace length and the separation of her feet as she moves. We can extend one-dimensional LERP blending to two dimensions in order to achieve these kinds of effects.

If we know that our 2D blend involves only four animation clips, and if those clips are positioned at the four corners of a square region, then we can find a blended pose by performing two 1D blends. Our generalized blend factor b becomes a two-dimensional blend vector $\mathbf{b} = [b_x \ b_y]$. If $\mathbf{b}$ lies within the square region bounded by our four clips, we can find the resulting pose by following these steps:

1. Using the horizontal blend factor b_x , find two intermediate poses, one between the top two animation clips and one between the bottom two clips. These two poses can be found by performing two simple one-

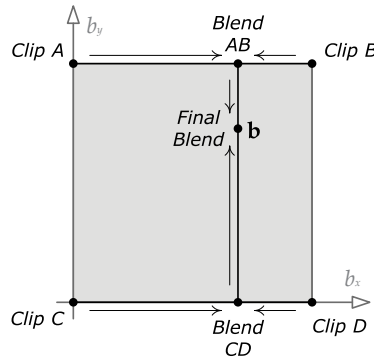


Figure 12.37. A simple formulation for 2D animation blending between four clips at the corners of a square region.

dimensional LERP blends.

2. Using the vertical blend factor b_y , find the final pose by LERP-blending the two intermediate poses together.

This technique is illustrated in Figure 12.37.

12.6.3.3 Triangular Two-Dimensional LERP Blending

The simple 2D blending technique we investigated in the previous section only works when the animation clips we wish to blend lie at the corners of a rectangular region. How can we blend between an arbitrary number of clips positioned at arbitrary locations in our 2D blend space?

Let's imagine that we have three animation clips that we wish to blend together. Each clip, designated by the index i , corresponds to a particular blend coordinate $\mathbf{b}_i = [b_{ix} \ b_{iy}]$ in our two-dimensional blend space; these three blend coordinates form a triangle within the blend space. Each of the three clips defines a set of joint poses $\{(\mathbf{P}_i)_j\}_{j=0}^{N-1}$, where $(\mathbf{P}_i)_j$ is the pose of joint j as defined by clip i , and N is the number of joints in the skeleton. We wish to find the interpolated pose of the skeleton corresponding to an arbitrary point $\mathbf{b}$ within the triangle, as illustrated in Figure 12.38.

But how can we calculate a LERP blend between three animation clips? Thankfully, the answer is simple: the LERP function can actually operate on any number of inputs, because it is really just a *weighted average*. As with any weighted average, the weights must add to one. In the case of a two-input LERP blend, we used the weights β and $(1 - \beta)$, which of course add to one. For a three-input LERP, we simply use three weights, α , β and $\gamma = (1 - \alpha - \beta)$.

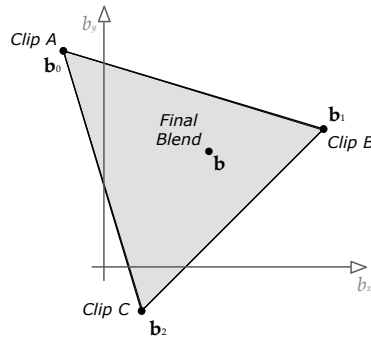


Figure 12.38. Two-dimensional animation blending between three animation clips.

Then we calculate the LERP as follows:

$$(\mathbf{P}_{\text{LERP}})_j = \alpha (\mathbf{P}_0)_j + \beta (\mathbf{P}_1)_j + \gamma (\mathbf{P}_2)_j. \quad (12.16)$$

Given the two-dimensional blend vector $\mathbf{b}$, we find the blend weights α , β and γ by finding the *barycentric coordinates* of the point $\mathbf{b}$ relative to the triangle formed by the three clips in two-dimensional blend space (http://en.wikipedia.org/wiki/Barycentric_coordinates_%28mathematics%29). In general, the barycentric coordinates of a point $\mathbf{b}$ within a triangle with vertices $\mathbf{b}_1$, $\mathbf{b}_2$ and $\mathbf{b}_3$ are three scalar values (α, β, γ) that satisfy the relations

$$\mathbf{b} = \alpha \mathbf{b}_0 + \beta \mathbf{b}_1 + \gamma \mathbf{b}_2, \quad (12.17)$$

and

$$\alpha + \beta + \gamma = 1.$$

These are exactly the weights we seek for our three-clip weighted average. Barycentric coordinates are illustrated in Figure 12.39.

Note that plugging the barycentric coordinate $(1, 0, 0)$ into Equation (12.17) yields $\mathbf{b}_0$, while $(0, 1, 0)$ gives us $\mathbf{b}_1$ and $(0, 0, 1)$ produces $\mathbf{b}_2$. Likewise, plugging these blend weights into Equation (12.16) gives us poses $(\mathbf{P}_0)_j$, $(\mathbf{P}_1)_j$ and $(\mathbf{P}_2)_j$ for each joint j , respectively. Furthermore, the barycentric coordinate $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ lies at the centroid of the triangle and gives us an *equal* blend between the three poses. This is exactly what we'd expect.

12.6.3.4 Generalized Two-Dimensional LERP Blending

The barycentric coordinate technique can be extended to an arbitrary number of animation clips positioned at arbitrary locations within the two-dimensional blend space. We won't describe it in its entirety here, but the basic idea is to use a technique known as *Delaunay triangulation* (<http://en.wikipedia.org/>

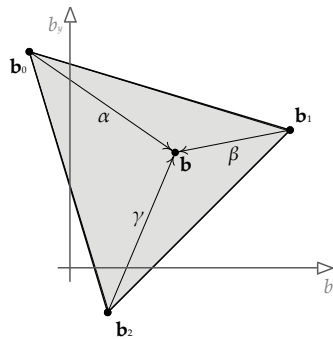


Figure 12.39. Various barycentric coordinates within a triangle.

wiki/Delaunay_triangulation) to find a set of triangles given the positions of the various animation clips $\mathbf{b}_i$. Once the triangles have been determined, we can find the triangle that encloses the desired point $\mathbf{b}$ and then perform a three-clip LERP blend as described above. This technique was used in FIFA soccer by EA Sports in Vancouver, implemented within their proprietary “ANT” animation framework. It is shown in Figure 12.40.

12.6.4 Partial-Skeleton Blending

A human being can control different parts of his or her body independently. For example, I can wave my right arm while walking and pointing at something with my left arm. One way to implement this kind of movement in a

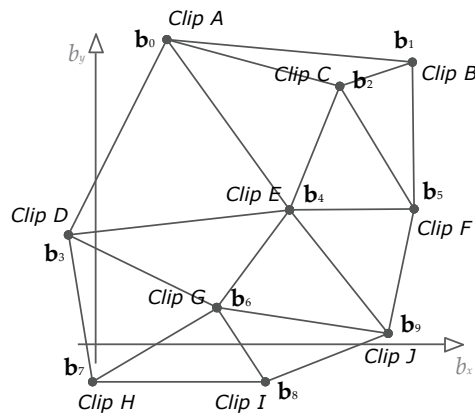


Figure 12.40. Delaunay triangulation between an arbitrary number of animation clips positioned at arbitrary locations in two-dimensional blend space.

game is via a technique known as *partial-skeleton blending*.

Recall from Equations (12.5) and (12.6) that when doing regular LERP blending, the same blend percentage β was used for every joint in the skeleton. Partial-skeleton blending extends this idea by permitting the blend percentage to vary on a per-joint basis. In other words, for each joint j , we define a separate blend percentage β_j . The set of all blend percentages for the entire skeleton $\{\beta_j\}_{j=0}^{N-1}$ is sometimes called a *blend mask* because it can be used to “mask out” certain joints by setting their blend percentages to zero.

As an example, let’s say we want our character to wave at someone using his right arm and hand. Moreover, we want him to be able to wave whether he’s walking, running or standing still. To implement this using partial blending, the animator defines three full-body animations: *Walk*, *Run* and *Stand*. The animator also creates a single waving animation, *Wave*. A blend mask is created in which the blend percentages are zero everywhere except for the right shoulder, elbow, wrist and finger joints, where they are equal to one:

$$\beta_j = \begin{cases} 1 & \text{when } j \text{ within right arm,} \\ 0 & \text{otherwise.} \end{cases}$$

When *Walk*, *Run* or *Stand* is LERP-blended with *Wave* using this blend mask, the result is a character who appears to be walking, running or standing while waving his right arm.

Partial blending is useful, but it has a tendency to make a character’s movements look unnatural. This occurs for two basic reasons:

- An abrupt change in the per-joint blend factors can cause the movements of one part of the body to appear disconnected from the rest of the body. In our example, the blend factors change abruptly at the right shoulder joint. Hence the animation of the upper spine, neck and head are being driven by one animation, while the right shoulder and arm joints are being entirely driven by a different animation. This can look odd. The problem can be mitigated somewhat by gradually changing the blend factors rather than doing it abruptly. (In our example, we might select a blend percentage of 0.9 at the right shoulder, 0.5 on the upper spine and 0.2 on the neck and mid-spine.)
- The movements of a real human body are never totally independent. For example, one would expect a person’s wave to look more “bouncy” and out of control when he or she is running than when he or she is standing still. Yet with partial blending, the right arm’s animation will be identical no matter what the rest of the body is doing. This problem is difficult to

overcome using partial blending. Instead, many game developers have turned to a more natural-looking technique known as *additive blending*.

12.6.5 Additive Blending

Additive blending approaches the problem of combining animations in a totally new way. It introduces a new kind of animation called a *difference clip*, which, as its name implies, represents the difference between two regular animation clips. A difference clip can be added onto a regular animation clip in order to produce interesting variations in the pose and movement of the character. In essence, a difference clip encodes the *changes* that need to be made to one pose in order to transform it into another pose. Difference clips are often called *additive animation clips* in the game industry. We'll stick with the term *difference clip* in this book because it more accurately describes what is going on.

Consider two input clips called the *source clip* (S) and the *reference clip* (R). Conceptually, the difference clip is $D = S - R$. If a difference clip D is added to its original reference clip, we get back the source clip ($S = D + R$). We can also generate animations that are partway between R and S by adding a percentage of D to R, in much the same way that LERP blending finds intermediate animations between two extremes. However, the real beauty of the additive blending technique is that once a difference clip has been created, it can be added to other unrelated clips, not just to the original reference clip. We'll call these animations *target clips* and denote them with the symbol T.

As an example, if the reference clip has the character running normally and the source clip has him running in a tired manner, then the difference clip will contain only the changes necessary to make the character look "tired" while running. If this difference clip is now applied to a clip of the character walking, the resulting animation can make the character look tired while walking. A whole host of interesting and very natural-looking animations can be created by adding a single difference clip onto various "regular" animation clips, or a collection of difference clips can be created, each of which produces a different effect when added to a single target animation.

12.6.5.1 Mathematical Formulation

A difference animation D is defined as the difference between some source animation S and some reference animation R. So conceptually, the difference pose (at a single point in time) is $D = S - R$. Of course, we're dealing with joint poses, not scalar quantities, so we cannot simply subtract the poses. In general, a joint pose is a 4×4 affine transformation matrix that transforms

points and vectors from the child joint's local space to the space of its parent joint. The matrix equivalent of subtraction is multiplication by the inverse matrix. So given the source pose $\mathbf{S}_j$ and the reference pose $\mathbf{R}_j$ for any joint j in the skeleton, we can define the difference pose $\mathbf{D}_j$ at that joint as follows. (For this discussion, we'll drop the $C \rightarrow P$ or $j \rightarrow p(j)$ subscript, as it is understood that we are dealing with child-to-parent pose matrices.)

$$\mathbf{D}_j = \mathbf{S}_j \mathbf{R}_j^{-1}.$$

"Adding" a difference pose $\mathbf{D}_j$ onto a target pose $\mathbf{T}_j$ yields a new additive pose $\mathbf{A}_j$. This is achieved by simply concatenating the difference transform and the target transform as follows:

$$\mathbf{A}_j = \mathbf{D}_j \mathbf{T}_j = \left(\mathbf{S}_j \mathbf{R}_j^{-1} \right) \mathbf{T}_j. \quad (12.18)$$

We can verify that this is correct by looking at what happens when the difference pose is "added" back onto the original reference pose:

$$\begin{aligned} \mathbf{A}_j &= \mathbf{D}_j \mathbf{R}_j \\ &= \mathbf{S}_j \mathbf{R}_j^{-1} \mathbf{R}_j \\ &= \mathbf{S}_j. \end{aligned}$$

In other words, adding the difference animation $\mathbf{D}$ back onto the original reference animation $\mathbf{R}$ yields the source animation $\mathbf{S}$, as we'd expect.

Temporal Interpolation of Difference Clips

As we learned in Section 12.4.1.1, game animations are almost never sampled on integer frame indices. To find a pose at an arbitrary time t , we must often *temporally interpolate* between adjacent pose samples at times t_1 and t_2 . Thankfully, difference clips can be temporally interpolated just like their non-additive counterparts. We can simply apply Equations (12.12) and (12.14) directly to our difference clips as if they were ordinary animations.

Note that a difference animation can only be found when the input clips $\mathbf{S}$ and $\mathbf{R}$ are of the same duration. Otherwise there would be a period of time during which either $\mathbf{S}$ or $\mathbf{R}$ is undefined, meaning $\mathbf{D}$ would be undefined as well.

Additive Blend Percentage

In games, we often wish to blend in only a percentage of a difference animation to achieve varying degrees of the effect it produces. For example, if a difference clip causes the character to turn his head 80 degrees to the right, blending in

50% of the difference clip should make him turn his head only 40 degrees to the right.

To accomplish this, we turn once again to our old friend LERP. We wish to interpolate between the unaltered target animation and the new animation that would result from a full application of the difference animation. To do this, we extend Equation (12.18) as follows:

$$\begin{aligned} \mathbf{A}_j &= \text{LERP}(\mathbf{T}_j, \mathbf{D}_j \mathbf{T}_j, \beta) \\ &= (1 - \beta) (\mathbf{T}_j) + \beta (\mathbf{D}_j \mathbf{T}_j). \end{aligned} \quad (12.19)$$

As we saw in Chapter 5, we cannot LERP matrices directly. So Equation (11.16) must be broken down into three separate interpolations for S, Q and T, just as we did in Equations (12.7) through (12.11).

12.6.5.2 Additive Blending versus Partial Blending

Additive blending is similar in some ways to partial blending. For example, we can take the difference between a standing clip and a clip of standing while waving the right arm. The result will be almost the same as using a partial blend to make the right arm wave. However, additive blends suffer less from the “disconnected” look of animations combined via partial blending. This is because, with an additive blend, we are not replacing the animation for a subset of joints or interpolating between two potentially unrelated poses. Rather, we are adding movement to the original animation—possibly across the entire skeleton. In effect, a difference animation “knows” how to change a character’s pose in order to get him to do something specific, like being tired, aiming his head in a certain direction, or waving his arm. These changes can be applied to a reasonably wide variety of animations, and the result often looks very natural.

12.6.5.3 Limitations of Additive Blending

Of course, additive animation is not a silver bullet. Because it adds movement to an existing animation, it can have a tendency to over-rotate the joints in the skeleton, especially when multiple difference clips are applied simultaneously. As a simple example, imagine a target animation in which the character’s left arm is bent at a 90 degree angle. If we add a difference animation that also rotates the elbow by 90 degrees, then the net effect would be to rotate the arm by $90 + 90 = 180$ degrees. This would cause the lower arm to interpenetrate the upper arm—not a comfortable position for most individuals!

Clearly we must be careful when selecting the reference clip and also when choosing the target clips to which to apply it. Here are some simple rules of thumb:

- Keep hip rotations to a minimum in the reference clip.
- The shoulder and elbow joints should usually be in neutral poses in the reference clip to minimize over-rotation of the arms when the difference clip is added to other targets.
- Animators should create a new difference animation for each core pose (e.g., standing upright, crouched down, lying prone, etc.). This allows the animator to account for the way in which a real human would move when in each of these stances.

These rules of thumb can be a helpful starting point, but the only way to really learn how to create and apply difference clips is by trial and error or by apprenticing with animators or engineers who have experience creating and applying difference animations. If your team hasn't used additive blending in the past, expect to spend a significant amount of time learning the art of additive blending.

12.6.6 Applications of Additive Blending

12.6.6.1 Stance Variation

One particularly striking application of additive blending is *stance variation*. For each desired stance, the animator creates a one-frame difference animation. When one of these single-frame clips is additively blended with a base animation, it causes the entire stance of the character to change drastically while he continues to perform the fundamental action he's supposed to perform. This idea is illustrated in Figure 12.41.

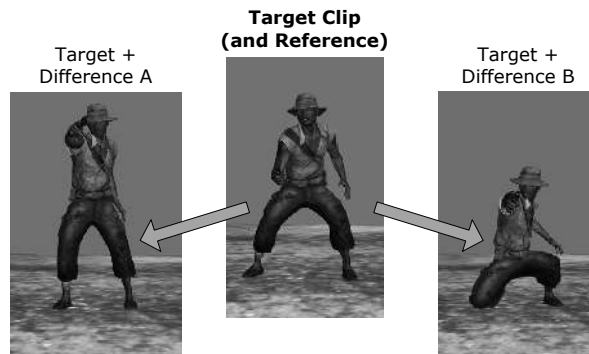


Figure 12.41. Two single-frame difference animations A and B can cause a target animation clip to assume two totally different stances. (Character from *Uncharted: Drake's Fortune*, © 2007/® SIE. Created and developed by Naughty Dog.)

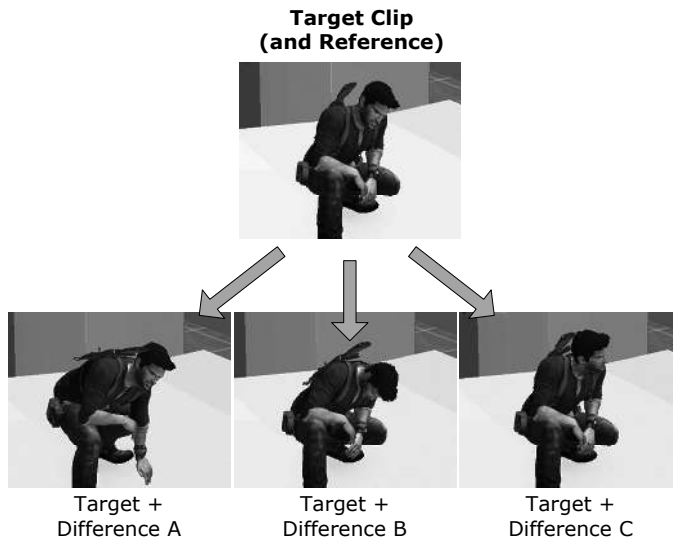


Figure 12.42. Additive blends can be used to add variation to a repetitive idle animation. Images courtesy of Naughty Dog, Inc., © 2014/™ SIE.

12.6.6.2 Locomotion Noise

Real humans don't run exactly the same way with every footfall—there is variation in their movement over time. This is especially true if the person is distracted (for example, by attacking enemies). Additive blending can be used to layer randomness, or reactions to distractions, on top of an otherwise entirely repetitive locomotion cycle. This is illustrated in Figure 12.42.

12.6.6.3 Aim and Look-At

Another common use for additive blending is to permit the character to look around or to aim his weapon. To accomplish this, the character is first animated doing some action, such as running, with his head or weapon facing straight ahead. Then the animator changes the direction of the head or the aim of the weapon to the extreme right and saves off a one-frame or multi-frame difference animation. This process is repeated for the extreme left, up and down directions. These four difference animations can then be additively blended onto the original straight-ahead animation clip, causing the character to aim right, left, up, down or anywhere in between.

The angle of the aim is governed by the additive blend factor of each clip. For example, blending in 100% of the right additive causes the character to aim as far right as possible. Blending 50% of the left additive causes him to aim at

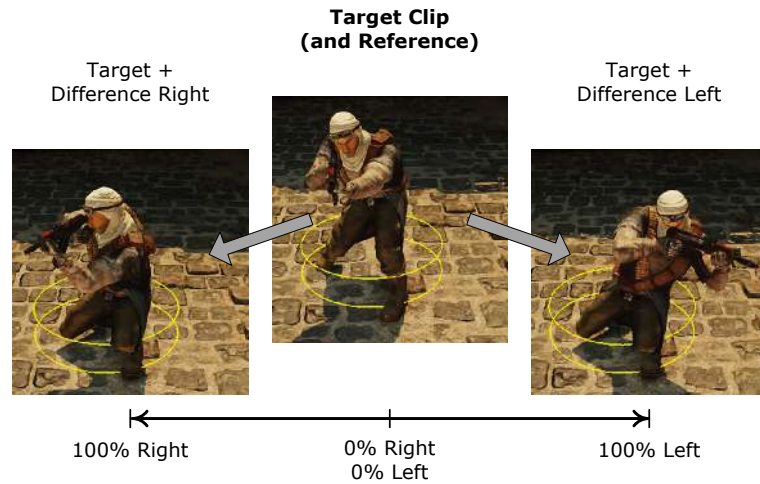


Figure 12.43. Additive blending can be used to aim a weapon. Screenshots courtesy of Naughty Dog, Inc., © 2014/™ SIE.

an angle that is one-half of his leftmost aim. We can also combine this with an up or down additive to aim diagonally. This is demonstrated in Figure 12.43.

12.6.6.4 Overloading the Time Axis

It's interesting to note that the time axis of an animation clip needn't be used to represent time. For example, a three-frame animation clip could be used to provide three aim poses to the engine—a left aim pose on frame 1, a forward aim pose on frame 2 and a right aim pose on frame 3. To make the character aim to the right, we can simply fix the local clock of the aim animation on frame 3. To perform a 50% blend between aiming forward and aiming right, we can dial in frame 2.5. This is a great example of leveraging existing features of the engine for new purposes.

12.7 Post-Processing

Once a skeleton has been posed by one or more animation clips and the results have been blended together using linear interpolation or additive blending, it is often necessary to modify the pose prior to rendering the character. This is called *animation post-processing*. In this section, we'll look at a few of the most common kinds of animation post-processing.

12.7.1 Procedural Animations

A *procedural animation* is any animation generated at runtime rather than being driven by data exported from an animation tool such as Maya. Sometimes, hand-animated clips are used to pose the skeleton initially, and then the pose is modified in some way via procedural animation as a post-processing step. A procedural animation can also be used as an input to the system in place of a hand-animated clip.

For example, imagine that a regular animation clip is used to make a vehicle appear to be bouncing up and down on the terrain as it moves. The direction in which the vehicle travels is under player control. We would like to adjust the rotation of the front wheels and steering wheel so that they move convincingly when the vehicle is turning. This can be done by post-processing the pose generated by the animation. Let's assume that the original animation has the front tires pointing straight ahead and the steering wheel in a neutral position. We can use the current angle of turn to create a quaternion about the vertical axis that will deflect the front tires by the desired amount. This quaternion can be multiplied with the front tire joints' Q channel to produce the final pose of the tires. Likewise, we can generate a quaternion about the axis of the steering column and multiply it into the steering wheel joint's Q channel to deflect it. These adjustments are made to the *local pose*, prior to global pose calculation and matrix palette generation (see Section 12.5).

As another example, let's say that we wish to make the trees and bushes in our game world sway naturally in the wind and get brushed aside when characters move through them. We can do this by modeling the trees and bushes as skinned meshes with simple skeletons. Procedural animation can be used, in place of or in addition to hand-animated clips, to cause the joints to move in a natural-looking way. We might apply one or more sinusoids, or a Perlin noise function, to the rotation of various joints to make them sway in the breeze, and when a character moves through a region containing a bush or grass, we can deflect its root joint quaternion radially outward to make it appear to be pushed over by the character.

12.7.2 Inverse Kinematics

Let's say we have an animation clip in which a character leans over to pick up an object from the ground. In Maya, the clip looks great, but in our production game level, the ground is not perfectly flat, so sometimes the character's hand misses the object or appears to pass through it. In this case, we would like to adjust the final pose of the skeleton so that the hand lines up exactly with the target object. A technique known as *inverse kinematics* (IK) can be used to make

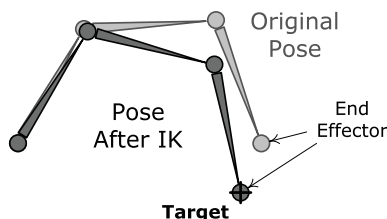


Figure 12.44. Inverse kinematics attempts to bring an end effector joint into a target global pose by minimizing the error between them.

this happen.

A regular animation clip is an example of *forward kinematics* (FK). In forward kinematics, the input is a set of local joint poses, and the output is a global pose and a skinning matrix for each joint. Inverse kinematics goes in the other direction: The input is the desired global pose of a single joint, which is known as the *end effector*. We solve for the *local* poses of other joints in the skeleton that will bring the end effector to the desired location.

Mathematically, IK boils down to an *error minimization* problem. As with most minimization problems, there might be one solution, many or none at all. This makes intuitive sense: If I try to reach a doorknob that is on the other side of the room, I won't be able to reach it without walking over to it. IK works best when the skeleton starts out in a pose that is reasonably close to the desired target. This helps the algorithm to focus in on the "closest" solution and to do so in a reasonable amount of processing time. Figure 12.44 shows IK in action.

Imagine a two-joint skeleton, each of which can rotate only about a single axis. The rotation of these two joints can be described by a two-dimensional angle vector $\theta = [\theta_1 \ \theta_2]$. The set of all possible angles for our two joints forms a two-dimensional space called *configuration space*. Obviously, for more complex skeletons with more degrees of freedom per joint, configuration space becomes multidimensional, but the concepts described here work equally well no matter how many dimensions we have.

Now imagine plotting a three-dimensional graph, where for each combination of joint rotations (i.e., for each point in our two-dimensional configuration space), we plot the distance from the end effector to the desired target. An example of this kind of plot is shown in Figure 12.45. The "valleys" in this three-dimensional surface represent regions in which the end effector is as close as possible to the target. When the height of the surface is zero, the end effector has reached its target. Inverse kinematics, then, attempts to find

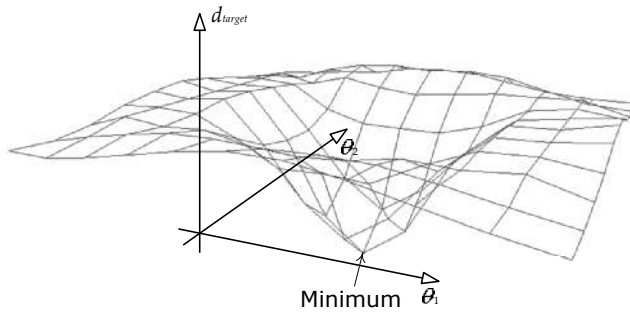


Figure 12.45. A three-dimensional plot of the distance from the end effector to the target for each point in two-dimensional configuration space. IK finds the local minimum.

minima (low points) on this surface.

We won't get into the details of solving the IK minimization problem here. You can read more about IK at http://en.wikipedia.org/wiki/Inverse_kinematics and in Jason Weber's article, "Constrained Inverse Kinematics" [47].

12.7.3 Rag Dolls

A character's body goes limp when he dies or becomes unconscious. In such situations, we want the body to react in a physically realistic way with its surroundings. To do this, we can use a *rag doll*. A rag doll is a collection of physically simulated rigid bodies, each one representing a semi-rigid part of the character's body, such as his lower arm or his upper leg. The rigid bodies are constrained to one another at the joints of the character in such a way as to produce natural-looking "lifeless" body movement. The positions and orientations of the rigid bodies are determined by the physics system and are then used to drive the positions and orientations of certain key joints in the character's skeleton. The transfer of data from the physics system to the skeleton is typically done as a post-processing step.

To really understand rag doll physics, we must first have an understanding of how the collision and physics systems work. Rag dolls are covered in more detail in Sections 13.4.8.7 and 13.5.3.8.

12.8 Compression Techniques

Animation data can take up a lot of memory. A single joint pose might be composed of ten floating-point channels (three for translation, four for rotation and

up to three more for scale). Assuming each channel contains a 4-byte floating-point value, a one-second clip sampled at 30 samples per second would occupy $4 \text{ bytes} \times 10 \text{ channels} \times 30 \text{ samples/second} = 1200 \text{ bytes per joint per second}$, or a data rate of about 1.17 KiB per joint per second. For a 100-joint skeleton (which is small by today's standards), an uncompressed animation clip would occupy 117 KiB per joint per second. If our game contained 1,000 seconds of animation (which is on the low side for a modern game), the entire dataset would occupy a whopping 114.4 MiB. That's quite a lot, considering that a PlayStation 3 has only 256 MiB of main RAM and 256 MiB of video RAM. Sure, the PS4 has 8 GiB of RAM. But even so—we would rather have much richer animations with a lot more variety than waste memory unnecessarily. Therefore, game engineers invest a significant amount of effort into compressing animation data in order to permit the maximum richness and variety of movement at the minimum memory cost.

12.8.1 Channel Omission

One simple way to reduce the size of an animation clip is to omit channels that are irrelevant. Many characters do not require nonuniform scaling, so the three scale channels can be reduced to a single uniform scale channel. In some games, the scale channel can actually be omitted altogether for all joints (except possibly the joints in the face). The bones of a humanoid character generally cannot stretch, so translation can often be omitted for all joints except the root, the facial joints and sometimes the collar bones. Finally, because quaternions are always normalized, we can store only three components per quat (e.g., x , y and z) and reconstruct the fourth component (e.g., w) at run-time.

As a further optimization, channels whose pose does not change over the course of the entire animation can be stored as a single sample at time $t = 0$ plus a single bit indicating that the channel is constant for all other values of t .

Channel omission can significantly reduce the size of an animation clip. A 100-joint character with no scale and no translation requires only 303 channels—three channels for the quaternions at each joint, plus three channels for the root joint's translation. Compare this to the 1,000 channels that would be required if all ten channels were included for all 100 joints.

12.8.2 Quantization

Another way to reduce the size of an animation is to reduce the size of each channel. A floating-point value is normally stored in 32-bit IEEE format. This format provides 23 bits of precision in the mantissa and an 8-bit exponent.

However, it's often not necessary to retain that kind of precision and range in an animation clip. When storing a quaternion, the channel values are guaranteed to lie in the range $[-1, 1]$. At a magnitude of 1, the exponent of a 32-bit IEEE float is zero, and 23 bits of precision give us accuracy down to the seventh decimal place. Experience shows that a quaternion can be encoded well with only 16 bits of precision, so we're really wasting 16 bits per channel if we store our quats using 32-bit floats.

Converting a 32-bit IEEE float into an n -bit integer representation is called *quantization*. There are actually two components to this operation: *Encoding* is the process of converting the original floating-point value to a quantized integer representation. *Decoding* is the process of recovering an approximation to the original floating-point value from the quantized integer. (We can only recover an *approximation* to the original data—quantization is a *lossy* compression method because it effectively reduces the number of bits of precision used to represent the value.)

To encode a floating-point value as an integer, we first divide the valid range of possible input values into N equally sized *intervals*. We then determine within which interval a particular floating-point value lies and represent that value by the *integer index* of its interval. To decode this quantized value, we simply convert the integer index into floating-point format and shift and scale it back into the original range. N is usually chosen to correspond to the range of possible integer values that can be represented by an n -bit integer. For example, if we're encoding a 32-bit floating-point value as a 16-bit integer, the number of intervals would be $N = 2^{16} = 65,536$.

Jonathan Blow wrote an excellent article on the topic of floating-point scalar quantization in the *Inner Product* column of Game Developer Magazine, available at <https://bit.ly/2J92oiU>. The article presents two ways to map a floating-point value to an interval during the encoding process: We can either *truncate* the float to the next lowest interval boundary (*T encoding*), or we can *round* the float to the center of the enclosing interval (*R encoding*). Likewise, it describes two approaches to reconstructing the floating-point value from its integer representation: We can either return the value of the *left-hand side* of the interval to which our original value was mapped (*L reconstruction*), or we can return the value of the center of the interval (*C reconstruction*). This gives us four possible encode/decode methods: TL, TC, RL and RC. Of these, TL and RC are to be avoided because they tend to remove or add energy to the dataset, which can often have disastrous effects. TC has the benefit of being the most efficient method in terms of bandwidth, but it suffers from a severe problem—there is no way to represent the value zero exactly. (If you encode 0.0f, it becomes a small positive value when decoded.) RL is therefore usually

the best choice and is the method we'll demonstrate here.

The article only talks about quantizing positive floating-point values, and in the examples, the input range is assumed to be $[0, 1]$ for simplicity. However, we can always shift and scale any floating-point range into the range $[0, 1]$. For example, the range of quaternion channels is $[-1, 1]$, but we can convert this to the range $[0, 1]$ by adding one and then dividing by two.

The following pair of routines encode and decode an input floating-point value lying in the range $[0, 1]$ into an n -bit integer, according to Jonathan Blow's RL method. The quantized value is always returned as a 32-bit unsigned integer (U32), but only the least-significant n bits are actually used, as specified by the `nBits` argument. For example, if you pass `nBits==16`, you can safely cast the result to a U16.

```
U32 CompressUnitFloatRL(F32 unitFloat, U32 nBits)
{
    // Determine the number of intervals based on the
    // number of output bits we've been asked to produce.
    U32 nIntervals = 1u << nBits;

    // Scale the input value from the range [0, 1] into
    // the range [0, nIntervals - 1]. We subtract one
    // interval because we want the largest output value
    // to fit into nBits bits.
    F32 scaled = unitFloat * (F32)(nIntervals - 1u);

    // Finally, round to the nearest interval center. We
    // do this by adding 0.5f and then truncating to the
    // next-lowest interval index (by casting to U32).
    U32 rounded = (U32)(scaled + 0.5f);

    // Guard against invalid input values.
    if (rounded > nIntervals - 1u)
        rounded = nIntervals - 1u;

    return rounded;
}

F32 DecompressUnitFloatRL(U32 quantized, U32 nBits)
{
    // Determine the number of intervals based on the
    // number of bits we used when we encoded the value.
    U32 nIntervals = 1u << nBits;

    // Decode by simply converting the U32 to an F32, and
    // scaling by the interval size.
```

```

    F32 intervalSize = 1.0f / (F32)(nIntervals - 1u);
    F32 approxUnitFloat = (F32)quantized * intervalSize;

    return approxUnitFloat;
}

```

To handle arbitrary input values in the range $[min, max]$, we can use these routines:

```

U32 CompressFloatRL(F32 value, F32 min, F32 max,
                   U32 nBits)
{
    F32 unitFloat = (value - min) / (max - min);
    U32 quantized = CompressUnitFloatRL(unitFloat,
                                       nBits);

    return quantized;
}

F32 DecompressFloatRL(U32 quantized, F32 min, F32 max,
                     U32 nBits)
{
    F32 unitFloat = DecompressUnitFloatRL(quantized,
                                       nBits);
    F32 value = min + (unitFloat * (max - min));
    return value;
}

```

Let's return to our original problem of animation channel compression. To compress and decompress a quaternion's four components into 16 bits per channel, we simply call `CompressFloatRL()` and `DecompressFloatRL()` with $min = -1$, $max = 1$ and $n = 16$:

```

inline U16 CompressRotationChannel(F32 qx)
{
    return (U16)CompressFloatRL(qx, -1.0f, 1.0f, 16u);
}

inline F32 DecompressRotationChannel(U16 qx)
{
    return DecompressFloatRL((U32)qx, -1.0f, 1.0f, 16u);
}

```

Compression of translation channels is a bit trickier than rotations, because unlike quaternion channels, the range of a translation channel could theoretically be unbounded. Thankfully, the joints of a character don't move very far in practice, so we can decide upon a reasonable range of motion and flag an

error if we ever see an animation that contains translations outside the valid range. In-game cinematics are an exception to this rule—when an IGC is animated in world space, the translations of the characters’ root joints can grow very large. To address this, we can select the range of valid translations on a per-animation or per-joint basis, depending on the maximum translations actually achieved within each clip. Because the data range might differ from animation to animation, or from joint to joint, we must store the range with the compressed clip data. This will add a tiny amount of data to each animation clip, but the impact is generally negligible.

```
// We'll use a 2 m range -- your mileage may vary.
F32 MAX_TRANSLATION = 2.0f;

inline U16 CompressTranslationChannel(F32 vx)
{
    // Clamp to valid range...
    if (vx < -MAX_TRANSLATION)
        vx = -MAX_TRANSLATION;
    if (vx > MAX_TRANSLATION)
        vx = MAX_TRANSLATION;

    return (U16)CompressFloatRL(vx,
                                -MAX_TRANSLATION, MAX_TRANSLATION, 16);
}

inline F32 DecompressTranslationChannel(U16 vx)
{
    return DecompressFloatRL((U32)vx,
                              -MAX_TRANSLATION, MAX_TRANSLATION, 16);
}
```

12.8.3 Sampling Frequency and Key Omission

Animation data tends to be large for three reasons: first, because the pose of each joint can contain upwards of ten channels of floating-point data; second, because a skeleton contains a large number of joints (250 or more for a humanoid character on PS3 or Xbox 360, and more than 800 on some PS4 and Xbox One games); third, because the pose of the character is typically sampled at a high rate (e.g., 30 frames per second). We’ve seen some ways to address the first problem. We can’t really reduce the number of joints for our high-resolution characters, so we’re stuck with the second problem. To attack the third problem, we can do two things:

- *Reduce the sample rate overall.* Some animations look fine when exported

at 15 samples per second, and doing so cuts the animation data size in half.

- *Omit some of the samples.* If a channel's data varies in an approximately linear fashion during some interval of time within the clip, we can omit all of the samples in this interval except the endpoints. Then, at runtime, we can use linear interpolation to recover the dropped samples.

The latter technique is a bit involved, and it requires us to store information about the *time* of each sample. This additional data can erode the savings we achieved by omitting samples in the first place. However, some game engines have used this technique successfully.

12.8.4 Curve-Based Compression

One of the most powerful, easiest-to-use and best-thought-out animation APIs I've ever worked with is Granny, by Rad Game Tools. Granny stores animations not as a regularly spaced sequence of pose samples but as a collection of n th-order, nonuniform, nonrational B-splines, describing the paths of a joint's S, Q and T channels over time. Using B-splines allows channels with a lot of curvature to be encoded using only a few data points.

Granny exports an animation by sampling the joint poses at regular intervals, much like traditional animation data. For each channel, Granny then fits a set of B-splines to the sampled dataset to within a user-specified tolerance. The end result is an animation clip that is usually significantly smaller than its uniformly sampled, linearly interpolated counterpart. This process is illustrated in Figure 12.46.

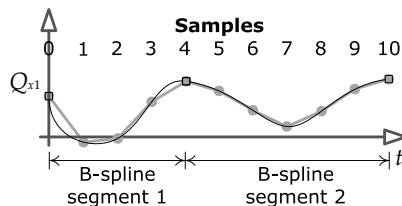


Figure 12.46. One form of animation compression fits B-splines to the animation channel data.

12.8.5 Wavelet Compression

Another way to compress animation data is to apply *signal processing theory* to the problem, via a technique known as *wavelet compression*. A wavelet is a function whose amplitude oscillates like a wave but whose duration is very

short, like a brief ripple in a pond. Wavelet functions are carefully crafted to give them desirable properties for use in signal processing.

In wavelet compression, an animation curve is decomposed into a sum of orthonormal wavelets, in much the same way that an arbitrary signal can be represented as a train of delta functions or a sum of sinusoids. We discuss signal processing and linear time-invariant systems in some depth in Section 14.2; the concepts presented there form the foundations necessary to understand wavelet compression. A full discussion of wavelet-based compression techniques is well beyond the scope of this book, but you can read more about it online. Search for “wavelet” to find introductory articles on the topic, and then try searching for “Animation Compression: Signal Processing” on Nicholas Frechette’s blog for a great article on how wavelet compression was implemented for *Thief* (2014) by Eidos Montreal.

12.8.6 Selective Loading and Streaming

The cheapest animation clip is the one that isn’t in memory at all. Most games don’t need every animation clip to be in memory simultaneously. Some clips apply only to certain classes of character, so they needn’t be loaded during levels in which that class of character is never encountered. Other clips apply to one-off moments in the game. These can be loaded or streamed into memory just before being needed and dumped from memory once they have played.

Most games load a core set of animation clips into memory when the game first boots and keep them there for the duration of the game. These include the player character’s core move set and animations that apply to objects that reappear over and over throughout the game, such as weapons or power-ups. All other animations are usually loaded on an as-needed basis. Some game engines load animation clips individually, but many package them together into logical groups that can be loaded and unloaded as a unit.

12.9 The Animation Pipeline

The operations performed by the low-level animation engine form a *pipeline* that transforms its inputs (animation clips and blend specifications) into the desired outputs (local and global poses, plus a matrix palette for rendering).

For each animating character and object in the game, the animation pipeline takes one or more animation clips and corresponding blend factors as input, blends them together, and generates a single local skeletal pose as output. It also calculates a global pose for the skeleton and a palette of skinning

matrices for use by the rendering engine. Post-processing hooks are usually provided, which permit the local pose to be modified prior to final global pose and matrix palette generation. This is where inverse kinematics (IK), rag doll physics and other forms of procedural animation are applied to the skeleton.

The stages of this pipeline are:

1. *Clip decompression and pose extraction.* In this stage, each individual clip's data is decompressed, and a static pose is extracted for the time index in question. The output of this phase is a local skeletal pose for each input clip. This pose might contain information for every joint in the skeleton (a *full-body pose*), for only a subset of joints (a *partial pose*), or it might be a *difference pose* for use in additive blending.
2. *Pose blending.* In this stage, the input poses are combined via full-body LERP blending, partial-skeleton LERP blending and/or additive blending. The output of this stage is a single local pose for all joints in the skeleton. This stage is of course only executed when blending more than one animation clip together—otherwise the output pose from stage 1 can be used directly.
3. *Global pose generation.* In this stage, the skeletal hierarchy is walked, and local joint poses are concatenated in order to generate a global pose for the skeleton.
4. *Post-processing.* In this optional stage, the local and/or global poses of the skeleton can be modified prior to finalization of the pose. Post-processing is used for inverse kinematics, rag doll physics and other forms of procedural animation adjustment.
5. *Recalculation of global poses.* Many types of post-processing require global pose information as input but generate local poses as output. After such a post-processing step has run, we must recalculate the global pose from the modified local pose. Obviously, a post-processing operation that does not require global pose information can be done between stages 2 and 3, thus avoiding the need for global pose recalculation.
6. *Matrix palette generation.* Once the final global pose has been generated, each joint's global pose matrix is multiplied by the corresponding inverse bind pose matrix. The output of this stage is a palette of skinning matrices suitable for input to the rendering engine.

A typical animation pipeline is depicted in Figure 12.47.

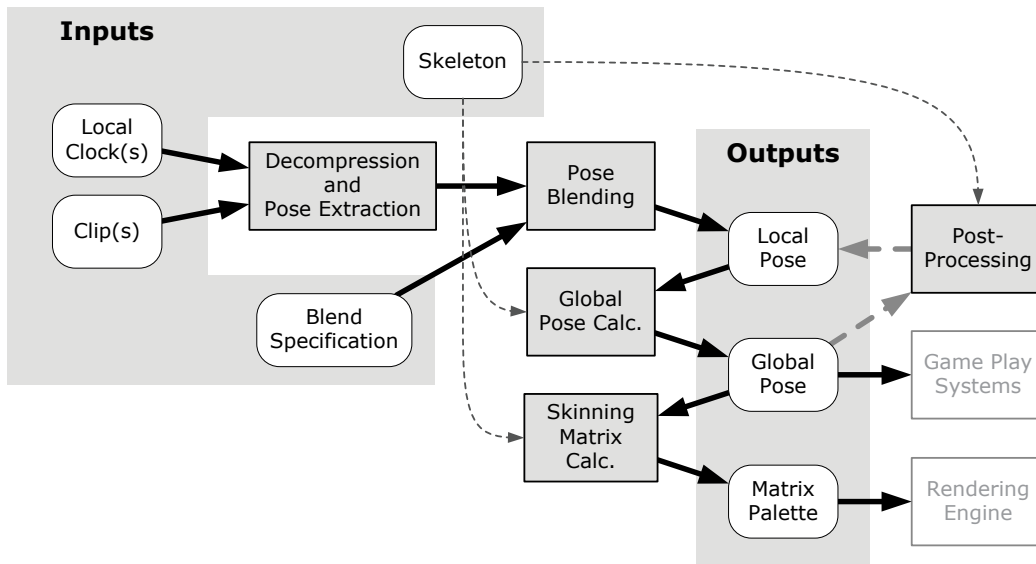


Figure 12.47. A typical animation pipeline.

12.10 Action State Machines

The actions of a game character (standing, walking, running, jumping, etc.) are usually best modeled via a finite state machine, commonly known as the *action state machine* (ASM). The ASM subsystem sits atop the animation pipeline and provides a state-driven animation interface for use by virtually all higher-level game code.

Each state in an ASM corresponds to an arbitrarily complex blend of simultaneous animation clips. Some states might be very simple—for example, the “idle” state might be comprised of a single full-body animation. Other states might be more complex. A “running” state might correspond to a semicircular blend, with strafing left, running forward and strafing right at the -90 degree, 0 degree and $+90$ degree points, respectively. The “running while shooting” state might include a semicircular directional blend, plus additive or partial-skeleton blend nodes for aiming the character’s weapon up, down, left and right, and additional blends to permit the character to look around with its eyes, head and shoulders. More additive animations might be included to control the character’s overall stance, gait and foot spacing while locomoting and to provide a degree of “humanness” through random movement variations.

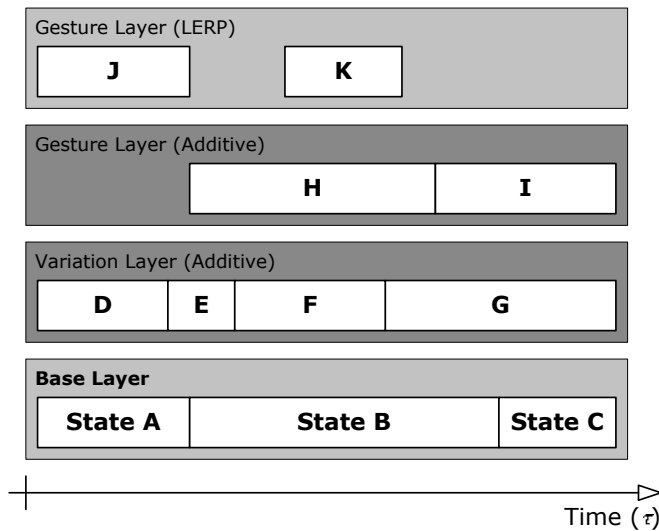


Figure 12.48. A layered action state machine, showing how each layer’s state transitions are temporally independent. In this example, the base layer describes the character’s full-body stance and movement. A variation layer provides variety by applying additive clips to the character’s pose. Finally, two gesture layers, one additive and one partial, permit the character to aim or point at objects in the world around it.

A character’s ASM also ensures that characters can *transition* smoothly from state to state. During a transition from state A to state B, the final output poses of both states are usually blended together to provide a smooth *cross-fade* between them.

Most high-quality animation engines also permit different parts of a character’s body to be doing different, independent or semi-independent actions simultaneously. For instance, a character might be running, aiming and firing a weapon with its arms, and speaking a line of dialog with its facial joints. The movements of different parts of the body aren’t generally in perfect sync either—certain parts of the body tend to “lead” the movements of other parts (e.g., the head leads a turn, followed by the shoulders, the hips and finally the legs). In traditional animation, this well-known technique is known as *anticipation* [51]. This kind of complex movement can be realized by allowing multiple independent state machines to control a single character. Usually each state machine exists in a separate *state layer*, as shown in Figure 12.48. The output poses from each layer’s ASM are blended together into a final composite pose.

All of this means that at any given moment in time, multiple animation

clips are contributing to the final pose of a character's skeleton. For each character, then, we need a way to track all of the currently-playing clips, and to describe how exactly they should be blended together in order to produce the character's final pose. Generally speaking, there are two ways to do this:

1. *Flat weighted average.* In this approach, the engine maintains a flat list of all animation clips that are currently contributing to a character's final pose, with one blend weight per clip. The animations are blended together as one big weighted average to produce the final pose.
2. *Blend trees.* In this approach, each contributing clip is represented by the leaf nodes of a tree. The interior nodes of this tree represent various blending operations that are being performed on the clips. Multiple blend operations are composed to form action states. Additional blend nodes are introduced to represent transient cross-fades. And in a layered ASM, the output poses obtained from the action states in each layer are blended together. The final pose of the character is thus produced at the root of this potentially complex blend tree.

12.10.1 The Flat Weighted Average Approach

In the flat weighted average approach, every animation clip that is currently playing on a given character is associated with a blend weight indicating how much it should contribute to its final pose. A flat list of all *active* animation clips (i.e., clips whose blend weights are nonzero) is maintained. To calculate the final pose of the skeleton, we extract a pose at the appropriate time index for each of the N active clips. Then, for each joint of the skeleton, we calculate a simple N -point weighted average of the translation vectors, rotation quaternions and scale factors extracted from the N active animations. This yields the final pose of the skeleton.

The equation for the weighted average of a set of N vectors $\{\mathbf{v}_i\}$ is as follows:

$$\mathbf{v}_{\text{avg}} = \frac{\sum_{i=0}^{N-1} w_i \mathbf{v}_i}{\sum_{i=0}^{N-1} w_i}.$$

If the weights are *normalized*, meaning they sum to one, then this equation can be simplified to the following:

$$\mathbf{v}_{\text{avg}} = \sum_{i=0}^{N-1} w_i \mathbf{v}_i, \quad \text{when } \sum_{i=0}^{N-1} w_i = 1.$$

In the case of $N = 2$, if we let $w_0 = (1 - \beta)$ and $w_1 = \beta$, the weighted average reduces to the familiar equation for the linear interpolation (LERP) between two vectors:

$$\begin{aligned}\mathbf{v}_{\text{avg}} &= w_0 \mathbf{v}_A + w_1 \mathbf{v}_B \\ &= (1 - \beta) \mathbf{v}_A + \beta \mathbf{v}_B \\ &= \text{LERP}[\mathbf{v}_A, \mathbf{v}_B, \beta].\end{aligned}$$

We can apply this same weighted average formulation equally well to quaternions by simply treating them as four-element vectors.

12.10.1.1 Example: OGRE

The OGRE animation system works in exactly this way. An `Ogre::Entity` represents an instance of a 3D mesh (e.g., one particular character walking around in the game world). The `Entity` aggregates an object called an `Ogre::AnimationStateSet`, which in turn maintains a list of `Ogre::AnimationState` objects, one for each active animation. The `Ogre::AnimationState` class is shown in the code snippet below. (A few irrelevant details have been omitted for clarity.)

```
/** Represents the state of an animation clip and the
    weight of its influence on the overall pose of the
    character.
*/
class AnimationState
{
protected:
    String          mAnimationName; // reference to
                                // clip
    Real            mTimePos;        // local clock
    Real            mWeight;         // blend weight
    bool            mEnabled;        // is this anim
                                // running?
    bool            mLoop;           // should the
                                // anim loop?

public:
    /// API functions...
};
```

Each `AnimationState` keeps track of one animation clip's local clock and its blend weight. When calculating the final pose of the skeleton for a particular `Ogre::Entity`, OGRE's animation system simply loops through each

active `AnimationState` in its `AnimationStateSet`. A skeletal pose is extracted from the animation clip corresponding to each state at the time index specified by that state's local clock. For each joint in the skeleton, an N -point weighted average is then calculated for the translation vectors, rotation quaternions and scales, yielding the final skeletal pose.

It is interesting to note that OGRE has no concept of a playback rate (R). If it did, we would have expected to see a data member like this in the `Ogre::AnimationState` class:

```
Real    mPlaybackRate;
```

Of course, we can still make animations play more slowly or more quickly in OGRE by simply scaling the amount of time we pass to the `addTime()` function, but unfortunately, OGRE does not support animation time scaling out of the box.

12.10.1.2 Example: Granny

The Granny animation system, by Rad Game Tools (<http://www.radgame.com/granny.html>), provides a flat, weighted average animation blending system similar to OGRE's. Granny permits any number of animations to be played on a single character simultaneously. The state of each active animation is maintained in a data structure known as a `granny_control`. Granny calculates a weighted average to determine the final pose, automatically normalizing the weights of all active clips. In this sense, its architecture is virtually identical to that of OGRE's animation system.

Where Granny really shines is in its handling of time. Granny uses the global clock approach discussed in Section 12.4.3. It allows each clip to be looped an arbitrary number of times or infinitely. Clips can also be time-scaled; a negative time scale allows an animation to be played in reverse.

12.10.1.3 Cross-Fades with a Flat Weighted Average

In an animation engine that employs the flat weighted average architecture, cross-fades are implemented by adjusting the weights of the clips themselves. Recall that any clip whose weight $w_i = 0$ will not contribute to the current pose of the character, while those whose weights are nonzero are averaged together to generate the final pose. If we wish to transition smoothly from clip A to clip B, we simply ramp up clip B's weight w_B , while simultaneously ramping down clip A's weight w_A . This is illustrated in Figure 12.49.

Cross-fading in a weighted average architecture becomes a bit trickier when we wish to transition from one complex blend to another. As an example, let's say we wish to transition the character from walking to jumping.

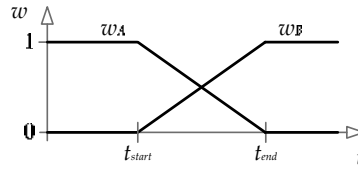


Figure 12.49. A simple cross-fade from clip A to clip B, as implemented in a weighted average animation architecture.

Let's assume that the walk movement is produced by a three-way average between clips A, B and C, and that the jump movement is produced by a two-way average between clips D and E.

We want the character to look like he's smoothly transitioning from walking to jumping, without affecting how the walk or jump animations look individually. So during the transition, we want to ramp down the ABC clips and ramp up the DE clips while keeping the *relative weights* of the ABC and DE clip groups constant. If the cross-fade's blend factor is denoted by λ , we can meet this requirement by simply setting the weights of *both* clip groups to their desired values and then multiplying the weights of the source group by $(1 - \lambda)$ and the weights of the destination group by λ .

Let's look at a concrete example to convince ourselves that this will work properly. Imagine that before the transition from ABC to DE, the nonzero weights are as follows: $w_A = 0.2$, $w_B = 0.3$ and $w_C = 0.5$. After the transition, we want the nonzero weights to be $w_D = 0.33$ and $w_E = 0.66$. So, we set the weights as follows:

$$\begin{aligned} w_A &= (1 - \lambda)(0.2), & w_D &= \lambda(0.33), \\ w_B &= (1 - \lambda)(0.3), & w_E &= \lambda(0.66). \\ w_C &= (1 - \lambda)(0.5), \end{aligned} \tag{12.20}$$

From Equations (12.20), you should be able to convince yourself of the following:

1. When $\lambda = 0$, the output pose is the correct blend of clips A, B and C, with zero contribution from clips D and E.
2. When $\lambda = 1$, the output pose is the correct blend of clips D and E, with no contribution from A, B or C.
3. When $0 < \lambda < 1$, the *relative* weights of both the ABC group and the DE group remain correct, although they no longer add to one. (In fact, group ABC's weights add to $(1 - \lambda)$, and group DE's weights add to λ .)

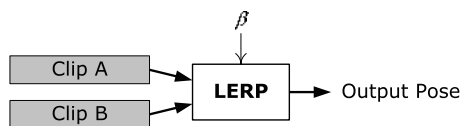


Figure 12.50. A binary LERP blend, represented by a binary expression tree.

For this approach to work, the implementation must keep track of the logical groupings between clips (even though, at the lowest level, all of the clips' states are maintained in one big, flat array—for example, the `Ogre::AnimationStateSet` in OGRE). In our example above, the system must “know” that A, B and C form a group, that D and E form another group, and that we wish to transition from group ABC to group DE. This requires additional metadata to be maintained, on top of the flat array of clip states.

12.10.2 Blend Trees

Some animation engines represent a character's clip state not as a flat weighted average but rather as a tree of blend operations. An animation blend tree is an example of what is known in compiler theory as an *expression tree* or a *syntax tree*. The interior nodes of such a tree are operators, and the leaf nodes serve as the inputs to those operators. (More correctly, the interior nodes represent the *nonterminals* of the grammar, while the leaf nodes represent the *terminals*.)

In the following sections, we'll briefly revisit the various kinds of animation blends we learned about in Sections 12.6.3 and 12.6.5 and see how each can be represented by an expression tree.

12.10.2.1 Binary LERP Blend Trees

As we saw in Section 12.6.1, a binary linear interpolation (LERP) blend takes two input poses and blends them together into a single output pose. A blend weight β controls the percentage of the second input pose that should appear at the output, while $(1 - \beta)$ specifies the percentage of the first input pose. This can be represented by the binary expression tree shown in Figure 12.50.

12.10.2.2 Generalized One-Dimensional Blend Trees

In Section 12.6.3.1, we learned that it can be convenient to define a generalized one-dimensional LERP blend by placing an arbitrary number of clips along a linear scale. A blend factor b specifies the desired blend along this scale. Such a blend can be pictured as an n -input operator, as shown in Figure 12.51.

Given a specific value for b , such a linear blend can always be transformed into a binary LERP blend. We simply use the two clips immediately adjacent to

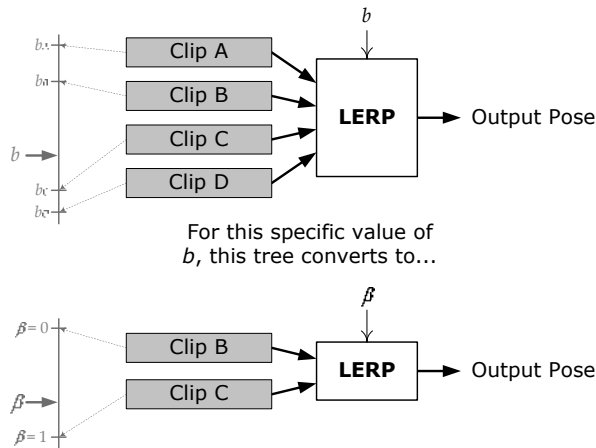


Figure 12.51. A multi-input expression tree can be used to represent a generalized ID blend. Such a tree can always be transformed into a binary expression tree for any specific value of the blend factor b .

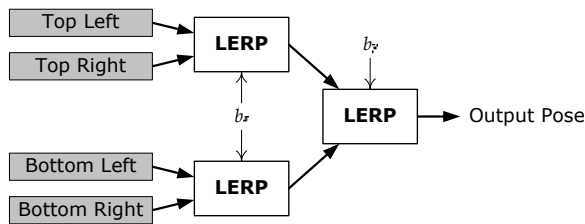


Figure 12.52. A simple 2D LERP blend, implemented as cascaded binary blends.

b as the inputs to the binary blend and calculate the blend weight β as specified in Equation (12.15)

12.10.2.3 Two-Dimensional LERP Blend Trees

In Section 12.6.3.2, we saw how a two-dimensional LERP blend can be realized by simply cascading the results of two binary LERP blends. Given a desired two-dimensional blend point $\mathbf{b} = [b_x \ b_y]$, Figure 12.52 shows how this kind of blend can be represented in tree form.

12.10.2.4 Additive Blend Trees

Section 12.6.5 described additive blending. This is a binary operation, so it can be represented by a binary tree node, as shown in Figure 12.53. A single blend weight β controls the amount of the additive animation that should appear at

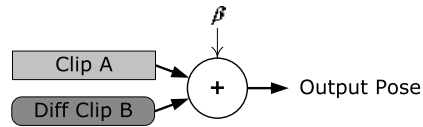


Figure 12.53. An additive blend represented as a binary tree.

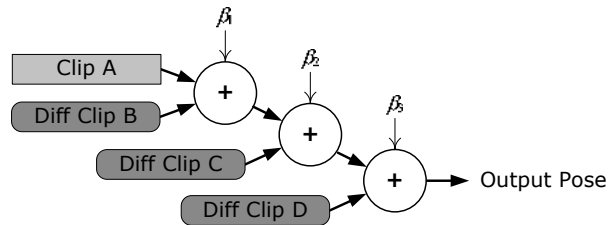


Figure 12.54. In order to additively blend more than one difference pose onto a regular “base” pose, a cascaded binary expression tree must be used.

the output—when $\beta = 0$, the additive clip does not affect the output at all, while when $\beta = 1$, the additive clip has its maximum effect on the output.

Additive blend nodes must be handled carefully, because the inputs are not interchangeable (as they are with most types of blend operators). One of the two inputs is a regular skeletal pose, while the other is a special kind of pose known as a *difference pose* (also known as an *additive pose*). A difference pose may *only* be applied to a regular pose, and the result of an additive blend is another regular pose. This implies that the additive input of a blend node must always be a leaf node, while the regular input may be a leaf or an interior node. If we want to apply more than one additive animation to our character, we must use a cascaded binary tree with the additive clips always applied to the additive inputs, as shown in Figure 12.54.

12.10.2.5 Layered Blend Trees

We said at the beginning of Section 12.10 that complex character movement can be produced by arranging multiple independent state machines into *state layers*. The output poses from each layer’s ASM are blended together into a final composite pose. When this is implemented using blend trees, the net effect is to combine the blend trees of each active state together into one über tree, as illustrated in Figure 12.55.

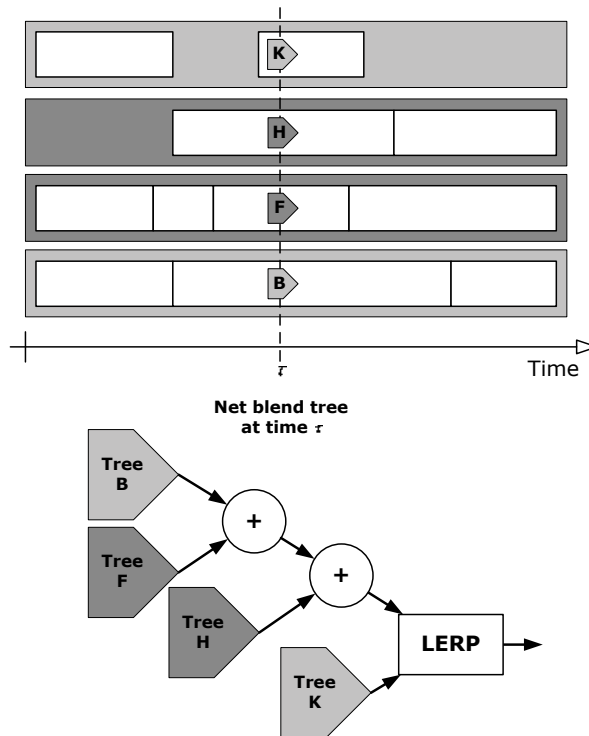


Figure 12.55. A layered state machine converts the blend trees from multiple states into a single, unified tree.

12.10.2.6 Cross-Fades with Blend Trees

As a character transitions from state to state within each layer of a layered ASM, we often wish to provide a smooth cross-fade between states. Implementing a cross-fade in an expression tree based ASM is a bit more intuitive than it is in a weighted average architecture. Whether we're transitioning from one clip to another or from one complex blend to another, the approach is always the same: We simply introduce a transient binary LERP node between the roots of the blend trees of each state to handle the cross-fade.

We'll denote the blend factor of the cross-fade node with the symbol λ as before. Its top input is the source state's blend tree (which can be a single clip or a complex blend), and its bottom input is the destination state's tree (again a clip or a complex blend). During the transition, λ is ramped from zero to one. Once $\lambda = 1$, the transition is complete, and the cross-fade LERP node and its top input tree can be retired. This leaves its bottom input tree as the root of

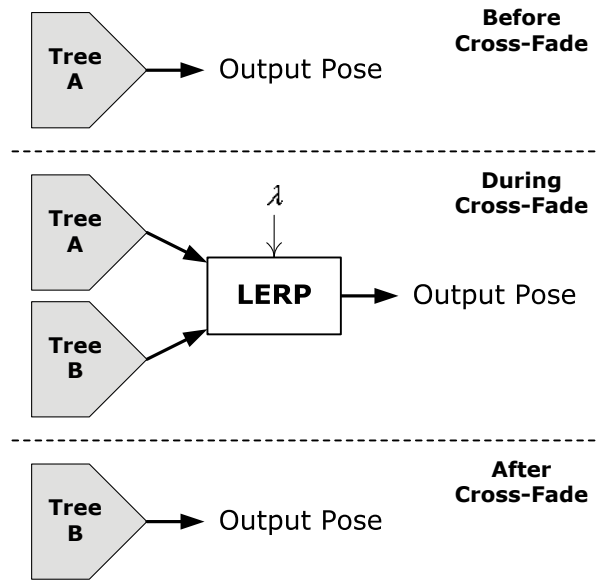


Figure 12.56. A cross-fade between two arbitrary blend trees A and B.

the overall blend tree for the given state layer, thus completing the transition. This process is illustrated in Figure 12.56.

12.10.3 State and Blend Tree Specifications

Animators, game designers and programmers usually cooperate to create the animation and control systems for the central characters in a game. These developers need a way to specify the states that make up a character's ASM, to lay out the tree structure of each blend tree, and to select the clips that will serve as their inputs. Although the states and blend trees could be hard-coded, most modern game engines provide a *data-driven* means of defining animation states. The goal of a data-driven approach is to permit a user to create new animation states, remove unwanted states, fine-tune existing states and then see the effects of his or her changes reasonably quickly. In other words, the central goal of a data-driven animation engine is to enable *rapid iteration*.

To build an arbitrarily complex blend tree, we really only require four atomic types of blend nodes: clips, binary LERP blends, binary additive blends and possibly ternary (triangular) LERP blends. Virtually any blend tree imaginable can be created as compositions of these atomic nodes.

A blend tree built exclusively from atomic nodes can quickly become large and unwieldy. As a result, many game engines permit custom compound node types to be predefined for convenience. The N -dimensional linear blend node discussed in Sections 12.6.3.4 and 12.10.2.2 is an example of a compound node. One can imagine myriad complex blend node types, each one addressing a particular problem specific to the particular game being made. A soccer game might define a node that allows the character to dribble the ball. A war game could define a special node that handles aiming and firing a weapon. A brawler could define custom nodes for each fight move the characters can perform. Once we have the ability to define custom node types, the sky's the limit.

The means by which the users enter animation state data varies widely. Some game engines employ a simple, bare-bones approach, allowing animation states to be specified in a text file with a simple syntax. Other engines provide a slick, graphical editor that permits animation states to be constructed by dragging atomic components such as clips and blend nodes onto a canvas and linking them together in arbitrary ways. Such editors usually provide a live preview of the character so that the user can see immediately how the character will look in the final game. In my opinion, the specific method chosen has little bearing on the quality of the final game—what matters most is that the user can make changes and see the results of those changes reasonably quickly and easily.

12.10.3.1 Example: The Naughty Dog Engine

The animation engine used in Naughty Dog's *Uncharted* and *The Last of Us* franchises employs a simple, text-based approach to specifying animation states. For reasons related to Naughty Dog's rich history with the Lisp language (see Section 16.9.5.1), state specifications in the Naughty Dog engine are written in a customized version of the Scheme programming language (which itself is a Lisp variant). Two basic state types can be used: *simple* and *complex*.

Simple States

A *simple* state contains a single animation clip. For example:

```
(define-state simple
  :name "pirate-b-bump-back"
  :clip "pirate-b-bump-back"
  :flags (anim-state-flag no-adjust-to-ground)
)
```

Don't let the Lisp-style syntax throw you. All this block of code does is to define a state named "pirate-b-bump-back" whose animation clip also happens to be named "pirate-b-bump-back." The `:flags` parameter allows users to specify various Boolean options on the state.

Complex States

A *complex* state contains an arbitrary tree of LERP or additive blends. For example, the following state defines a tree that contains a single binary LERP blend node, with two clips ("walk-l-to-r" and "run-l-to-r") as its inputs:

```
(define-state complex
  :name "move-l-to-r"
  :tree
    (anim-node-lerp
      (anim-node-clip "walk-l-to-r")
      (anim-node-clip "run-l-to-r")
    )
)
```

The `:tree` argument allows the user to specify an arbitrary blend tree, composed of LERP or additive blend nodes and nodes that play individual animation clips.

From this, we can see how the `(define-state simple ...)` example shown above might really work under the hood—it probably defines a complex blend tree containing a single "clip" node, like this:

```
(define-state complex
  :name "pirate-b-unimog-bump-back"
  :tree (anim-node-clip "pirate-b-unimog-bump-back")
  :flags (anim-state-flag no-adjust-to-ground)
)
```

The following complex state shows how blend nodes can be cascaded into arbitrarily deep blend trees:

```
(define-state complex
  :name "move-b-to-f"
  :tree
    (anim-node-lerp
      (anim-node-additive
        (anim-node-additive
          (anim-node-clip "move-f")
          (anim-node-clip "move-f-look-lr")
        )
      )
    )
)
```

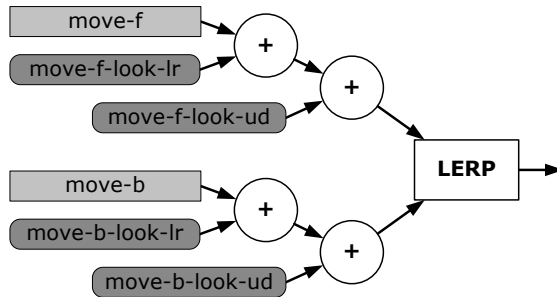



Figure 12.57. Blend tree corresponding to the example state “move-b-to-f.”

```

        (anim-node-clip "move-f-look-ud")
    )
    (anim-node-additive
        (anim-node-additive
            (anim-node-clip "move-b")
            (anim-node-clip "move-b-look-lr")
        )
        (anim-node-clip "move-b-look-ud")
    )
)
)
)

```

This corresponds to the tree shown in Figure 12.57.

Rapid Iteration

Naughty Dog’s animation team achieves rapid iteration with the help of four important tools:

1. An in-game animation viewer allows a character to be spawned into the game and its animations controlled via an in-game menu.
2. A simple command-line tool allows animation scripts to be recompiled and reloaded into the running game on the fly. To tweak a character’s animations, the user can make changes to the text file containing the animation state specifications, quickly reload the animation states and immediately see the effects of his or her changes on an animating character in the game.
3. The engine continually keeps track of all state transitions performed by each character during the last few seconds of gameplay. This allows us to pause the game and then literally rewind the animations to scrutinize them and debug problems that are noticed while playing.

4. The Naughty Dog engine also offers a host of “live update” tools. For example, animators can tweak their animations in Maya and see them update virtually instantaneously in the game.

12.10.3.2 Example: Unreal Engine 4

Unreal Engine 4 (UE4) provides its users with five tools for working with skeletal animations and skeletal meshes: The Skeleton Editor, the Skeletal Mesh Editor, the Animation Editor, the Animation Blueprint Editor, and the Physics Editor.

- The Skeleton Editor is essentially a rigging tool. It allows users to view and modify skeletons, add *sockets* to joints, and test out the movement of the skeleton. A socket is sometimes called an *attach point* in other engines (see Section 12.11.1).
- The Skeletal Mesh Editor allows users to edit properties of the meshes that are skinned to animating skeletons.
- The Animation Editor allows users to import, create and manage animation assets. In this editor, the compression and timing of animation clips (which UE4 calls Sequences) can be adjusted. Clips can be combined into predefined Blend Spaces, and in-game cinematics can be defined by creating Animation Montages.
- The Animation Blueprint Editor allows users to apply the power of Unreal Engine’s Blueprints visual scripting system to controlling characters’ animation state machines. This editor is depicted in Figure 12.58.
- The Physics Editor allows users to model a hierarchy of rigid bodies that drive the skeleton’s motion when ragdoll physics is active.

A complete discussion of Unreal Engine’s animation tools is beyond our scope here, but you can read more about it by searching for “Unreal Skeletal Mesh Animation System” online.

12.10.4 Transitions

To create a high-quality animating character, we must carefully manage the *transitions* between states in the action state machine to ensure that the splices between animations do not have a jarring and unpolished appearance. Most modern animation engines provide a data-driven mechanism for specifying exactly how transitions should be handled. In this section, we’ll explore how this mechanism works.

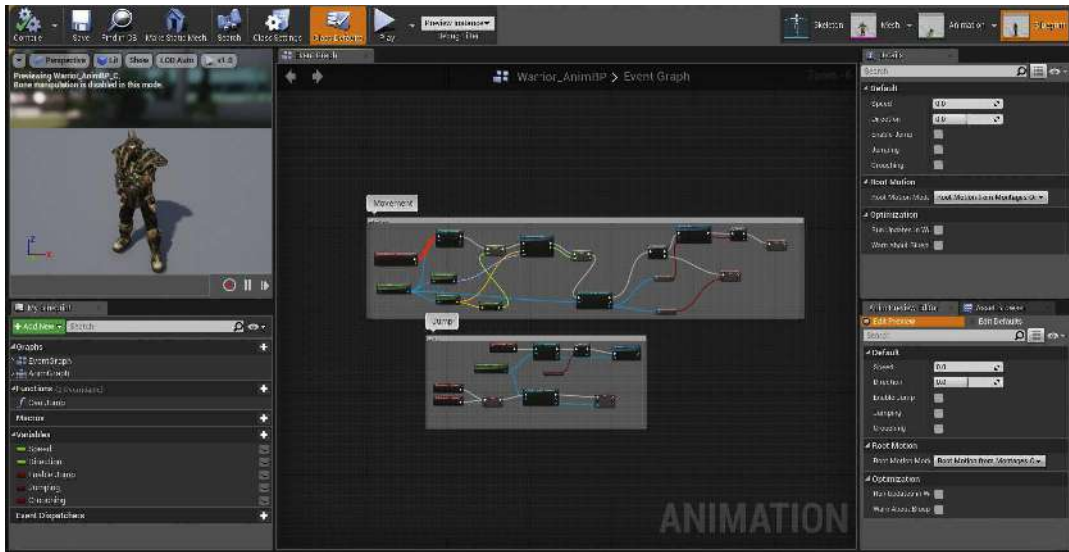


Figure 12.58. The Unreal Engine 4 animation Blueprints editor. (See Color Plate XXVI.)

12.10.4.1 Kinds of Transitions

There are many different ways to manage the transition between states. If we know that the final pose of the source state exactly matches the first pose of the destination state, we can simply “pop” from one state to another. Otherwise, we can cross-fade from one state to the next. Cross-fading is not always a suitable choice when transitioning from state to state. For example, there is no way that a cross-fade can produce a realistic transition from lying on the ground to standing upright. For this kind of state transition, we need one or more custom animations. This kind of transition is often implemented by introducing special *transitional states* into the state machine. These states are intended for use only when going from one state to another—they are never used as a steady-state node. But because they are full-fledged states, they can be comprised of arbitrarily complex blend trees. This provides maximum flexibility when authoring custom-animated transitions.

12.10.4.2 Transition Parameters

When describing a particular transition between two states, we generally need to specify various parameters, controlling exactly how the transition will occur. These include but are not limited to the following.

- *Source and destination states.* To which state(s) does this transition apply?

- *Transition type.* Is the transition immediate, cross-faded or performed via a transitional state?
- *Duration.* For cross-faded transitions, we need to specify how long the cross-fade should take.
- *Ease-in/ease-out curve type.* In a cross-faded transition, we may wish to specify the type of ease-in/ease-out curve to use to vary the blend factor during the fade.
- *Transition window.* Certain transitions can only be taken when the source animation is within a specified window of its local timeline. For example, a transition from a punch animation to an impact reaction might only make sense when the arm is in the second half of its swing. If an attempt to perform the transition is made during the first half of the swing, the transition would be disallowed (or a different transition might be selected instead).

12.10.4.3 The Transition Matrix

Specifying transitions between states can be challenging, because the number of possible transitions is usually very large. In a state machine with n states, the worst-case number of possible transitions is n^2 . We can imagine a two-dimensional square matrix with every possible state listed along both the vertical and horizontal axes. Such a table can be used to specify all of the possible transitions from any state along the vertical axis to any other state along the horizontal axis.

In a real game, this *transition matrix* is usually quite sparse, because not all state-to-state transitions are possible. For example, transitions are usually disallowed from a death state to any other state. Likewise, there is probably no way to go from a driving state to a swimming state (without going through at least one intermediate state that causes the character to jump out of his vehicle). The number of unique transitions in the table may be significantly less even than the number of valid transitions between states. This is because we can often reuse a single transition specification between many different pairs of states.

12.10.4.4 Implementing a Transition Matrix

There are all sorts of ways to implement a transition matrix. We could use a spreadsheet application to tabulate all the transitions in matrix form, or we might permit transitions to be authored in the same text file used to author our action states. If a graphical user interface is provided for state editing, transitions could be added to this GUI as well. In the following sections, we'll

take a brief look at a few transition matrix implementations from real game engines.

Example: Wildcarded Transitions in Medal of Honor: Pacific Assault

On *Medal of Honor: Pacific Assault* (MOHPA), we used the sparseness of the transition matrix to our advantage by supporting wildcarded transition specifications. For each transition specification, the names of both the source and destination states could contain asterisks (*) as a wildcard character. This allowed us to specify a single default transition from any state to any other state (via the syntax `from="*" to="*"`) and then refine this global default easily for entire categories of states. The refinement could be taken all the way down to custom transitions between specific state pairs when necessary. The MOHPA transition matrix looked something like this:

```
<transitions>
  <!-- global default -->
  <trans from="*" to="*"
    type=frozen duration=0.2>

  <!-- default for any walk to any run -->
  <trans from="walk*" to="run*"
    type=smooth
    duration=0.15>

  <!-- special handling from any prone to any getting-up
    -- action (only valid from 2 sec to 7.5 sec on the
    -- local timeline) -->
  <trans from="*prone" to="*get-up"
    type=smooth
    duration=0.1
    window-start=2.0
    window-end=7.5>
  ...
</transitions>
```

Example: First-Class Transitions in Uncharted

In some animation engines, high-level game code requests transitions from the current state to a new state by naming the destination state explicitly. The problem with this approach is that the calling code must have intimate knowledge of the names of the states and of which transitions are valid when in a particular state.

In Naughty Dog's engine, this problem is overcome by turning state transitions from secondary implementation details into first-class entities. Each state provides a list of valid transitions to other states, and each transition is given a unique name. The names of the transitions are standardized in order to make the *effect* of each transition predictable. For example, if a transition is called "walk," then it *always* goes from the current state to a walking state of some kind, no matter what the current state is. Whenever the high-level animation control code wants to transition from state A to state B, it asks for a transition by name (rather than requesting the destination state explicitly). If such a transition can be found and is valid, it is taken; otherwise, the request fails.

The following example state defines four transitions named "reload," "step-left," "step-right" and "fire." The (transition-group ...) line invokes a previously defined group of transitions; it is useful when the same set of transitions is to be used in multiple states. The (transition-end ...) command specifies a transition that is taken upon reaching the end of the state's local timeline if no other transition has been taken before then.

```
(define-state complex
  :name "s_turret-idle"
  :tree (aim-tree
    (anim-node-clip "turret-aim-all--base")
    "turret-aim-all--left-right"
    "turret-aim-all--up-down"
  )
  :transitions (
    (transition "reload" "s_turret-reload"
      (range - -) :fade-time 0.2)

    (transition "step-left" "s_turret-step-left"
      (range - -) :fade-time 0.2)

    (transition "step-right" "s_turret-step-right"
      (range - -) :fade-time 0.2)

    (transition "fire" "s_turret-fire"
      (range - -) :fade-time 0.1)

    (transition-group "combat-gunout-idle^move")

    (transition-end "s_turret-idle")
  )
)
```

The beauty of this approach may be difficult to see at first. Its primary

purpose is to allow transitions and states to be modified in a data-driven manner, without requiring changes to the C++ source code in many cases. This degree of flexibility is accomplished by shielding the animation control code from knowledge of the structure of the state graph. For example, let's say that we have ten different walking states (normal, scared, crouched, injured and so on). All of them can transition into a jumping state, but different kinds of walks might require different jump animations (e.g., normal jump, scared jump, jump from crouch, injured jump, etc.). For each of the ten walking states, we define a transition simply called "jump." At first, we can point all of these transitions to a single generic "jump" state, just to get things up and running. Later, we can fine-tune some of these transitions so that they point to custom jump states. We can even introduce transitional states between some of the "walk" states and their corresponding "jump" states. All sorts of changes can be made to the structure of the state graph and the parameters of the transitions without affecting the C++ source code—as long as the *names* of the transitions don't change.

12.10.5 Control Parameters

From a software engineering perspective, it can be challenging to orchestrate all of the blend weights, playback rates and other control parameters of a complex animating character. Different blend weights have different effects on the way the character animates. For example, one weight might control the character's movement direction, while others control its movement speed, horizontal and vertical weapon aim, head/eye look direction and so on. We need some way of exposing all of these blend weights to the code that is responsible for controlling them.

In a flat weighted average architecture, we have a flat list of all the animation clips that could possibly be played on the character. Each clip state has a blend weight, a playback rate and possibly other control parameters. The code that controls the character must look up individual clip states by name and adjust each one's blend weight appropriately. This makes for a simple interface, but it shifts most of the responsibility for controlling the blend weights to the character control system. For example, to adjust the direction in which a character is running, the character control code must know that the "run" action is comprised of a group of animation clips, named something like "StrafeLeft," "RunForward," "StrafeRight" and "RunBackward." It must look up these clip states by name and manually control all four blend weights in order to achieve a particular angled run animation. Needless to say, controlling animation parameters in such a fine-grained way can be tedious and can lead to difficult-

to-understand source code.

In a blend tree, a different set of problems arise. Thanks to the tree structure, the clips are grouped naturally into functional units. Custom tree nodes can encapsulate complex character motions. These are both helpful advantages over the flat weighted average approach. However, the control parameters are buried within the tree. Code that wishes to control the horizontal look-at direction of the head and eyes needs a priori knowledge of the structure of the blend tree so that it can find the appropriate nodes in the tree in order to control their parameters.

Different animation engines solve these problems in different ways. Here are some examples:

- *Node search.* Some engines provide a way for higher-level code to *find* blend nodes in the tree. For example, relevant nodes in the tree can be given special names, such as “HorizAim” for the node that controls horizontal weapon aiming. The control code can simply search the tree for a node of a particular name; if one is found, then we know what effect adjusting its blend weight will have.
- *Named variables.* Some engines allow names to be assigned to the individual control parameters. The controlling code can look up a control parameter by name in order to adjust its value.
- *Control structure.* In other engines, a simple data structure, such as an array of floating-point values or a C struct, contains all of the control parameters for the entire character. The nodes in the blend tree(s) are connected to particular control parameters, either by being hard-coded to use certain struct members or by looking up the parameters by name or index.

Of course, there are many other alternatives as well. Every animation engine tackles this problem in a slightly different way, but the net effect is always roughly the same.

12.11 Constraints

We’ve seen how action state machines can be used to specify complex blend trees and how a transition matrix can be used to control how transitions between states should work. Another important aspect of character animation control is to constrain the movement of the characters and/or objects in the scene in various ways. For example, we might want to constrain a weapon so that it always appears to be in the hand of the character who is carrying it.

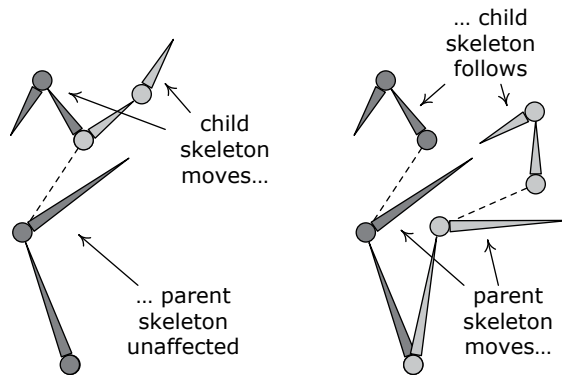


Figure 12.59. An attachment, showing how movement of the parent automatically produces movement of the child but not vice versa.

We might wish to constrain two characters so that they line up properly when shaking hands. A character's feet are often constrained so that they line up with the floor, and its hands might be constrained to line up with the rungs on a ladder or the steering wheel of a vehicle. In this section, we'll take a brief look at how these constraints are handled in a typical animation system.

12.11.1 Attachments

Virtually all modern game engines permit objects to be attached to one another. At its simplest, object-to-object attachment involves constraining the position and/or orientation of a particular joint J_A within the skeleton of object A so that it coincides with a joint J_B in the skeleton of object B. An attachment is usually a parent-child relationship. When the parent's skeleton moves, the child object is adjusted to satisfy the constraint. However, when the child moves, the parent's skeleton is usually not affected. This is illustrated in Figure 12.59.

Sometimes it can be convenient to introduce an *offset* between the parent joint and the child joint. For example, when placing a gun into a character's hand, we could constrain the "Grip" joint of the gun so that it coincides with the "RightWrist" joint of the character. However, this might not produce the correct alignment of the gun with the hand. One solution to this problem is to introduce a special joint into one of the two skeletons. For example, we could add a "RightGun" joint to the character's skeleton, make it a child of the "RightWrist" joint, and position it so that when the "Grip" joint of the gun is constrained to it, the gun looks like it is being held naturally by the character. The problem with this approach, however, is that it increases the

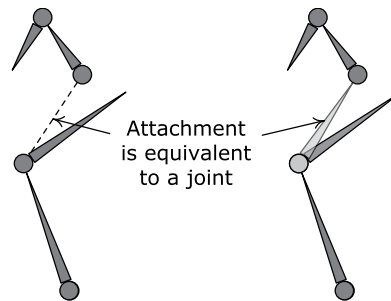


Figure 12.60. An attach point acts like an extra joint between the parent and the child.

number of joints in the skeleton. Each joint has a processing cost associated with animation blending and matrix palette calculation and a memory cost for storing its animation keys. So adding new joints is often not a viable option.

We know that an additional joint added for attachment purposes will not contribute to the pose of the character—it merely introduces an additional transform between the parent and child joint in an attachment. What we really want, then, is a way to mark certain joints so that they can be ignored by the animation blending pipeline but can still be used for attachment purposes. Such special joints are sometimes called *attach points*. They are illustrated in Figure 12.60.

Attach points might be modeled in Maya just like regular joints or locators, although many game engines define attach points in a more convenient manner. For example, they might be specified as part of the action state machine text file or via a custom GUI within the animation authoring tool. This allows the animators to focus only on the joints that affect the look of the character, while the power to control attachments is put conveniently into the hands of the people who need it—the game designers and the engineers.

12.11.2 Interobject Registration

The interactions between game characters and their environments is growing ever more complex and nuanced with each new title. Hence, it is important to have a system that allows characters and objects to be aligned with one another when animating. Such a system can be used for in-game cinematics and interactive gameplay elements alike.

Imagine that an animator, working in Maya or some other animation tool, sets up a scene involving two characters and a door object. The two characters shake hands, and then one of them opens the door and they both walk through it. The animator can ensure that all three actors in the scene line up perfectly.

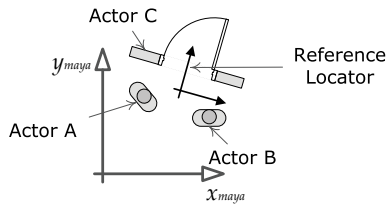


Figure 12.61. Original Maya scene containing three actors and a reference locator.

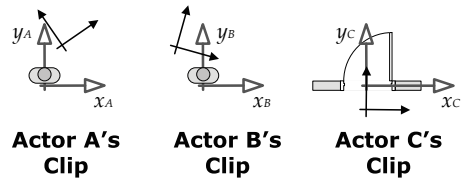


Figure 12.62. The reference locator is encoded in each actor's animation file.

However, when the animations are exported, they become three separate clips, to be played on three separate objects in the game world. The two characters might have been under AI or player control prior to the start of this animated sequence. How, then, can we ensure that the three objects line up correctly with one another when the three clips are played back in-game?

12.11.2.1 Reference Locators

One good solution is to introduce a common reference point into all three animation clips. In Maya, the animator can drop a *locator* (which is just a 3D transform, much like a skeletal joint) into the scene, placing it anywhere that seems convenient. Its location and orientation are actually irrelevant, as we'll see. The locator is tagged in some way to tell the animation export tools that it is to be treated specially.

When the three animation clips are exported, the tools store the position and orientation of the reference locator, expressed in coordinates that are relative to the local object space of each actor, into all three clip's data files. Later, when the three clips are played back in-game, the animation engine can look up the relative position and orientation of the reference locator in all three clips. It can then transform the origins of the three objects in such a way as to make all three reference locators coincide in world space. The reference locator acts much like an *attach point* (Section 12.11.1) and, in fact, could be implemented as one. The net effect—all three actors now line up with one another, exactly as they had been aligned in the original Maya scene.

Figure 12.61 illustrates how the door and the two characters from the above example might be set up in a Maya scene. As shown in Figure 12.62, the reference locator appears in each exported animation clip (expressed in that actor's local space). In-game, these local-space reference locators are aligned to a fixed world-space locator in order to realign the actors, as shown in Figure 12.63.

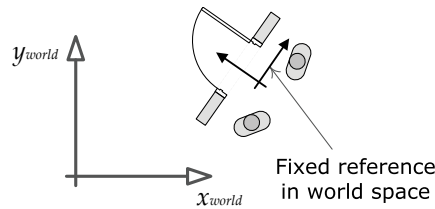


Figure 12.63. At runtime, the local-space reference transforms are aligned to a world-space reference locator, causing the actors to line up properly.

12.11.2.2 Finding the World-Space Reference Location

We've glossed over one important detail here—who decides what the world-space position and orientation of the reference locator should be? Each animation clip provides the reference locator's transform in the coordinate space of *its actor*. But we need some way to define where that reference locator should be in world space.

In our example with the door and the two characters shaking hands, one of the actors is fixed in the world (the door). So one viable solution is to ask the door for the location of the reference locator and then align the two characters to it. The commands to accomplish this might look similar to the following pseudocode.

```
void playShakingHandsDoorSequence(
    Actor& door,
    Actor& characterA,
    Actor& characterB)
{
    // Find the world-space transform of the reference
    // locator as specified in the door's animation.
    Transform refLoc = getReferenceLocatorWs(door,
        "shake-hands-door");

    // Play the door's animation in-place. (It's
    // already in the correct place.)
    playAnimation("shake-hands-door", door);

    // Play the two characters' animations relative to
    // the world-space reference locator obtained from
    // the door.
    playAnimationRelativeToReference(
        "shake-hands-character-a", characterA, refLoc);
    playAnimationRelativeToReference(
        "shake-hands-character-b", characterB, refLoc);
}
```

Another option is to define the world-space transform of the reference locator independently of the three actors in the scene. We could place the reference locator into the world using our world-building tool, for example (see Section 15.3). In this case, the pseudocode above should be changed to look something like this:

```
void playShakingHandsDoorSequence(
    Actor& door,
    Actor& characterA,
    Actor& characterB,
    Actor& refLocatorActor)
{
    // Find the world-space transform of the reference
    // locator by simply querying the transform of an
    // independent actor (presumably placed into the
    // world manually).
    Transform refLoc = getActorTransformWs(
        refLocatorActor);

    // Play all animations relative to the world-space
    // reference locator obtained above.
    playAnimationRelativeToReference("shake-hands-door",
        door, refLoc);
    playAnimationRelativeToReference(
        "shake-hands-character-a", characterA, refLoc);
    playAnimationRelativeToReference(
        "shake-hands-character-b", characterB, refLoc);
}
```

12.11.3 Grabbing and Hand IK

Even after using an attachment to connect two objects, we sometimes find that the alignment does not look exactly right in-game. For example, a character might be holding a rifle in her right hand, with her left hand supporting the stock. As the character aims the weapon in various directions, we may notice that the left hand no longer aligns properly with the stock at certain aim angles. This kind of joint misalignment is caused by LERP blending. Even if the joints in question are aligned perfectly in clip A and in clip B, LERP blending does not guarantee that those joints will be in alignment when A and B are blended together.

One solution to this problem is to use *inverse kinematics* (IK) to correct the position of the left hand. The basic approach is to determine the desired target position for the joint in question. IK is then applied to a short chain of joints (usually two, three or four joints), starting with the joint in question and

progressing up the hierarchy to its parent, grandparent and so on. The joint whose position we are trying to correct is known as the *end effector*. The IK solver adjusts the orientations of the end effector's parent joint(s) in order to get the end effector as close as possible to the target.

The API for an IK system usually takes the form of a request to enable or disable IK on a particular chain of joints, plus a specification of the desired target point. The actual IK calculation is usually done internally by the low-level animation pipeline. This allows it to do the calculation at the proper time—namely, after intermediate local and global skeletal poses have been calculated but before the final matrix palette calculation.

Some animation engines allow IK chains to be defined a priori. For example, we might define one IK chain for the left arm, one for the right arm and two for the two legs. Let's assume for the purposes of this example that a particular IK chain is identified by the name of its end-effector joint. (Other engines might use an index or handle or some other unique identifier, but the concept remains the same.) The function to enable an IK calculation might look something like this:

```
void enableIkChain(Actor& actor,
                  const char* endEffectorJointName,
                  const Vector3& targetLocationWs);
```

and the function to disable an IK chain might look like this:

```
void disableIkChain(Actor& actor,
                   const char* endEffectorJointName);
```

IK is usually enabled and disabled relatively infrequently, but the world-space target location must be kept up-to-date every frame (if the target is moving). Therefore, the low-level animation pipeline always provides some mechanism for updating an active IK target point. For example, the pipeline might allow us to call `enableIkChain()` multiple times. The first time it is called, the IK chain is enabled, and its target point is set. All subsequent calls simply update the target point. Another way to keep IK targets up-to-date is to link them to dynamic objects in the game. For example, an IK target might be specified as a handle to a rigid game object, or a joint within an animated object.

IK is well-suited to making minor corrections to joint alignment when the joint is already reasonably close to its target. It does not work nearly as well when the error between a joint's desired location and its actual location is large. Note also that most IK algorithms solve only for the *position* of a joint. You may

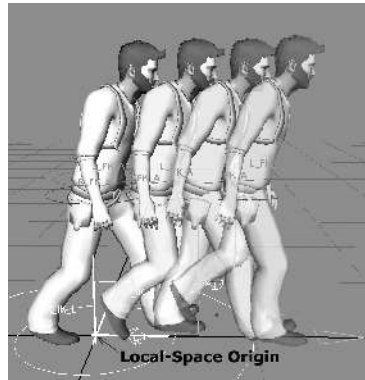


Figure 12.64. In the animation authoring package, the character moves forward in space, and its feet appear grounded. Image courtesy of Naughty Dog, Inc. (UNCHARTED: Drake's Fortune © 2007/© SIE. Created and developed by Naughty Dog.)

need to write additional code to ensure that the *orientation* of the end effector aligns properly with its target as well. IK is not a cure-all, and it may have significant performance costs. So always use it judiciously.

12.11.4 Motion Extraction and Foot IK

In games, we usually want the locomotion animations of our characters to look realistic and “grounded.” One of the biggest factors contributing to the realism of a locomotion animation is whether or not the feet slide around on the ground. *Foot sliding* can be overcome in a number of ways, the most common of which are *motion extraction* and *foot IK*.

12.11.4.1 Motion Extraction

Let’s imagine how we’d animate a character walking forward in a straight line. In Maya (or his or her animation package of choice), the animator makes the character take one complete step forward, first with the left foot and then with the right foot. The resulting animation clip is known as a *locomotion cycle*, because it is intended to be looped indefinitely, for as long as the character is walking forward in-game. The animator takes care to ensure that the feet of the character appear grounded and don’t slide as it moves. The character moves from its initial location on frame 0 to a new location at the end of the cycle. This is shown in Figure 12.64.

Notice that the local-space origin of the character remains fixed during the entire walk cycle. In effect, the character is “leaving his origin behind him” as he takes his step forward. Now imagine playing this animation as a loop. We



Figure 12.65. Walk cycle after zeroing out the root joint's forward motion. Image courtesy of Naughty Dog, Inc. (UNCHARTED: Drake's Fortune © 2007/© SIE. Created and developed by Naughty Dog.)

would see the character take one complete step forward, and then pop back to where he was on the first frame of the animation. Clearly this won't work in-game.

To make this work, we need to remove the forward motion of the character, so that his local-space origin remains roughly under the center of mass of the character at all times. We could do this by zeroing out the forward translation of the root joint of the character's skeleton. The resulting animation clip would make the character look like he's "moonwalking," as shown in Figure 12.65.

In order to get the feet to appear to "stick" to the ground the way they did in the original Maya scene, we need the character to move forward by just the right amount each frame. We could look at the distance the character moved, divide by the amount of time it took for him to get there, and hence find his average movement speed. But a character's forward speed is not constant when walking. This is especially evident when a character is limping (quick forward motion on the injured leg, followed by slower motion on the "good" leg), but it is true for all natural-looking walk cycles.

Therefore, before we zero out the forward motion of the root joint, we first save the animation data in a special "extracted motion" channel. This data can be used in-game to move the local-space origin of the character forward by the exact amount that the root joint had moved in Maya each frame. The net result is that the character will walk forward exactly as he was authored, but now his local-space origin comes along for the ride, allowing the animation to loop properly. This is shown in Figure 12.66.

If the character moves forward by 4 feet in the animation and the animation

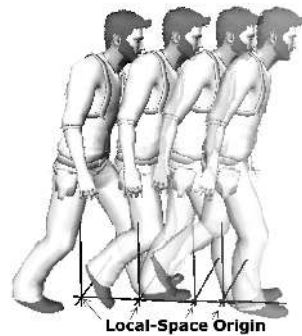


Figure 12.66. Walk cycle in-game, with extracted root motion data applied to the local-space origin of the character. Image courtesy of Naughty Dog, Inc. (UNCHARTED: Drake's Fortune © 2007/© SIE. Created and developed by Naughty Dog.)

takes one second to complete, then we know that the character is moving at an average speed of 4 feet/second. To make the character walk at a different speed, we can simply scale the playback rate of the walk cycle animation. For example, to make the character walk at 2 feet/second, we can simply play the animation at half speed ($R = 0.5$).

12.II.4.2 Foot IK

Motion extraction does a good job of making a character's feet appear grounded when it is moving in a straight line (or, more correctly, when it moves in a path that exactly matches the path animated by the animator). However, a real game character must be turned and moved in ways that don't coincide with the original hand-animated path of motion (e.g., when moving over uneven terrain). This results in additional foot sliding.

One solution to this problem is to use IK to correct for any sliding in the feet. The basic idea is to analyze the animations to determine during which periods of time each foot is fully in contact with the ground. At the moment a foot contacts the ground, we note its world-space location. For all subsequent frames while that foot remains on the ground, we use IK to adjust the pose of the leg so that the foot remains fixed to the proper location. This technique sounds easy enough, but getting it to look and feel right can be very challenging. It requires a lot of iteration and fine-tuning. And some natural human motions—like leading into a turn by increasing your stride—cannot be produced by IK alone.

In addition, there is a big trade-off between the *look* of the animations and the *feel* of the character, particularly for a human-controlled character. It's generally more important for the player character control system to feel responsive

and fun than it is for the character's animations to look perfect. The upshot is this: Do not take the task of adding foot IK or motion extraction to your game lightly. Budget time for a lot of trial and error, and be prepared to make trade-offs to ensure that your player character not only looks good but *feels* good as well.

12.11.5 Other Kinds of Constraints

There are plenty of other possible kinds of constraint systems that can be added to a game animation engine. Some examples include:

- *Look-at.* This is the ability for characters to look at points of interest in the environment. A character might look at a point with only his or her eyes, with eyes and head, or with eyes, head and a twist of the entire upper body. Look-at constraints are sometimes implemented using IK or procedural joint offsets, although a more natural look can often be achieved via additive blending.
- *Cover registration.* This is the ability for a character to align perfectly with an object that is serving as cover. This is often implemented via the reference locator technique described above.
- *Cover entry and departure.* If a character can take cover, animation blending and custom entry and departure animations must usually be used to get the character into and out of cover.
- *Traversal aids.* The ability for a character to navigate over, under, around or through obstacles in the environment can add a lot of life to a game. This is often done by providing custom animations and using a reference locator to ensure proper registration with the obstacle being overcome.

13

Collision and Rigid Body Dynamics

In the real world, solid objects are inherently, well...solid. They generally avoid doing impossible things, like passing through one another, all by themselves. But in a virtual game world, objects don't do anything unless we tell them to, and game programmers must make an explicit effort to ensure that objects do not pass through one another. This is the role of one of the central components of any game engine—the *collision detection system*.

A game engine's collision system is often closely integrated with a *physics engine*. Of course, the field of physics is vast, and what most of today's game engines call "physics" is more accurately described as a *rigid body dynamics* simulation. A *rigid body* is an idealized, infinitely hard, non-deformable solid object. The term *dynamics* refers to the process of determining how these rigid bodies *move* and *interact* over time under the influence of *forces*. A rigid body dynamics simulation allows motion to be imparted to objects in the game in a highly interactive and naturally chaotic manner—an effect that is much more difficult to achieve when using canned animation clips to move things about.

A dynamics simulation makes heavy use of the collision detection system in order to properly simulate various physical behaviors of the objects in the simulation, including bouncing off one another, sliding under friction, rolling and coming to rest. Of course, a collision detection system can be used stand-alone, without a dynamics simulation—many games do not have a "physics"

system at all. But all games that involve objects moving about in two- or three-dimensional space have some form of collision detection.

In this chapter, we'll investigate the architecture of both a typical collision detection system and a typical physics (rigid body dynamics) system. As we investigate the components of these two closely interrelated systems, we'll take a look at the mathematics and the theory that underlie them.

13.1 Do You Want Physics in Your Game?

Nowadays, most game engines have some kind of physical simulation capabilities. Some physical effects, like rag doll deaths, are simply expected by gamers. Other effects, like ropes, cloth, hair or complex physically driven machinery can add that *je ne sais quoi* that sets a game apart from its competitors. In recent years, some game studios have started experimenting with advanced physical simulations, including approximate real-time fluid mechanics effects and simulations of deformable bodies. But adding physics to a game is not without costs, and before we commit ourselves to implementing an exhaustive list of physics-driven features in our game, we should (at the very least) understand the trade-offs involved.

13.1.1 Things You Can Do with a Physics System

Here are just a few of the things you can do or have with a game physics system.

- Detect collisions between dynamic objects and static world geometry.
- Simulate free rigid bodies under the influence of gravity and other forces.
- Spring-mass systems.
- Destructible buildings and structures.
- Ray and shape casts (to determine line of sight, bullet impacts, etc.).
- Trigger volumes (determine when objects enter, leave or are inside pre-defined regions in the game world).
- Complex machines (cranes, moving platform puzzles and so on).
- Traps (such as an avalanche of boulders).
- Drivable vehicles with realistic suspensions.
- Rag doll character deaths.
- Powered rag doll: a realistic blend between traditional animation and rag doll physics.

- Dangling props (canteens, necklaces, swords), semi-realistic hair, clothing movements.
- Cloth simulations.
- Water surface simulations and buoyancy.
- Audio propagation.

And the list goes on.

We should note here that in addition to running a physics simulation at runtime in our game, we can also run a simulation as part of an offline pre-processing step in order to generate an animation clip. A number of physics plug-ins are available for animation tools like Maya. This is also the approach taken by the *Endorphin*¹ package by NaturalMotion, Inc. (<http://www.naturalmotion.com/endorphin.htm>). In this chapter, we'll restrict our discussion to runtime rigid body dynamics simulations, but offline tools are a powerful option, of which we should always remain aware as we plan our game projects.

13.1.2 Is Physics Fun?

The presence of a rigid body dynamics system in a game does not necessarily make the game fun. More often than not, the inherently chaotic behavior of a physics sim can actually detract from the gameplay experience rather than enhancing it. The fun derived from physics depends on many factors, including the quality of the simulation itself, the care with which it has been integrated with other engine systems, the selection of physics-driven gameplay elements versus elements that are controlled in a more direct manner, how the physical elements interact with the goals of the player and the abilities of the player character, and the genre of game being made.

Let's take a look at a few broad game genres and how a rigid body dynamics system might fit into each one.

13.1.2.1 Simulations (Sims)

The primary goal of a sim is to accurately reproduce a real-life experience. Examples include the *Flight Simulator*, *Gran Turismo* and *NASCAR Racing* series of games. Clearly, the realism provided by a rigid body dynamics system fits extremely well into these kinds of games.

¹NaturalMotion also offers a runtime version of Endorphin called *Euphoria*.

13.1.2.2 Physics Puzzle Games

The whole idea of a physics puzzle is to let the user play around with dynamically simulated toys. So obviously this kind of game relies almost entirely on physics for its core mechanic. Examples of this genre include *Bridge Builder*, *The Incredible Machine*, the online game *Fantastic Contraption*, and *Crayon Physics* for the iPhone.

13.1.2.3 Sandbox Games

In a sandbox game, there may be no objectives at all, or there may be a large number of optional objectives. The player's primary objective is usually to "mess around" and explore what the objects in the game world can be made to do. Examples of sandbox games include *Besiege*, *Spore*, the *LittleBigPlanet* series, and of course *Minecraft*.

Sandbox games can put a realistic dynamics simulation to good use, especially if much of the fun is derived from playing with realistic (or semi-realistic) interactions between objects in the game world. So in these contexts, physics can be fun in and of itself. However, many games trade realism for an increased fun factor (e.g., larger-than-life explosions, gravity that is stronger or weaker than normal, etc.). So the dynamics simulation may need to be tweaked in various ways to achieve the right "feel."

13.1.2.4 Goal-Based and Story-Driven Games

A goal-based game has rules and specific objectives that the player must accomplish in order to progress; in a story-driven game, telling a story is of paramount importance. Integrating a physics system into these kinds of games can be tricky. We generally give away *control* in exchange for a *realistic simulation*, and this loss of control can inhibit the player's ability to accomplish goals or the game's ability to tell the story.

For example, in a character-based platformer game, we want the player character to move in ways that are fun and easy to control but not necessarily physically realistic. In a war game, we might want a bridge to explode in a realistic way, but we also may want to ensure that the debris doesn't end up blocking the player's only path forward. In these kinds of games, physics is often not necessarily fun, and in fact it can often get in the way of fun when the player's goals are at odds with the physically simulated behaviors of the objects in the game world. Therefore, developers must be careful to apply physics judiciously and take steps to control the behavior of the simulation in various ways to ensure it doesn't get in the way of gameplay. It's usually a good idea to provide the player with a way out of difficult situations, too.

A good example of this can be found in the *Halo* series of games, where the player can press X to flip over a vehicle that has landed upside-down.

13.1.3 Impact of Physics on a Game

Adding a physics simulation to a game can have all sorts of impacts on the project and the gameplay. Here are a few examples across various game development disciplines.

13.1.3.1 Design Impacts

- *Predictability.* The inherent chaos and variability that sets a physically simulated behavior apart from an animated one is also a source of unpredictability. If something absolutely must happen a certain way every time, it's usually better to animate it than to try to coerce your dynamics simulation into producing the motion reliably.
- *Tuning and control.* The laws of physics (when modeled accurately) are fixed. In a game, we can tweak the value of gravity or the coefficient of restitution of a rigid body, which gives back some degree of control. However, the results of tweaking physics parameters are often indirect and difficult to visualize. It's much harder to tweak a force in order to get a character to move in the desired direction than it is to tweak an animation of a character walking.
- *Emergent behaviors.* Sometimes physics introduces unexpected features into a game—for example, the rocket-launcher jump trick in *Team Fortress Classic*, the high-flying exploding Warthog in *Halo* and the flying “surfboards” in *PsyOps*.

In general, the game design should usually drive the physics requirements of a game engine—not the other way around.

13.1.3.2 Engineering Impacts

- *Tools pipeline.* A good collision/physics pipeline takes time to build and maintain.
- *User interface.* How does the player control the physics objects in the world? Does he or she shoot them? Walk into them? Pick them up? Does he or she hold them using virtual arms, as in *Trespasser*? Or using a “gravity gun,” as in *Half-Life 2*?
- *Collision detection.* Collision models intended for use within a dynamics simulation may need to be more detailed and more carefully constructed than their non-physics-driven counterparts.

- *AI.* Pathing may not be predictable in the presence of physically simulated objects. The engine may need to handle dynamic cover points that can move or blow up. Can the AI use the physics to its advantage?
- *Misbehaved objects.* Animation-driven objects can clip slightly through one another with few or no ill effects. But when driven by a dynamics simulation, objects may bounce off one another in unexpected ways or jitter badly. Collision filtering may need to be applied to permit objects to interpenetrate slightly. Mechanisms may need to be put in place to ensure that objects settle and go to sleep properly.
- *Rag doll physics.* Rag dolls require a lot of fine-tuning and often suffer from instability in the simulation. An animation may drive parts of a character's body into penetration with other collision volumes—when the character turns into a rag doll, these interpenetrations can cause enormous instability. Steps must be taken to avoid this.
- *Graphics.* Physics-driven motion can have an effect on renderable objects' bounding volumes (where they would otherwise be static or more predictable). The presence of destructible buildings and objects can invalidate some kinds of precomputed lighting and shadow methods.
- *Networking and multiplayer.* Physics effects that do not affect gameplay may be simulated exclusively (and independently) on each client machine. However, physics that has an effect on gameplay (such as the trajectory that a grenade follows) must be simulated on the server and accurately replicated on all clients.
- *Record and playback.* The ability to record gameplay and play it back at a later time is very useful as a debugging/testing aid, and it can also serve as a fun game feature. This feature is difficult to implement because it requires every engine system to behave in a deterministic manner, so that everything will play out exactly in the same way during playback as it did when the recording was made. If your physics simulation isn't deterministic, this can become a major fly in the ointment.

13.1.3.3 Art Impacts

- *Additional tool and workflow complexity.* The need to rig up objects with mass, friction, constraints and other attributes for consumption by the dynamics simulation makes the art department's job more difficult as well.
- *More complex content.* We may need multiple visually identical versions of an object with different collision and dynamics configurations for dif-

ferent purposes—for example, a pristine version and a destructible version.

- *Loss of control.* The unpredictability of physics-driven objects can make it difficult to control the artistic composition of a scene.

13.1.3.4 Other Impacts

- *Interdisciplinary impacts.* The introduction of a dynamics simulation into your game requires close cooperation between engineering, art, audio and design.
- *Production impacts.* Physics can add to a project's development costs, technical and organizational complexity and risk.

Having explored the impacts, most teams today do choose to integrate a rigid body dynamics system into their games. With some careful planning and wise choices along the way, adding physics to your game can be rewarding and fruitful. And as we'll see below, third-party middleware is making physics more accessible than ever.

13.2 Collision/Physics Middleware

Writing a collision system and rigid body dynamics simulation is challenging and time-consuming work. The collision/physics system of a game engine can account for a significant percentage of the source code in a typical game engine. That's a lot of code to write and maintain!

Thankfully, a number of robust, high-quality collision/physics engines are now available, either as commercial products or in open source form. Some of these are listed below. For a discussion of the pros and cons of various physics SDKs, check out the online game development forums (e.g., http://www.gamedev.net/community/forums/topic.asp?topic_id=463024).

13.2.1 ODE

ODE stands for “Open Dynamics Engine” (<http://www.ode.org>). As its name implies, ODE is an open source collision and rigid body dynamics SDK. Its feature set is similar to a commercial product like Havok. Its benefits include being free (a big plus for small game studios and school projects!) and the availability of full source code (which makes debugging much easier and opens up the possibility of modifying the physics engine to meet the specific needs of a particular game).

13.2.2 Bullet

Bullet is an open source collision detection and physics library used by both the game and film industries. Its collision engine is integrated with its dynamics simulation, but hooks are provided so that the collision system can be used stand-alone or integrated with other physics engines. It supports *continuous collision detection* (CCD)—also known as *time of impact* (TOI) collision detection—which, as we’ll see below, can be extremely helpful when a simulation includes small, fast-moving objects. The Bullet SDK is available for download at <http://code.google.com/p/bullet/>, and the Bullet wiki is located at http://www.bulletphysics.com/mediawiki-1.5.8/index.php?title=Main_Page.

13.2.3 TrueAxis

TrueAxis is another collision/physics SDK. It is free for non-commercial use. You can learn more about TrueAxis at <http://trueaxis.com>.

13.2.4 PhysX

PhysX started out as a library called Novodex, produced and distributed by Ageia as part of their strategy to market their dedicated physics coprocessor. It was bought by NVIDIA and retooled so that it can run using NVIDIA’s GPUs as a coprocessor. (It can also run entirely on a CPU, without GPU support.) It is available at http://www.nvidia.com/object/nvidia_physx.html. Part of Ageia’s and NVIDIA’s marketing strategy has been to provide the CPU version of the SDK entirely for free, in order to drive the physics coprocessor market forward. Developers can also pay a fee to obtain full source code and the ability to customize the library as needed. PhysX is now combined with APEX, NVIDIA’s scalable multiplatform dynamics framework. PhysX/APEX is available for Windows, Linux, Mac, Android, Xbox 360, PlayStation 3, Xbox One, PlayStation 4 and Wii.

13.2.5 Havok

Havok is the gold standard in commercial physics SDKs, providing one of the richest feature sets available and boasting excellent performance characteristics on all supported platforms. (It’s also the most expensive solution.) Havok is comprised of a core collision/physics engine, plus a number of optional add-on products including a vehicle physics system, a system for modeling destructible environments and a fully featured animation SDK with direct integration into Havok’s rag doll physics system. It is available on Xbox 360,

PlayStation 3, Xbox One, PlayStation 4, PlayStation Vita, Wii, Wii U, Windows 8, Android, Apple Mac and iOS. You can learn more about Havok at <http://www.havok.com>.

13.2.6 Physics Abstraction Layer (PAL)

The Physics Abstraction Layer (PAL) is an open source library that allows developers to work with more than one physics SDK on a single project. It provides hooks for PhysX (Novodex), Newton, ODE, OpenTissue, Tokamak, TrueAxis and a few other SDKs. You can read more about PAL at <http://www.adrianboeing.com/pal/index.html>.

13.2.7 Digital Molecular Matter (DMM)

Pixelux Entertainment S.A., located in Geneva, Switzerland, has produced a unique physics engine that uses finite element methods to simulate the dynamics of deformable bodies and breakable objects, called Digital Molecular Matter (DMM). The engine has both an offline and a runtime component. It was released in 2008 and can be seen in action in LucasArts' *Star Wars: The Force Unleashed*. A discussion of deformable body mechanics is beyond our scope here, but you can read more about DMM at <http://www.pixeluxentertainment.com>.

13.3 The Collision Detection System

The primary purpose of a game engine's collision detection system is to determine whether any of the objects in the game world have come into *contact*. To answer this question, each logical object is represented by one or more geometric *shapes*. These shapes are usually quite simple, such as spheres, boxes and capsules. However, more complex shapes can also be used. The collision system determines whether or not any of the shapes are *intersecting* (i.e., overlapping) at any given moment in time. So a collision detection system is essentially a glorified geometric intersection tester.

Of course, the collision system does more than answer yes/no questions about shape intersection. It also provides relevant information about the nature of each contact. Contact information can be used to prevent unrealistic visual anomalies on-screen, such as objects *interpenetrating* one another. This is generally accomplished by moving all interpenetrating objects apart prior to rendering the next frame. Collisions can provide *support* for an object—one or more contacts that together allow the object to come to rest, in equilibrium with gravity and/or any other forces acting on it. Collisions can also be used

for other purposes, such as to cause a missile to explode when it strikes its target or to give the player character a health boost when he passes through a floating health pack. A rigid body dynamics simulation is often the most demanding client of the collision system, using it to mimic physically realistic behaviors like bouncing, rolling, sliding and coming to rest. But, of course, even games that have no physics system can still make heavy use of a collision detection engine.

In this chapter, we'll go on a brief high-level tour of how collision detection engines work. For an in-depth treatment of this topic, a number of excellent books on real-time collision detection are available, including [14], [48] and [11].

13.3.1 Collidable Entities

If we want a particular logical object in our game to be capable of colliding with other objects, we need to provide it with a *collision representation*, describing the object's shape and its position and orientation in the game world. This is a distinct data structure, separate from the object's *gameplay representation* (the code and data that define its role and behavior in the game) and separate from its *visual representation* (which might be an instance of a triangle mesh, a subdivision surface, a particle effect or some other visual representation).

From the point of view of detecting intersections, we generally favor shapes that are geometrically and mathematically simple. For example, a rock might be modeled as a sphere for collision purposes; the hood of a car might be represented by a rectangular box; a human body might be approximated by a collection of interconnected *capsules* (pill-shaped volumes). Ideally, we should resort to a more complex shape only when a simpler representation proves inadequate to achieve the desired behavior in the game. Figure 13.1 shows a few examples of using simple shapes to approximate object volumes for collision detection purposes.

Havok uses the term *collidable* to describe a distinct, rigid object that can take part in collision detection. It represents each collidable with an instance of the C++ class `hkpCollidable`. PhysX calls its rigid objects *actors* and represents them as instances of the class `NxActor`. In both of these libraries, a collidable entity contains two basic pieces of information—a *shape* and a *transform*. The shape describes the collidable's geometric form, and the transform describes the shape's position and orientation in the game world. Collidables need transforms for three reasons:

1. Technically speaking, a shape only describes the form of an object (i.e., whether it is a sphere, a box, a capsule or some other kind of volume).

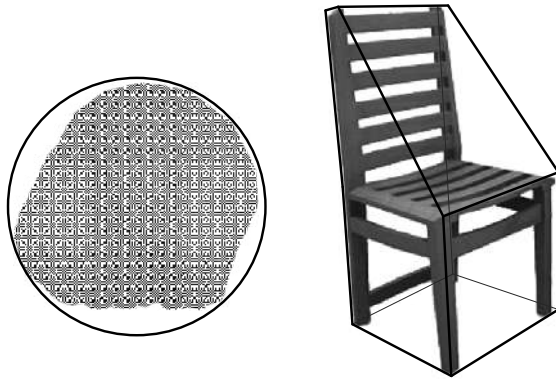


Figure 13.1. Simple geometric shapes are often used to approximate the collision volumes of the objects in a game.

It may also describe the object's size (e.g., the radius of a sphere or the dimensions of a box). But a shape is usually defined with its center at the origin and in some sort of canonical orientation relative to the coordinate axes. To be useful, a shape must therefore be transformed in order to position and orient it appropriately in world space.

2. Many of the objects in a game are dynamic. Moving an arbitrarily complex shape through space could be expensive if we had to move the *features* of the shape (vertices, planes, etc.) individually. But with a transform, any shape can be moved in space inexpensively, no matter how simple or complex the shape's features may be.
3. The information describing some of the more complex kinds of shapes can take up a nontrivial amount of memory. So, it can be beneficial to permit more than one collidable to share a single shape description. For example, in a racing game, the shape information for many of the cars might be identical. In that case, all of the car collidables in the game can share a single car shape.

Any particular object in the game may have no collidable at all (if it doesn't require collision detection services), a single collidable (if the object is a simple rigid body) or multiple collidables (each representing one rigid component of an articulated robot arm, for example).

13.3.2 The Collision/Physics World

A collision system typically keeps track of all of its collidable entities via a singleton data structure known as the *collision world*. The collision world is a complete representation of the game world designed explicitly for use by the collision detection system. Havok's collision world is an instance of the class `hkpWorld`. Likewise, the PhysX world is an instance of `NxScene`. ODE uses an instance of class `dSpace` to represent the collision world; it is actually the root of a hierarchy of geometric volumes representing all the collidable shapes in the game.

Maintaining all collision information in a private data structure has a number of advantages over attempting to store collision information with the game objects themselves. For one thing, the collision world need only contain collidables for those game objects that can potentially collide with one another. This eliminates the need for the collision system to iterate over any irrelevant data structures. This design also permits collision data to be organized in the most efficient manner possible. The collision system can take advantage of cache coherency to maximize performance, for example. The collision world is also an effective encapsulation mechanism, which is generally a plus from the perspectives of understandability, maintainability, testability and the potential for software reuse.

13.3.2.1 The Physics World

If a game has a rigid body dynamics system, it is usually tightly integrated with the collision system. It typically shares its “world” data structure with the collision system, and each rigid body in the simulation is usually associated with a single collidable in the collision system. This design is commonplace among physics engines because of the frequent and detailed collision queries required by the physics system. It's typical for the physics system to actually *drive* the operation of the collision system, instructing it to run collision tests at least once, and sometimes multiple times, per simulation time step. For this reason, the collision world is often called the *collision/physics world* or sometimes just the *physics world*.

Each dynamic rigid body in the physics simulation is usually associated with a single collidable object in the collision system (although not all collidables need be dynamic rigid bodies). For example, in Havok, a rigid body is represented by an instance of the class `hkpRigidBody`, and each rigid body has a pointer to exactly one `hkpCollidable`. In PhysX, the concepts of collidable and rigid body are comingled—the `NxActor` class serves both purposes (although the physical properties of the rigid body are stored separately, in

an instance of `NxBodyDesc`). In both SDKs, it is possible to tell a rigid body that its location and orientation are to be fixed in space, meaning that it will be omitted from the dynamics simulation and will serve as a collidable only.

Despite this tight integration, most physics SDKs do make at least some attempt to separate the collision library from the rigid body dynamics simulation. This permits the collision system to be used as a stand-alone library (which is important for games that don't need physics but do need to detect collisions). It also means that a game studio could *theoretically* replace a physics SDK's collision system entirely, without having to rewrite the dynamics simulation. (Practically speaking, this may be a bit harder than it sounds!)

13.3.3 Shape Concepts

A rich body of mathematical theory underlies the everyday concept of *shape* (see <http://en.wikipedia.org/wiki/Shape>). For our purposes, we can think of a shape simply as a region of space described by a boundary, with a definite *inside* and *outside*. In two dimensions, a shape has area, and its boundary is defined either by a curved line or by three or more straight edges (in which case it's a *polygon*). In three dimensions, a shape has volume, and its boundary is either a curved surface or is composed of polygons (in which case it is called a *polyhedron*).

It's important to note that some kinds of game objects, like terrain, rivers or thin walls, might be best represented by *surfaces*. In three-space, a surface is a two-dimensional geometric entity with a *front* and a *back* but no inside or outside. Examples include planes, triangles, subdivision surfaces and surfaces constructed from a group of connected triangles or other polygons. Most collision SDKs provide support for surface primitives and extend the term *shape* to encompass both closed volumes and open surfaces.

It's commonplace for collision libraries to allow surfaces to be given volume via an optional extrusion parameter. Such a parameter specifies how "thick" a surface should be. Doing this helps reduce the occurrence of missed collisions between small, fast-moving objects and infinitesimally thin surfaces (the so-called "bullet through paper" problem—see Section 13.3.5.7).

13.3.3.1 Intersection

We all have an intuitive notion of what an *intersection* is. Technically speaking, the term comes from set theory ([http://en.wikipedia.org/wiki/Intersection_\(set_theory\)](http://en.wikipedia.org/wiki/Intersection_(set_theory))). The intersection of two sets is comprised of the subset of members that are common to both sets. In geometrical terms, the intersection between two shapes is just the (infinitely large!) set of all points that lie inside both shapes.

13.3.3.2 Contact

In games, we're not usually interested in finding the intersection in the strictest sense, as a set of points. Instead, we want to know simply whether or not two objects are intersecting. In the event of a collision, the collision system will usually provide additional information about the nature of the contact. This information allows us to separate the objects in a physically plausible and efficient way, for example.

Collision systems usually package contact information into a convenient data structure that can be instanced for each contact detected. For example, Havok returns contacts as instances of the class `hkContactPoint`. Contact information often includes a *separating vector*—a vector along which we can slide the objects in order to efficiently move them out of collision. It also typically contains information about which two collidables were in contact, including which individual shapes were intersecting and possibly even which individual features of those shapes were in contact. The system may also return additional information, such as the velocity of the bodies projected onto the separating normal.

13.3.3.3 Convexity

One of the most important concepts in the field of collision detection is the distinction between *convex* and *non-convex* (i.e., *concave*) shapes. Technically, a convex shape is defined as one for which no ray originating inside the shape will pass through its surface more than once. A simple way to determine if a shape is convex is to imagine shrink-wrapping it with plastic film—if it's convex, no air pockets will be left under the film. So in two dimensions, circles, rectangles and triangles are all convex, but Pac Man is not. The concept extends equally well to three dimensions.

The property of *convexity* is important because, as we'll see, it's generally simpler and less computationally intensive to detect intersections between convex shapes than concave ones. See <http://en.wikipedia.org/wiki/Convex> for more information about convex shapes.

13.3.4 Collision Primitives

Collision detection systems can usually work with a relatively limited set of shape types. Some collision systems refer to these shapes as *collision primitives* because they are the fundamental building blocks out of which more complex shapes can be constructed. In this section, we'll take a brief look at some of the most common types of collision primitives.

13.3.4.1 Spheres

The simplest three-dimensional volume is a sphere. And as you might expect, spheres are the most efficient kind of collision primitive. A sphere is represented by a center point and a radius. This information can be conveniently packed into a four-element floating-point vector—a format that works particularly well with SIMD math libraries.

13.3.4.2 Capsules

A capsule is a pill-shaped volume, composed of a cylinder and two hemispherical end caps. It can be thought of as a *swept sphere*—the shape that is traced out as a sphere moves from point A to point B. (There are, however, some important differences between a static capsule and a sphere that sweeps out a capsule-shaped volume over time, so the two are not identical.) Capsules are often represented by two points and a radius (Figure 13.2). Capsules are more efficient to intersect than cylinders or boxes, so they are often used to model objects that are roughly cylindrical, such as the limbs of a human body.

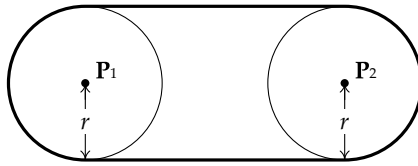


Figure 13.2. A capsule can be represented by two points and a radius.

13.3.4.3 Axis-Aligned Bounding Boxes

An axis-aligned bounding box (AABB) is a rectangular volume (technically known as a *cuboid*) whose faces are parallel to the axes of the coordinate system. Of course, a box that is axis-aligned in one coordinate system will not necessarily be axis-aligned in another. So we can only speak about an AABB in the context of the particular coordinate frame(s) with which it aligns.

An AABB can be conveniently defined by two points: one containing the minimum coordinates of the box along each of the three principal axes and the other containing its maximum coordinates. This is depicted in Figure 13.3.

The primary benefit of axis-aligned boxes is that they can be tested for interpenetration with other axis-aligned boxes in a highly efficient manner. The big limitation of using AABBs is that they must remain axis-aligned at all times if their computational advantages are to be maintained. This means that if an AABB is used to approximate the shape of an object in the game,

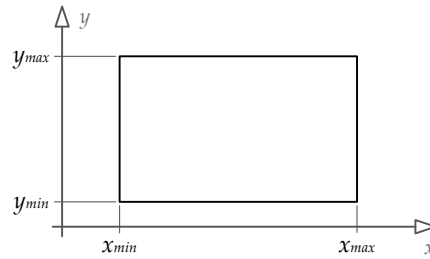


Figure 13.3. An axis-aligned box.

the AABB will have to be recalculated whenever that object rotates. Even if an object is roughly box-shaped, its AABB may degenerate into a very poor approximation to its shape when the object rotates off-axis. This is shown in Figure 13.4.

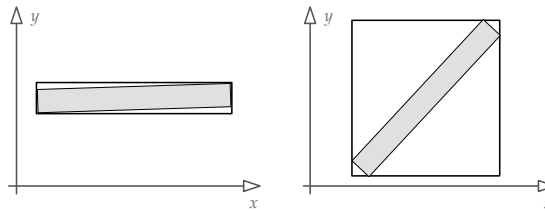


Figure 13.4. An AABB is only a good approximation to a box-shaped object when the object's principal axes are roughly aligned with the coordinated system's axes.

13.3.4.4 Oriented Bounding Boxes

If we permit an axis-aligned box to rotate relative to its coordinate system, we have what is known as an oriented bounding box (OBB). It is often represented by three half-dimensions (half-width, half-depth and half-height) and a transformation, which positions the center of the box and defines its orientation relative to the coordinate axes. Oriented boxes are a commonly used collision primitive because they do a better job at fitting arbitrarily oriented objects, yet their representation is still quite simple.

13.3.4.5 Discrete Oriented Polytopes (DOP)

A discrete oriented polytope (DOP) is a more-general case of the AABB and OBB. It is a convex polytope that approximates the shape of an object. A DOP can be constructed by taking a number of planes at infinity and sliding them along their normal vectors until they come into contact with the object whose

shape is to be approximated. An AABB is a 6-DOP in which the plane normals are taken parallel to the coordinate axes. An OBB is also a 6-DOP in which the plane normals are parallel to the object's natural principal axes. A k -DOP is constructed from an arbitrary number of planes k . A common method of constructing a DOP is to start with an OBB for the object in question and then bevel the edges and/or corners at 45 degrees with additional planes in an attempt to yield a tighter fit. An example of a k -DOP is shown in Figure 13.5.

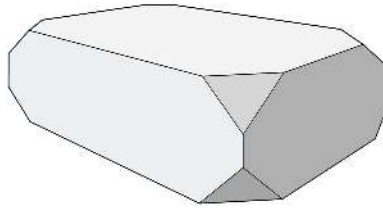


Figure 13.5. An OBB that has been beveled on all eight corners is known as a 14-DOP.

13.3.4.6 Arbitrary Convex Volumes

Most collision engines permit arbitrary convex volume to be constructed by a 3D artist in a package like Maya. The artist builds the shape out of polygons (triangles or quads). An offline tool analyzes the triangles to ensure that they actually do form a convex polyhedron. If the shape passes the convexity test, its triangles are converted into a collection of planes (essentially a k -DOP), represented by k plane equations, or k points and k normal vectors. (If it is found to be non-convex, it can still be represented by a polygon soup—described in the next section.) This approach is depicted in Figure 13.6.

Convex volumes are more expensive to intersection-test than the simpler geometric primitives we've discussed thus far. However, as we'll see in Section

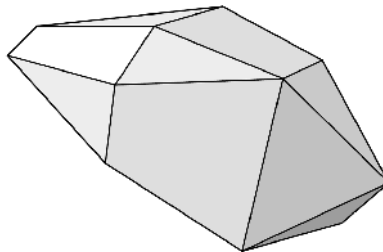


Figure 13.6. An arbitrary convex volume can be represented by a collection of intersecting planes.

13.3.5.5, certain highly efficient intersection-finding algorithms such as GJK are applicable to these shapes because they are convex.

13.3.4.7 Poly Soup

Some collision systems also support totally arbitrary, non-convex shapes. These are usually constructed out of triangles or other simple polygons. For this reason, this type of shape is often called a *polygon soup*, or *poly soup* for short. Poly soups are often used to model complex static geometry, such as terrain and buildings (Figure 13.7).

As you might imagine, detecting collisions with a poly soup is the most expensive kind of collision test. In effect, the collision engine must test every individual triangle, and it must also properly handle spurious intersections with triangle edges that are shared between adjacent triangles. As a result, most games try to limit the use of poly soup shapes to objects that will not take part in the dynamics simulation.

Does a Poly Soup Have an Inside?

Unlike convex and simple shapes, a poly soup does not necessarily represent a volume—it can represent an open surface as well. Poly soup shapes often don't include enough information to allow the collision system to differentiate between a closed volume and an open surface. This can make it difficult to know in which direction to push an object that is interpenetrating a poly soup in order to bring the two objects out of collision.

Thankfully, this is by no means an intractable problem. Each triangle in

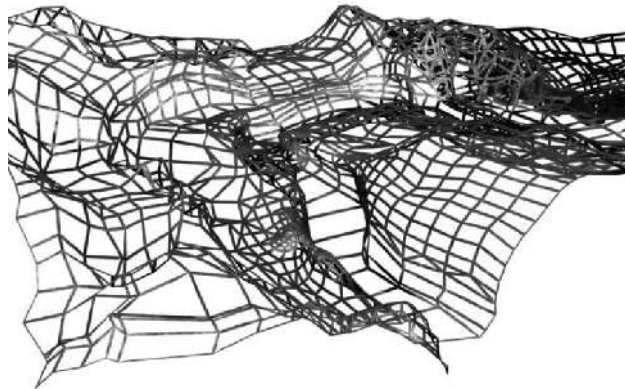


Figure 13.7. A poly soup is often used to model complex static surfaces such as terrain or buildings.

a poly soup has a front and a back, as defined by the winding order of its vertices. Therefore, it is possible to carefully construct a poly soup shape so that all of the polygons' vertex winding orders are consistent (i.e., adjacent triangles always "face" in the same direction). This gives the entire poly soup a notion of "front" and "back." If we also store information about whether a given poly soup shape is open or closed (presuming that this fact can be ascertained by offline tools), then for closed shapes, we can interpret "front" and "back" to mean "outside" and "inside" (or vice versa, depending on the conventions used when constructing the poly soup).

We can also "fake" an inside and outside for certain kinds of *open* poly soup shapes (i.e., surfaces). For example, if the terrain in our game is represented by an open poly soup, then we can decide arbitrarily that the front of the surface always points away from the Earth. This implies that "front" should always correspond to "outside." Practically speaking, to make this work, we would probably need to customize the collision engine in some way in order to make it aware of our particular choice of conventions.

13.3.4.8 Compound Shapes

Some objects that cannot be adequately approximated by a single shape can be approximated well by a *collection* of shapes. For example, a chair might be modeled out of two boxes—one for the back of the chair and one enclosing the seat and all four legs. This is shown in Figure 13.8.

A compound shape can often be a more-efficient alternative to a poly soup for modeling non-convex objects; two or more convex volumes can often outperform a single poly soup shape. What's more, some collision systems can take advantage of the convex bounding volume of the compound shape as a whole when testing for collisions. In Havok, this is called *midphase* collision detection. As the example in Figure 13.9 shows, the collision system first tests the convex bounding volumes of the two compound shapes. If they do not intersect, the system needn't test the subshapes for collisions at all.

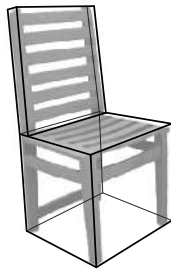


Figure 13.8. A chair can be modeled using a pair of interconnected box shapes.

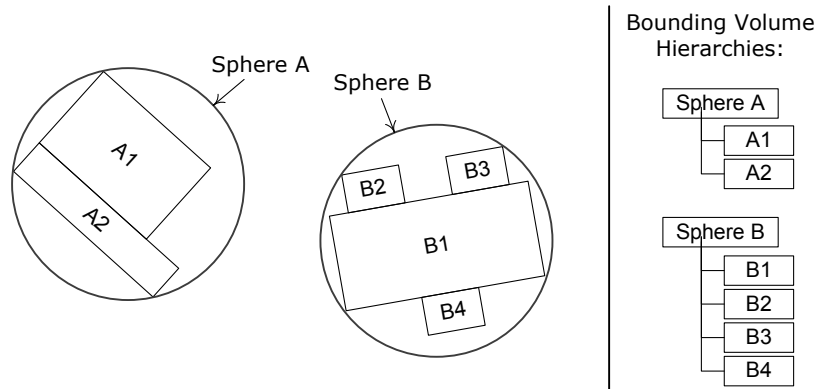


Figure 13.9. A collision system need only test the subshapes of a pair of compound shapes when their convex bounding volumes (in this case, Sphere A and Sphere B) are found to be intersecting.

13.3.5 Collision Testing and Analytical Geometry

A collision system makes use of *analytical geometry*—mathematical descriptions of three-dimensional volumes and surfaces—in order to detect intersections between shapes computationally. See http://en.wikipedia.org/wiki/Analytic_geometry for more details on this profound and broad area of research. In this section, we'll briefly introduce the concepts behind analytical geometry, show a few common examples and then discuss the generalized GJK intersection testing algorithm for arbitrary convex polyhedra.

13.3.5.1 Point versus Sphere

We can determine whether a point $\mathbf{p}$ lies within a sphere by simply forming the separation vector $\mathbf{s}$ between the point and the sphere's center $\mathbf{c}$ and then checking its length. If it is greater than the radius of the sphere r , then the point lies outside the sphere; otherwise, it lies inside:

$$\mathbf{s} = \mathbf{c} - \mathbf{p};$$

$$\text{if } |\mathbf{s}| \leq r, \text{ then } \mathbf{p} \text{ is inside.}$$

13.3.5.2 Sphere versus Sphere

Determining if two spheres intersect is almost as simple as testing a point against a sphere. Again, we form a vector $\mathbf{s}$ connecting the center points of the two spheres. We take its length and compare it with the sum of the radii of the two spheres. If the length of the separating vector is less than or equal to the sum of the radii, the spheres intersect; otherwise, they do not:

$$\mathbf{s} = \mathbf{c}_1 - \mathbf{c}_2; \tag{13.1}$$

$$\text{if } |\mathbf{s}| \leq (r_1 + r_2), \text{ then spheres intersect.}$$

To avoid the square root operation inherent in calculating the length of vector $\mathbf{s}$, we can simply square the entire equation. So Equation (13.1) becomes

$$\begin{aligned}\mathbf{s} &= \mathbf{c}_1 - \mathbf{c}_2; \\ |\mathbf{s}|^2 &= \mathbf{s} \cdot \mathbf{s}; \\ \text{if } |\mathbf{s}|^2 &\leq (r_1 + r_2)^2, \text{ then spheres intersect.}\end{aligned}$$

13.3.5.3 The Separating Axis Theorem

Most collision detection systems make heavy use of a theorem known as the *separating axis theorem* (http://en.wikipedia.org/wiki/Separating_axis_theorem). It states that if an axis can be found along which the *projections* of two convex shapes do not overlap, then we can be certain that the two shapes do not intersect at all. If such an axis does not exist *and* the shapes are convex, then we know for certain that they do intersect. (If the shapes are concave, then they may not be interpenetrating despite the lack of a separating axis. This is one reason why we tend to favor convex shapes in collision detection.)

This theorem is easiest to visualize in two dimensions. Intuitively, it says that if a line can be found, such that object A is entirely on one side of the line and object B is entirely on the other side, then objects A and B do not overlap. Such a line is called a *separating line*, and it is always *perpendicular* to the separating axis. So once we've found a separating line, it's a lot easier to convince ourselves that the theory is in fact correct by looking at the projections of our shapes onto the axis that is perpendicular to the separating line.

The projection of a two-dimensional *convex* shape onto an axis acts like the shadow that the object would leave on a thin wire. It is always a line segment, lying on the axis, that represents the maximum extents of the object in the direction of the axis. We can also think of a projection as a minimum and maximum coordinate along the axis, which we can write as the fully closed interval $[c_{\min}, c_{\max}]$. As you can see in Figure 13.10, when a separating line exists between two shapes, their projections do not overlap along the separating axis. However, the projections may overlap along other, non-separating axes.

In three dimensions, the separating line becomes a separating plane, but the separating axis is still an axis (i.e., an infinite line). Again, the projection of a three-dimensional *convex* shape onto an axis is a line segment, which we can represent by the fully closed interval $[c_{\min}, c_{\max}]$.

Some types of shapes have properties that make the potential separating axes obvious. To detect intersections between two such shapes A and B, we can project the shapes onto each potential separating axis in turn and then check whether or not the two projection intervals, $[c_{\min}^A, c_{\max}^A]$ and $[c_{\min}^B, c_{\max}^B]$,

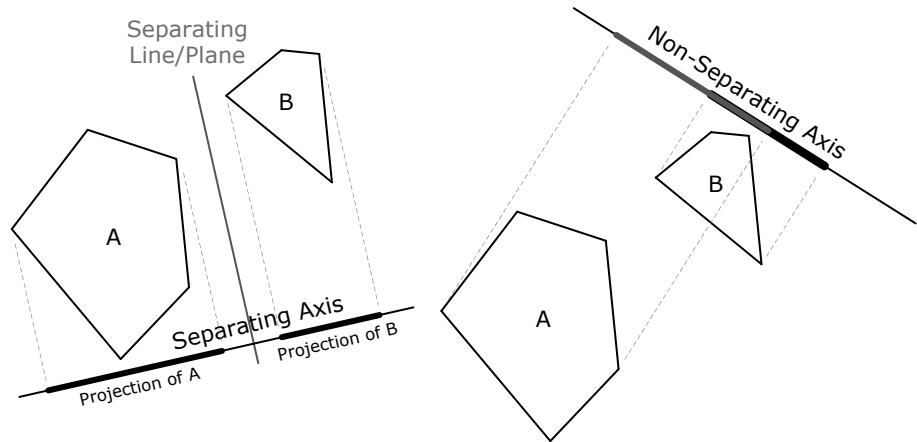


Figure 13.10. The projections of two shapes onto a separating axis are always two disjoint line segments. The projections of these same shapes onto a non-separating axis are not necessarily disjoint. If no separating axis exists, the shapes intersect.

are disjoint (i.e., do not overlap). In math terms, the intervals are disjoint if $c_{\max}^A < c_{\min}^B$ or if $c_{\max}^B < c_{\min}^A$. If the projection intervals along one of the potential separating axes are disjoint, then we've found a separating axis, and we know the two shapes do not intersect.

One example of this principle in action is the sphere-versus-sphere test. If two spheres do not intersect, then the axis parallel to the line segment joining the spheres' center points will always be a valid separating axis (although other separating axes may exist, depending on how far apart the two spheres are). To visualize this, consider the limit when the two spheres are just about to touch but have not yet come into contact. In that case, the *only* separating axis is the one parallel to the center-to-center line segment. As the spheres move apart, we can rotate the separating axis more and more in either direction. This is shown in Figure 13.11.

13.3.5.4 AABB versus AABB

To determine whether two AABBs are intersecting, we can again apply the separating axis theorem. The fact that the faces of both AABBs are guaranteed to lie parallel to a common set of coordinate axes tells us that if a separating axis exists, it will be one of these three coordinate axes.

So, to test for intersections between two AABBs, which we'll call A and B, we merely inspect the minimum and maximum coordinates of the two boxes along each axis independently. Along the x -axis, we have the two intervals

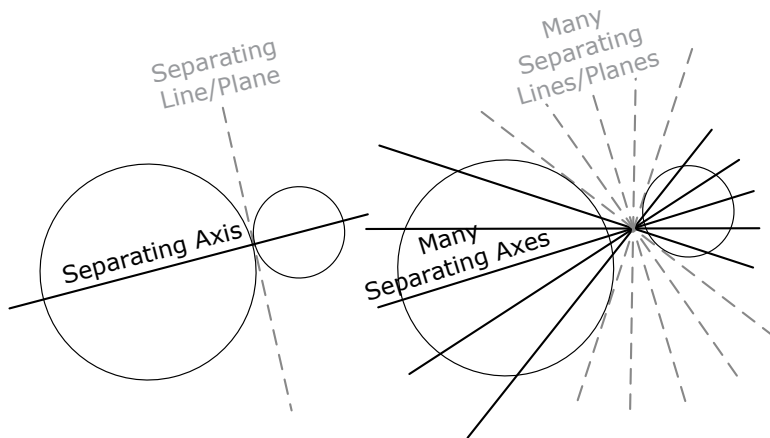


Figure 13.11. When two spheres are an infinitesimal distance apart, the only separating axis lies parallel to the line segment formed by the two spheres' center points.

$[x_{\min}^A, x_{\max}^A]$ and $[x_{\min}^B, x_{\max}^B]$, and we have corresponding intervals for the y - and z -axes. If the intervals overlap along *all three axes*, then the two AABBs are intersecting—in all other cases, they are not. Examples of intersecting and non-intersecting AABBs are shown in Figure 13.12 (simplified to two dimensions for the purposes of illustration). For an in-depth discussion of AABB collision, see http://www.gamasutra.com/features/20000203/lander_01.htm.

13.3.5.5 Detecting Convex Collisions: The GJK Algorithm

A very efficient algorithm exists for detecting intersections between arbitrary convex *polytopes* (i.e., convex polygons in two dimensions, or convex polyhedra in three dimensions). It is known as the GJK algorithm, named after its inventors, E. G. Gilbert, D. W. Johnson and S. S. Keerthi of the University of Michigan. Many papers have been written on the algorithm and its vari-

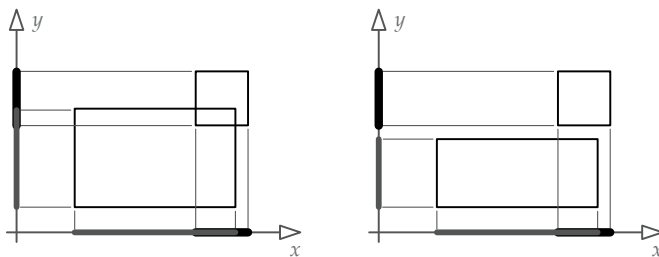


Figure 13.12. A two-dimensional example of intersecting and non-intersecting AABBs. Notice that even though the second pair of AABBs are intersecting along the x -axis, they are not intersecting along the y -axis.

ants, including the original paper (http://ieeexplore.ieee.org/xpl/freeabs_all.jsp?arnumber=2083), an excellent SIGGRAPH PowerPoint presentation by Christer Ericson (http://realtimecollisiondetection.net/pubs/SIGGRAPH04_Ericson_the_GJK_algorithm.ppt) and another great PowerPoint presentation by Gino van den Bergen (www.laas.fr/~nic/MOVIE/Workshop/Slides/Gino.vander.Bergen.ppt). However, the easiest-to-understand (and most entertaining) description of the algorithm is probably Casey Muratori's instructional video entitled "Implementing GJK," available online at <http://mollyrocket.com/849>. Because these descriptions are so good, I'll just give you a feel for the essence of the algorithm here and then direct you to the Molly Rocket website and the other references cited above for additional details.

The GJK algorithm relies on a geometric operation known as the *Minkowski difference*. This fancy-sounding operation is really quite simple: We take every point that lies within shape B and subtract it pairwise from every point inside shape A. The resulting set of points $\{(\mathbf{A}_i - \mathbf{B}_j)\}$ is the Minkowski difference.

The useful thing about the Minkowski difference is that, when applied to two convex shapes, it will *contain the origin* if and only if those two shapes intersect. Proof of this statement is a bit beyond our scope, but we can intuit why it is true by remembering that when we say two shapes A and B intersect, we really mean that there are points within A that are *also* within B. During the process of subtracting every point in B from every point in A, we would expect to eventually hit one of those shared points that lies within both shapes. A point minus itself is all zeros, so the Minkowski difference will contain the origin if (and only if) sphere A and sphere B have points in common. This is illustrated in Figure 13.13.

The Minkowski difference of two convex shapes is itself a convex shape. All we care about is the *convex hull* of the Minkowski difference, not all of the interior points. The basic procedure of GJK is to try to find a *tetrahedron* (i.e., a four-sided shape made out of triangles) that lies on the convex hull of the Minkowski difference and that encloses the origin. If one can be found, then the shapes intersect; if one cannot be found, then they don't.

A tetrahedron is just one case of a geometrical object known as a *simplex*. But don't let that name scare you—a simplex is just a collection of points. A single-point simplex is a point, a two-point simplex is a line segment, a three-point simplex is a triangle and a four-point simplex is a tetrahedron (see Figure 13.14).

GJK is an iterative algorithm that starts with a one-point simplex lying anywhere within the Minkowski difference hull. It then attempts to build higher-order simplexes that might potentially contain the origin. During each iteration of the loop, we take a look at the simplex we currently have and determine

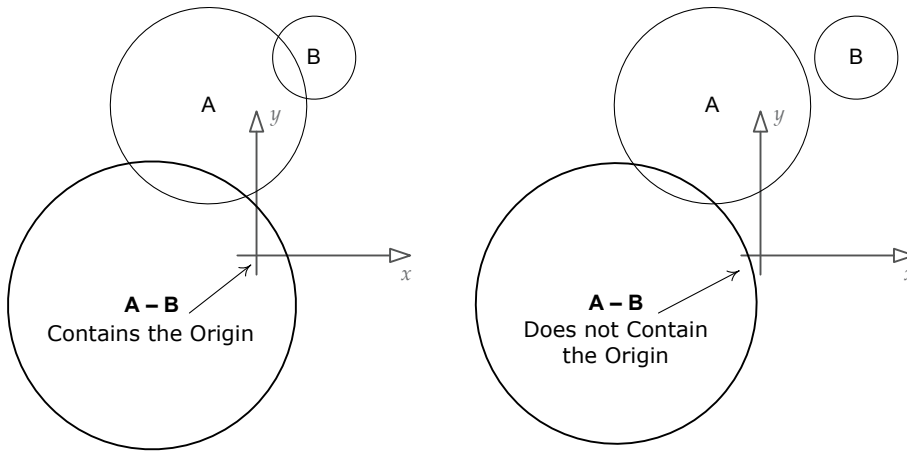


Figure 13.13. The Minkowski difference of two intersecting convex shapes contains the origin, but the Minkowski difference of two non-intersecting shapes does not.

in which direction the origin lies relative to it. We then find a *supporting vertex* of the Minkowski difference in that direction—i.e., the vertex of the convex hull that is closest to the origin in the direction we’re currently going. We add that new point to the simplex, creating a higher-order simplex (i.e., a point becomes a line segment, a line segment becomes a triangle and a triangle becomes a tetrahedron). If the addition of this new point causes the simplex to surround the origin, then we’re done—we know the two shapes intersect. On the other hand, if we are unable to find a supporting vertex that is closer to the origin than the current simplex, then we know that we can never get there, which implies that the two shapes do *not* intersect. This idea is illustrated in Figure 13.15.

To truly understand the GJK algorithm, you’ll need to check out the papers and video I referenced previously. But hopefully this description will whet your appetite for deeper investigation. Or, at the very least, you can impress

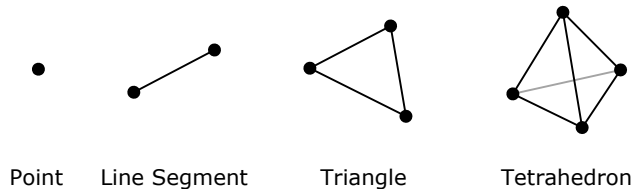


Figure 13.14. Simplexes containing one, two, three and four points.

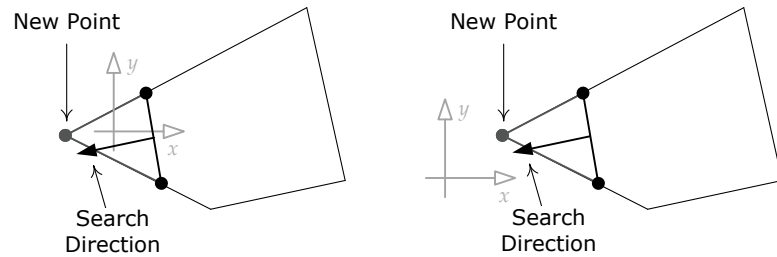


Figure 13.15. In the GJK algorithm, if adding a point to the current simplex creates a shape that contains the origin, we know the shapes intersect; if there is no supporting vertex that will bring the simplex any closer to the origin, then the shapes do not intersect.

your friends by dropping the name “GJK” at parties. (Just don’t try this at job interviews unless you really do understand the algorithm!)

13.3.5.6 Other Shape-Shape Combinations

We won’t cover any of the other shape-shape intersection combinations here, as they are covered well in other texts such as [14], [48] and [11]. The key point to recognize here, however, is that the *number* of shape-shape combinations is very large. In fact, for N shape types, the number of pairwise tests required is $O(N^2)$. Much of the complexity of a collision engine arises because of the sheer number of intersection cases it must handle. This is one reason why the authors of collision engines usually try to limit the number of primitive types—doing so drastically reduces the number of cases the collision detector must handle. (This is also why GJK is popular—it handles collision detection between *all* convex shape types in one fell swoop. The only thing that differs from shape type to shape type is the *support function* used in the algorithm.)

There’s also the practical matter of how to implement the code that selects the appropriate collision-testing function given two arbitrary shapes that are to be tested. Many collision engines use a *double dispatch* method (http://en.wikipedia.org/wiki/Double_dispatch). In single dispatch (i.e., virtual functions), the type of a single object is used to determine which concrete implementation of a particular abstract function should be called at runtime. Double dispatch extends the virtual function concept to two object types. It can be implemented via a two-dimensional function look-up table keyed by the types of the two objects being tested. It can also be implemented by arranging for a virtual function based on the type of object A to call a second virtual function based on the type of object B.

Let’s take a look at a real-world example. Havok uses objects known as *collision agents* (classes derived from `hkpCollisionAgent`) to handle specific

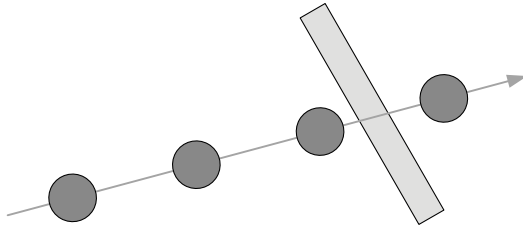


Figure 13.16. A small, fast-moving object can leave gaps in its motion path between consecutive snapshots of the collision world, meaning that collisions might be missed entirely.

intersection test cases. Concrete agent classes include `hkpSphereSphereAgent`, `hkpSphereCapsuleAgent`, `hkpGskConvexConvexAgent` and so on. The agent types are referenced by what amounts to a two-dimensional dispatch table, managed by the class `hkpCollisionDispatcher`. As you'd expect, the dispatcher's job is to efficiently look up the appropriate agent given a pair of collidables that are to be collision-tested and then call it, passing the two collidables as arguments.

13.3.5.7 Detecting Collisions between Moving Bodies

Thus far, we've considered only static intersection tests between stationary objects. When objects move, this introduces some additional complexity. Motion in games is usually simulated in discrete time steps. So one simple approach is to treat the positions and orientations of each rigid body as stationary at each time step and use static intersection tests on each "snapshot" of the collision world. This technique works as long as objects aren't moving too fast relative to their sizes. In fact, it works so well that many collision/physics engines, including Havok, use this approach by default.

However, this technique breaks down for small, fast-moving objects. Imagine an object that is moving so fast that it covers a distance *larger than its own size* (measured in the direction of travel) between time steps. If we were to overlay two consecutive snapshots of the collision world, we'd notice that there is now a gap between the fast-moving object's images in the two snapshots. If another object happens to lie within this gap, we'll miss the collision with it entirely. This problem, illustrated in Figure 13.16, is known as the "bullet through paper" problem, also known as "tunneling." The following sections describe a number of common ways to overcome this problem.

Swept Shapes

One way to avoid tunneling is to make use of *swept shapes*. A swept shape

is a new shape formed by the motion of a shape from one point to another over time. For example, a swept sphere is a capsule, and a swept triangle is a triangular prism (see Figure 13.17).

Rather than testing static snapshots of the collision world for intersections, we can test the swept shapes formed by moving the shapes from their positions and orientations in the previous snapshot to their positions and orientations in the current snapshot. This approach amounts to *linearly interpolating* the motion of the collidables between snapshots, because we generally sweep the shapes along line segments from snapshot to snapshot.

Of course, linear interpolation may not be a good approximation of the motion of a fast-moving collidable. If the collidable is following a curved path, then theoretically we should sweep its shape along that curved path. Unfortunately, a convex shape that has been swept along a curve is not itself convex, so this can make our collision tests much more complex and computationally intensive.

In addition, if the convex shape we are sweeping is rotating, the resulting swept shape is not necessarily convex, even when it is swept along a line segment. As Figure 13.18 shows, we *can* always form a convex shape by linearly extrapolating the extreme features of the shapes from the previous and current snapshots—but the resulting convex shape is not necessarily an accurate representation of what the shape really would have done over the time step. Put another way, a linear interpolation is not appropriate in general for rotating shapes. So unless our shapes are not permitted to rotate, intersection testing of swept shapes becomes much more complex and computationally intensive than its static snapshot-based counterpart.

Swept shapes can be a useful technique for ensuring that collisions are not missed between static snapshots of the collision world state. However, the results are generally inaccurate when linearly interpolating curved paths or

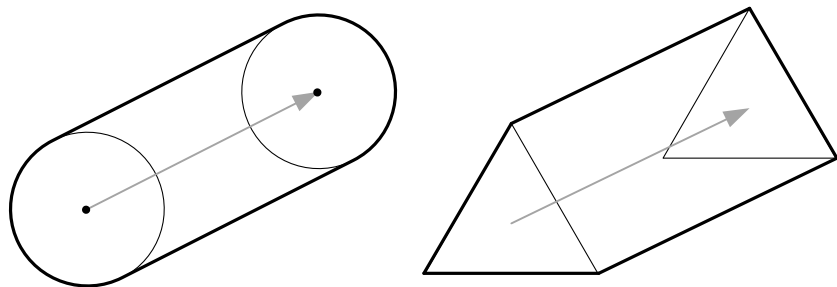


Figure 13.17. A swept sphere is a capsule; a swept triangle is a triangular prism.

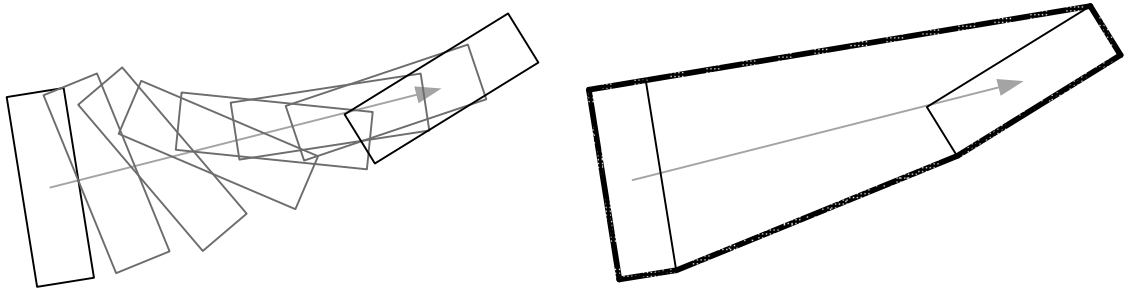


Figure 13.18. A rotating object swept along a line segment does not necessarily generate a convex shape (left). A linear interpolation of the motion does form a convex shape (right), but it can be a fairly inaccurate approximation of what actually happened during the time step.

rotating collidables, so more-detailed techniques may be required depending on the needs of the game.

Continuous Collision Detection (CCD)

Another way to deal with the tunneling problem is to employ a technique known as *continuous collision detection* (CCD). The goal of CCD is to find the earliest *time of impact* (TOI) between two moving objects over a given time interval.

CCD algorithms are generally iterative in nature. For each collidable, we maintain both its position and orientation at the previous time step and its position and orientation at the current time. This information can be used to linearly interpolate the position and rotation independently, yielding an approximation of the collidable's transform at any time between the previous and current time steps. The algorithm then searches for the earliest TOI along the motion path. A number of search algorithms are commonly used, including Brian Mirtich's *conservative advancement* method, performing a ray cast on the Minkowski sum, or considering the minimum TOI of individual feature pairs. Erwin Coumans of Sony Interactive Entertainment describes some of these algorithms in <http://gamedevs.org/uploads/continuous-collision-detection-and-physics.pdf> along with his own novel variation on the conservative advancement approach.

13.3.6 Performance Optimizations

Collision detection is a CPU-intensive task for two reasons:

1. The calculations required to determine whether two shapes intersect are themselves nontrivial.

2. Most game worlds contain a large number of objects, and the number of intersection tests required grows rapidly as the number of objects increases.

To detect intersections between n objects, the brute-force technique would be to test every possible pair of objects, yielding an $O(n^2)$ algorithm. However, much more efficient algorithms are used in practice. Collision engines typically employ some form of spatial hashing (<http://bit.ly/1fLtX1D>), spatial subdivision or hierarchical bounding volumes in order to reduce the number of intersection tests that must be performed.

13.3.6.1 Temporal Coherency

One common optimization technique is to take advantage of *temporal coherency*, also known as *frame-to-frame coherency*. When collidables are moving at reasonable speeds, their positions and orientations are usually quite similar from time step to time step. We can often avoid recalculating certain kinds of information every frame by caching the results across multiple time steps. For example, in Havok, collision agents (`hkpCollisionAgent`) are usually persistent between frames, allowing them to reuse calculations from previous time steps as long as the motion of the collidables in question hasn't invalidated those calculations.

13.3.6.2 Spatial Partitioning

The basic idea of spatial partitioning is to greatly reduce the number of collidables that need to be checked for intersection by dividing space into a number of smaller regions. If we can determine (in an inexpensive manner) that a pair of collidables do not occupy the same region, then we needn't perform more-detailed intersection tests on them.

Various hierarchical partitioning schemes, such as octrees, binary space partitioning trees (BSPs), kd-trees or sphere trees, can be used to subdivide space for the purposes of collision detection optimization. These trees subdivide space in different ways, but they all do so in a hierarchical fashion, starting with a gross subdivision at the root of the tree and further subdividing each region until sufficiently fine-grained regions have been obtained. The tree can then be walked in order to find and test groups of potentially colliding objects for actual intersections. Because the tree partitions space, we know that when we traverse down one branch of the tree, the objects in that branch cannot be colliding with objects in other sibling branches.

13.3.6.3 Broad Phase, Midphase and Narrow Phase

Havok uses a three-tiered approach to prune the set of collidables that need to be tested for collisions during each time step.

- First, gross AABB tests are used to determine which collidables are potentially intersecting. This is known as *broad phase* collision detection.
- Second, the coarse bounding volumes of compound shapes are tested. This is known as *midphase* collision detection. For example, in a compound shape composed of three spheres, the bounding volume might be a fourth, larger sphere that encloses the other spheres. A compound shape may contain other compound shapes, so in general a compound collidable has a bounding volume hierarchy. The midphase traverses this hierarchy in search of subshapes that are potentially intersecting.
- Finally, the collidables' individual primitives are tested for intersection. This is known as *narrow phase* collision detection.

The Sweep and Prune Algorithm

In all of the major collision/physics engines (e.g., Havok, ODE, PhysX), broad phase collision detection employs an algorithm known as *sweep and prune* (http://en.wikipedia.org/wiki/Sweep_and_prune). The basic idea is to sort the minimum and maximum dimensions of the collidables' AABBs along the three principal axes, and then check for overlapping AABBs by traversing the sorted lists. Sweep and prune algorithms can make use of frame-to-frame coherency (see Section 13.3.6.1) to reduce an $O(n \log n)$ sort operation to an expected $O(n)$ running time. Frame coherency can also aid in the updating of AABBs when objects rotate.

13.3.7 Collision Queries

Another responsibility of the collision detection system is to answer hypothetical questions about the collision volumes in the game world. Examples include the following:

- If a bullet travels from the player's weapon in a given direction, what is the first target it will hit, if any?
- Can a vehicle move from point A to point B without striking anything along the way?
- Find all enemy objects within a given radius of a character.

In general, such operations are known as *collision queries*.

The most common kind of query is a *collision cast*, sometimes just called a *cast*. (The terms *trace* and *probe* are other common synonyms for “cast.”) A cast determines what, if anything, a hypothetical object would hit if it were to be placed into the collision world and moved along a ray or line segment. Casts are different from regular collision detection operations because the entity being cast is not really in the collision world—it cannot affect the other objects in the world in any way. This is why we say that a collision cast answers *hypothetical* questions about the collidables in the world.

13.3.7.1 Ray Casting

The simplest type of collision cast is a *ray cast*, although this name is actually a bit of a misnomer. What we’re really casting is a *directed line segment*—in other words, our casts always have a start point ($\mathbf{p}_0$) and an end point ($\mathbf{p}_1$). The cast line segment is tested against the collidable objects in the collision world. If it intersects any of them, the contact point or points are returned.

Ray casting systems typically describe the line segment via its start point $\mathbf{p}_0$ and a delta vector $\mathbf{d}$ that, when added to $\mathbf{p}_0$, yields the end point $\mathbf{p}_1$. Any point on this line segment can be found via the following *parametric equation*, where the parameter t is permitted to vary between zero and one:

$$\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{d}, \quad t \in [0, 1].$$

Clearly, $\mathbf{p}_0 = \mathbf{p}(0)$ and $\mathbf{p}_1 = \mathbf{p}(1)$. In addition, any contact point along the segment can be uniquely described by specifying the value of the parameter t corresponding to the contact. Most ray casting APIs return their contact points as “ t values,” or they permit a contact point to be converted into its corresponding t by making an additional function call.

Most collision detection systems are capable of returning the *earliest contact*—i.e., the contact point that lies closest to $\mathbf{p}_0$ and corresponds to the smallest value of t . Some systems are also capable of returning a complete list of all collidables that were intersected by the ray or line segment. The information returned for each contact typically includes the t value, some kind of unique identifier for the collidable entity that was hit, and possibly other information such as the surface normal at the point of contact or other relevant properties of the shape or surface that was struck. One possible contact point data structure is shown below.

```
struct RayCastContact
{
    F32    m_t;                // the t value for this
                                // contact
```

```

    U32      m_collidableId; // which collidable did we
                          // hit?

    Vector m_normal;        // surface normal at
                          // contact pt.

    // other information...
};

```

Applications of Ray Casts

Ray casts are used heavily in games. For example, we might want to ask the collision system whether character A has a direct line of sight to character B. To determine this, we simply cast a directed line segment from the eyes of character A to the chest of character B. If the ray hits character B, we know that A can “see” B. But if the ray strikes some other object *before* reaching character B, we know that the line of sight is being blocked by that object. Ray casts are used by weapon systems (e.g., to determine bullet hits), player mechanics (e.g., to determine whether or not there is solid ground beneath the character’s feet), AI systems (e.g., line of sight checks, targeting, movement queries, etc.), vehicle systems (e.g., to locate and snap the vehicle’s tires to the terrain) and so on.

13.3.7.2 Shape Casting

Another common query involves asking the collision system how far an imaginary convex shape would be able to travel along a directed line segment before it hits something solid. This is known as a *sphere cast* when the volume being cast is a sphere, or a *shape cast* in general. (Havok calls them *linear casts*.) As with ray casts, a shape cast is usually described by specifying the start point $\mathbf{p}_0$, the distance to travel $\mathbf{d}$ and of course the type, dimensions and orientation of the shape we wish to cast.

There are two cases to consider when casting a convex shape.

1. The cast shape is already interpenetrating or contacting at least one other collidable, preventing it from moving away from its starting location.
2. The cast shape is not intersecting with anything else at its starting location, so it is free to move a nonzero distance along its path.

In the first scenario, the collision system typically reports the contact(s) between the cast shape and all of the collidables with which it is initially interpenetrating. These contacts might be *inside* the cast shape or on its *surface*, as shown in Figure 13.19.

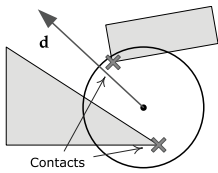


Figure 13.19. A cast sphere that starts in penetration will be unable to move, and possibly many contact points will lie inside the cast shape in general.

In the second case, the shape can move a nonzero distance along the line segment before striking something. Presuming that it hits something, it will usually hit only a single collidable. However, it is possible for a cast shape to strike more than one collidable simultaneously if its trajectory is just right. And of course, if the impacted collidable is a non-convex poly soup, the cast shape may end up touching more than one part of the poly soup simultaneously. We can safely say that no matter what kind of convex shape is cast, it is *possible* for the cast to generate multiple contact points. The contacts will always be on the *surface* of the cast shape in this case, never inside it (because we know that the cast shape was not interpenetrating anything when it started its journey). This case is illustrated in Figure 13.20.

As with ray casts, some shape casting APIs report only the *earliest* contact experienced by the cast shape, while others allow the shape to continue along its hypothetical path, returning all the contacts it experiences on its journey. This is illustrated in Figure 13.21.

The contact information returned by a shape cast is necessarily a bit more complex than it is for a ray cast. We cannot simply return one or more t values, because a t value only describes the location of the center point of the shape along its path. It tells us nothing of where, on the surface or interior of the shape, it came into contact with the impacted collidable. As a result, most shape casting APIs return both a t value and the actual contact point, along with other relevant information (such as which collidable was struck, the surface normal at the contact point, etc.).

Unlike ray casting APIs, a shape casting system must always be capable of reporting multiple contacts. This is because even if we only report the contact with the earliest t value, the shape may have touched multiple distinct collidables in the game world, or it may be touching a single non-convex collidable at more than one point. As a result, collision systems usually return an array

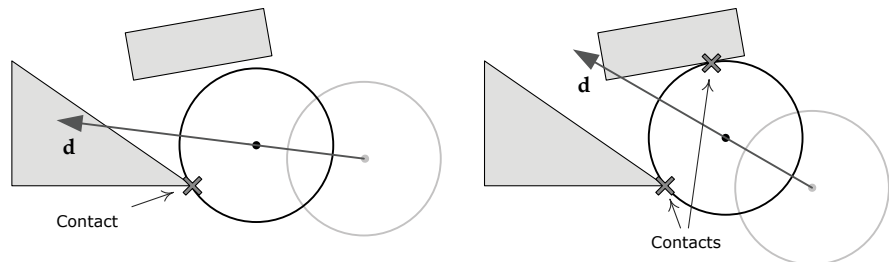


Figure 13.20. If the starting location of a cast shape is not interpenetrating anything, then the shape will move a nonzero distance along its line segment, and its contacts (if any) will always be on its surface.

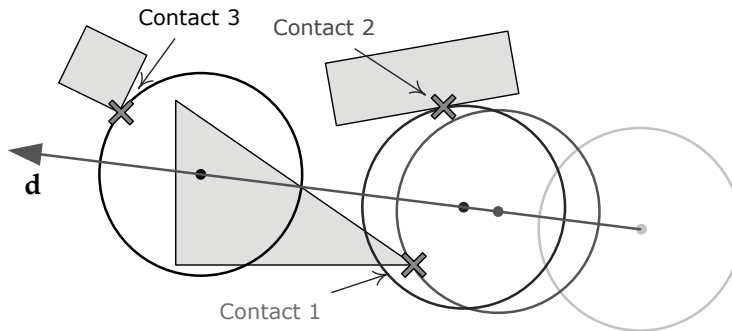


Figure 13.21. A shape casting API might return all contacts instead of only the earliest contact.

or list of contact point data structures, each of which might look something like this:

```
struct ShapeCastContact
{
    F32    m_t;           // the t value for this
                        // contact

    U32    m_collidableId; // which collidable did we
                        // hit?

    Point  m_contactPoint; // location of actual
                        // contact

    Vector m_normal;      // surface normal at
                        // contact pt.

    // other information...
};
```

Given a list of contact points, we often want to distinguish between the groups of contact points for each distinct t value. For example, the earliest contact is actually described by the group of contact points that all share the minimum t in the list. It's important to realize that collision systems may or may not return their contact points sorted by t . If it does not, it's almost always a good idea to sort the results by t manually. This ensures that if one looks at the first contact point in the list, it will be guaranteed to be among the earliest contact points along the shape's path.

Applications of Shape Casts

Shape casts are extremely useful in games. Sphere casts can be used to determine whether the virtual camera is in collision with objects in the game world. Sphere or capsule casts are also commonly used to implement character movement. For example, in order to slide the character forward on uneven terrain, we can cast a sphere or capsule that lies between the character's feet in the direction of motion. We can adjust it up or down via a second cast, to ensure that it remains in contact with the ground. If the sphere hits a very short vertical obstruction, such as a street curb, it can "pop up" over the curb. If the vertical obstruction is too tall, like a wall, the cast sphere can be slid horizontally along the wall. The final resting place of the cast sphere becomes the character's new location next frame.

13.3.7.3 Phantoms

Sometimes, games need to determine which collidable objects lie within some specific volume in the game world. For example, we might want the list of all enemies that are within a certain radius of the player character. Havok supports a special kind of collidable object known as a *phantom* for this purpose.

A phantom acts much like a shape cast whose distance vector $\mathbf{d}$ is zero. At any moment, we can ask the phantom for a list of its contacts with other collidables in the world. It returns this data in essentially the same format that would be returned by a zero-distance shape cast.

However, unlike a shape cast, a phantom is persistent in the collision world. This means that it can take full advantage of the temporal coherency optimizations used by the collision engine when detecting collisions between "real" collidables. In fact, the only difference between a phantom and a regular collidable is that it is "invisible" to all other collidables in the collision world (and it does not take part in the dynamics simulation). This allows it to answer hypothetical questions about what objects it would collide with were it a "real" collidable, but it is guaranteed not to have any effect on the other collidables—including other phantoms—in the collision world.

13.3.7.4 Other Types of Queries

Some collision engines support other kinds of queries in addition to casts. For example, Havok supports *closest point* queries, which are used to find the set of points on other collidables that are closest to a given collidable in the collision world.

13.3.8 Collision Filtering

It is quite common for game developers to want to enable or disable collisions between certain kinds of objects. For example, most objects are permitted to

pass through the surface of a body of water—we might employ a buoyancy simulation to make them float, or they might just sink to the bottom, but in either case we do not want the water’s surface to appear solid. Most collision engines allow contacts between collidables to be accepted or rejected based on game-specific criteria. This is known as *collision filtering*.

13.3.8.1 Collision Masking and Layers

One common filtering approach is to categorize the objects in the world and then use a look-up table to determine whether certain categories are permitted to collide with one another or not. For example, in Havok, a collidable can be a member of one (and only one) collision layer. The default collision filter in Havok, represented by an instance of the class `hkpGroupFilter`, maintains a 32-bit mask for each layer, each bit of which tells the system whether or not that particular layer can collide with one of the other layers.

13.3.8.2 Collision Callbacks

Another filtering technique is to arrange for the collision library to invoke a *callback function* whenever a collision is detected. The callback can inspect the specifics of the collision and make the decision to either allow or reject the collision based on suitable criteria. Havok also supports this kind of filtering. When contact points are first added to the world, the `contactPointAdded()` callback is invoked. If the contact point is later determined to be valid (it may not be if an earlier TOI contact was found), the `contactPointConfirmed()` callback is invoked. The application may reject contact points in these callbacks if desired.

13.3.8.3 Game-Specific Collision Materials

Game developers often need to categorize the collidable objects in the game world, in part to control how they collide (as with collision filtering) and in part to control other secondary effects, such as the sound that is made or the particle effect that is generated when one type of object hits another. For example, we might want to differentiate between wood, stone, metal, mud, water and human flesh.

To accomplish this, many games implement a collision shape categorization mechanism similar in many respects to the *material system* used in the rendering engine. In fact, some game teams use the term *collision material* to describe this categorization. The basic idea is to associate with each collidable surface a set of properties that defines how that particular surface should behave from a physical and collision standpoint. Collision properties can include sound and particle effects, physical properties like coefficient of restitution or

friction coefficients, collision filtering information and whatever other information the game might require.

For simple convex primitives, the collision properties are usually associated with the shape as a whole. For polygon soup shapes, the properties might be specified on a per-triangle basis. Because of this latter usage, we usually try to keep the binding between the collision primitive and its collision material as compact as possible. A typical approach is to bind collision primitives to collision materials via an 8-, 16- or 32-bit integer, or a pointer to the material data. This integer indexes into a global array of data structures containing the detailed collision properties themselves.

13.4 Rigid Body Dynamics

In a game engine, we are particularly concerned with the *kinematics* of objects—how they move over time. Many game engines include a *physics system* for the purposes of simulating the motion of the objects in the virtual game world in a somewhat physically realistic way. Technically speaking, game physics engines are typically concerned with a particular field of physics known as *dynamics*. This is the study of how *forces* affect the movement of objects. Until very recently, game physics systems have been focused almost exclusively on a specific subdiscipline known as *classical rigid body dynamics*. This name implies that in a game's physics simulation, two important simplifying assumptions are made:

- *Classical (Newtonian) mechanics.* The objects in the simulation are assumed to obey Newton's laws of motion. The objects are large enough that there are no quantum effects, and their speeds are low enough that there are no relativistic effects.
- *Rigid bodies.* All objects in the simulation are perfectly solid and cannot be deformed. In other words, their shape is constant. This idea meshes well with the assumptions made by the collision detection system. Furthermore, the assumption of rigidity greatly simplifies the mathematics required to simulate the dynamics of solid objects.

Game physics engines are also capable of ensuring that the motions of the rigid bodies in the game world conform to various *constraints*. The most common constraint is that of non-penetration—in other words, objects aren't allowed to pass through one another. Hence the physics system attempts to provide realistic *collision responses* whenever bodies are found to be interpen-

etrating.² This is one of the primary reasons for the tight interconnection between the physics engine and the collision detection system.

Most physics systems also allow game developers to set up other kinds of constraints in order to define realistic interactions between physically simulated rigid bodies. These may include hinges, prismatic joints (sliders), ball joints, wheels, “rag dolls” to emulate unconscious or dead characters and so on.

The physics system usually shares the collision world data structure, and in fact it usually drives the execution of the collision detection algorithm as part of its time step update routine. There is typically a one-to-one mapping between the rigid bodies in the dynamics simulation and the collidables managed by the collision engine. For example, in Havok, an `hkpRigidBody` object maintains a reference to one and only one `hkpCollidable` (although it is possible to create a collidable that has no rigid body). In PhysX, the two concepts are a bit more tightly integrated—an `NxActor` serves both as a collidable object and as a rigid body for the purposes of the dynamics simulation. These rigid bodies and their corresponding collidables are usually maintained in a singleton data structure known as the *collision/physics world*, or sometimes just the *physics world*.

The rigid bodies in the physics engine are typically distinct from the logical objects that make up the virtual world from a gameplay perspective. The positions and orientations of game objects can be driven by the physics simulation. To accomplish this, we query the physics engine every frame for the transform of each rigid body, and apply it in some way to the transform of the corresponding game object. It’s also possible for a game object’s motion, as determined by some other engine system (such as the animation system or the character control system) to drive the position and rotation of a rigid body in the physics world. As mentioned in Section 13.3.1, a single logical game object may be represented by one rigid body in the physics world, or by many. A simple object like a rock, weapon or barrel, might correspond to one rigid body. But an articulated character or a complex machine might be composed of many interconnected rigid pieces.

The remainder of this chapter will be devoted to investigating how game physics engines work. We’ll briefly introduce the theory that underlies rigid body dynamics simulations. Then we’ll investigate some of the most common features of a game physics system and have a look at how a physics engine might be integrated into a game.

²Or in the case of continuous collision detection, the collision response actually *prevents* the penetration from occurring.

13.4.1 Some Foundations

A great many excellent books, articles and slide presentations have been written on the topic of classical rigid body dynamics. A solid foundation in analytical mechanics theory can be obtained from [17]. Even more relevant to our discussion are texts like [39], [13] and [29], which have been written specifically about the kind of physics simulations done by games. Other texts, like [2], [11] and [32], include chapters on rigid body dynamics for games. Chris Hecker wrote a series of helpful articles on the topic of game physics for *Game Developer Magazine*; Chris has posted these and a variety of other useful resources at http://chrishecker.com/Rigid_Body_Dynamics. An informative slide presentation on dynamics simulation for games was produced by Russell Smith, the primary author of ODE; it is available at <http://www.ode.org/slides/parc/dynamics.pdf>.

In this section, I'll summarize the fundamental theoretical concepts that underlie the majority of game physics engines. This will be a whirlwind tour only, and by necessity I'll have to omit some details. Once you've read this chapter, I strongly encourage you to read at least a few of the additional resources cited previously.

13.4.1.1 Units

Most rigid body dynamics simulations operate in the MKS system of units. In this system, distance is measured in meters (abbreviated "m"), mass is measured in kilograms (abbreviated "kg") and time is measured in seconds (abbreviated "s"). Hence the name MKS.

You could configure your physics system to use other units if you wanted to, but if you do this, you need to make sure everything in the simulation is consistent. For example, constants like the acceleration due to gravity $\mathbf{g}$, which is measured in m/s^2 in the MKS system, would have to be re-expressed in whatever unit system you select. Most game teams just stick with MKS to keep life simple.

13.4.1.2 Separability of Linear and Angular Dynamics

An *unconstrained* rigid body is one that can translate freely along all three Cartesian axes and that can rotate freely about these three axes as well. We say that such a body has six *degrees of freedom* (DOF).

It is perhaps somewhat surprising that the motion of an unconstrained rigid body can be separated into two independent components:

- *Linear dynamics*. This is a description of the motion of the body when we ignore all rotational effects. (We can use linear dynamics alone to

describe the motion of an idealized *point mass*—i.e., a mass that is infinitesimally small and cannot rotate.)

- *Angular dynamics.* This is a description of the rotational motion of the body.

As you can well imagine, this ability to separate the linear and angular components of a rigid body's motion is extremely helpful when analyzing or simulating its behavior. It means that we can calculate a body's linear motion without regard to rotation—as if it were an idealized point mass—and then layer its angular motion on top in order to arrive at a complete description of the body's motion.

13.4.1.3 Center of Mass

For the purposes of *linear* dynamics, an unconstrained rigid body acts as though all of its mass were concentrated at a single point known as the *center of mass* (abbreviated CM, or sometimes COM). The center of mass is essentially the balancing point of the body for all possible orientations. In other words, the mass of a rigid body is distributed evenly around its center of mass in all directions.

For a body with uniform density, the center of mass lies at the *centroid* of the body. That is, if we were to divide the body up into N very small pieces, add up the positions of all these pieces as a vector sum and then divide by the number of pieces, we'd end up with a pretty good approximation to the location of the center of mass. If the body's density is not uniform, the position of each little piece would need to be *weighted* by that piece's mass, meaning that in general the center of mass is really a *weighted average* of the pieces' positions. So we have

$$\mathbf{r}_{\text{CM}} = \frac{\sum_{\forall i} m_i \mathbf{r}_i}{\sum_{\forall i} m_i} = \frac{\sum_{\forall i} m_i \mathbf{r}_i}{m},$$

where the symbol m represents the total mass of the body, and the symbol $\mathbf{r}$ represents a *radius vector* or *position vector*—i.e., a vector extending from the world-space origin to the point in question. (These sums become integrals in the limit as the sizes and masses of the little pieces approach zero.)

The center of mass always lies inside a convex body, although it may actually lie outside the body if it is concave. (For example, where would the center of mass of the letter “C” lie?)

13.4.2 Linear Dynamics

For the purposes of linear dynamics, the position of a rigid body can be fully described by a position vector $\mathbf{r}_{\text{CM}}$ that extends from the world-space origin to the center of mass of the body, as shown in Figure 13.22. Since we're using the MKS system, position is measured in meters (m). For the remainder of this discussion, we'll drop the CM subscripts, as it is understood that we are describing the motion of the body's center of mass.

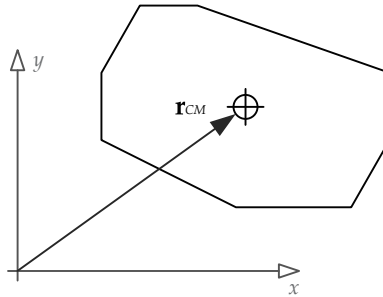


Figure 13.22. For the purposes of linear dynamics, the position of a rigid body can be fully described by the position of its center of mass.

13.4.2.1 Linear Velocity and Acceleration

The *linear velocity* of a rigid body defines the speed and direction in which the body's CM is moving. It is a vector quantity, typically measured in meters per second (m/s). Velocity is the first time derivative of position, so we can write

$$\mathbf{v}(t) = \frac{d\mathbf{r}(t)}{dt} = \dot{\mathbf{r}}(t),$$

where the dot over the vector $\mathbf{r}$ denotes taking the derivative with respect to time. Differentiating a vector is the same as differentiating each component independently, so

$$v_x(t) = \frac{dr_x(t)}{dt} = \dot{r}_x(t),$$

and so on for the y - and z -components.

Linear acceleration is the first derivative of linear velocity with respect to time, or the second derivative of the position of a body's CM versus time. Acceleration is a vector quantity, usually denoted by the symbol $\mathbf{a}$. So we can

write

$$\begin{aligned}\mathbf{a}(t) &= \frac{d\mathbf{v}(t)}{dt} = \dot{\mathbf{v}}(t) \\ &= \frac{d^2\mathbf{r}(t)}{dt^2} = \ddot{\mathbf{r}}(t).\end{aligned}$$

13.4.2.2 Force and Momentum

A *force* is defined as anything that causes an object with mass to accelerate or decelerate. A force has both a magnitude and a direction in space, so all forces are represented by vectors. A force is often denoted by the symbol $\mathbf{F}$. When N forces are applied to a rigid body, their net effect on the body's linear motion is found by simply adding up the force vectors:

$$\mathbf{F}_{\text{net}} = \sum_{i=1}^N \mathbf{F}_i.$$

Newton's famous Second Law states that force is proportional to acceleration and mass:

$$\mathbf{F}(t) = m\mathbf{a}(t) = m\ddot{\mathbf{r}}(t). \quad (13.2)$$

As Newton's law implies, force is measured in units of kilogram-meters per second squared ($\text{kg}\cdot\text{m}/\text{s}^2$). This unit is also called the Newton.

When we multiply a body's linear velocity by its mass, the result is a quantity known as *linear momentum*. It is customary to denote linear momentum with the symbol $\mathbf{p}$:

$$\mathbf{p}(t) = m\mathbf{v}(t).$$

When mass is constant, Equation (13.2) holds true. But if mass is not constant, as would be the case for a rocket whose fuel is being gradually used up and converted into energy, Equation (13.2) is not exactly correct. The proper formulation is actually as follows:

$$\mathbf{F}(t) = \frac{d\mathbf{p}(t)}{dt} = \frac{d(m(t)\mathbf{v}(t))}{dt}.$$

which of course reduces to the more familiar $\mathbf{F} = m\mathbf{a}$ when the mass is constant and can be brought outside the derivative. Linear momentum is not of much concern to us. However, the concept of momentum will become relevant when we discuss angular dynamics.

13.4.3 Solving the Equations of Motion

The central problem in rigid body dynamics is to solve for the motion of the body, given a set of known forces acting on it. For linear dynamics, this means finding $\mathbf{v}(t)$ and $\mathbf{r}(t)$ given knowledge of the net force $\mathbf{F}_{\text{net}}(t)$ and possibly other information, such as the position and velocity at some previous time. As we'll see below, this amounts to solving a pair of ordinary differential equations—one to find $\mathbf{v}(t)$ given $\mathbf{a}(t)$ and the other to find $\mathbf{r}(t)$ given $\mathbf{v}(t)$.

13.4.3.1 Force as a Function

A force can be constant, or it can be a function of time as shown above. A force can also be a function of the position of the body, its velocity, or any number of other quantities. So in general, the expression for force should really be written as follows:

$$\mathbf{F}(t, \mathbf{r}(t), \mathbf{v}(t), \dots) = m\mathbf{a}(t). \quad (13.3)$$

This can be rewritten in terms of the position vector and its first and second derivatives as follows:

$$\mathbf{F}(t, \mathbf{r}(t), \dot{\mathbf{r}}(t), \dots) = m\ddot{\mathbf{r}}(t).$$

For example, the force exerted by a spring is proportional to how far it has been stretched away from its natural resting position. In one dimension, with the spring's resting position at $x = 0$, we can write

$$F(t, x(t)) = -kx(t),$$

where k is the *spring constant*, a measure of the spring's stiffness.

As another example, the damping force exerted by a mechanical viscous damper (a so-called dashpot) is proportional to the velocity of the damper's piston. So in one dimension, we can write

$$F(t, v(t)) = -bv(t),$$

where b is a *viscous damping coefficient*.

13.4.3.2 Ordinary Differential Equations

In general, an *ordinary differential equation* (ODE) is an equation involving a function of one independent variable and various derivatives of that function. If our independent variable is time and our function is $x(t)$, then an ODE is a relation of the form

$$\frac{d^n x}{dt^n} = f\left(t, x(t), \frac{dx(t)}{dt}, \frac{d^2 x(t)}{dt^2}, \dots, \frac{d^{n-1} x(t)}{dt^{n-1}}\right).$$

Put another way, the n th derivative of $x(t)$ is expressed as a function f whose arguments can be time (t), position ($x(t)$), and any number of derivatives of $x(t)$ as long as those derivatives are of *lower order* than n .

As we saw in Equation (13.3), force is a function of time, position and velocity in general:

$$\ddot{\mathbf{r}}(t) = \frac{1}{m} \mathbf{F}(t, \mathbf{r}(t), \dot{\mathbf{r}}(t)).$$

This clearly qualifies as an ODE. We wish to solve this ODE in order to find $\mathbf{v}(t)$ and $\mathbf{r}(t)$.

13.4.3.3 Analytical Solutions

In some rare situations, the differential equations of motion can be solved *analytically*, meaning that a simple, closed-form function can be found that describes the body's position for *all possible values* of time t . A common example is the vertical motion of a projectile under the influence of a constant acceleration due to gravity, $\mathbf{a}(t) = [0, g, 0]$, where $g = -9.8\text{m/s}^2$. In this case, the ODE of motion boils down to

$$\ddot{y}(t) = g.$$

Integrating once yields

$$\dot{y}(t) = gt + v_0,$$

where v_0 is the vertical velocity at time $t = 0$. Integrating a second time yields the familiar solution

$$y(t) = \frac{1}{2}gt^2 + v_0t + y_0,$$

where y_0 is the initial vertical position of the object.

However, analytical solutions are almost never possible in game physics. This is due in part to the fact that closed-form solutions to some differential equations are simply not known. Moreover, a game is an interactive simulation, so we usually cannot predict how the forces in a game will behave over time. This makes it impossible to find simple, closed-form expressions for the positions and velocities of the objects in the game as functions of time.

There are of course exceptions to this rule of thumb. For example, it's pretty common to solve for a closed-form expression in order to determine with what velocity a projectile must be launched in order to hit a predefined target.

13.4.4 Numerical Integration

For the reasons cited above, game physics engines turn to a technique known as *numerical integration*. With this technique, we solve our differential equations in a *time-stepped* manner—using the solution from a previous time step

to arrive at the solution for the next time step. The duration of the time step is usually taken to be (roughly) constant and is denoted by the symbol Δt . Given that we know the body's position and velocity at the current time t_1 and that the force is known as a function of time, position and/or velocity, we wish to find the position and velocity at the next time step $t_2 = t_1 + \Delta t$. In other words, given $\mathbf{r}(t_1)$, $\mathbf{v}(t_1)$ and $\mathbf{F}(t, \mathbf{r}, \mathbf{v})$, the problem is to find $\mathbf{r}(t_2)$ and $\mathbf{v}(t_2)$.

13.4.4.1 Explicit Euler

One of the simplest numerical solutions to an ODE is known as the *explicit Euler method*. This is the intuitive approach often taken by new game programmers. Let's assume for the moment that we already know the current velocity and that we wish to solve the following ODE to find the body's position on the next frame:

$$\mathbf{v}(t) = \dot{\mathbf{r}}(t). \quad (13.4)$$

Using the explicit Euler method, we simply convert the velocity from meters per second into meters per frame by multiplying by the time delta, and then we add "one frame's worth" of velocity onto the current position in order to find the new position on the next frame. This yields the following approximate solution to the ODE given by Equation (13.4):

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t. \quad (13.5)$$

We can take an analogous approach to find the body's velocity next frame given the net force acting this frame. Hence, the approximate explicit Euler solution to the ODE

$$\mathbf{a}(t) = \frac{\mathbf{F}_{\text{net}}(t)}{m} = \dot{\mathbf{v}}(t)$$

is as follows:

$$\mathbf{v}(t_2) = \mathbf{v}(t_1) + \frac{\mathbf{F}_{\text{net}}(t)}{m}\Delta t. \quad (13.6)$$

Interpretations of Explicit Euler

What we're really doing in Equation (13.5) is assuming that the velocity of the body is constant during the time step. Therefore, we can use the *current* velocity to predict the body's position on the *next* frame. The change in position $\Delta \mathbf{r}$ between times t_1 and t_2 is hence $\Delta \mathbf{r} = \mathbf{v}(t_1)\Delta t$. Graphically, if we imagine a plot of the position of the body versus time, we are taking the *slope* of the function at time t_1 (which is just $\mathbf{v}(t_1)$) and extrapolating it linearly to the next time step t_2 . As we can see in Figure 13.23, linear extrapolation does not necessarily provide us with a particularly good estimate of the true position at the

next time step $\mathbf{r}(t_2)$, but it does work reasonably well as long as the velocity is roughly constant.

Figure 13.23 suggests another way to interpret the explicit Euler method—as an approximation of a derivative. By definition, any derivative is the quotient of two infinitesimally small differences (in our case, $d\mathbf{r}/dt$). The explicit Euler method approximates this using the quotient of two *finite differences*. In other words, $d\mathbf{r}$ becomes $\Delta\mathbf{r}$ and dt becomes Δt . This yields

$$\frac{d\mathbf{r}}{dt} \approx \frac{\Delta\mathbf{r}}{\Delta t};$$

$$\mathbf{v}(t_1) \approx \frac{\mathbf{r}(t_2) - \mathbf{r}(t_1)}{t_2 - t_1},$$

which again simplifies to Equation (13.5). This approximation is really only valid when the velocity is constant over the time step. It is also valid in the limit as Δt tends toward zero (at which point it becomes *exactly* right). Obviously, this same analysis can be applied to Equation (13.6) as well.

13.4.4.2 Properties of Numerical Methods

We’ve implied that the explicit Euler method is not particularly accurate. Let’s pin this idea down more concretely. A numerical solution to an ordinary differential equation actually has three important and interrelated properties:

- *Convergence.* As the time step Δt tends toward zero, does the approximate solution get closer and closer to the real solution?
- *Order.* Given a particular numerical approximation to the solution of an ODE, how “bad” is the error? Errors in numerical ODE solutions are typically proportional to some power of the time step duration Δt , so they are often written using “big O” notation (e.g., $O(\Delta t^2)$). We say

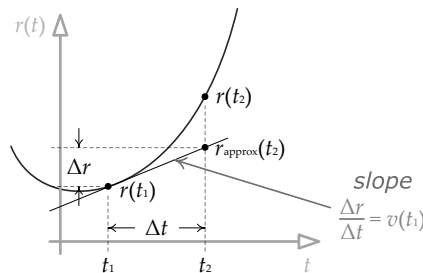


Figure 13.23. In the explicit Euler method, the slope of $\mathbf{r}(t)$ at time t_1 is used to linearly extrapolate from $\mathbf{r}(t_1)$ to an estimate of the true value of $\mathbf{r}(t_2)$.

that a particular numerical method is of “order n ” when its error term is $O(\Delta t^{(n+1)})$.

- *Stability.* Does the numerical solution tend to “settle down” over time? If a numerical method adds energy into the system, object velocities will eventually “explode,” and the system will become *unstable*. On the other hand, if a numerical method tends to remove energy from the system, it will have an overall damping effect, and the system will be *stable*.

The concept of order warrants a little more explanation. We usually measure the error of a numerical method by comparing its approximate equation with the infinite Taylor series expansion of the exact solution to the ODE. We then cancel terms by subtracting the two equations. The remaining Taylor terms represent the error inherent in the method. For example, the explicit Euler equation is

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t.$$

The infinite Taylor series expansion of the exact solution is

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 + \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + \dots,$$

where $\mathbf{r}^{(3)}$ represents the third derivative with respect to time. Therefore, the error is represented by all of the terms after the $\mathbf{v}\Delta t$ term, which is of order $O(\Delta t^2)$ (because this term dwarfs the other higher-order terms):

$$\begin{aligned} \mathbf{E} &= \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 + \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + \dots \\ &= O(\Delta t^2). \end{aligned}$$

To make the error of a method explicit, we’ll often write its equation with the error term added in “big O” notation at the end. For example, the explicit Euler method’s equation is most accurately written as follows:

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t + O(\Delta t^2).$$

We say that the explicit Euler method is a “first-order” method because it is accurate up to *and including* the Taylor series term involving Δt to the first power. In general, if a method’s error term is $O(\Delta t^{(n+1)})$, then it is said to be an “order n ” method.

13.4.4.3 Alternatives to Explicit Euler

The explicit Euler method sees quite a lot of use for simple integration tasks in games, producing the best results when the velocity is nearly constant. However, it is not used in general-purpose dynamics simulations because of its

high error and poor stability. There are all sorts of other numerical methods for solving ODEs, including backward Euler (another first-order method), midpoint Euler (a second-order method) and the family of Runge-Kutta methods. (The fourth-order Runge-Kutta, often abbreviated “RK4,” is particularly popular.) We won’t describe these in any detail here, as you can find voluminous amounts of information about them online and in the literature. The Wikipedia page http://en.wikipedia.org/wiki/Numerical_ordinary_differential_equations serves as an excellent jumping-off point for learning these methods.

13.4.4.4 Verlet Integration

The numerical ODE method most often used in interactive games these days is probably the Verlet method, so I’ll take a moment to describe it in some detail. There are actually two variants of this method: regular Verlet and the so-called *velocity Verlet*. I’ll present both methods here, but I’ll leave the theory and deep explanations to the myriad papers and Web pages available on the topic. (For a start, check out http://en.wikipedia.org/wiki/Verlet_integration.)

The regular Verlet method is attractive because it achieves a high order (low error), is relatively simple and inexpensive to evaluate, and produces a solution for position directly in terms of acceleration in one step (as opposed to the two steps normally required to go from acceleration to velocity and then from velocity to position). The formula is derived by adding two Taylor series expansions, one going forward in time and one going backward in time:

$$\mathbf{r}(t_1 + \Delta t) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 + \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + O(\Delta t^4);$$

$$\mathbf{r}(t_1 - \Delta t) = \mathbf{r}(t_1) - \dot{\mathbf{r}}(t_1)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 - \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + O(\Delta t^4).$$

Adding these expressions causes the negative terms to cancel with the corresponding positive ones. The result gives us the position at the next time step in terms of the acceleration and the two (known) positions at the current and previous time steps. This is the regular Verlet method:

$$\mathbf{r}(t_1 + \Delta t) = 2\mathbf{r}(t_1) - \mathbf{r}(t_1 - \Delta t) + \mathbf{a}(t_1)\Delta t^2 + O(\Delta t^4).$$

In terms of net force, the Verlet method becomes

$$\mathbf{r}(t_1 + \Delta t) = 2\mathbf{r}(t_1) - \mathbf{r}(t_1 - \Delta t) + \frac{\mathbf{F}_{\text{net}}(t_1)}{m}\Delta t^2 + O(\Delta t^4).$$

The velocity is conspicuously absent from this expression. However, it can be found using the following somewhat inaccurate approximation (among other alternatives):

$$\mathbf{v}(t_1 + \Delta t) = \frac{\mathbf{r}(t_1 + \Delta t) - \mathbf{r}(t_1)}{\Delta t} + O(\Delta t).$$

13.4.4.5 Velocity Verlet

The more commonly used *velocity Verlet* method is a four-step process in which the time step is divided into two parts to facilitate the solution. Given that $\mathbf{a}(t_1) = \frac{1}{m}\mathbf{F}(t_1, \mathbf{r}(t_1), \mathbf{v}(t_1))$ is known, we do the following:

1. Calculate $\mathbf{r}(t_1 + \Delta t) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t + \frac{1}{2}\mathbf{a}(t_1)\Delta t^2$.
2. Calculate $\mathbf{v}(t_1 + \frac{1}{2}\Delta t) = \mathbf{v}(t_1) + \frac{1}{2}\mathbf{a}(t_1)\Delta t$.
3. Determine $\mathbf{a}(t_1 + \Delta t) = \mathbf{a}(t_2) = \frac{1}{m}\mathbf{F}(t_2, \mathbf{r}(t_2), \mathbf{v}(t_2))$.
4. Calculate $\mathbf{v}(t_1 + \Delta t) = \mathbf{v}(t_1 + \frac{1}{2}\Delta t) + \frac{1}{2}\mathbf{a}(t_1 + \Delta t)\Delta t$.

Notice in the third step that the force function depends on the position and velocity on the *next* time step, $\mathbf{r}(t_2)$ and $\mathbf{v}(t_2)$. We already calculated $\mathbf{r}(t_2)$ in step 1, so we have all the information we need as long as the force is not velocity-dependent. If it is velocity-dependent, then we must approximate the next frame's velocity, perhaps using the explicit Euler method.

13.4.5 Angular Dynamics in Two Dimensions

Up until now, we've focused on analyzing the linear motion of a body's center of mass (which acts as if it were a point mass). As I said earlier, an unconstrained rigid body will rotate about its center of mass. This means that we can layer the angular motion of a body on top of the linear motion of its center of mass in order to arrive at a complete description of the body's overall motion. The study of a body's rotational motion in response to applied forces is called *angular dynamics*.

In two dimensions, angular dynamics works almost identically to linear dynamics. For each linear quantity, there's an angular analog, and the mathematics works out quite neatly. So let's investigate two-dimensional angular dynamics first. As we'll see, when we extend the discussion into three dimensions, things get a bit messier, but we'll burn that bridge when we get to it!

13.4.5.1 Orientation and Angular Speed

In two dimensions, every rigid body can be treated as a thin sheet of material. (Some physics texts refer to such a body as a *plane lamina*.) All linear motion occurs in the xy -plane, and all rotations occur about the z -axis. (Visualize wooden puzzle pieces sliding about on an air hockey table.)

The orientation of a rigid body in 2D is fully described by an angle θ , measured in radians relative to some agreed-upon zero rotation. For example, we

might specify that $\theta = 0$ when a race car is facing directly down the positive x -axis in world space. This angle is of course a time-varying function, so we denote it $\theta(t)$.

13.4.5.2 Angular Speed and Acceleration

Angular velocity measures the rate at which a body's rotation angle changes over time. In two dimensions, angular velocity is a scalar, more correctly called angular *speed*, since the term "velocity" really only applies to vectors. It is denoted by the scalar function $\omega(t)$ and measured in radians per second (rad/s). Angular speed is the derivative of the orientation angle $\theta(t)$ with respect to time:

Angular:	Linear:
$\omega(t) = \frac{d\theta(t)}{dt} = \dot{\theta}(t)$	$\mathbf{v}(t) = \frac{d\mathbf{r}(t)}{dt} = \dot{\mathbf{r}}(t).$

And as we'd expect, angular acceleration, denoted $\alpha(t)$ and measured in radians per second squared (rad/s²), is the rate of change of angular speed:

Angular:	Linear:
$\alpha(t) = \frac{d\omega(t)}{dt} = \dot{\omega}(t) = \ddot{\theta}(t)$	$\mathbf{a}(t) = \frac{d\mathbf{v}(t)}{dt} = \dot{\mathbf{v}}(t) = \ddot{\mathbf{r}}(t).$

13.4.5.3 Moment of Inertia

The rotational equivalent of mass is a quantity known as the *moment of inertia*. Just as mass describes how easy or difficult it is to change the linear velocity of a point mass, the moment of inertia measures how easy or difficult it is to change the angular speed of a rigid body about a particular axis. If a body's mass is concentrated near an axis of rotation, it will be relatively easier to rotate about that axis, and it will hence have a smaller moment of inertia than a body whose mass is spread out away from that axis.

Since we're focusing on two-dimensional angular dynamics right now, the axis of rotation is always z , and a body's moment of inertia is a simple scalar value. Moment of inertia is usually denoted by the symbol I . We won't get into the details of how to calculate the moment of inertia here. For a full derivation, see [17].

13.4.5.4 Torque

Until now, we've assumed that all forces are applied to the center of mass of a rigid body. However, in general, forces can be applied at arbitrary points on a

body. If the line of action of a force passes through the body's center of mass, then the force will produce linear motion only, as we've already seen. Otherwise, the force will introduce a rotational force known as a *torque* in addition to the linear motion it normally causes. This is illustrated in Figure 13.24.

We can calculate torque using a cross product. First, we express the location at which the force is applied as a vector $\mathbf{r}$ extending from the body's center of mass to the point of application of the force. (In other words, the vector $\mathbf{r}$ is in *body space*, where the origin of body space is defined to be the center of mass.) This is illustrated in Figure 13.25. The torque $\mathbf{N}$ caused by a force $\mathbf{F}$ applied at a location $\mathbf{r}$ is

$$\mathbf{N} = \mathbf{r} \times \mathbf{F}. \quad (13.7)$$

Equation (13.7) implies that torque increases as the force is applied farther from the center of mass. This explains why a lever can help us to move a heavy object. It also explains why a force applied directly through the center of mass produces no torque and no rotation—the magnitude of the vector $\mathbf{r}$ is zero in this case.

When two or more forces are applied to a rigid body, the torque vectors produced by each one can be summed, just as we can sum forces. So in general we are interested in the net torque, $\mathbf{N}_{\text{net}}$.

In two dimensions, the vectors $\mathbf{r}$ and $\mathbf{F}$ must both lie in the xy -plane, so $\mathbf{N}$ will always be directed along the positive or negative z -axis. As such, we'll denote a two-dimensional torque via the scalar N_z , which is just the z -component of the vector $\mathbf{N}$.

Torque is related to angular acceleration and moment of inertia in much

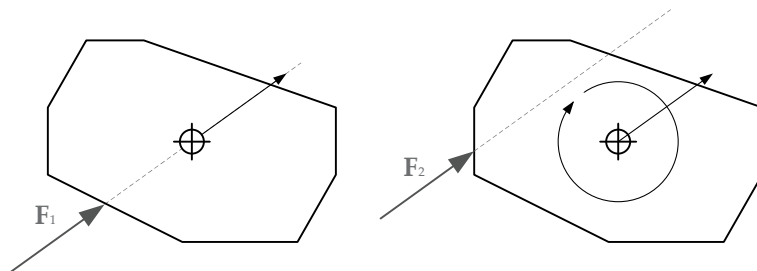


Figure 13.24. On the left, a force applied to a body's CM produces purely linear motion. On the right, a force applied off-center will give rise to a torque, producing rotational motion as well as linear motion.

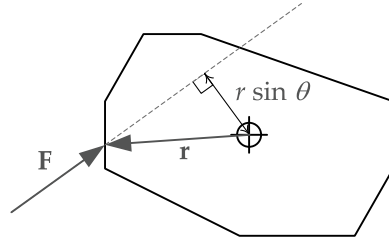


Figure 13.25. Torque is calculated by taking the cross product between a force's point of application in body space (i.e., relative to the center of mass) and the force vector. The vectors are shown here in two dimensions for ease of illustration; if it could be drawn, the torque vector would be directed into the page.

the same way that force is related to linear acceleration and mass:

Angular:		Linear:	
$N_z(t) = I\alpha(t) = I\dot{\omega}(t) = I\ddot{\theta}(t)$		$\mathbf{F}(t) = m\mathbf{a}(t) = m\dot{\mathbf{v}}(t) = m\ddot{\mathbf{r}}(t).$	(13.8)

13.4.5.5 Solving the Angular Equations of Motion in Two Dimensions

For the two-dimensional case, we can solve the angular equations of motion using exactly the same numerical integration techniques we applied to the linear dynamics problem. The pair of ODEs that we wish to solve is as follows:

Angular:		Linear:
$N_{\text{net}}(t) = I\dot{\omega}(t)$		$\mathbf{F}_{\text{net}}(t) = m\dot{\mathbf{v}}(t)$
$\omega(t) = \dot{\theta}(t)$		$\mathbf{v}(t) = \dot{\mathbf{r}}(t),$

and their approximate explicit Euler solutions are

Angular:		Linear:
$\omega(t_2) = \omega(t_1) + I^{-1}N_{\text{net}}(t_1)\Delta t$		$\mathbf{v}(t_2) = \mathbf{v}(t_1) + m^{-1}\mathbf{F}_{\text{net}}(t_1)\Delta t$
$\theta(t_2) = \theta(t_1) + \omega(t_1)\Delta t$		$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t.$

Of course, we could apply any of the other more-accurate numerical methods as well, such as the velocity Verlet method (I've omitted the linear case here for compactness, but compare this to the steps given in Section 13.4.4.5):

1. Calculate $\theta(t_1 + \Delta t) = \theta(t_1) + \omega(t_1)\Delta t + \frac{1}{2}\alpha(t_1)\Delta t^2$.
2. Calculate $\omega(t_1 + \frac{1}{2}\Delta t) = \omega(t_1) + \frac{1}{2}\alpha(t_1)\Delta t$.

3. Calculate $\alpha(t_1 + \Delta t) = \alpha(t_2) = I^{-1}N_{\text{net}}(t_2, \theta(t_2), \omega(t_2))$.
4. Calculate $\omega(t_1 + \Delta t) = \omega(t_1 + \frac{1}{2}\Delta t) + \frac{1}{2}\alpha(t_1 + \Delta t)\Delta t$.

13.4.6 Angular Dynamics in Three Dimensions

Angular dynamics in three dimensions is a somewhat more complex topic than its two-dimensional counterpart, although the basic concepts are of course very similar. In the following section, I'll give a very brief overview of how angular dynamics works in 3D, focusing primarily on the things that are typically confusing to someone who is new to the topic. For further information, check out Glenn Fiedler's series of articles on the topic, available at <http://gafferongames.com/game-physics/physics-in-3d/>. Another helpful resource is the paper entitled "An Introduction to Physically Based Modeling" by David Baraff of the Robotics Institute at Carnegie Mellon University, available at <http://www-2.cs.cmu.edu/~baraff/sigcourse/notesd1.pdf>.

13.4.6.1 The Inertia Tensor

A rigid body may have a very different distribution of mass about the three coordinate axes. As such, we should expect a body to have different moments of inertia about different axes. For example, a long thin rod should be relatively easy to make rotate about its long axis because all the mass is concentrated very close to the axis of rotation. Likewise, the rod should be relatively more difficult to make rotate about its short axis because its mass is spread out farther from the axis. This is indeed the case, and it is why a figure skater spins faster when she tucks her limbs in close to her body.

In three dimensions, the rotational mass of a rigid body is represented by a 3×3 matrix known as its *inertia tensor*. It is usually represented by the symbol $\mathbf{I}$ (as before, we won't describe how to calculate the inertia tensor here; see [17] for details):

$$\mathbf{I} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}.$$

The elements lying along the diagonal of this matrix are the moments of inertia of the body about its three principal axes, I_{xx} , I_{yy} and I_{zz} . The off-diagonal elements are called *products of inertia*. They are zero when the body is symmetrical about all three principal axes (as would be the case for a rectangular box). When they are nonzero, they tend to produce physically realistic yet somewhat unintuitive motions that the average game player would probably think were "wrong" anyway. Therefore, the inertia tensor is often

simplified down to the three-element vector $[I_{xx} \ I_{yy} \ I_{zz}]$ in game physics engines.

13.4.6.2 Orientation in Three Dimensions

In two dimensions, we know that the orientation of a rigid body can be described by a single angle θ , which measures rotation about the z -axis (assuming the motion is taking place in the xy -plane). In three dimensions, a body's orientation could be represented using three Euler angles $[\theta_x \ \theta_y \ \theta_z]$, each representing the body's rotation about one of the three Cartesian axes. However, as we saw in Chapter 5, Euler angles suffer from gimbal lock problems and can be difficult to work with mathematically. Therefore, the orientation of a body is more often represented using either a 3×3 matrix $\mathbf{R}$ or a unit quaternion $\mathbf{q}$. We'll use the quaternion form exclusively in this chapter.

Recall that a quaternion is a four-element vector whose x -, y - and z -components can be interpreted as a unit vector $\mathbf{u}$ lying along the axis of rotation, scaled by the sine of the half-angle and whose w component is the cosine of the half-angle:

$$\begin{aligned} \mathbf{q} &= [q_x \ q_y \ q_z \ q_w] \\ &= [\mathbf{q} \ q_w] \\ &= [\mathbf{u} \sin \frac{\theta}{2} \ \cos \frac{\theta}{2}]. \end{aligned}$$

A body's orientation is of course a function of time, so we should write it $\mathbf{q}(t)$.

Again, we need to select an arbitrary direction to be our zero rotation. For example, we might say that by default, the front of every object will lie along the positive z -axis in world space, with y up and x to the left. Any non-identity quaternion will serve to rotate the object away from this canonical world-space orientation. The choice of the canonical orientation is arbitrary, but of course it's important to be consistent across all assets in the game.

13.4.6.3 Angular Velocity and Momentum in Three Dimensions

In three dimensions, angular velocity is a vector quantity, denoted by $\boldsymbol{\omega}(t)$. The angular velocity vector can be visualized as a unit-length vector $\mathbf{u}$ that defines the axis of rotation, scaled by the two-dimensional angular velocity $\omega_u = \dot{\theta}_u$ of the body about the u -axis. Hence,

$$\boldsymbol{\omega}(t) = \omega_u(t)\mathbf{u} = \dot{\theta}_u(t)\mathbf{u}.$$

In linear dynamics, we saw that if there are no forces acting on a body, then the linear acceleration is zero, and linear velocity is constant. In two-dimensional angular dynamics, this again holds true: If there are no torques

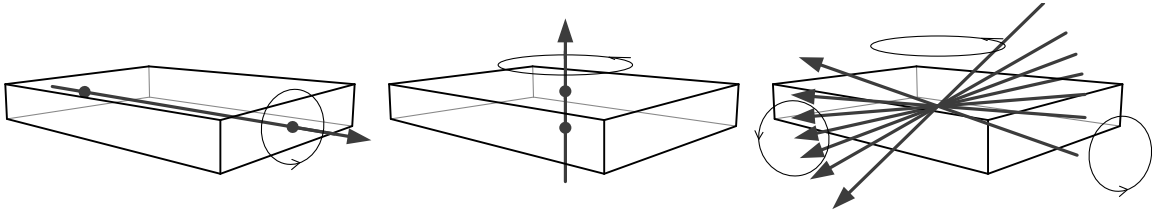


Figure 13.26. A rectangular object that is spun about its shortest or longest axis has a constant angular velocity vector. However, when spun about its medium-sized axis, the direction of the angular velocity vector changes wildly.

acting on a body in two dimensions, then the angular acceleration α is zero, and the angular speed ω about the z -axis is constant.

Unfortunately, this is *not* the case in three dimensions. It turns out that even when a rigid body is rotating in the absence of all forces, its angular velocity vector $\boldsymbol{\omega}(t)$ may not be constant because the axis of rotation can continually change direction. You can see this effect in action when you try to spin a rectangular object, like a block of wood, in mid-air in front of you. If you throw the block so that it is rotating about its shortest axis, it will spin in a stable way. The orientation of the axis stays roughly constant. The same thing happens if you try to spin the block about its longest axis. But if you try to spin the block around the remaining axis (the one that's neither the shortest nor the longest), the rotation will be utterly unstable. (Try it! Go steal a wooden block from a baby and spin it in various ways. On second thought, make sure to give it back when you're done.) The axis of rotation itself changes direction wildly as the object spins. This is illustrated in Figure 13.26.

The fact that the angular velocity vector can change in the absence of torques is another way of saying that angular velocity is not *conserved*. However, a related quantity called the *angular momentum* does remain constant in the absence of forces and hence *is* conserved. Angular momentum is the rotational equivalent of linear momentum:

Angular:	Linear:
$\mathbf{L}(t) = \mathbf{I}\boldsymbol{\omega}(t)$	$\mathbf{p}(t) = m\mathbf{v}(t).$

Like the linear case, angular momentum $\mathbf{L}(t)$ is a three-element vector. However, unlike the linear case, rotational mass (the inertia tensor) is not a scalar but rather a 3×3 matrix. As such, the expression $\mathbf{I}\boldsymbol{\omega}$ is computed via a

matrix multiplication:

$$\begin{bmatrix} L_x(t) \\ L_y(t) \\ L_z(t) \end{bmatrix} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix} \begin{bmatrix} \omega_x(t) \\ \omega_y(t) \\ \omega_z(t) \end{bmatrix}.$$

Because the angular velocity $\boldsymbol{\omega}$ is not conserved, we do not treat it as a primary quantity in our dynamics simulations the way we do the linear velocity $\mathbf{v}$. Instead, we treat angular momentum $\mathbf{L}$ as the primary quantity. The angular velocity is a secondary quantity, determined only after we have determined the value of $\mathbf{L}$ at each time step of the simulation.

13.4.6.4 Torque in Three Dimensions

In three dimensions, we still calculate torque as the cross product between the radial position vector of the point of force application and the force vector itself ($\mathbf{N} = \mathbf{r} \times \mathbf{F}$). Equation (13.8) still holds, but we always write it in terms of the angular momentum because angular velocity is not a conserved quantity:

$$\begin{aligned} \mathbf{N} &= \mathbf{I}\boldsymbol{\alpha}(t) \\ &= \mathbf{I} \frac{d\boldsymbol{\omega}(t)}{dt} \\ &= \frac{d}{dt}(\mathbf{I}\boldsymbol{\omega}(t)) \\ &= \frac{d\mathbf{L}(t)}{dt}. \end{aligned}$$

13.4.6.5 Solving the Equations of Angular Motion in Three Dimensions

When solving the equations of angular motion in three dimensions, we might be tempted to take exactly the same approach we used for linear motion and two-dimensional angular motion. We might guess that the differential equations of motion should be written

Angular 3D?	Linear:
$\mathbf{N}_{\text{net}}(t) = \mathbf{I}\dot{\boldsymbol{\omega}}(t)$	$\mathbf{F}_{\text{net}}(t) = m\dot{\mathbf{v}}(t)$
$\boldsymbol{\omega}(t) = \dot{\boldsymbol{\theta}}(t)$	$\mathbf{v}(t) = \dot{\mathbf{r}}(t),$

and using the explicit Euler method, we might guess that the approximate solutions to these ODEs would look something like this:

Angular 3D?	Linear:
$\boldsymbol{\omega}(t_2) = \boldsymbol{\omega}(t_1) + \mathbf{I}^{-1}\mathbf{N}_{\text{net}}(t_1)\Delta t$	$\mathbf{v}(t_2) = \mathbf{v}(t_1) + m^{-1}\mathbf{F}_{\text{net}}(t)\Delta t$
$\boldsymbol{\theta}(t_2) = \boldsymbol{\theta}(t_1) + \boldsymbol{\omega}(t_1)\Delta t$	$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t.$

However, this is *not* actually correct. The differential equations of three-dimensional angular motion differ from their linear and two-dimensional angular counterparts in two important ways:

1. Instead of solving for the angular velocity $\boldsymbol{\omega}$, we solve for the angular momentum $\mathbf{L}$ directly. We then calculate the angular velocity vector as a secondary quantity using $\mathbf{I}$ and $\mathbf{L}$. We do this because angular momentum is conserved, while angular velocity is not.
2. When solving for the orientation given the angular velocity, we have a problem: The angular velocity is a three-element *vector*, while the orientation is a four-element *quaternion*. How can we write an ODE relating a quaternion to a vector? The answer is that we cannot, at least not directly. But what we can do is convert the angular velocity vector into quaternion form and then apply a slightly odd-looking equation that relates the orientation quaternion to the angular velocity quaternion.

It turns out that when we express a rigid body's orientation as a quaternion, the derivative of this quaternion is related to the body's angular velocity vector in the following way. First, we construct an *angular velocity quaternion*. This quaternion contains the three components of the angular velocity vector in x , y and z , with its w -component set to zero:

$$\boldsymbol{\omega} = [\omega_x \quad \omega_y \quad \omega_z \quad 0].$$

Now the differential equation relating the orientation quaternion to the angular velocity quaternion is (for reasons we won't get into here) as follows:

$$\frac{d\boldsymbol{\omega}(t)}{dt} = \dot{\mathbf{q}}(t) = \frac{1}{2}\boldsymbol{\omega}(t)\mathbf{q}(t).$$

It's important to remember here that $\boldsymbol{\omega}(t)$ is the angular velocity *quaternion* as described above and that the product $\boldsymbol{\omega}(t)\mathbf{q}(t)$ is a *quaternion* product (see Section 5.4.2.1 for details).

So, we actually need to write the ODEs of motion as follows (note that I've recast the linear ODEs in terms of linear momentum as well, to underscore the similarities between the two cases):

Angular 3D:	Linear:
$\mathbf{N}_{\text{net}}(t) = \dot{\mathbf{L}}(t)$	$\mathbf{F}_{\text{net}}(t) = \dot{\mathbf{p}}(t)$
$\boldsymbol{\omega}(t) = \mathbf{I}^{-1}\mathbf{L}(t)$	$\mathbf{v}(t) = m^{-1}\mathbf{p}(t)$
$\boldsymbol{\omega}(t) = [\boldsymbol{\omega}(t) \quad 0]$	$\mathbf{v}(t) = \dot{\mathbf{r}}(t).$
$\frac{1}{2}\boldsymbol{\omega}(t)\mathbf{q}(t) = \dot{\mathbf{q}}(t)$	

Using the explicit Euler method, the final approximate solution to the angular ODEs in three dimensions is actually as follows:

$$\begin{aligned}\mathbf{L}(t_2) &= \mathbf{L}(t_1) + \mathbf{N}_{\text{net}}(t_1)\Delta t && \text{(vectors)} \\ &= \mathbf{L}(t_1) + \Delta t \sum_{\forall i} (\mathbf{r}_i \times \mathbf{F}_i(t_1)); && \text{(vectors)} \\ \omega(t_2) &= [\mathbf{I}^{-1}\mathbf{L}(t_2) \quad 0]; && \text{(quaternions)} \\ \mathbf{q}(t_2) &= \mathbf{q}(t_1) + \frac{1}{2}\omega(t_1)\mathbf{q}(t_1)\Delta t. && \text{(quaternions)}\end{aligned}$$

The orientation quaternion $\mathbf{q}(t)$ should be renormalized periodically to reverse the effects of the inevitable accumulation of floating-point error.

As always, the explicit Euler method is being used here just as an example. In a real engine, we would employ velocity Verlet, RK4 or some other more-stable and more-accurate numerical method.

13.4.7 Collision Response

Everything we've discussed so far assumes that our rigid bodies are neither colliding with anything, nor is their motion constrained in any other way. When bodies collide with one another, the dynamics simulation must take steps to ensure that they respond realistically to the collision and that they are never left in a state of interpenetration after the simulation step has been completed. This is known as *collision response*.

13.4.7.1 Energy

Before we discuss collision response, we must understand the concept of *energy*. When a force moves a body over a distance, we say that the force does *work*. Work represents a change in energy—that is, a force either adds energy to a system of rigid bodies (e.g., an explosion) or it removes energy from the system (e.g., friction). Energy comes in two forms. The potential energy V of a body is the energy it has simply because of where it is relative to a force field such as a gravitational or a magnetic field. (For example, the higher up a body is above the surface of the Earth, the more gravitational potential energy it has.) The kinetic energy of a body T represents the energy arising from the fact that it is moving relative to other bodies in a system. The total energy $E = V + T$ of an isolated system of bodies is a *conserved* quantity, meaning that it remains constant unless energy is being drained from the system or added from outside the system.

The kinetic energy arising from linear motion can be written

$$T_{\text{linear}} = \frac{1}{2}mv^2,$$

or in terms of the linear momentum and velocity vectors:

$$T_{\text{linear}} = \frac{1}{2} \mathbf{p} \cdot \mathbf{v}.$$

Analogously, the kinetic energy arising from a body's rotational motion is as follows:

$$T_{\text{angular}} = \frac{1}{2} \mathbf{L} \cdot \boldsymbol{\omega}.$$

Energy and its conservation can be extremely useful concepts when solving all sorts of physics problems. We'll see the role that energy plays in the determination of collision responses in the following section.

13.4.7.2 Impulsive Collision Response

When two bodies collide in the real world, a complex set of events takes place. The bodies compress slightly and then rebound, changing their velocities and losing energy to sound and heat in the process. Most real-time rigid body dynamics simulations approximate all of these details with a simple model based on an analysis of the momenta and kinetic energies of the colliding objects, called *Newton's law of restitution for instantaneous collisions with no friction*. It makes the following simplifying assumptions about the collision:

- The collision force acts over an infinitesimally short period of time, turning it into what we call an idealized *impulse*. This causes the velocities of the bodies to change *instantaneously* as a result of the collision.
- There is no friction at the point of contact between the objects' surfaces. This is another way of saying that the impulse acting to separate the bodies during the collision is normal to both surfaces—there is no tangential component to the collision impulse. (This is just an idealization of course; we'll get to friction in Section 13.4.7.5.)
- The nature of the complex submolecular interactions between the bodies during the collision can be approximated by a single quantity known as the *coefficient of restitution*, customarily denoted by the symbol ϵ . This coefficient describes how much energy is lost during the collision. When $\epsilon = 1$, the collision is perfectly elastic, and no energy is lost. (Picture two billiard balls colliding in mid-air.) When $\epsilon = 0$, the collision is perfectly *inelastic*, also known as perfectly *plastic* and the kinetic energy of both bodies is lost. The bodies will stick together after the collision, continuing to move in the direction that their mutual center of mass had been moving before the collision. (Picture pieces of putty being slammed together.)

All collision analysis is based around the idea that linear momentum is conserved. So for two bodies 1 and 2, we can write

$$\mathbf{p}_1 + \mathbf{p}_2 = \mathbf{p}'_1 + \mathbf{p}'_2, \quad \text{or} \\ m_1 \mathbf{v}_1 + m_2 \mathbf{v}_2 = m'_1 \mathbf{v}'_1 + m'_2 \mathbf{v}'_2$$

where the primed symbols represent the momenta and velocities after the collision. The kinetic energy of the system is conserved as well, but we must account for the energy lost due to heat and sound by introducing an additional energy loss term T_{lost} :

$$\frac{1}{2} m_1 v_1^2 + \frac{1}{2} m_2 v_2^2 = \frac{1}{2} m'_1 v'^2_1 + \frac{1}{2} m'_2 v'^2_2 + T_{\text{lost}}.$$

If the collision is perfectly *elastic*, the energy loss T_{lost} is zero. If it is perfectly *plastic*, the energy loss is equal to the original kinetic energy of the system, the primed kinetic energy sum becomes zero and the bodies stick together after the collision.

To resolve a collision using Newton's law of restitution, we apply an idealized *impulse* to the two bodies. An impulse is like a force that acts over an infinitesimally short period of time and thereby causes an instantaneous change in the velocity of the body to which it is applied. We could denote an impulse with the symbol $\Delta \mathbf{p}$, since it is a change in momentum ($\Delta \mathbf{p} = m \Delta \mathbf{v}$). However, most physics texts use the symbol $\hat{\mathbf{p}}$ (pronounced "p-hat") instead, so we'll do the same.

Because we assume that there is no friction involved in the collision, the impulse vector must be normal to both surfaces at the point of contact. In other words, $\hat{\mathbf{p}} = \hat{p} \mathbf{n}$, where $\mathbf{n}$ is the unit vector normal to both surfaces. This is illustrated in Figure 13.27. If we assume that the surface normal points toward body 1, then body 1 experiences an impulse of $\hat{\mathbf{p}}$, and body 2 experiences an equal but opposite impulse $-\hat{\mathbf{p}}$. Hence, the momenta of the two bodies after the collision can be written in terms of their momenta prior to the collision and the impulse $\hat{\mathbf{p}}$ as follows:

$$\begin{aligned} \mathbf{p}'_1 &= \mathbf{p}_1 + \hat{\mathbf{p}} & \mathbf{p}'_2 &= \mathbf{p}_2 - \hat{\mathbf{p}} \\ m_1 \mathbf{v}'_1 &= m_1 \mathbf{v}_1 + \hat{\mathbf{p}} & m_2 \mathbf{v}'_2 &= m_2 \mathbf{v}_2 - \hat{\mathbf{p}} \\ \mathbf{v}'_1 &= \mathbf{v}_1 + \frac{\hat{\mathbf{p}}}{m_1} \mathbf{n} & \mathbf{v}'_2 &= \mathbf{v}_2 + \frac{\hat{\mathbf{p}}}{m_2} \mathbf{n}. \end{aligned} \quad (13.9)$$

The *coefficient of restitution* provides the key relationship between the relative velocities of the bodies before and after the collision. Given that the centers of mass of the bodies have velocities before the collision and afterward, the coefficient of restitution ε is defined as follows:

$$(\mathbf{v}'_2 - \mathbf{v}'_1) = \varepsilon(\mathbf{v}_2 - \mathbf{v}_1). \quad (13.10)$$

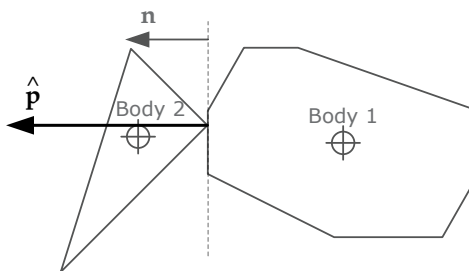


Figure 13.27. In a frictionless collision, the impulse acts along a line normal to both surfaces at the point of contact. This line is defined by the unit normal vector $\mathbf{n}$.

Solving Equations (13.9) and (13.10) under the temporary assumption that the bodies cannot rotate yields

$$\hat{\mathbf{p}} = \hat{p}\mathbf{n} = \frac{(\varepsilon + 1)(\mathbf{v}_2 \cdot \mathbf{n} - \mathbf{v}_1 \cdot \mathbf{n})}{\frac{1}{m_1} + \frac{1}{m_2}}\mathbf{n}.$$

Notice that if the coefficient of restitution is one (perfectly elastic collision) and if the mass of body 2 is effectively infinite (as it would be for, say, a concrete driveway), then $(1/m_2) = 0$, $\mathbf{v}_2 = 0$, and this expression reduces to a reflection of the other body's velocity vector about the contact normal, as we'd expect:

$$\begin{aligned}\hat{\mathbf{p}} &= -2m_1(\mathbf{v}_1 \cdot \mathbf{n})\mathbf{n}; \\ \mathbf{v}'_1 &= \frac{\mathbf{p}_1 + \mathbf{p}_2}{m_1} \\ &= \frac{m_1\mathbf{v}_1 - 2m_1(\mathbf{v}_1 \cdot \mathbf{n})\mathbf{n}}{m_1} \\ &= \mathbf{v}_1 - 2m_1(\mathbf{v}_1 \cdot \mathbf{n})\mathbf{n}.\end{aligned}$$

The solution gets a bit hairier when we take the rotations of the bodies into account. In this case, we need to look at the velocities of the points of contact on the two bodies rather than the velocities of their centers of mass, and we need to calculate the impulse in such a way as to impart a realistic rotational effect as a result of the collision. We won't get into the details here, but Chris Hecker's article, available at <http://chrishecker.com/images/e/e7/Gdmphys3.pdf>, does an excellent job of describing both the linear and the rotational aspects of collision response. The theory behind collision response is explained more fully in [17].

13.4.7.3 Penalty Forces

Another approach to collision response is to introduce imaginary forces called *penalty forces* into the simulation. A penalty force acts like a stiff damped spring attached to the contact points between two bodies that have just interpenetrated. Such a force induces the desired collision response over a short but finite period of time. Using this approach, the spring constant k effectively controls the duration of the interpenetration, and the damping coefficient b acts a bit like the restitution coefficient. When $b = 0$, there is no damping—no energy is lost, and the collision is perfectly elastic. As b increases, the collision becomes more plastic.

Let's take a brief look at some of the pros and cons of the penalty force approach to resolving collisions. On the positive side, penalty forces are easy to implement and understand. They also work well when three or more bodies are interpenetrating each other. This problem is very difficult to solve when resolving collisions one pair at a time. A good example is the Sony PS3 demo in which a huge number of rubber duckies are poured into a bathtub—the simulation was nice and stable despite the very large number of collisions. The penalty force method is a great way to achieve this.

Unfortunately, because penalty forces respond to penetration (i.e., relative position) rather than to relative velocity, the forces may not align with the direction we would intuitively expect, especially during a high-speed collision. A classic example is a car driving head-on into a truck. The car is low while the truck is tall. Using only the penalty force method, it is easy to arrive at a situation in which the penalty force is vertical, rather than horizontal as we would expect given the velocities of the two vehicles. This can cause the truck to pop its nose up into the air while the car drives under it.

In general, the penalty force technique works well for low-speed impacts, but it does not work well at all when objects are moving quickly. It is possible to combine the penalty force method with other collision resolution approaches in order to strike a balance between stability in the presence of large numbers of interpenetrations and responsiveness and more-intuitive behavior at high velocities.

13.4.7.4 Using Constraints to Resolve Collisions

As we'll investigate in Section 13.4.8, most physics systems permit various kinds of constraints to be imposed on the motion of the bodies in the simulation. If collisions are treated as constraints that disallow object interpenetration, then they can be resolved by simply running the simulation's general-purpose constraint solver. If the constraint solver is fast and produces high-

quality visual results, this can be an effective way to resolve collisions.

13.4.7.5 Friction

Friction is a force that arises between two bodies that are in continuous contact, resisting their movement relative to one another. There are a number of types of friction. *Static friction* is the resistance one feels when trying to start a stationary object sliding along a surface. *Dynamic friction* is a resisting force that arises when objects are actually moving relative to one another. *Sliding friction* is a type of dynamic friction that resists movement when an object slides along a surface. *Rolling friction* is a type of static or dynamic friction that acts at the point of contact between a wheel or other round object and the surface it is rolling on. When the surface is very rough, the rolling friction is exactly strong enough to cause the wheel to roll without sliding, and it acts as a form of static friction. If the surface is somewhat smooth, the wheel may slip, and a dynamic form of rolling friction comes into play. *Collision friction* is the friction that acts instantaneously at the point of contact when two bodies collide while moving. (This is the friction force that we ignored when discussing Newton's law of restitution in Section 13.4.7.1.) Various kinds of *constraints* can have friction as well. For example, a rusted hinge or axle might resist being turned by introducing a friction torque.

Let's look at an example to understand the essence of how friction works. Linear sliding friction is proportional to the component of an object's weight that is acting normal to the surface on which it is sliding. The weight of an object is just the force due to gravity, $\mathbf{G} = m\mathbf{g}$, which is always directed downward. The component of this force normal to an inclined surface that makes an angle θ with the horizontal is just $G_N = mg \cos \theta$. The friction force f is then

$$f = \mu mg \cos \theta,$$

where the constant of proportionality μ is called the *coefficient of friction*. This force acts tangentially to the surface, in a direction opposite to the attempted or actual motion of the object. This is illustrated in Figure 13.28.

Figure 13.28 also shows the component of the gravitational force acting tangent to the surface, $G_T = mg \sin \theta$. This force tends to make the object accelerate down the plane, but in the presence of sliding friction, it is counteracted by f . Hence, the net force tangent to the surface is

$$F_{\text{net}} = G_T - f = mg(\sin \theta - \mu \cos \theta).$$

If the angle of inclination is such that the expression in parentheses is zero, the object will slide at a constant speed (if already moving) or be at rest. If the

expression is greater than zero, the object will accelerate down the surface. If it is less than zero, the object will decelerate and eventually come to rest.

13.4.7.6 Welding

An additional problem arises when an object is sliding across a polygon soup. Recall that a polygon soup is just what its name implies—a soup of essentially unrelated polygons (usually triangles). As an object slides from one triangle to the next, the collision detection system will generate additional spurious contacts because it will think that the object is about to hit the edge of the next triangle. This is illustrated in Figure 13.29.

There are a number of solutions to this problem. One is to analyze the set of contacts and discard ones that appear to be spurious, based on various heuristics and possibly some knowledge of the object's contacts on a previous frame (e.g., if we know the object was sliding along a surface and a contact normal arises that is due to the object being near the edge of its current triangle, then discard that contact normal). Versions of Havok prior to 4.5 employed this approach.

Starting with Havok 4.5, a new technique was implemented that essentially annotates the mesh with triangle adjacency information. The collision detection system therefore “knows” which edges are interior edges and can discard spurious collisions reliably and quickly. Havok describes this solution as *welding*, because in effect the edges of the triangles in the poly soup are welded to one another.

13.4.7.7 Coming to Rest, Islands and Sleeping

When energy is removed from a simulated system via friction, damping or other means, moving objects will eventually come to rest. This seems like a natural consequence of the simulation—something that would just “fall out” of the differential equations of motion. Unfortunately, in a real computerized

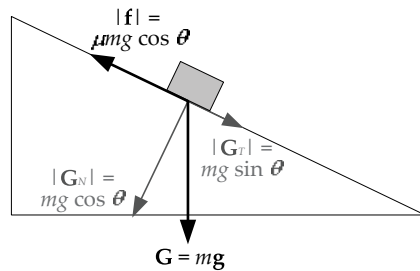


Figure 13.28. The force of friction f is proportional to the normal component of the object's weight. The proportionality constant μ is called the coefficient of friction.

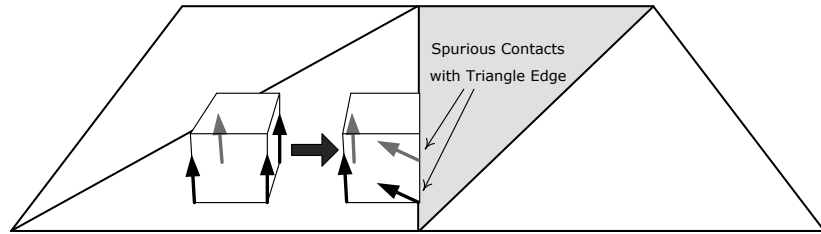


Figure 13.29. When an object slides between two adjacent triangles, spurious contacts with the new triangle's edge can be generated.

simulation, coming to rest is never quite that simple. Various factors such as floating-point error, inaccuracies in the calculation of restitution forces and numerical instability can cause objects to jitter forever rather than coming to rest as they should. For this reason, most physics engines use various heuristic methods to detect when objects are oscillating instead of coming to rest as they should. Additional energy can be removed from the system to ensure that such objects eventually settle down, or they can simply be stopped abruptly once their average velocity drops below a threshold.

When an object really does stop moving (finds itself in a state of equilibrium), there is no reason to continue integrating its equations of motion every frame. To optimize performance, most physics engines allow dynamic objects in the simulation to be *put to sleep*. This excludes them from the simulation temporarily, although sleeping objects are still active from a collision standpoint. If any force or impulse begins acting on a sleeping object, or if the object loses one of the contacts that was holding it in equilibrium, it will be awoken so that its dynamic simulation can be resumed.

Sleep Criteria

Various criteria can be used to determine whether or not a body qualifies for sleep. It's not always easy to make this determination in a robust manner for all situations. For example, a long pendulum might have very low angular momentum and yet still be moving visibly on-screen.

The most commonly used criteria for equilibrium detection include:

- The body is *supported*. This means it has three or more contact points (or one or more *planar* contacts) that allow it to attain equilibrium with gravity and any other forces that might be affecting it.
- The body's *linear and angular momentum* are below a predefined threshold.

- A *running average* of the linear and angular momentum are below a predefined threshold.
- The total *kinetic energy* of the body ($T = \frac{1}{2}\mathbf{p} \cdot \mathbf{v} + \frac{1}{2}\mathbf{L} \cdot \boldsymbol{\omega}$) is below a predefined threshold. The kinetic energy is usually mass-normalized so that a single threshold can be used for all bodies regardless of their masses.

The motion of a body that is about to go to sleep might be *progressively damped* so that it comes to a smooth stop rather than stopping abruptly.

Simulation Islands

Both Havok and PhysX further optimize their performance by automatically grouping objects that either are interacting or have the potential to interact in the near future into sets called *simulation islands*. Each simulation island can be simulated independently of all the other islands—an approach that is highly conducive to cache coherency optimizations and parallel processing.

Havok and PhysX both put entire islands to sleep rather than individual rigid bodies. This approach has its pros and cons. The performance boost is obviously larger when a whole group of interacting objects can be put to sleep. On the other hand, if even one object in an island is awake, the entire island is awake. Overall, it seems that the pros tend to outweigh the cons, so the simulation island design is one we're likely to continue to see in future versions of these SDKs.

13.4.8 Constraints

An unconstrained rigid body has six degrees of freedom (DOF): It can translate in three dimensions, and it can rotate about the three Cartesian axes. *Constraints* restrict an object's motion, reducing its degrees of freedom either partially or completely. Constraints can be used to model all sorts of interesting behaviors in a game. Here are a few examples:

- a swinging chandelier (point-to-point constraint);
- a door that can be kicked, slammed, blown off its hinges (hinge constraint);
- a vehicle's wheel assembly (axle constraint with damped springs for suspension);
- a train or a car pulling a trailer (stiff spring/rod constraint);
- a rope or chain (chain of stiff springs or rods); and

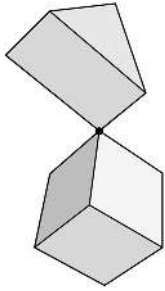


Figure 13.30. A point-to-point constraint requires that a point on body A aligns with a point on body B.

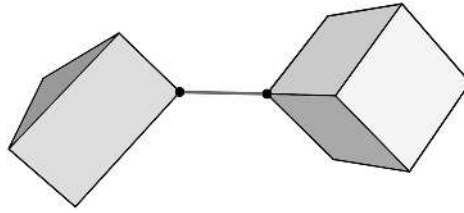


Figure 13.31. A stiff spring constraint requires that a point on body A be separated from a point on body B by a user-specified distance.

- a rag doll (specialized constraints that mimic the behavior of various joints in the human skeleton).

In the sections that follow, we'll briefly investigate these and some of the other most common kinds of constraints typically provided by a physics SDK.

13.4.8.1 Point-to-Point Constraints

A point-to-point constraint is the simplest type of constraint. It acts like a ball-and-socket joint—bodies can move in any way they like, as long as a specified point on one body lines up with a specified point on the other body. This is illustrated in Figure 13.30.

13.4.8.2 Stiff Springs

A stiff spring constraint is a lot like a point-to-point constraint except that it keeps the two points separated by a specified distance. This kind of constraint acts like an invisible rod between the two constrained points. Figure 13.31 illustrates this constraint.

13.4.8.3 Hinge Constraints

A hinge constraint limits rotational motion to only a single degree of freedom, about the hinge's axis. An *unlimited hinge* acts like an axle, allowing the constrained object to complete an unlimited number of full rotations. It's common to define *limited hinges* that can only move through a predefined range of angles about the one allowed axis. For example, a one-way door can only move through a 180 degree arc, because otherwise it would pass through the adjacent wall. Likewise, a two-way door is constrained to move through a ± 180 degree arc. Hinge constraints may also be given a degree of friction in the form of a torque that resists rotation about the hinge's axis. A limited hinge constraint is shown in Figure 13.32.

13.4.8.4 Prismatic Constraints

Prismatic constraints act like a piston: A constrained body's motion is restricted to a single translational degree of freedom. A prismatic constraint may or may not permit rotation about the translation axis of the piston. Prismatic constraints can of course be limited or unlimited and may or may not include friction. A prismatic constraint is illustrated in Figure 13.33.

13.4.8.5 Other Common Constraint Types

Many other types of constraints are possible, of course. Here are just a few examples:

- *Planar*. Objects are constrained to move in a two-dimensional plane.
- *Wheel*. This is typically a hinge constraint with unlimited rotation, coupled with some form of vertical suspension simulated via a spring-damper assembly.
- *Pulley*. In this specialized constraint, an imaginary rope passes through a pulley and is attached to two bodies. The bodies move along the line of the rope via a leverage ratio.

Constraints may be breakable, meaning that after enough force is applied, they automatically come apart. Alternatively, the game can turn the constraint on and off at will, using its own criteria for when the constraint should break.

13.4.8.6 Constraint Chains

Long chains of linked bodies are sometimes difficult to simulate in a stable manner because of the iterative nature of the constraint solver. A *constraint chain* is a specialized group of constraints with information that tells the constraint solver how the objects are connected. This allows the solver to deal with the chain in a more stable manner than would otherwise be possible.

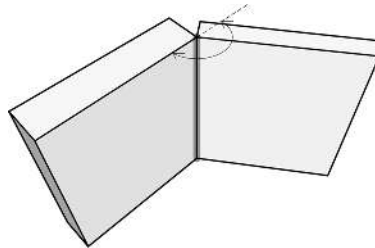


Figure 13.32. A limited hinge constraint mimics the behavior of a door.

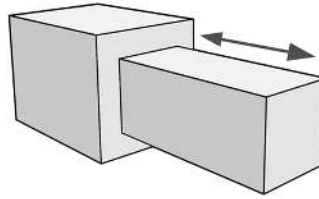


Figure 13.33. A prismatic constraint acts like a piston.

13.4.8.7 Rag Dolls

A rag doll is a physical simulation of the way a human body might move when it is dead or unconscious and hence entirely limp. Rag dolls are created by linking together a collection of rigid bodies, one for each semi-rigid part of the body. For example, we might have capsules for the feet, calves, thighs, hands, upper and lower arms and head and possibly a few for the torso to simulate the flexibility of the spine.

The rigid bodies in a rag doll are connected to one another via constraints. Rag doll constraints are specialized to mimic the kinds of motions the joints in a real human body can perform. We usually make use of constraint chains to improve the stability of the simulation.

A rag doll simulation is always tightly integrated with the animation system. As the rag doll moves in the physics world, we extract the positions and rotations of the rigid bodies, and we use this information to drive the positions and orientations of certain joints in the animated skeleton. So in effect, a rag doll is really just a form of *procedural animation* that happens to be driven by the physics system. (See Chapter 12 for more details on skeletal animation.)

Of course, implementing a rag doll is not quite as simple as I've made it sound here. For one thing, there's usually not a one-to-one mapping between the rigid bodies in the rag doll and the joints in the animated skeleton—the skeleton usually has more joints than the rag doll has bodies. Therefore, we need a system that can map rigid bodies to joints (i.e., one that “knows” to which joint each rigid body in the rag doll corresponds). There may be additional joints between those that are being driven by the rag doll bodies, so the mapping system must also be capable of determining the correct pose transforms for these intervening joints. This is not an exact science. We must apply artistic judgment and/or some knowledge of human biomechanics in order to achieve a natural-looking rag doll.

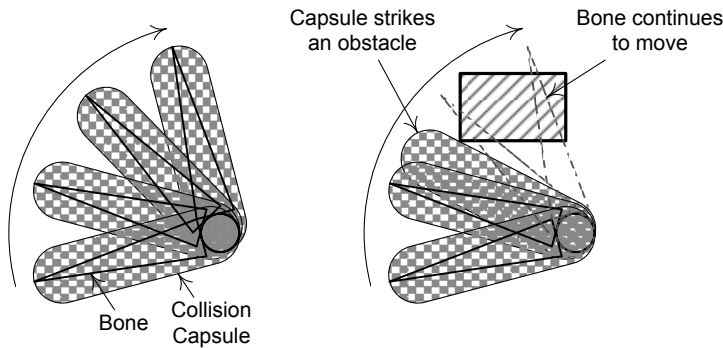


Figure 13.34. With a powered rag doll constraint, and in the absence of any additional forces or torques, a rigid body representing the lower arm can be made to exactly track the movements of an animated elbow joint (left). If an obstacle blocks the motion of the body, it will diverge from that of the animated elbow joint in a realistic way (right).

13.4.8.8 Powered Constraints

Constraints can also be “powered,” meaning that an external engine system such as the animation system can indirectly control the translations and orientations of the rigid bodies in the rag doll.

Let’s take an elbow joint as an example. An elbow acts pretty much like a limited hinge, with a little less than 180 degrees of free rotation. (Actually, an elbow can also rotate axially, but we’ll ignore that for the purposes of this discussion.) To power this constraint, we model the elbow as a *rotational spring*. Such a spring exerts a torque proportional to the spring’s angle of deflection away from some predefined rest angle, $N = -k(\theta - \theta_{\text{rest}})$. Now imagine changing the rest angle externally, say by ensuring that it always matches the angle of the elbow joint in an animated skeleton. As the rest angle changes, the spring will find itself out of equilibrium, and it will exert a torque that tends to rotate the elbow back into alignment with θ_{rest} . In the absence of any other forces or torques, the rigid bodies will exactly track the motion of the elbow joint in the animated skeleton. But if other forces are introduced (for example, the lower arm comes in contact with an immovable object), then these forces will play into the overall motion of the elbow joint, allowing it to diverge from the animated motion in a somewhat realistic manner. As illustrated in Figure 13.34, this provides the illusion of a human who is trying her best to move in a certain way (i.e., the “ideal” motion provided by the animation) but who is sometimes unable to do so due to the limitations of the physical world (e.g., her arm gets caught on something as she tries to swing it forward).

13.4.9 Controlling the Motions of Rigid Bodies

Most game designs call for a degree of control over the way rigid bodies move over and above the way they would move naturally under the influence of gravity and in response to collisions with other objects in the scene. For example:

- An air vent applies an upward force to any object that enters its shaft of influence.
- A car is coupled to a trailer and exerts a pulling force on it as it moves.
- A tractor beam exerts a force on an unwitting spacecraft.
- An anti-gravity device causes objects to hover.
- The flow of a river creates a force field that causes objects floating in the river to move downstream.

And the list goes on. Most physics engines typically provide their users with a number of ways to exert control over the bodies in the simulation. We'll outline the most common of these mechanisms in the following sections.

13.4.9.1 Gravity

Gravity is ubiquitous in most games that take place on the surface of the Earth or some other planet (or on a spacecraft with simulated gravity). Gravity is technically not a force but rather a (roughly) constant acceleration, so it affects all bodies equally regardless of their mass. Because of its ubiquitous and special nature, the magnitude and direction of the gravitational acceleration is specified via a global setting in most SDKs. (If you're writing a space game, you can always set gravity to zero to eliminate it from the simulation.)

13.4.9.2 Applying Forces

Any number of forces can be applied to the bodies in a game physics simulation. A force always acts over a finite time interval. (If it acted instantaneously, it would be called an *impulse*—more on that in Section 13.4.9.4 below.) The forces in a game are often dynamic in nature—they often change their directions and/or their magnitudes every frame. So the force-application function in most physics SDKs is designed to be called once per frame for the duration of the force's influence. The signature of such a function usually looks something like this: `applyForce(const Vector& forceInNewtons)`, where the duration of the force is assumed to be Δt .

13.4.9.3 Applying Torques

When a force is applied such that its line of action passes through the center of mass of a body, no torque is generated, and only the body's linear acceleration is affected. If it is applied off-center, it will induce *both* a linear and a rotational acceleration. A *pure torque* can be applied to a body as well by applying two equal and opposite forces to points equidistant from the center of mass. The linear motions induced by such a pair of forces will cancel each other out (since for the purposes of linear dynamics, the forces both act at the center of mass). This leaves only their rotational effects. A pair of torque-inducing forces like this is known as a *couple* ([http://en.wikipedia.org/wiki/Couple_\(mechanics\)](http://en.wikipedia.org/wiki/Couple_(mechanics))). A special function such as `applyTorque(const Vector& torque)` may be provided for this purpose. However, if your physics SDK provides no `applyTorque()` function, you can always write one and have it generate a suitable couple instead.

13.4.9.4 Applying Impulses

As we saw in Section 13.4.7.2, an *impulse* is an instantaneous change in velocity (or actually, a change in momentum). Technically speaking, an impulse is a force that acts for an infinitesimal amount of time. However, the shortest possible duration of force application in a time-stepped dynamics simulation is Δt , which is not short enough to simulate an impulse adequately. As such, most physics SDKs provide a function with a signature such as `applyImpulse(const Vector& impulse)` for the purposes of applying impulses to bodies. Of course, impulses come in two flavors—linear and angular—and a good SDK should provide functions for applying both types.

13.4.10 The Collision/Physics Step

Now that we've covered the theory and some of the technical details behind implementing a collision and physics system, let's take a brief look at how these systems actually perform their updates every frame.

Every collision/physics engine performs the following basic tasks during its update step. Different physics SDKs may perform these phases in different orders. That said, the technique I've seen used most often goes something like this:

1. The forces and torques acting on the bodies in the physics world are integrated forward by Δt in order to determine their tentative positions and orientations next frame.

2. The collision detection library is called to determine if any new contacts have been generated between any of the objects as a result of their tentative movement. (The bodies normally keep track of their contacts in order to take advantage of temporal coherency. Hence, at each step of the simulation, the collision engine need only determine whether any previous contacts have been lost and whether any new contacts have been added.)
3. Collisions are resolved, often by applying impulses or penalty forces or as part of the constraint-solving step below. Depending on the SDK, this phase may or may not include continuous collision detection (CCD, otherwise known as time of impact detection or TOI).
4. Constraints are satisfied by the constraint solver.

At the conclusion of step 4, some of the bodies may have moved away from their tentative positions as determined in step 1. This movement may cause additional interpenetrations between objects or cause other previously satisfied constraints to be broken. Therefore, steps 1 through 4 (or sometimes only 2 through 4, depending on how collisions and constraints are resolved) are repeated until either (a) all collisions have been successfully resolved and all constraints are satisfied, or (b) a predefined maximum number of iterations has been exceeded. In the latter case, the solver effectively “gives up,” with the hope that things will resolve themselves naturally during subsequent frames of the simulation. This helps to avoid performance spikes by amortizing the cost of collision and constraint resolution over multiple frames. However, it can lead to incorrect-looking behavior if the errors are too large or if the time step is too long or is inconsistent. Penalty forces can be blended into the simulation in order to gradually resolve these problems over time.

13.4.10.1 The Constraint Solver

A constraint solver is essentially an iterative algorithm that attempts to satisfy a large number of constraints simultaneously by *minimizing the error* between the actual positions and rotations of the bodies in the physics world and their ideal positions and rotations as defined by the constraints. As such, constraint solvers are essentially iterative error-minimization algorithms.

Let’s take a look first at how a constraint solver works in the trivial case of a single pair of bodies connected by a single hinge constraint. During each step of the physics simulation, the numerical integrator will find new tentative transforms for the two bodies. The constraint solver then evaluates their

relative positions and calculates the error between the positions and orientations of their shared axis of rotation. If any error is detected, the solver moves the bodies in such a way as to minimize or eliminate it. Since there are no other bodies in the system, the second iteration of the step should discover no new contacts, and the constraint solver will find that the one hinge constraint is now satisfied. Hence the loop can exit without further iterations.

When more than one constraint must be satisfied simultaneously, more iterations may be required. During each iteration, the numerical integrator will sometimes tend to move the bodies out of alignment with their constraints, while the constraint solver tends to put them back into alignment. With luck, and a carefully designed approach to minimizing error in the constraint solver, this feedback loop should eventually settle into a valid solution. However, the solution may not always be exact. This is why, in games with physics engines, you sometimes witness seemingly impossible behaviors, like chains that stretch (opening up little gaps between the links), objects that interpenetrate briefly or hinges that momentarily move beyond their allowable ranges. The goal of the constraint solver is to minimize error—it's not always possible to eliminate it completely.

13.4.10.2 Variations between Engines

The description given above is of course an over-simplification of what really goes on in a physics/collision engine every frame. The way in which the various phases of computation are performed, and their order relative to one another, may vary from physics SDK to physics SDK. For example, some kinds of constraints are modeled as forces and torques that are taken care of by the numerical integration step rather than being resolved by the constraint solver. Collision may be run before the integration step rather than after. Collisions may be resolved in any number of different ways. Our goal here is merely to give you a taste of how these systems work. For a detailed understanding of how any one SDK operates, you'll want to read its documentation and probably also inspect its source code (presuming the relevant bits are available for you to read). The curious and industrious reader can get a good start by downloading and experimenting with Open Dynamics Engine (ODE) and/or PhysX, as these two SDKs are available for free. You can also learn a great deal from ODE's wiki, which is available at http://opende.sourceforge.net/wiki/index.php/Main_Page.

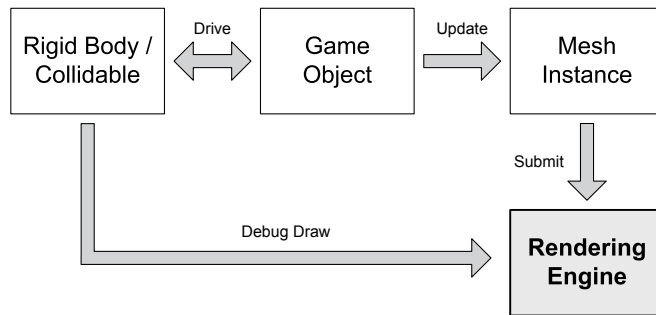


Figure 13.35. Rigid bodies are linked to their visual representations by way of game objects. An optional direct rendering path is usually provided so that the locations of the rigid bodies can be visualized for debugging purposes.

13.5 Integrating a Physics Engine into Your Game

Obviously, a collision/physics engine is of little use by itself—it must be integrated into your game engine. In this section, we’ll discuss the most common interface points between the collision/physics engine and the rest of the game code.

13.5.1 Linking Game Objects and Rigid Bodies

The rigid bodies and collidables in the collision/physics world are nothing more than abstract mathematical descriptions. In order for them to be useful in the context of a game, we need to link them in some way to their visual representations on-screen. Usually, we don’t draw the rigid bodies directly (except for debugging purposes). Instead, the rigid bodies are used to describe the shape, size, and physical behavior of the logical objects that make up the virtual game world. We’ll discuss game objects in depth in Chapter 16, but for the time being, we’ll rely on our intuitive notion of what a game object is—a logical entity in the game world, such as a character, a vehicle, a weapon, a floating power-up and so on. So the linkage between a rigid body in the physics world and its visual representation on-screen is usually indirect, with the logical game object serving as the hub that links the two together. This is illustrated in Figure 13.35.

In general, a game object is represented in the collision/physics world by zero or more rigid bodies. The following list describes three possible scenarios:

- *Zero rigid bodies.* Game objects without any rigid bodies in the physics

world act as though they are not solid, because they have no collision representation at all. Decorative objects with which the player or non-player characters cannot interact, such as birds flying overhead or portions of the game world that can be seen but never reached, might have no collision. This scenario can also apply to objects whose collision detection is handled manually (without the help of the collision/physics engine) for some reason.

- *One rigid body.* Most simple game objects need only be represented by a single rigid body. In this case, the shape of the rigid body's collidable is chosen to closely approximate the shape of the game object's visual representation, and the rigid body's position and orientation exactly match the position and orientation of the game object itself.
- *Multiple rigid bodies.* Some complex game objects are represented by multiple rigid bodies in the collision/physics world. Examples include characters, machinery, vehicles or any object that is composed of multiple solid pieces that can move relative to one another. Such game objects usually make use of a skeleton (i.e., a hierarchy of affine transforms) to track the locations of their component pieces (although other means are certainly possible as well). The rigid bodies are usually linked to the joints of the skeleton in such a way that the position and orientation of each rigid body corresponds to the position and orientation of one of the joints. The joints in the skeleton might be driven by an animation, in which case the associated rigid bodies simply come along for the ride. Alternatively, the physics system might drive the locations of rigid bodies and hence indirectly control the locations of the joints. The mapping from joints to rigid bodies may or may not be one-to-one—some joints might be controlled entirely by animation, while others are linked to rigid bodies.

The linkage between game objects and rigid bodies must be managed by the engine, of course. Typically, each game object will manage its own rigid bodies, creating and destroying them when necessary, adding and removing them from the physics world as needed, and maintaining the connection between each rigid body's location and the location of the game object and/or one of its joints. For complex game objects consisting of multiple rigid bodies, a wrapper class of some kind may be used to manage them. This insulates the game objects from the nitty-gritty details of managing a collection of rigid bodies and allows different kinds of game objects to manage their rigid bodies in a consistent way.

13.5.1.1 Physics-Driven Bodies

If our game has a rigid body dynamics system, then presumably we want the motions of at least some of the objects in the game to be driven entirely by the simulation. Such game objects are called *physics-driven* objects. Bits of debris, exploding buildings, rocks rolling down a hillside, empty magazines and shell casings—these are all examples of physics-driven objects.

A physics-driven rigid body is linked to its game object by stepping the simulation and then querying the physics system for the body's position and orientation. This transform is then applied either to the game object as a whole or to a joint or some other data structure within the game object.

Example: Building a Safe with a Detachable Door

When physics-driven rigid bodies are linked to the joints of a skeleton, the bodies are often *constrained* to produce a desired kind of motion. As an example, let's look at how a safe with a detachable door might be modeled.

Visually, let's assume that the safe consists of a single triangle mesh with two submeshes, one for the housing and one for the door. A two-joint skeleton is used to control the motions of these two pieces. The root joint is bound to the housing of the safe, while the child joint is bound to the door in such a way that rotating the door joint causes the door submesh to swing open and shut in a suitable way.

The collision geometry for the safe is broken into two independent pieces as well, one for the housing and one for the door. These two pieces are used to create two totally separate rigid bodies in the collision/physics world. The rigid body for the safe's housing is attached to the root joint in the skeleton, and the door's rigid body is linked to the door joint. A hinge constraint is then added to the physics world to ensure that the door body swings properly relative to the housing when the dynamics of the two rigid bodies are simulated. The motions of the two rigid bodies representing the housing and the door are used to update the transforms of the two joints in the skeleton. Once the skeleton's matrix palette has been generated by the animation system, the rendering engine will end up drawing the housing and door submeshes in the locations of the rigid bodies within the physics world.

If the door needs to be blown off at some point, the constraint can be broken, and impulses can be applied to the rigid bodies to send them flying. Visibly, it will appear to the human player that the door and the housing have become separate objects. But in reality, it's still a single game object and a single triangle mesh with two joints and two rigid bodies.

13.5.1.2 Game-Driven Bodies

In most games, certain objects in the game world need to be moved about in a non-physical way. The motions of such objects might be determined by an animation or by following a spline path, or they might be under the control of the human player. We often want these objects to participate in collision detection—to be capable of pushing the physics-driven objects out of their way, for example—but we do not want the physics system to interfere with their motion in any way. To accommodate such objects, most physics SDKs provide a special type of rigid body known as a *game-driven body*. (Havok calls these “key framed” bodies.)

Game-driven bodies do not experience the effects of gravity. They are also considered to be infinitely massive by the physics system (usually denoted by a mass of zero, since this is an invalid mass for a physics-driven body). The assumption of infinite mass ensures that forces and collision impulses within the simulation can never change the velocity of a game-driven body.

To move a game-driven body around in the physics world, we cannot simply set its position and orientation every frame to match the location of the corresponding game object. Doing so would introduce discontinuities that would be very difficult for the physical simulation to resolve. (For example, a physics-driven body might find itself suddenly interpenetrating a game-driven body, but it would have no information about the game-driven body’s momentum with which to resolve the collision.) As such, game-driven bodies are usually moved using impulses—instantaneous changes in velocity that, when integrated forward in time, will position the bodies in the desired places at the end of the time step. Most physics SDKs provide a convenience function that will calculate the linear and angular impulses required in order to achieve a desired position and orientation on the next frame. When moving a game-driven body, we do have to be careful to zero out its velocity when it is supposed to stop. Otherwise, the body will continue forever along its last nonzero trajectory.

Example: Animated Safe Door

Let’s continue our example of the safe with a detachable door. Imagine that we want a character to walk up to the safe, dial the combination, open the door, deposit some money and close and lock the door again. Later, we want a different character to get the money in a rather less-civilized manner—by blowing the door off the safe. To do this, the safe would be modeled with an additional submesh for the dial and an additional joint that allows the dial to be rotated. No rigid body is required for the dial, however, unless of course

we want it to fly off when the door explodes.

During the animated sequence of the person opening and closing the safe, its rigid bodies can be put into game-driven mode. The animation now drives the joints, which in turn drive the rigid bodies. Later, when the door is to be blown off, we can switch the rigid bodies into physics-driven mode, break the hinge constraint, apply the impulse and watch the door fly.

As you've probably already noticed, the hinge constraint is not actually needed in this particular example. It would only be required if the door is to be left open at some point and we want to see the door swinging naturally in response to the safe being moved or the door being bumped.

13.5.1.3 Fixed Bodies

Most game worlds are composed of both static geometry and dynamic objects. To model the static components of the game world, most physics SDKs provide a special kind of rigid body known as a *fixed body*. Fixed bodies act a bit like game-driven bodies, but they do not take part in the dynamics simulation at all. They are, in effect, collision-only bodies. This optimization can give a big performance boost to most games, especially those whose worlds contain only a small number of dynamic objects moving around within a large static world.

13.5.1.4 Havok's Motion Type

In Havok, all types of rigid body are represented by instances of the class `hkpRigidBody`. Each instance contains a field that specifies its *motion type*. The motion type tells the system whether the body is fixed, game-driven (what Havok calls "key framed") or physics-driven (what Havok calls "dynamic"). If a rigid body is created with the fixed motion type, its type can never be changed. Otherwise, the motion type of a body can be changed dynamically at runtime. This feature can be incredibly useful. For example, an object that is in a character's hand would be game-driven. But as soon as the character drops or throws the object, it would be changed to physics-driven so the dynamics simulation can take over its motion. This is easily accomplished in Havok by simply changing the motion type at the moment of release.

The motion type also doubles as a way to give Havok some hints about the inertia tensor of a dynamic body. As such, the "dynamic" motion type is broken into subcategories such as "dynamic with sphere inertia," "dynamic with box inertia" and so on. Using the body's motion type, Havok can decide to apply various optimizations based on assumptions about the internal structure of the inertia tensor.

13.5.2 Updating the Simulation

The physics simulation must of course be updated periodically, usually once per frame. This does not merely involve stepping the simulation (numerically integrating, resolving collisions and applying constraints). The linkages between the game objects and their rigid bodies must be maintained as well. If the game needs to apply any forces or impulses to any of the rigid bodies, this must also be done every frame. The following steps are required to completely update the physics simulation:

- *Update game-driven rigid bodies.* The transforms of all game-driven rigid bodies in the physics world are updated so that they match the transforms of their counterparts (game objects or joints) in the game world.
- *Update phantoms.* A phantom shape acts like a game-driven collidable with no corresponding rigid body. It is used to perform certain kinds of collision queries. The locations of all phantoms are updated prior to the physics step, so that they will be in the right places when collision detection is run.
- *Update forces, apply impulses and adjust constraints.* Any forces being applied by the game are updated. Any impulses caused by game events that occurred this frame are applied. Constraints are adjusted if necessary. (For example, a breakable hinge might be checked to determine if it has been broken; if so, the physics engine is instructed to remove the constraint.)
- *Step the simulation.* We saw in Section 13.4.10 that the collision and physics engines must both be updated periodically. This involves *numerically integrating* the equations of motion to find the physical state of all bodies on the next frame, running the *collision detection* algorithm to add and remove contacts from all rigid bodies in the physics world, *resolving collisions* and *applying constraints*. Depending on the SDK, these update phases may be hidden behind a single atomic `step()` function, or it may be possible to run them individually.
- *Update physics-driven game objects.* The transforms of all physics-driven objects are extracted from the physics world, and the transforms of the corresponding game objects or joints are updated to match.
- *Query phantoms.* The contacts of each phantom shape are read after the physics step and used to make decisions.
- *Perform collision cast queries.* Ray casts and shape casts are kicked off, either synchronously or asynchronously. When the results of these queries

become available, they are used by various engine systems to make decisions.

These tasks are usually performed in the order shown above, with the exception of ray and shape casts, which can theoretically be done at any time during the game loop. Clearly it makes sense to update game-driven bodies and apply forces and impulses prior to the step, so that the effects will be “seen” by the simulation. Likewise, physics-driven game objects should always be updated after the step, to ensure that we’re using the most up-to-date body transforms. Rendering typically happens after everything else in the game loop. This ensures that we are rendering a consistent view of the game world at a particular instant in time.

13.5.2.1 Timing Collision Queries

In order to query the collision system for up-to-date information, we need to run our collision queries (ray and shape casts) after the physics step has run during the frame. However, the physics step is usually run toward the end of the frame, after the game logic has made most of its decisions and the new locations of any game-driven physics bodies have been determined. When, then, should collision queries be run?

This question does not have an easy answer. We have a number of options, and most games end up using some or all of them:

- *Base decisions on last frame’s state.* In many cases, decisions can be made correctly based on last frame’s collision information. For example, we might want to know whether or not the player was standing on something last frame, in order to decide whether or not he should start falling this frame. In this case, we can safely run our collision queries prior to the physics step.
- *Accept a one-frame lag.* Even if we really want to know what is happening *this frame*, we may be able to tolerate a one-frame lag in our collision query results. This is usually only true if the objects in question aren’t moving too fast. For example, we might move one object forward in time and then want to know whether or not that object is now in the player’s line of sight. A one-frame-off error in this kind of query may not be noticeable to the player. If this is the case, we can run the collision query prior to the physics step (returning collision information from the previous frame) and then use these results as if they were an *approximation* to the collision state at the end of the current frame.
- *Run the query after the physics step.* Another approach is to run certain queries after the physics step. This is feasible when the decisions being

made based on the results of the query can be deferred until late in the frame. For example, a rendering effect that depends on the results of a collision query could be implemented this way.

13.5.2.2 Single-Threaded Updating

A very simple single-threaded game loop might look something like this:

```
F32 dt = 1.0f/30.0f;

for (;;) // main game loop
{
    g_hidManager->poll();

    g_gameObjectManager->preAnimationUpdate(dt);
    g_animationEngine->updateAnimations(dt);
    g_gameObjectManager->postAnimationUpdate(dt);

    g_physicsWorld->step(dt);
    g_animationEngine->updateRagDolls(dt);

    g_gameObjectManager->postPhysicsUpdate(dt);
    g_animationEngine->finalize();

    g_effectManager->update(dt);

    g_audioEngine->update(dt);

    // etc.

    g_renderManager->render();

    dt = calcDeltaTime();
}
```

In this example, our game objects are updated in three phases: once before animation runs (during which they can queue up new animations, for example), once after the animation system has calculated final local poses and a tentative global pose (but before the final global pose and matrix palette has been generated) and once after the physics system has been stepped.

- The locations of all game-driven rigid bodies are generally updated in `preAnimationUpdate()` or `postAnimationUpdate()`. Each game-driven body's transform is set to match the location of either the game object that owns it or a joint in the owner's skeleton.

- The location of each physics-driven rigid body is generally read in `post-PhysicsUpdate()` and used to update the location of either the game object or one of the joints in its skeleton.

One important concern is the frequency with which you are stepping the physics simulation. Most numerical integrators, collision detection algorithms and constraint solvers operate best when the time between steps (Δt) is constant. It's usually a good idea to step your physics/collision SDK with an ideal 1/30 second or 1/60 second time delta and then govern the frame rate of your overall game loop. If your game drops below its target frame rate, it's better to let the physics slow down visually than to try to adjust the simulation time step to match the actual frame rate.

13.5.3 Example Uses of Collision and Physics in a Game

To make our discussion of collision and physics more concrete, let's take a high-level look at a few common examples of how collision and/or physics simulations are commonly used in real games.

13.5.3.1 Simple Rigid Body Game Objects

Many games include simple physically simulated objects like weapons, rocks that can be picked up and thrown, empty magazines, furniture, objects on shelves that can be shot and so on. Such objects might be implemented by creating a custom game object class and giving it a reference to a rigid body in the physics world (e.g., `hkpRigidBody` if we're using Havok). Or we might create an add-on component class that handles simple rigid body collision and physics, allowing this feature to be added to virtually any type of game object in the engine.

Simple physics objects usually change their motion type at runtime. They are game-driven when being held in a character's hand and physics-driven when in free fall after having been dropped.

13.5.3.2 Bullet Traces

Whether or not you approve of game violence, the fact remains that laser guns and projectile weapons of one form or another are a big part of most games. Let's look at how these are typically implemented.

Sometimes projectiles are implemented using ray casts. On the frame that the weapon is fired, we shoot off a ray cast, determine what object was hit and immediately impart the impact to the affected object.

Unfortunately, the ray cast approach does not account for the travel time of the projectile. It also does not account for the slight downward trajectory

caused by the influence of gravity. If these details are important to the game, we can model our projectiles using real rigid bodies that move through the collision/physics world over time. This is especially useful for slower-moving projectiles, like thrown objects or rockets. The thrown bricks in Naughty Dog's *The Last of Us* used such an approach.

There are plenty of issues to consider and deal with when implementing laser beams and projectiles. A few of the most common ones are discussed below.

Bullet Ray Casting

When using ray casting to check for bullet hits, the question arises: Does the ray come from the camera focal point or from the tip of the gun in the player character's hands? This is especially problematic in a *third-person* shooter, where the ray coming out of the player's gun usually does not align with the ray coming from the camera focal point through the reticle in the center of the screen. This can lead to situations in which the reticle appears to be on top of a target, yet the third-person character is clearly behind an obstacle and would not be able to shoot that target from his point of view. Various "tricks" must usually be employed to ensure that the player feels like he or she is shooting what he or she is aiming at while maintaining plausible visuals on the screen.

Mismatches between Collision and Visible Geometry

Mismatches between collision geometry and visible geometry can lead to situations in which the player can see the target through a small crack or just over the edge of some other object, and yet the collision geometry is solid, so the bullet cannot reach the target. (This is usually only a problem for the player character.) One solution to this problem is to use a render query instead of a collision query to determine if the ray actually hit the target. For example, during one of the rendering passes, we could generate a texture in which each pixel stores the unique identifier of the game object to which it corresponds. We can then query this texture to determine whether or not an enemy character or other suitable target occupies the pixel(s) underneath the weapon's reticle.

Aiming in a Dynamic Environment

AI-controlled characters may need to "lead" their shots if projectiles take a finite amount of time to reach their targets.

Impact Effects

When bullets hit their targets, we may want to trigger a sound or a particle effect, lay down a decal or perform other tasks.

In the Unreal engine, this is accomplished via a system of *physical materials*. Visible geometry can be tagged not only with visual materials, but with physical materials as well. The former defines how the surface looks, the latter defines how it reacts to physical interactions, including impact sounds, bullet “squib” particle effects, decals and so on. (See <http://udn.epicgames.com/Three/PhysicalMaterialSystem.html> for more details.)

At Naughty Dog, we use a very similar system: Collision geometry can be tagged with *polygon attributes* (PATs for short), which define certain physical behaviors like footstep sounds. But bullet impacts are treated in a special way, because we need them to interact directly with visible geometry rather than the crude collision geo. As such, visible materials can be tagged with an optional *bullet effect* that defines which bullet squib, impact sound and decal should be laid down for each possible type of projectile that might impact that surface.

13.5.3.3 Grenades

Grenades in games are sometimes implemented as free-moving physics objects. However, this leads to a significant loss of control. Some control can be regained by imposing various artificial forces or impulses on the grenade. For example, we could apply an extreme air drag once the grenade bounces for the first time, in an attempt to limit the distance it can bounce away from its target.

Some game teams actually go so far as to manage the grenade’s motion entirely manually. The arc of a grenade’s trajectory can be calculated beforehand, using a series of ray casts to determine what target it would hit if released. The trajectory can even be shown to the player via some kind of on-screen display. When the grenade is thrown, the game moves it along its arc and can then carefully control the bounce so that it never goes too far away from its target, while still looking natural.

13.5.3.4 Explosions

In a game, an explosion typically has a few components: some kind of visual effect like a fireball and smoke, audio effects to mimic the sound of the explosion and its impacts with objects in the world, and a growing damage radius that affects any objects in its wake.

When an object finds itself in the radius of an explosion, its health is typically reduced, and we often also want to impart some motion to mimic the effect of the shock wave. This might be done via an animation. (For example, the reaction of a character to an explosion might best be done this way.) We might also wish to allow the impact reaction to be driven entirely by the dynamics simulation. We can accomplish this by having the explosion apply impulses to any suitable objects within its radius. It's pretty easy to calculate direction of these impulses—they are typically radial, calculated by normalizing the vector from the center of the explosion to the center of the impacted object and then scaling this vector by the magnitude of the explosion (and perhaps falling off as the distance from the epicenter increases).

Explosions may interact with other engine systems as well. For example, we might want to impart a “force” to the animated foliage system, causing grass, plants and trees to momentarily bend as a result of the explosion's shock wave.

13.5.3.5 Destructible Objects

Destructible objects are commonplace in many games. These objects are peculiar because they start out in an undamaged state in which they must appear to be a single cohesive object, and yet they must be capable of breaking into many separate pieces. We may want the pieces to break off one by one, allowing the object to be “whittled down” gradually, or we may only require a single catastrophic explosion.

Deformable body simulations like DMM can handle destruction naturally. However, we can also implement breakable objects using rigid body dynamics. This is typically done by dividing a model into a number of breakable pieces and assigning a separate rigid body to each one. This is the approach taken by Havok Destruction, for example.

For reasons of performance optimization and/or visual quality, we might decide to use special “undamaged” versions of the visual and collision geometry, each of which is constructed as a single solid piece. This model can be swapped out for the damaged version when the object needs to start breaking apart. In other cases, we may want to model the object as separate pieces at all times. This might be appropriate if the object is a stack of bricks or a pile of pots and pans, for example.

To model a multi-piece object, we could simply stack a bunch of rigid bodies and let the physics simulation take care of it. This can be made to work in good-quality physics engines (although it's not always trivial to get right). However, we may want some Hollywood-style effects that cannot be achieved with a simple stack of rigid bodies.

For example, we may want to define the structure of the object. Some pieces might be *indestructible*, like the base of a wall or the chassis of a car. Others might be *non-structural*—they just fall off when hit by bullets or other objects. Still other pieces might be *structural*—if they are hit, not only do they fall, but they also impart forces to other pieces lying on top of them. Some pieces could be *explosive*—when they are hit, they create secondary explosions or propagate damage throughout the structure. We may want some pieces to act as valid *cover* points for some characters, but not others. This implies that our breakable object system may have some connections to the cover system.

We might also want our breakable objects to have a notion of health. Damage might build up until eventually the whole thing collapses, or each piece might have a health, requiring multiples shots or impacts before it is allowed to break. Constraints might also be employed to allow broken pieces to hang off the object rather than coming away from it completely.

We may also want our structures to take time to collapse completely. For example, if a long bridge is hit by an explosion at one end, the collapse should slowly propagate from one end to the other so that the bridge looks massive. This is another example of a feature the physics system won't give you for free—it would just wake up all rigid bodies in the simulation island simultaneously. These kinds of effects can be implemented through judicious use of the game-driven motion type.

13.5.3.6 Character Mechanics

For a game like bowling, pinball or *Marble Madness*, the “main character” is a ball that rolls around in an imaginary game world. For this kind of game, we could very well model the ball as a free-moving rigid body in the physics simulation and control its movements by applying forces and impulses to it during gameplay.

In character-based games, however, we usually don't take this kind of approach. The locomotion of a humanoid or animal character is usually far too complex to be controlled adequately with forces and impulses. Instead, we usually model characters as a set of game-driven capsule-shaped rigid bodies, each one linked to a joint in the character's animated skeleton. These bodies are primarily used for bullet hit detection or to generate secondary effects such as when a character's arm bumps an object off a table. Because these bodies are game-driven, they won't avoid interpenetrations with immovable objects in the physics world, so it is up to the animator to ensure that the character's movements appear believable.

To move the character around in the game world, most games use sphere or capsule casts to probe in the direction of desired motion. Collisions are

resolved manually. This allows us to do cool stuff like:

- having the character slide along walls when he runs into them at an oblique angle;
- allowing the character to “pop up” over low curbs rather than getting stuck;
- preventing the character from entering a “falling” state when he walks off a low curb;
- preventing the character from walking up slopes that are too steep (most games have a cutoff angle after which the character will slide back rather than being able to walk up the slope); and
- adjusting animations to accommodate collisions.

As an example of this last point, if the character is running directly into a wall at a roughly 90 degree angle, we can let the character “moonwalk” into the wall forever, or we can slow down his animation. We can also do something even more slick, like playing an animation in which the character sticks out his hand and touches the wall and then idles sensibly until the movement direction changes.

Havok provides a character controller system that handles many of these things. In Havok’s system, illustrated in Figure 13.36, a character is modeled as a capsule phantom that is moved each frame to find a potential new location. A collision contact manifold (i.e., a collection of contact planes, cleaned up to eliminate noise) is maintained for the character. This manifold can be analyzed each frame in order to determine how best to move the character, adjust animations and so on.

13.5.3.7 Camera Collision

In many games, the camera follows the player’s character or vehicle around in the game world, and it can often be rotated or controlled in limited ways by the person playing the game. It’s important in such games to never permit the camera to interpenetrate geometry in the scene, as this would break the illusion of realism. The camera system is therefore another important client of the collision engine in many games.

The basic idea behind most camera collision systems is to surround the virtual camera with one or more sphere phantoms or sphere cast queries that can detect when it is getting close to colliding with something. The system can respond by adjusting the camera’s position and/or orientation in some way to avoid the potential collision before the camera actually passes through the object in question.

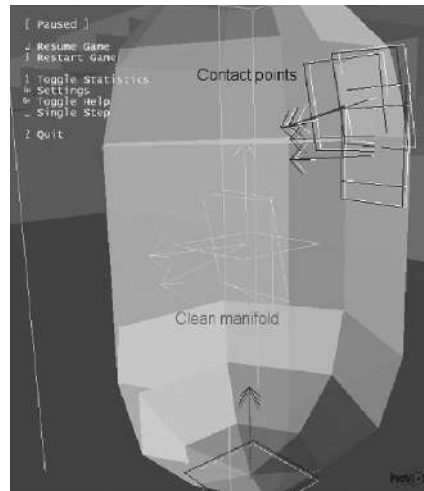


Figure 13.36. Havok's character controller models a character as a capsule-shaped phantom. The phantom maintains a noise-reduced collision manifold (a collection of contact planes) that can be used by the game to make movement decisions.

This sounds simple enough, but it is actually an incredibly tricky problem requiring a great deal of trial and error to get right. To give you a feel for how much effort can be involved, many game teams have a dedicated engineer working on the camera system for the entire duration of the project. We can't possibly cover camera collision detection and resolution in any depth here, but the following list should give you a sense of some of the most pertinent issues to be aware of:

- Zooming the camera in to avoid collisions works well in a wide variety of situations. In a third-person game, you can zoom all the way in to a first-person view without causing too much trouble (other than making sure the camera doesn't interpenetrate the character's head in the process).
- It's usually a bad idea to drastically change the horizontal angle of the camera in response to collisions, as this tends to mess with camera-relative player controls. However, some degree of horizontal adjustment can work well, depending on what the player is expected to be doing at the time. If she is aiming at a target, she'll be angry with you if you throw off her aim to bring the camera out of collision. But if she's just locomoting through the world, the change in camera orientation may feel entirely natural. Because of this, you might want to allow adjustments to the horizontal angle of the camera only when the main character is not

in the heat of a battle.

- You can adjust the vertical angle of the camera to some degree, but it's important not to do too much of this, or the player will lose track of the horizon and end up looking down onto the top of the player character's head!
- Some games allow the camera to move along an arc lying in a vertical plane, perhaps described by a spline. This permits a single HID control such as the vertical deflection of the left thumb stick to control both the zoom and the vertical angle of the camera in an intuitive way. (The camera in the Naughty Dog engine works this way.) When the camera comes into collision with objects in the world, it can be automatically moved along this same arc to avoid the collision, the arc might be compressed horizontally, or any number of other approaches might be taken.
- It's important to consider not only what's behind and beside the camera but what is in front of it as well. For example, what should happen if a pillar or another character comes between the camera and the player character? In some games, the offending object becomes translucent; in others, the camera zooms in or swings around to avoid the collision. This may or may not feel good to the person playing the game! How you handle these kinds of situations can make or break the perceived quality of your game.

Even after taking account of these and many other problematic situations, your camera may not look or feel right! Always budget plenty of time for trial and error when implementing a camera collision system.

13.5.3.8 Rag Doll Integration

In Section 13.4.8.7, we learned how special types of constraints can be used to link a collection of rigid bodies together to mimic the behavior of a limp (dead or unconscious) human body. In this section, we'll investigate a few of the issues that arise when integrating rag doll physics into your game.

As we saw in Section 13.5.3.6, the gross movements of a conscious character are usually determined by performing shape casts or moving a phantom shape around in the game world. The detailed movements of the character's body are typically driven by animations. Game-driven rigid bodies are sometimes attached to the limbs for the purposes of weapons targeting or to allow the character to knock over other objects in the world.

When a character becomes unconscious, the rag doll system kicks in. The character's limbs are modeled as capsule-shaped rigid bodies connected via constraints and linked to joints in the character's animated skeleton. The

physics system simulates the motions of these bodies, and we update the skeletal joints to match, thereby allowing physics to move the character's body.

The set of rigid bodies used for rag doll physics might not be the same ones affixed to the character's limbs when it was alive. This is because the two collision models have very different requirements. When the character is alive, its rigid bodies are game-driven, so we don't care if they interpenetrate. And in fact, we usually want them to overlap, so there aren't any holes through which an enemy character might shoot. But when the character turns into a rag doll, it's important that the rigid bodies do not interpenetrate, as this would cause the collision resolution system to impart large impulses that would tend to make the limbs explode outward! For these reasons, it's actually quite common for characters to have entirely different collision/physics representations depending on whether they're conscious or unconscious.

Another issue is how to transition from the conscious state to the unconscious state. A simple LERP blend between animation-generated and physics-generated poses usually doesn't work very well, because the physics pose very quickly diverges from the animation pose. (A blend between two totally unrelated poses usually doesn't look natural.) As such, we may want to use powered constraints during the transition (see Section 13.4.8.8).

Characters often interpenetrate background geometry when they are conscious (i.e., when their rigid bodies are game-driven). This means that the rigid bodies might be inside another solid object when the character transitions to rag doll (physics-driven) mode. This can give rise to huge impulses that cause rather wild-looking rag doll behavior in-game. To avoid these problems, it is best to author death animations carefully, so that the character's limbs are kept out of collision as best as possible. It's also important to detect collisions via phantoms or collision callbacks during the game-driven mode so that you can drop the character into rag doll mode the moment any part of his body touches something solid.

Even when these steps are taken, rag dolls have a tendency to get stuck inside other objects. Single-sided collision can be an incredibly important feature when trying to make rag dolls look good. If a limb is partly embedded in a wall, it will tend to be pushed out of the wall rather than staying stuck inside it. However, even single-sided collision doesn't solve all problems. For example, when the character is moving quickly or if the transition to rag doll isn't executed properly, one rigid body in the rag doll can end up on the far side of a thin wall. This causes the character to hang in mid-air rather than falling properly to the ground.

Another rag doll feature that can be useful is the ability for unconscious

characters to regain consciousness and get back up. To implement this, we need a way to search for a suitable “stand up” animation. We want to find an animation whose pose on frame zero most closely matches the rag doll’s pose after it has come to rest (which is totally unpredictable in general). This can be done by matching the poses of only a few key joints, like the upper thighs and the upper arms. Another approach is to manually guide the rag doll into a pose suitable for getting up by the time it comes to rest, using powered constraints.

As a final note, we should mention that setting up a rag doll’s constraints can be a tricky business. We generally want the limbs to move freely but without doing anything biomechanically impossible. This is one reason specialized types of constraints are often used when constructing rag dolls. Nonetheless, you shouldn’t assume that your rag dolls will look great without some effort. High-quality physics engines like Havok provide a rich set of content creation tools that allow an artist to set up constraints within a DCC package like Maya and then test them in real time to see how they might look in-game.

All in all, getting rag doll physics to *work* in your game isn’t particularly difficult, but getting it to look *good* can take a lot of work! As with many things in game programming, it’s a good idea to budget plenty of time for trial and error, especially when it’s your first time working with rag dolls.

13.6 Advanced Physics Features

A rigid body dynamics simulation with constraints can cover an amazing range of physics-driven effects in a game. However, such a system clearly has its limitations. Recent research and development is seeking to expand physics engines beyond constrained rigid bodies. Here are just a few examples:

- *Deformable bodies.* As hardware capabilities improve and more-efficient algorithms are developed, physics engines are beginning to provide support for deformable bodies. DMM is an excellent example of such an engine.
- *Cloth.* Cloth can be modeled as a sheet of point masses, connected by stiff springs. Cloth is notoriously difficult to get right, as many difficulties arise with respect to collision between cloth and other objects, numerical stability of the simulation, etc. That being said, many games and third-party physics SDKs like Havok now provide powerful and well-behaved cloth simulations for use in games and other real-time applications.

- *Hair.* Hair can be modeled by a large number of small physically simulated filaments, or a simpler approach can be used in which sheets of cloth are texture-mapped to look like hair, and the cloth simulation is tuned to make the character's hair move in a believable way. This is how Chloe's hair in *Uncharted: The Lost Legacy* works. Hair simulation and rendering remains an active area of research, and the quality of hair in games will certainly continue to improve.
- *Water surface simulations and buoyancy.* Games have been doing water surface simulations and buoyancy for some time now. Buoyancy can be implemented via a special-case system (not part of the physics engine per se), or it can be modeled via forces within the physics simulation. Organic movement of the water surface is often a rendering effect only and does not affect the physics simulation at all. From the point of view of physics, the water surface is often modeled as a plane. For large displacements in the water surface, the entire plane might be moved. However, some game teams and researchers are pushing the limits of these simulations, allowing for dynamic water surfaces, waves that crest, realistic current simulations and more. One good example is the water in the god game *From Dust*.
- *General fluid dynamics simulations.* Right now, fluid dynamics falls primarily into the realm of specialized simulation libraries. However, this is an active area of research and development, and some games already make use of fluid simulations to produce some astounding visual effects. For example, the *LittleBigPlanet* series makes use of a 2D fluid simulation for its smoke and fire effects; and the PhysX SDK offers a 3D *position based* fluid simulation that produces stunningly realistic results.
- *Physically based audio synthesis.* When physically simulated objects collide, bounce, roll and slide, it's important to be able to generate appropriate audio to reinforce the believability of the simulation. These sounds can be created in games via controlled playback of pre-recorded audio clips. But dynamic synthesis of such sounds is becoming a viable alternative, and is currently an active area of research.
- *GPGPU.* As GPUs become more and more powerful, there has been a shift toward harnessing their awesome parallel processing power for tasks other than graphics. One obvious application of general-purpose GPU (GPGPU) computing is for collision and physics simulation. For example, Naughty Dog's cloth simulation engine was ported to run entirely on the GPU for PlayStation 4.

14

Audio

If you've ever watched a horror film with your speakers muted, you know just how important audio is to immersiveness. (If not, try it! It's a real ear-opener.) Be it a film or a video game, sound can quite literally make the difference between a gripping, emotional, unforgettable multimedia experience and a lackluster yawnfest.

Modern games immerse the player in a realistic (or a semi-realistic but stylized) virtual environment. The graphics engine is charged with the task of reproducing as accurately and believably as possible what the player would actually see, if he or she were present within this virtual environment (while remaining true to the art style of the game). In exactly the same sense, the audio engine is charged with the task of accurately and believably reproducing what the player would actually hear, if he or she were present in the game world (while remaining true to the fiction and tonal style of the game). Sound programmers today use the term audio *rendering* engine to underscore its many parallels with the graphics rendering engine.

In this chapter, we're going to explore both the theory and practice of creating audio for a triple-A game. We'll introduce an area of mathematics called *signal processing theory* that underlies almost every aspect of digital audio technology, including digital sound recording and playback, filtering, reverb and other digital signal processor (DSP) effects. We'll explore game audio from a software engineering standpoint, by investigating a number of widely used

audio APIs, breaking down the components that comprise a typical audio rendering engine and learning how the audio system is interconnected with other game engine systems. We'll also see how environmental acoustic modeling and character dialog were handled in Naughty Dog's popular game *The Last of Us*. So hold on tight, keep your hands inside the car at all times and enjoy the noisy ride!

14.1 The Physics of Sound

Sound is a compression wave that travels through the air (or some other compressible medium). A sound wave gives rise to alternating regions of air compression and decompression (also known as *rarefaction*) relative to the average atmospheric pressure. As such, we measure the *amplitude* of a sound wave in units of *pressure*. In SI units, pressure is measured in Pascals, abbreviated Pa. One Pascal is the force of one Newton applied over an area of one square meter ($1 \text{ Pa} = 1 \text{ N/m}^2 = 1 \text{ kg}/(\text{m} \cdot \text{s}^2)$).

The *instantaneous acoustic pressure* is the ambient atmospheric pressure (considered a constant for our purposes) plus the perturbation caused by the sound wave at one specific instant in time:

$$p_{\text{inst}} = p_{\text{atmos}} + p_{\text{sound}}.$$

Of course, sound is a dynamic phenomenon—the sound pressure varies over time. We can plot the instantaneous sound pressure as a function of time, $p_{\text{inst}}(t)$. In *signal processing theory*—the area of mathematics that underlies virtually every aspect of digital audio technology—such a time-varying function is called a *signal*. Figure 14.1 illustrates a typical sound wave signal $p(t)$, oscillating about the average atmospheric pressure.

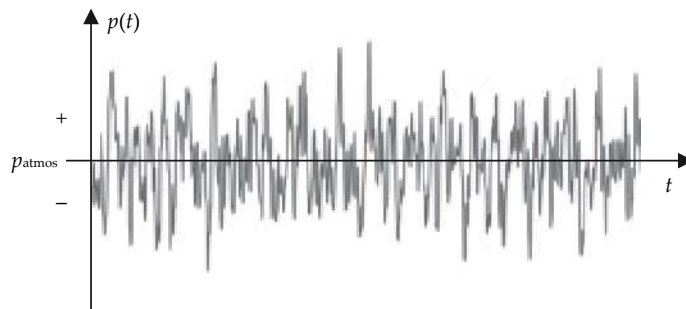


Figure 14.1. A signal $p(t)$ can be used to model the time-varying acoustic pressure of a sound.

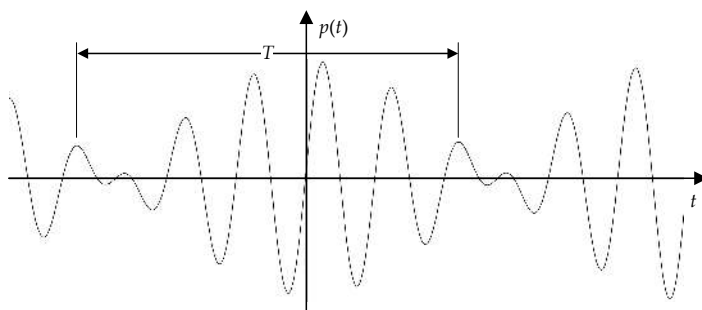


Figure 14.2. The period T of an arbitrary periodic signal is the minimum time between repeated patterns in the waveform.

14.1.1 Properties of Sound Waves

When a musical instrument plays a long steady note, the resulting sound pressure signal is *periodic*, meaning the waveform consists of a *repeating pattern* characteristic to that particular kind of instrument. The *period* T of any repeating pattern describes the minimum amount of time that passes between successive instances of the pattern. For example, for a sinusoidal sound wave the period measures the time between successive peaks or troughs. In SI units, period is typically measured in seconds (s). This is illustrated in Figure 14.2.

The *frequency* of a wave is just the inverse of its period ($f = 1/T$). Frequency is measured in Hertz (Hz), which means “cycles per second.” A “cycle” is technically a dimensionless quantity, so the Hertz is the inverse of the second ($\text{Hz} = 1/\text{s}$).

Many scientists and mathematicians make use of a quantity known as the *angular frequency*, typically denoted by the symbol ω . The angular frequency is just the rate of oscillation measured in *radians* per second instead of *cycles* per second. Since one complete circular rotation is 2π radians, $\omega = 2\pi f = 2\pi/T$. Angular frequency is very useful when analyzing sinusoidal waves, because a circular motion in two dimensions gives rise to a sinusoidal motion when projected onto a single dimensional axis.

The amount by which a periodic signal such as a sine wave is *shifted* left or right along the time axis is known as its *phase*. Phase is a relative term. For example, $\sin(t)$ is really just a version of $\cos(t)$ that has been *phase-shifted* by $+\frac{\pi}{2}$ along the t axis (i.e., $\sin(t) = \cos(t - \frac{\pi}{2})$). Likewise $\cos(t)$ is just $\sin(t)$ phase-shifted by $-\frac{\pi}{2}$ (i.e., $\cos(t) = \sin(t + \frac{\pi}{2})$). Phase is illustrated in Figure 14.3.

The *speed* v at which a sound wave propagates through its medium depends upon the material and physical properties of the medium, including phase (solid, gas or liquid), temperature, pressure and density. In 20°C dry

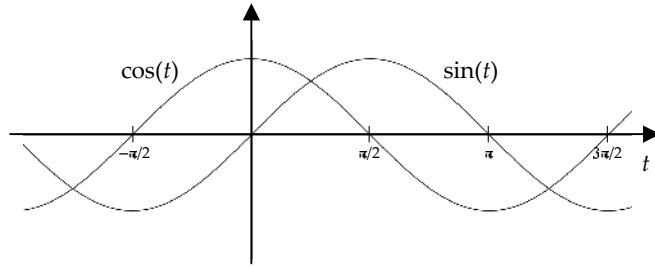


Figure 14.3. The sine and cosine functions are just phase-shifted versions of one another.

air, the speed of sound is approximately 343.2 m/s, which is 767.7 mph or 1235.6 km/h.

The *wavelength* λ of a sinusoidal wave measures the spatial distance between successive peaks or troughs. It depends in part on the frequency of the wave, but because it is a *spatial* measurement it also depends on the speed of the wave. Specifically, $\lambda = v/f$ where v is the speed of the wave (measured in m/s) and f is the frequency (measured in Hz or 1/s). The seconds in the numerator and denominator cancel one another, and we are left with wavelength measured in units of meters.

14.1.2 Perceived Loudness and the Decibel

In order to judge the “loudness” of the sounds we hear, our ears continuously *average* the amplitude of the incoming sound signal over a short, sliding time window. This averaging effect is modeled well by a quantity known as the *effective sound pressure*. This is defined as the *root mean square* (RMS) of the instantaneous sound pressure measured over a specific interval of time.

If we were to take a series of n discrete sound pressure measurements p_i , equally spaced in time, the RMS sound pressure p_{rms} would be

$$p_{\text{rms}} = \sqrt{\frac{1}{n} \sum_{i=1}^n p_i^2}. \quad (14.1)$$

However, our ears take pressure measurements continuously, rather than at discrete points in time. If we imagine measuring the instantaneous sound pressure continuously, starting at time T_1 and lasting until time T_2 , the summation in Equation (14.1) would become an integral as follows:

$$p_{\text{rms}} = \sqrt{\frac{1}{T_2 - T_1} \int_{T_1}^{T_2} (p(t))^2 dt}. \quad (14.2)$$

However, the story doesn't end here. Perceived loudness is actually proportional to the *acoustic intensity* I , which is itself proportional to the *square* of the RMS sound pressure:

$$I \propto p_{\text{rms}}^2.$$

Humans can perceive a very wide range of sound pressure variations—from the flutter of a piece of paper falling to the ground to the boom of an aircraft breaking mach one. In order to manage such a wide dynamic range, we normally measure sound intensity in units of *decibels* (dB). The decibel is a *logarithmic* unit that expresses the *ratio* between two values. By employing a logarithmic scale, the decibel allows a wide range of measurements to be represented by a relatively narrow range of values. A decibel is actually one-tenth of a *bel*, a unit named in honor of Alexander Graham Bell.

When sound intensity is measured in decibels, it is called *sound pressure level* (SPL) and represented by the symbol L_p . Sound pressure level is defined as the ratio of the acoustic intensity (i.e., the squared pressure) of a sound relative to a reference intensity p_{ref} that represents the lower limit of human hearing. So we have:

$$\begin{aligned} L_p &= 10 \log_{10} \left(\frac{p_{\text{rms}}^2}{p_{\text{ref}}^2} \right) \text{ dB} \\ &= 20 \log_{10} \left(\frac{p_{\text{rms}}}{p_{\text{ref}}} \right) \text{ dB}, \end{aligned}$$

where the 20 arises because when we take the square outside the logarithm, it becomes a multiplication by two. The commonly used reference sound pressure in air is $p_{\text{ref}} = 20 \mu\text{Pa}$ (RMS). For more information on sound pressure, the physics of sound and human aural perception, see [6].

By the way, if you're feeling a bit rusty, the following identities may help to refresh your memory on logarithms. In Equations (14.3), b , x and y are positive real numbers with $b \neq 1$, c and d are any real numbers, $c = \log_b x$, and $d = \log_b y$ (or written another way, $b^c = x$ and $b^d = y$).

$\log_b x = c$	when	$b^c = x$	(definition);	
$\log_b 1 = 0$	because	$b^0 = 1$;		
$\log_b b = 1$	because	$b^1 = b$;		
$\log_b (x \cdot y) = \log_b x + \log_b y$	because	$b^c \cdot b^d = b^{c+d}$;		(14.3)
$\log_b (x/y) = \log_b x - \log_b y$	because	$b^c / b^d = b^{c-d}$;		
$\log_b x^d = d \log_b x$	because	$(b^c)^d = b^{cd}$.		

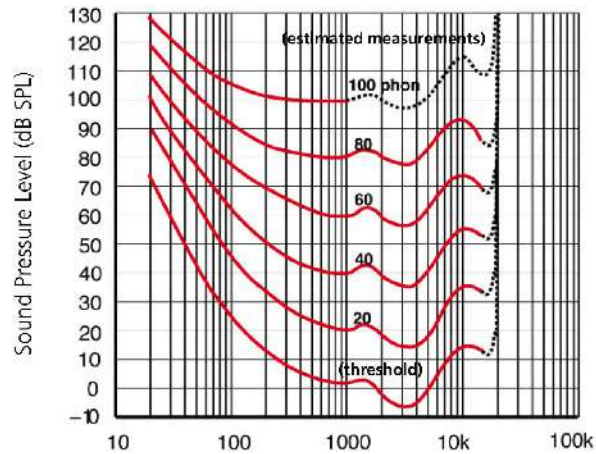


Figure 14.4. The human ear is most sensitive in the frequency range between 2 and 5 kHz. As the frequency decreases or increases beyond this range, more and more acoustic pressure is required to produce the same perception of “loudness.”

14.1.2.1 Equal-Loudness Contours

The human ear does not have the same response to sound waves of different frequencies. The human ear is most sensitive in the frequency range between 2 and 5 kHz. As the frequency decreases or increases beyond this range, more and more acoustic intensity (i.e., pressure) is required to produce the same perception of “loudness.”

Figure 14.4 shows a number of equal-loudness countours, each corresponding to a different perceived loudness level. These curves show that more pressure is required at low and high frequencies to achieve the same perceived loudness than is needed at mid-range frequencies. Or put another way, if we were to keep the amplitude of an acoustic pressure wave the same while varying the frequency, the human ear would actually perceive the lower and higher frequencies as “less loud” than the mid-range frequencies. The lowest equal-loudness contour represents the quietest audible tone and is also known as the absolute threshold of hearing. The highest contour passes through the human threshold of pain, which lies roughly at the 120 dB level for audible sounds.

For more information on equal-loudness contours, and the Fletcher-Munson curves on which they are based, see <https://bit.ly/2HfCjCs>.

14.1.2.2 The Audible Frequency Band

A typical adult can hear sounds with frequencies as low as 20 Hz and as high as 20,000 Hz (20 kHz) (although the upper limit generally decreases with age). The equal-loudness contours help to explain why the human ear can perceive sounds only within this limited “band” of frequencies. As the frequency becomes lower or higher, more and more acoustic pressure is required to produce the same perceived loudness. As the frequency approaches the lower or upper limits of human hearing, the contours become asymptotically vertical, meaning we’d need an effectively infinite acoustic pressure to produce any perception of loudness at all. Or put another way, human audio perception drops off to effectively zero outside the audible frequency band.

14.1.3 Sound Wave Propagation

Like any kind of wave, an acoustic pressure wave propagates through space and can be *absorbed* or *reflected* by surfaces, *diffracted* around corners and through narrow “slits,” and *refracted* as it passes across the boundary between different transmission media. Sound waves exhibit no *polarization*¹ because the acoustic pressure oscillation occurs in the direction of wave travel (this is known as a *longitudinal* wave), rather than perpendicular to it as with light waves (a *transverse* wave). In games, we typically model the absorption, reflection and sometimes the diffraction (e.g., bending slightly around corners) of our virtual sound waves, but we generally ignore refraction effects because these effects are not easily noticed by a human listener.

14.1.3.1 Fall-Off with Distance

In an open space with otherwise perfectly still air, and assuming a sound source that radiates equally in all directions, the intensity of the sound pressure wave it produces falls off with distance, following a $1/r^2$ law, while pressure follows a $1/r$ law.

$$p(r) \propto \frac{1}{r};$$

$$I(r) \propto \frac{1}{r^2}.$$

Here, r measures the radial distance of the listener or microphone from the sound source, and both pressure and intensity are expressed as functions of r .

¹Sound waves in solids can be transverse and therefore can exhibit polarization.

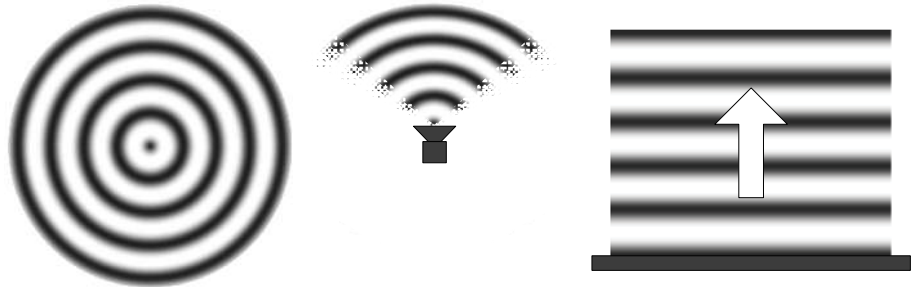


Figure 14.5. Three types of sound sources and their sound radiation patterns (in two dimensions for ease of illustration). From left to right: omnidirectional, conical and directional.

More precisely, the sound pressure level for a spherically radiating (omnidirectional) sound wave in open space can be written as follows:

$$\begin{aligned} L_p(r) &= L_p(0) + 10 \log_{10} \left(\frac{1}{4\pi r^2} \right) \text{ dB} \\ &= L_p(0) - 10 \log_{10} (4\pi r^2) \text{ dB}, \end{aligned}$$

where $L_p(r)$ is the SPL at the listener as a function of its radial distance from the sound source, and $L_p(0)$ represents the unattenuated or “natural” sound intensity of the source.

Sound sources are not always omnidirectional. For example, when a large, flat wall reflects sound waves, it acts like a purely *directional* sound source—the reflected waves propagate in a single direction, and the pressure wavefronts are essentially parallel.

A bullhorn projects sound in a particular direction but with a *conical* fall-off, meaning that the intensity of the sound waves is maximum along the centerline of the projection “cone,” but falls off as the angle between the listener and this centerline increases.

Various sound radiation patterns are illustrated in Figure 14.5.

14.1.3.2 Atmospheric Absorption

The $1/r$ fall-off of sound pressure with distance arises because energy is dissipated as the waveform expands geometrically. This fall-off affects sounds of all frequencies equally. Sound intensity also falls off with distance due to energy absorption by the atmosphere. Atmospheric absorption effects are *not* uniform across the entire frequency spectrum. In general, the absorption effect becomes greater as the frequency of the sound increases.

I’m reminded of a story I heard when I was in high school: A woman was walking down a quiet village street at night. She heard a sporadic sequence of

low tones with long, silent gaps between them. Curious what might be making these strange tones, she walked toward them. As she walked, the tones became louder and the spaces between the tones seemed to get shorter. After a few minutes' walk, the tones had resolved into a beautiful piece of music. The woman arrived at an open window to discover a viola player practicing within. The musician stopped playing to say "hello," and the woman asked him why he had been playing random notes a few minutes before. He replied, "I haven't been playing random notes—I've been playing this piece the whole time." The explanation for what the woman heard, of course, is that lower-frequency sounds can be heard over longer distances than higher-frequency sounds due to atmospheric absorption. You can learn more about sound wave propagation at http://www.sfu.ca/sonic-studio/handbook/Sound_Propagation.html.

Other factors also affect the intensity of sound waves as they propagate through their medium. In general, fall-off depends on distance, frequency, temperature and humidity. See <http://sengpielaudio.com/calculator-air.htm> for an online calculator that lets you experiment with the effects of these factors.

14.1.3.3 Phase Shift and Interference

When multiple sound waves overlap in space, their amplitudes add together—this is called *superposition*. Consider two periodic sound waves with the same frequency. (The simplest example would be two sinusoids.) If the waves are *in phase*—that is, their peaks and troughs line up—then the waves will *positively reinforce* each other, and the result is a wave with larger amplitude than either of the original waves. Likewise, if the waves are *out of phase*, the peaks of one wave can tend to *cancel* the troughs of the other and vice versa, and the result is a wave with lower (or even zero) amplitude.

When multiple waves interact, we call this *interference*. *Constructive interference* describes the case in which the waves reinforce one another and the amplitude increases. *Destructive interference* occurs when the waves cancel each other out, resulting in lower amplitude.

The frequency of the waves has an important effect on this phenomenon: If the frequencies of the two waves match closely, the interference simply increases or decreases the overall amplitude. If the frequencies differ significantly, we can get an effect called *beating*, wherein the frequency difference causes alternating periods of the waves being in and out of phase, resulting in alternating periods of higher and lower amplitude.

Interference can occur between two totally unrelated sound signals, or it can occur if a single sound signal takes multiple paths from the source to the

listener. In the latter case, the difference in path lengths introduces a phase shift that can cause either constructive or destructive interference, depending on the amount of the phase shift.

Comb Filtering

Interference can lead to an effect known as *comb filtering*. This is caused when sound waves reflect off surfaces in such a way as to either almost completely cancel or completely reinforce certain frequencies. The result is a *frequency response* (see Section 14.2.5.7) with lots of narrow peaks and troughs, which when plotted look a bit like a comb (hence the name). This effect can have a big impact on audio reproduction and recording—sometimes it is an undesirable artifact, and sometimes it is used as a tool. The existence of comb filtering is also one of the key reasons why it is generally better to spend money on acoustic room treatment than to spend money on high-end audio equipment: If the room exhibits comb effects, you’re wasting your time trying to get a flat response from your gear. See http://www.realtraps.com/video_comb.htm for a great video by Ethan Winer on the subject.

14.1.3.4 Reverb and Echo

In any environment containing sound-reflective surfaces, a listener generally receives three kinds of sound waves from a sound source:

- *Direct (dry)*. Sound waves that arrive at the listener via a direct, unobstructed path from the source are collectively known as *direct* or *dry* sound.
- *Early reflections (echo)*. Sound waves that arrive at the listener via an indirect path, after being reflected from and partially absorbed by surrounding surfaces, take a longer time to reach the listener because their path is longer. As such, there will be a *delay* between the arrival of the direct sound waves and the arrival of the reflected waves. The first group of reflected sound waves arriving at the ear have only interacted with one or two surfaces. As such, they are relatively “clean” signals, and we perceive them as distinct new “copies” of the sound or *echos*.
- *Late reverberations (tail)*. Once the sound waves have bounced around the listening space more than a few times, they superimpose and interfere with one another so much that the brain can no longer detect distinct echos. These are known as *late reverberations* or the *diffuse tail*. The properties of the reflective surfaces cause the amplitudes of the waves to be attenuated by varying amounts. And because the reflected sound waves

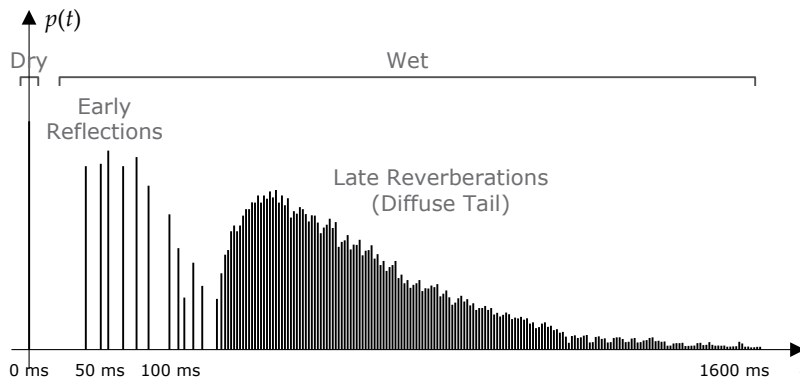


Figure 14.6. Direct sound waves, early reflections and late reverberations.

are delayed, phase shifts occur causing the waves to interfere with one another. This causes certain frequencies to be attenuated relative to the others. When we speak of the *acoustics* of a space, we are largely speaking about the effects of late reverberations on the perceived “quality” or “timbre” of the sound.

Collectively, the echos and the tail combine with the dry sound to create what is known as *wet* sound. Figure 14.6 illustrates the wet and dry components of a single abrupt clap.

The early reflections and late reverberations provide the brain with a wealth of cues that tell us quite a lot about the type of space we’re in. The *pre-delay* is the time interval between the arrival of the direct sound waves and the arrival of the very first reflected waves. From the pre-delay, the brain can determine the approximate size of the room or space in which we are listening. The *decay* is the time it takes for the reflected sound waves to die away. This tells our brains how much of the sound has been absorbed by the surroundings, and so indirectly tells us something about the materials that make up the space we’re in. For example, a small tiled bathroom would produce late reverberations with a very short pre-delay (due to its small size) and a long decay (due to the tile’s ability to reflect sound waves efficiently, with little absorption). A large granite-walled room like Grand Central Terminal (a.k.a. Grand Central Station) in New York City will have a much longer pre-delay and a lot more echos, but the decay will be similar to that of the tiled bathroom.

If we were to hang curtains in that bathroom, or if the walls were covered with wood panels instead of tile, the pre-delay would remain the same, but

the decay, along with other factors such as *density* (how closely spaced in time the individual reflections are) and *diffusion* (the rate at which the reflections increase in density over time), would change. This explains how a person can guess where they are even when blindfolded, or how the blind can learn to navigate with only a cane to aid them. Sound provides us with a *lot* of information about our surroundings!

The term *reverb* is used to describe the quality of a sound in terms of its wet components. In the early days of audio recording, sound engineers had little control over reverb, relying entirely on the shape and construction of the room in which the recording was made. Later, simple artificial reverb devices were created, from the use of a speaker and microphone in a bathroom by Bill Putman Sr. (founder of Universal Audio), to the use of a long metal plate or spring to introduce a delay in a sound signal, to modern digital techniques. Today, digital signal processor (DSP) chips and/or software are used not only to recreate natural reverb effects in recorded sound effects and music, but also to augment recordings with all sorts of interesting effects that are not normally heard in nature. We'll learn more about digital signal processing in Section 14.2. You can read more about reverb at <http://www.uaudio.com/blog/the-basics-of-reverb>.

An *anechoic chamber* is a room especially designed to entirely eliminate reflected sound waves. This is accomplished by lining the walls, floor and ceiling of the room with thick corrugated foam padding that absorbs essentially all of the reflected sound waves. As a result, only the direct (dry) sound reaches the listener or microphone. Sound in an anechoic chamber has a completely "dead" timbre. Anechoic chambers are useful for recording "pure" sounds that contain no reverb. Such pure sounds are often perfect candidates for input into a digital signal processing pipeline, giving a sound designer maximum flexibility to control the timbre of the sound.

14.1.3.5 Sound in Motion: The Doppler Effect

If you've ever stood at a railway crossing when a train goes by, you've heard the *Doppler effect* in action. The sound of the train seems higher pitched when it is approaching you, and becomes lower pitched as it races off into the distance. Sound waves travel at a roughly constant speed through the air, but the sound source (in this case, the train) is also moving. The sound waves that are moving in the same direction as the train become "squashed together," and the waves that are moving opposite to the motion of the train become "spread out," each by an amount proportional to the difference between the speed of sound in air and the speed of the train through the air. The frequency of the squashed waves is therefore increased, because the space between the peaks and troughs

of the sound waves has been effectively reduced, resulting in a higher-pitched sound. Likewise, the frequency of the spread-out waves is decreased, resulting in a lower-pitched sound. The Doppler effect was named after the Austrian physicist Christian Doppler, who identified it in 1842.

The Doppler effect also occurs when the listener is moving but the sound source is stationary. In general, the Doppler shift is dependent upon the *relative velocity* (as a vector) between the listener and the sound source. In one dimension, the Doppler shift amounts to a change in frequency, and can be quantified as follows:

$$f' = \left(\frac{c + v_l}{c + v_s} \right) f,$$

where f is the original frequency, f' is the Doppler-shifted (observed) frequency at the listener, c is the speed of sound in air and v_l and v_s are the speeds of the listener and sound source, respectively. If the speeds of the sound sources are very small relative to the speed of sound, we can approximate this relationship as follows:

$$\begin{aligned} f' &= \left(\frac{1 + (v_l - v_s)}{c} \right) f \\ &= \left(\frac{1 + \Delta v}{c} \right) f. \end{aligned}$$

This expression makes the relative velocity, Δv , apparent. The Doppler effect can be easily visualized by looking at the following animated GIF: <http://en.wikipedia.org/wiki/File:Dopplereffectsourcemovingrightatmach0.7.gif>.

14.1.4 Perception of Position

The human auditory system has evolved to allow a reasonably accurate perception of the position of sounds in the space around us. A number of factors contribute to our perception of sound position:

- *Fall-off with distance* provides us with a rough idea of how far away the source of a sound is. In order for this to work, we must have some idea of the loudness of the sound when heard at close range to serve as a “baseline.”
- *Atmospheric absorption* causes the higher frequencies in a sound to drop out as the source moves farther away from the listener. This can serve as an important cue in perceiving the difference between, for example, a person speaking at normal volume but far away and a person speaking at reduced volume up close.

- *Having two ears*, one on the left and one on the right, gives us a great deal of positional information. A sound that is to our right will sound louder in the right ear than in the left. An *interaural time difference* (ITD) of approximately one millisecond also arises, because a sound to one side of your head will take just a little bit longer to reach the opposite ear. Finally, the head itself obstructs sounds, so the ear opposite to the sound source will perceive a slightly muffled version of the sound reaching the near ear. This is known as interaural intensity difference (IID).
- *Ear shape* has an effect as well. Our ears are cupped slightly forward, so sounds coming from behind us are very slightly muffled relative to those coming from in front of us.
- *The head-related transfer function* (HRTF) is a mathematical model of the minute effects that the folds of our ears (the *pinnae*) have on sounds coming from different directions.

14.2 The Mathematics of Sound

Signal processing and systems theory is the area of mathematics that lies at the heart of virtually all modern audio technology. It is also extensively used in a wide variety of other technological and engineering endeavors, including image processing and machine vision, aeronautics, electronics, fluid dynamics, and the list goes on. In this section, we'll embark on a whirlwind tour of the key concepts in signals and systems theory, because it will help us to understand some of the more advanced topics in game audio later in the chapter. (It's also an important area of mathematical theory that can benefit any game programmer—so what the heck!) An in-depth treatment of the topic can be obtained from [41].

14.2.1 Signals

A *signal* is any function of one or more independent variables, typically describing the behavior of some kind of physical phenomenon. In Section 14.1, we used the signal $p(t)$ to represent the time-varying acoustic pressure of an audio compression wave. Of course, many other kinds of signals are possible. A signal $v(t)$ might represent the voltage produced by a microphone over time, while $w(t)$ might model the time-varying water pressure in a system of pipes, or we could use $f(t)$ to represent the varying population of foxes in an ecosystem.

In studying signal theory, we often refer to the independent variable as “time” and represent it with the symbol t —but of course the independent vari-

able might represent some other quantity, and there may be more than one independent variable. For example, one can think of a 2D greyscale image as a signal $i(x, y)$, where the two independent variables, x and y , represent the orthogonal coordinate axes, and the signal value i represents the intensity of the greyscale image at each pixel. A color image could be similarly represented by three signals, $r(x, y)$ for the red channel, $g(x, y)$ for the green channel and $b(x, y)$ for the blue channel.

14.2.1.1 Be Discrete, Continuously

The 2D image examples above bring to light an important distinction between two fundamental kinds of signal: *continuous* and *discrete*.

- If the independent variable is a *real number* ($t \in \mathbb{R}$), we call the signal a *continuous-time signal*. In this chapter, we'll use the symbol t to represent continuous "time," and we'll use round parentheses for our functional notation (e.g., $x(t)$) to remind us that we're working with a continuous-time signal.
- If the independent variable(s) is an *integer* ($n \in \mathbb{I}$), we call the signal a *discrete-time signal*. We'll use the symbol n to represent discrete "time," and we'll use square brackets for our functional notation (e.g., $x[n]$) to remind us that we're working with a discrete-time signal. Note that the *value* of a discrete-time signal might still be a real number ($x[n] \in \mathbb{R}$)—the only thing the term "discrete-time signal" prescribes is that the *independent* variable is an integer ($n \in \mathbb{I}$).

In Figure 14.1, we saw that we can visualize a continuous-time signal as an ordinary function plot, with time t on the horizontal axis and the signal value $p(t)$ on the vertical axis. We can plot a discrete-time signal $x[n]$ in similar fashion, although the function's values are only defined for integer values of the independent variable n (see Figure 14.7). One common way to think of a discrete-time signal is as a *sampled* version of a continuous-time signal. The sampling process (also known as *digitization* or *analog-to-digital conversion*) lies at the heart of digital audio recording and playback. See Section 14.3.2.1 for more information on sampling.

14.2.2 Manipulating Signals

It will be important in the following discussions for us to understand some basic ways to manipulate a signal by making changes to its independent variable. For example, to *reflect* a signal about $t = 0$, we simply replace t with $-t$ in the signal's equation. To *time-shift* the entire signal to the *right* (i.e., in the

positive direction) by a distance s , we replace t with $t - s$ in the signal's equation. (Time shifting to the left/negative direction is accomplished by replacing t with $t + s$.) We can also expand or compress the domain of the signal by scaling the independent variable. These simple transformations are illustrated in Figure 14.8.

14.2.3 Linear Time-Invariant (LTI) Systems

In the context of signal processing theory, a *system* is defined as any device or process that transforms an *input* signal into a new *output* signal. The mathematical concept of a system can be used to describe, analyze and manipulate many real-world systems that arise in audio processing, including microphones, speakers, analog-to-digital converters, reverb units, equalizers and filters and even the acoustics of a room.

As a simple example, an *amplifier* is a system that increases the amplitude of its input signal by a factor A known as the *gain* of the amp. Given an input signal $x(t)$, such an amplification system would produce an output signal $y(t) = Ax(t)$.

A *time-invariant* system is one for which a time shift in the input signal causes an equal time shift in the output signal. In other words, the behavior of the system does not change over time.

A *linear* system is one that possesses the property of *superposition*. This means that if an input signal consists of a *weighted sum* of other signals, then the output is a weighted sum of the individual outputs that would have been produced, had each of the other signals been fed through the system independently.

Linear time-invariant (LTI) systems are extremely useful for two reasons.

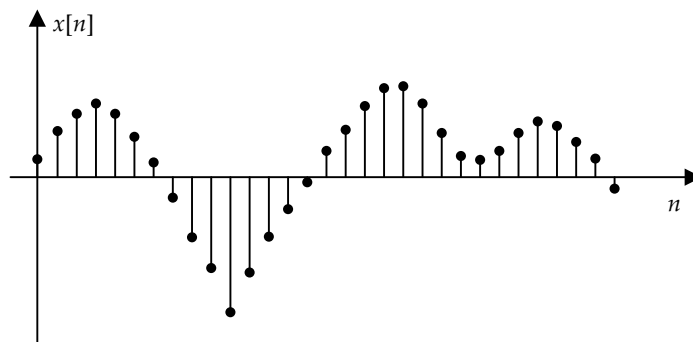


Figure 14.7. The value of a discrete-time signal $x[n]$ is defined only for integer values of n .

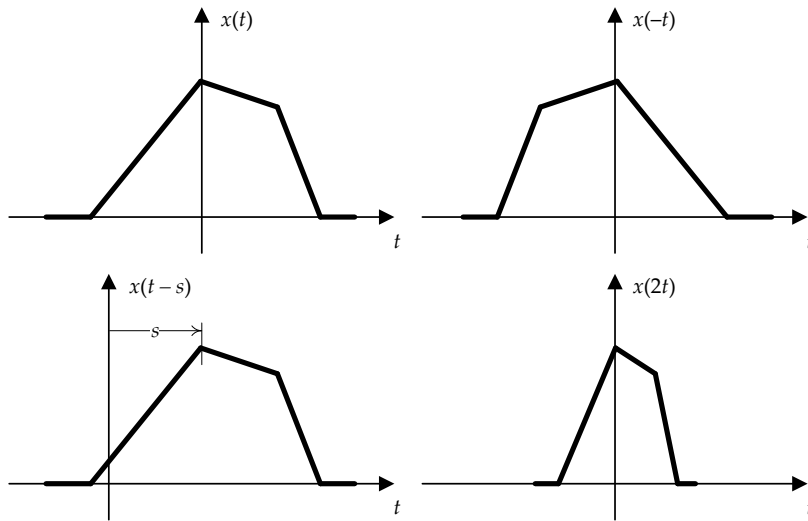


Figure 14.8. Simple manipulations of a signal's independent variable.

First, their behaviors are well understood and relatively easy to work with mathematically. Second, many real physical systems in the fields of audio propagation theory, electronics, mechanics, fluid flow, etc. can be modeled accurately using LTI systems. As such, we will restrict ourselves to a discussion of LTI systems for our purposes of understanding audio technology.

We can visualize any system as a black box with an input signal and an output signal, as shown in Figure 14.9.

Using this black-box notation, simple systems can be conveniently interconnected to construct more complex systems. For example:

- The output of system A could be connected to the input of system B, yielding a composite system that performs operation A followed by operation B. This is called a *serial connection*.
- The outputs of two systems could be added together.
- The output of a system could be *fed back* into an earlier input, yielding what is known as a *feedback loop*.

See Figure 14.10 for examples of all of these kinds of connections.

One very important property of all LTI systems is that their interconnections are *order-independent*. So if we have a serial connection of system A followed by system B, we can reverse the order of the two systems and the output will remain unchanged.

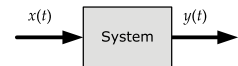


Figure 14.9. A system as a black box.

14.2.4 Impulse Response of an LTI System

It's all fine and dandy to talk about systems that convert an input signal into an output signal, and it's even pretty intuitive to draw diagrams of system interconnections. But how can we describe the operation of a system mathematically?

Recall from Section 14.2.3 that for a *linear* system, if the input consists of a linear combination (weighted sum) of input signals, the output will be a linear combination (weighted sum) of the individual outputs (had each of the input signals been fed into the system independently). So, if we can figure out a way to represent an arbitrary input signal as a weighted sum of very simple signals, we should be able to describe the behavior of the system by describing *only* its response to those very simple signals.

14.2.4.1 The Unit Impulse

If we are going to describe an input signal as a linear combination of simple signals, the question arises: Which simple signal shall we use? For reasons that will become clear in a moment, our signal of choice is going to be the *unit impulse*. This signal is one of a family of related functions known as *singularity functions* because they all contain at least one discontinuity or “singularity.”

In discrete time, the unit impulse $\delta[n]$ is as simple as it gets: It is a signal whose value is zero everywhere except at $n = 0$, where its value is one:

$$\delta[n] = \begin{cases} 1 & \text{if } n = 0, \\ 0 & \text{otherwise.} \end{cases}$$

The discrete-time unit impulse is illustrated in Figure 14.11.

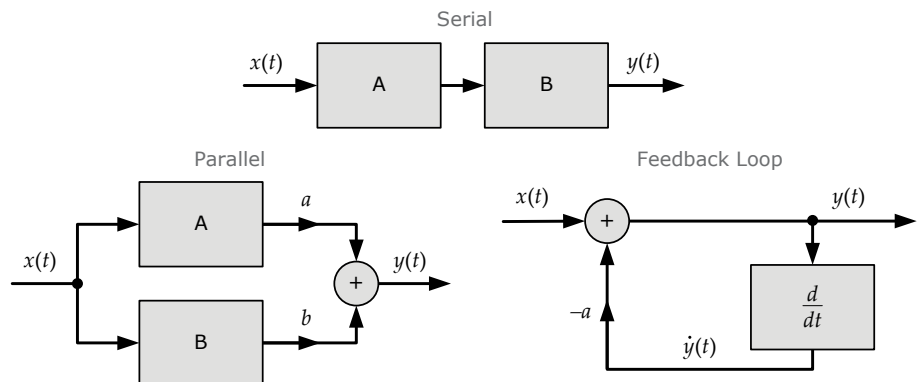


Figure 14.10. Various ways to interconnect systems. In the serial connection, $y(t) = B(A(x(t)))$. In the parallel connection, $y(t) = aA(x(t)) + bB(x(t))$. In the feedback loop, $y(t) = x(t) - a\dot{y}(t)$.

In continuous time, the unit impulse $\delta(t)$ is a bit trickier to define. It is a function whose value is zero everywhere except at $t = 0$, where its value is infinite—but the *area* under the curve is equal to one.

To see how such a strange beast of a function might be formally defined, imagine a “box” function $b(t)$, whose value is zero everywhere except in the interval $[0, T)$, where its value is $1/T$. The area under this curve is just the area of the box, width times height, or $T \times \frac{1}{T} = 1$. Now imagine the limit as $T \rightarrow 0$. As this happens, the width of the box approaches zero and its height approaches infinity, but its area remains equal to 1. This is shown in Figure 14.12.

The unit impulse function is typically denoted by the symbol $\delta(t)$. It can be formally defined as follows:

$$\delta(t) = \lim_{T \rightarrow 0} b(t),$$

where

$$b(t) = \begin{cases} 1/T & \text{if } t \geq 0 \text{ and } t < T, \\ 0 & \text{otherwise.} \end{cases}$$

As shown in Figure 14.13, we typically plot the unit impulse by drawing an arrow whose height represents the area under the curve (since the actual “height” of the function at $t = 0$ is infinite).

14.2.4.2 Using an Impulse Train to Represent a Signal

Now that we know what the unit impulse signal is, let’s see if we can describe an arbitrary signal $x[n]$ as a linear combination of unit impulses. (Spoiler alert: It turns out we can.)

The function $\delta[n - k]$ is a time-shifted discrete unit impulse, whose value is zero everywhere except at time $n = k$, where it is equal to one. In other words, the unit impulse $\delta[n - k]$ is “positioned” at time k . Consider an impulse at one particular value of k (say, $k = 3$). Let’s make sure that the “height” of that impulse matches the value of the original function at $k = 3$ by “scaling” the impulse by $x[3]$, yielding $x[3]\delta[n - 3]$. If we rinse and repeat for all possible

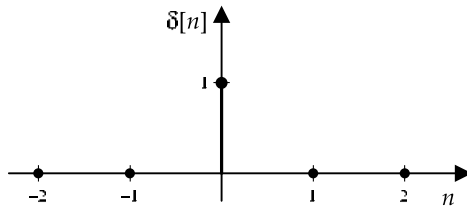


Figure 14.11. The unit impulse in discrete time.

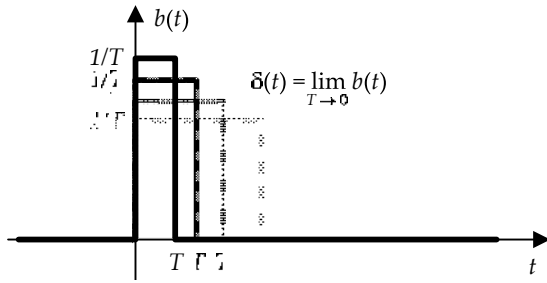


Figure 14.12. The unit impulse can be defined as the limit of a box function $b(t)$ whose width approaches zero.

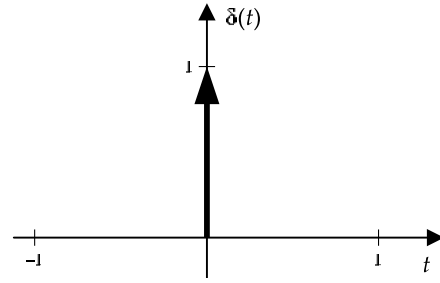


Figure 14.13. The value of the unit impulse function $\delta(t)$ is zero everywhere except at $t = 0$, where it is infinite. It is drawn as an arrow of unit height to indicate that the area under the curve is 1.

values of k , we get a train of impulses of the form $x[k]\delta[n - k]$. Adding all these scaled, time-shifted impulse functions together is just another way of writing the original signal $x[n]$:

$$x[n] = \sum_{k=-\infty}^{+\infty} x[k]\delta[n - k]. \quad (14.4)$$

We won't give a rigorous proof here, but it's probably not too hard to believe that doing this in continuous time works in pretty much the same way. The only difficulty is that for continuous time, the sum in Equation (14.4) becomes an integral. Let's imagine an infinite sequence of time-shifted unit impulses $\delta(t - \tau)$, each one located at a different time τ . We can build up an arbitrary signal $x(t)$ in an analogous fashion to the discrete-time case, as follows:

$$x(t) = \int_{\tau=-\infty}^{+\infty} x(\tau)\delta(t - \tau) d\tau. \quad (14.5)$$

14.2.4.3 Convolution

Equation (14.4) tells us how to represent a signal $x[n]$ as a *linear combination* of simple, time-shifted unit impulse signals $\delta[n - k]$. Let's imagine putting just *one* of these weighted impulse inputs ($x[k]\delta[n - k]$) through the system. It doesn't matter which one we choose, so let's select the one at $k = 0$. This gives us the input signal $x[0]\delta[n]$.

We'll use the notation $x[n] \Rightarrow y[n]$ to indicate that an input signal $x[n]$ is being transformed by an LTI system into an output signal $y[n]$. So we can write:

$$x[0]\delta[n] \Rightarrow y[n].$$

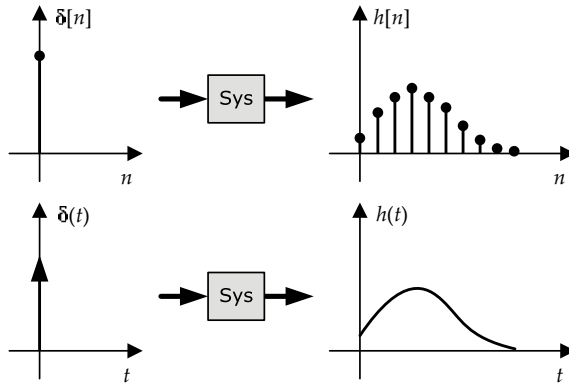


Figure 14.14. Examples of the impulse response of a system in discrete and continuous time.

The value of $x[0]$ is just a constant, so because we're dealing with a linear system, the output $y[n]$ will just be that same constant times the system's response to the unit impulse $\delta[n]$. Let's use the signal $h[n]$ to represent the system's response to a "bare" unit impulse: $\delta[n] \Rightarrow h[n]$. The signal $h[n]$ is called the *impulse response* of the system. So we can write the system's response to our simple input signal as follows:

$$x[0]\delta[n] \Rightarrow x[0]h[n].$$

The concept of impulse response is illustrated in Figure 14.14.

The response of an LTI system to a time-shifted unit impulse is just a time-shifted impulse response ($\delta[n - k] \Rightarrow h[n - k]$). So for values of k other than zero, everything works out exactly the same except that now the input and output signals are both *time-shifted* by k :

$$x[k]\delta[n - k] \Rightarrow x[k]h[n - k].$$

To find the system's response to the entire input signal $x[n]$, we just sum up the responses to each individual time-shifted component, like this:

$$\sum_{k=-\infty}^{+\infty} x[k] \delta[n - k] \Rightarrow \sum_{k=-\infty}^{+\infty} x[k] h[n - k].$$

In other words, the output of our system can be written as follows:

$$y[n] = \sum_{k=-\infty}^{+\infty} x[k] h[n - k]. \quad (14.6)$$

This very important equation is known as the *convolution sum*. It's convenient to introduce a new mathematical operator $*$ to represent the operation

of convolution:

$$x[n] * h[n] = \sum_{k=-\infty}^{+\infty} x[k] h[n - k]. \quad (14.7)$$

Equations (14.6) and (14.7) give us a way to calculate an LTI system's response $y[n]$ to any arbitrary input signal $x[n]$, given only the *impulse response* of the system, $h[n]$. In other words, for LTI systems, the impulse response signal $h[n]$ *completely describes the system*. Pretty cool stuff.

Convolution in Continuous Time

In our discussions above, we worked in discrete time to keep things simple. In continuous time, everything works out in pretty much the same way. The only difference is that summations become integrals, and we need to remember to include the differential $d\tau$ in our equations.

When we apply an arbitrary signal $x(t)$ to the input of a continuous-time LTI system, the output signal can be written as follows:

$$y(t) = \int_{\tau=-\infty}^{+\infty} x(\tau) h(t - \tau) d\tau. \quad (14.8)$$

As before, we'll use the operator $*$ as a shorthand for convolution:

$$x(t) * h(t) = \int_{\tau=-\infty}^{+\infty} x(\tau) h(t - \tau) d\tau. \quad (14.9)$$

Analogous to the convolution sum, the integral in Equations (14.8) and (14.9) is known as the *convolution integral*.

14.2.4.4 Visualizing Convolution

Let's try to visualize the convolution operation in the continuous time case. To evaluate $y(t) = x(t) * h(t)$ for one specific value of t (say, $t = 4$), we perform the following steps as illustrated in Figure 14.15:

1. Plot $x(\tau)$, using τ as the time variable because t is fixed (at $t = 4$ in this example).
2. Plot $h(t - \tau)$. We can rewrite this as $h(-\tau + t)$. Because τ is negated, we know that the impulse response has been flipped about $\tau = 0$. And because we've added t to the independent variable, we know the signal has been shifted to the *left* by $t = 4$ units.
3. Multiply the two signals together across the entire τ axis.

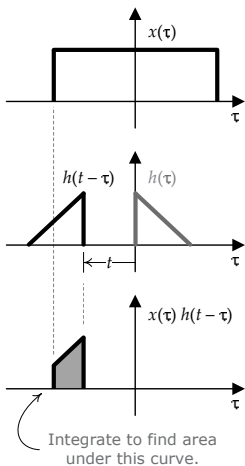


Figure 14.15. Visualization of the convolution operation in continuous time.

4. Integrate from $-\infty$ to $+\infty$ along the τ axis to find the area under the resulting curve. This is the value of $y(t)$ at this one specific value of t (in this example, $t = 4$).

Remember that we must repeat this procedure for every possible value of t in order to determine the complete output signal $y(t)$.

14.2.4.5 Some Properties of Convolution

The properties of the convolution operation are surprisingly analogous to those of ordinary multiplication. Convolution is:

- *commutative*: $x(t) * h(t) = h(t) * x(t)$;
- *associative*: $x(t) * (h_1(t) * h_2(t)) = (x(t) * h_1(t)) * h_2(t)$; and
- *distributive*: $x(t) * (h_1(t) + h_2(t)) = (x(t) * h_1(t)) + (x(t) * h_2(t))$.

14.2.5 The Frequency Domain and the Fourier Transform

In order to arrive at the concepts of impulse response and convolution, we described a signal as a weighted sum of unit impulses. We can also represent a signal as a weighted sum of sinusoids. Representing a signal in this manner essentially breaks it up into its *frequency components*. This will allow us to derive another incredibly powerful mathematical tool—the *Fourier transform*.

14.2.5.1 The Sinusoidal Signal

A sinusoidal signal is produced when a circular motion in two dimensions is projected onto a single axis. An audio signal in the form of a sinusoid produces a “pure” tone at one specific frequency.

The most basic sinusoidal signal is the sine (or cosine) function. The signal $x(t) = \sin t$ takes on the value 0 at $t = 0, \pi$ and 2π , has a value of 1 at $t = \frac{\pi}{2}$ and has a value of -1 at $t = \frac{3\pi}{2}$.

The most general form of a real-valued sinusoidal signal is

$$x(t) = A \cos(\omega_0 t + \phi). \quad (14.10)$$

Here, A represents the *amplitude* of the sine wave (i.e., the peaks and troughs of the cosine wave hit maximum and minimum values of A and $-A$, respectively). The *angular frequency* is ω_0 , measured in radians/second (see Section 14.1.1 for a discussion of frequency and angular frequency). ϕ represents a *phase offset* (also measured in radians) that shifts the cosine wave to the left or right along the time axis.

When $A = 1$, $\omega_0 = 1$ and $\phi = 0$, Equation (14.10) reduces to $x(t) = \cos t$. When $\phi = \frac{\pi}{2}$, the expression becomes $x(t) = \sin t$. The cos function represents the projection of a circular motion onto the horizontal axis, while sin represents its projection onto the vertical axis.

14.2.5.2 The Complex Exponential Signal

The cosine function isn't actually the best tool for representing a signal as a sum of sinusoids. The math is much simpler and more elegant if we make use of *complex numbers* instead. In order to understand how this works, we need to review complex math, and take a look at how multiplication of complex numbers works. So bear with me here—all will become clear by the time we're done.

A Brief Review of Complex Numbers

You'll probably remember from high-school math class that a complex number is a kind of two-dimensional quantity consisting of a real part and an imaginary part. Any complex number can be written as follows: $c = a + jb$, where a and b are real numbers and $j = \sqrt{-1}$ is the *imaginary unit*. The real part of c is $a = \Re(c)$, and its imaginary part is $b = \Im(c)$.

You can visualize a complex number as a kind of “vector” $[a, b]$ in a two-dimensional space known as the *Argand plane*. It's important to remember, however, that complex numbers and vectors are *not* interchangeable—their mathematical behaviors are quite different.

We define the *magnitude* of a complex number as the length of its 2D “vector” representation in the complex plane: $|c| = \sqrt{a^2 + b^2}$. The angle the vector makes with the real axis is known as its *argument*: $\arg c = \tan^{-1}(b/a)$. (The argument of a complex number is sometimes called its *phase*. As we'll see, the term “phase” is closely related to the *phase offset* ϕ in Equation (14.10).) The magnitude and argument of a complex number are depicted in Figure 14.16.

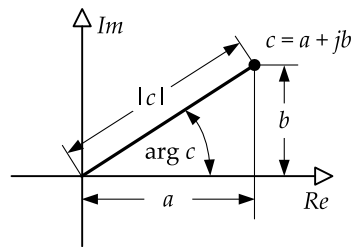


Figure 14.16. The magnitude $|c| = \sqrt{a^2 + b^2}$ of a complex number is its length in the complex plane, and its argument $\arg c = \tan^{-1}(b/a)$ is the angle it makes with the $\Re$ axis.

Complex Multiplication and Rotation

We won't get into all of the properties of complex numbers here. Check out <http://www.math.wisc.edu/~angenent/Free-Lecture-Notes/freecomplexnumbers.pdf> for an in-depth discussion of complex number theory. However, there is one mathematical operation that does concern us here: the operation of complex *multiplication*.

Complex numbers are multiplied algebraically (no dot or cross products here):

$$\begin{aligned} c_1 c_2 &= (a_1 + j b_1)(a_2 + j b_2) \\ &= (a_1 a_2) + j(a_1 b_2 + a_2 b_1) + j^2 b_1 b_2 \\ &= (a_1 a_2 - b_1 b_2) + j(a_1 b_2 + a_2 b_1). \end{aligned} \quad (14.11)$$

If you work out² the magnitude and argument (angle) of the product $c_1 c_2$, you'll find that the magnitude is equal to the *product* of the two input magnitudes, and the argument is the *sum* of the input arguments:

$$\begin{aligned} |c_1 c_2| &= |c_1| |c_2|; \\ \arg(c_1 c_2) &= \arg c_1 + \arg c_2. \end{aligned} \quad (14.12)$$

The fact that multiplication of complex numbers causes their angles (arguments) to *add* means that complex multiplication produces a *rotation* in the complex plane. If the magnitude of c_1 is unity ($|c_1| = 1$), then the magnitude of the product will be equal to the magnitude of c_2 ($|c_1 c_2| = |c_2|$). In this case, the product represents a *pure rotation* of c_2 by an angle equal to $\arg c_1$ (see Figure 14.17). If $|c_1| \neq 1$, then the product's magnitude will be *scaled* by $|c_1|$, and the result is that c_2 undergoes a *spiral motion* in the complex plane.

This explains why *unit-length* quaternions operate as rotations in 3D space! A quaternion is essentially a four-dimensional complex number, with one real part and three imaginary parts. So a quaternion follows the same basic rules in three dimensions that a regular complex number follows in two dimensions.

The fact that complex multiplication produces a rotation makes sense when we consider what happens when we multiply j by itself many times:

$$\begin{aligned} 1 \times j &= j, \\ j \times j &= \sqrt{-1} \sqrt{-1} = -1, \\ -1 \times j &= -j, \\ -j \times j &= 1, \\ &\dots \end{aligned}$$

²Yikes—this sounds an awful lot like an exercise for the reader...

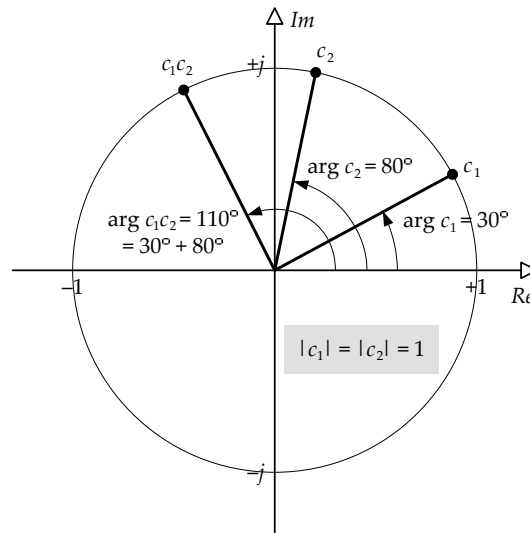


Figure 14.17. Multiplying two complex numbers together that both have magnitudes of 1 produces a *pure rotation* in the complex plane.

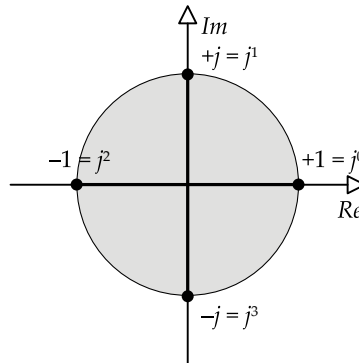


Figure 14.18. Multiplying the imaginary number $j = \sqrt{-1}$ by itself acts like rotating a unit vector by 90 degrees in the complex plane.

So multiplying j by itself is like *rotating* a unit vector by 90 degrees in the complex plane. In fact, multiplying *any* complex number by j has the effect of rotating it by 90 degrees. This is illustrated in Figure 14.18.

The Complex Exponential and Euler's Formula

For any complex number c with $|c| = 1$, the function $f(n) = c^n$, with n taking on a sequence of increasing positive real values, will trace out a *circular path* in

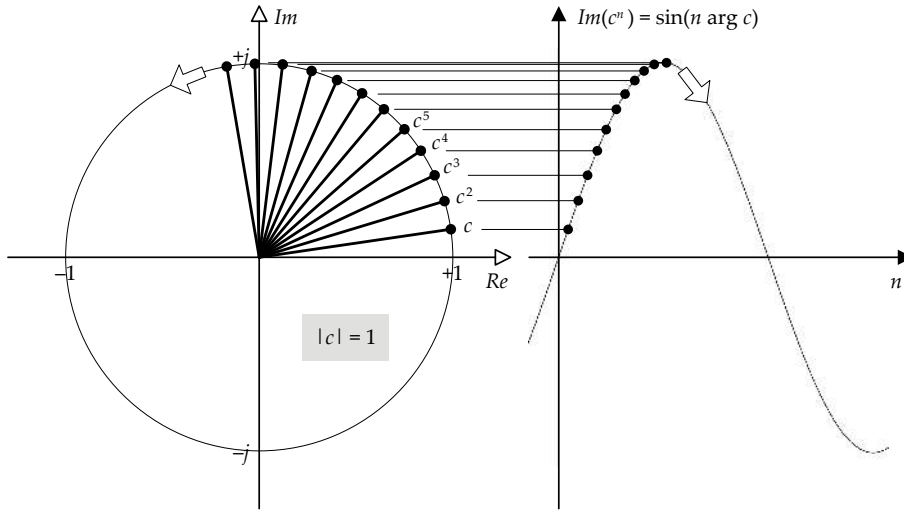


Figure 14.19. Multiplying a complex number by itself repeatedly traces out a circular path in the Argand plane, producing a sinusoid when projected onto any axis through the origin.

the complex plane. Any circular path in two dimensions traces out a sine curve along the vertical axis, and a corresponding cosine curve along the horizontal axis. This is illustrated in Figure 14.19.

Raising a *complex* number to a *real* power (c^n) produces rotation in the complex plane, and therefore yields a sinusoid when projected onto any axis in the plane. As it turns out, we can also get this rotational effect by raising a *real* number to a *complex* power (n^c). This means that we can write Equation (14.10) in terms of complex numbers as follows:

$$\begin{aligned} e^{j\omega_0 t} &= \cos \omega_0 t + j \sin \omega_0 t, \quad t \in \mathbb{R}; \\ \Re[e^{j\omega_0 t}] &= \cos \omega_0 t; \\ \Im[e^{j\omega_0 t}] &= \sin \omega_0 t, \end{aligned} \quad (14.13)$$

where $e \approx 2.71828$ is the real transcendental number that defines the base of the natural logarithm function.

Equation (14.13) is one of the most important equations in all of mathematics. It is known as *Euler's formula*. Why it works is a bit of a mystery (even to some seasoned mathematicians). The theorem can be explained by looking at the Taylor series expansion of e^{jt} , or by considering the derivative of e^x and then allowing x to become a complex number. But for our purposes, it should suffice to rely on the intuitions we gained from looking at how complex multiplication results in rotation in the complex plane.

14.2.5.3 The Fourier Series

Now that we have the mathematical tools we need to represent sinusoids as complex numbers, let's turn our attention again to the task of representing a signal as a sum of sinusoids.

Doing this is easiest when the signal is *periodic*. In this case, we can write the signal as a sum of *harmonically related* sinusoids:

$$x(t) = \sum_{k=-\infty}^{+\infty} a_k e^{j(k\omega_0)t}. \quad (14.14)$$

We call this the *Fourier series* representation of the signal. Here, the complex exponential functions $e^{j(k\omega_0)t}$ are the sinusoidal components from which we are building up the signal. These components are *harmonically related*, in that each one has a frequency that is an integer multiple k of the so-called *fundamental frequency* ω_0 . The coefficients a_k represent the “amount” of each harmonic present in the signal $x(t)$.

14.2.5.4 The Fourier Transform

A full explanation of this topic is beyond the scope of this book, but for our purposes it will suffice to state (without any proof whatsoever!) that *any* reasonably well-behaved signal,³ even signals that are *non-periodic*, can be represented as a linear combination of sinusoids. In general, an arbitrary signal may contain components at *any* frequency, not just frequencies that are harmonically related. As such, the discrete set of harmonic coefficients a_k in Equation (14.14) becomes a continuum of values representing “how much” of each frequency the signal contains.

We can envision a new function $X(\omega)$ whose independent variable is the frequency ω rather than time t , and whose value represents the amount of each frequency present in the original signal $x(t)$. We say that $x(t)$ is the *time domain* representation of the signal, while $X(\omega)$ is its *frequency domain* representation.

Mathematically, we can find the frequency domain representation of a signal from its time domain representation, and vice versa, by using the *Fourier transform*:

$$X(\omega) = \int_{-\infty}^{+\infty} x(t) e^{-j\omega t} dt; \quad (14.15)$$

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} X(\omega) e^{j\omega t} d\omega. \quad (14.16)$$

³All signals that meet the so-called *Dirichlet conditions* have Fourier transforms and are therefore “reasonably well-behaved” for our purposes.

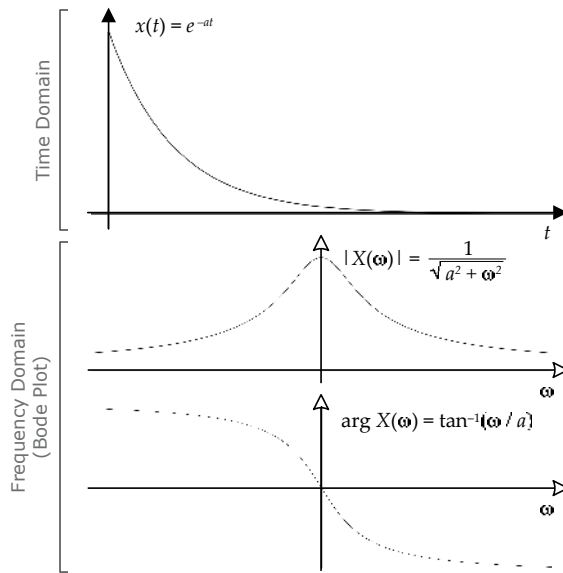


Figure 14.20. The Fourier transform yields a complex-valued frequency domain signal. A Bode plot is used to visualize this complex-valued signal in terms of its magnitude and its phase (or argument).

If you compare Equation (14.16) to the Fourier series from Equation (14.14), you can see the similarity. Rather than describing the “amounts” of the frequency components via a discrete series of coefficients a_k , we’re now describing them using the continuous function $X(\omega)$. But in both cases we’re representing $x(t)$ as a “sum” of sinusoids.

14.2.5.5 Bode Plots

In general the Fourier transform of a real-valued signal is a *complex-valued* signal ($X(\omega) \in \mathbb{C}$). When visualizing the Fourier transform, we often draw it using two plots. For example, we might plot its real and imaginary components. Or we might plot its magnitude and its argument (angle) on two different plots—a visualization known as a *Bode plot* (pronounced “Boh-dee”). Figure 14.20 shows an example of a signal and its Bode plot.

14.2.5.6 The Fast Fourier Transform (FFT)

A collection of fast algorithms exist for calculating the Fourier transform in discrete time. This family of algorithms is called, aptly enough, the *fast Fourier transform* or FFT. You can read more about the FFT at http://en.wikipedia.org/wiki/Fast_Fourier_transform.

14.2.5.7 Fourier Transforms and Convolution

It is interesting to note that convolution in the time domain corresponds to multiplication in the frequency domain and vice versa. Given a system whose impulse response is $h(t)$, we know that we can find the output of the system $y(t)$ in response to an input $x(t)$ as follows:

$$y(t) = x(t) * h(t).$$

In the frequency domain, given the Fourier transforms of the impulse response $H(\omega)$ and the input $X(\omega)$, we can find the Fourier transform of the output as follows:

$$Y(\omega) = X(\omega)H(\omega).$$

This result is pretty incredible, and it's also very handy. Sometimes it is more convenient to perform a convolution on the time axis using a system's impulse response $h(t)$, while at other times it's more convenient to perform a multiplication in the frequency domain using the system's *frequency response* $H(\omega)$.

As it turns out, LTI systems exhibit a property called *duality*, which says that you can reverse the roles of time and frequency and virtually the same mathematical rules continue to apply. So, for example, we can understand how signal *modulation* (the multiplication of one signal by another) works in the time domain by looking at what happens when we convolve the Fourier transforms of the two signals on the frequency axis. Having two ways of tackling a problem is always better than one!

14.2.5.8 Filtering

The Fourier transform allows us to visualize the set of frequencies that make up virtually any audio signal. A *filter* is an LTI system that attenuates a selected range of input frequencies while leaving all other frequencies unaltered. A *low-pass* filter retains low frequencies while attenuating high frequencies. A *high-pass* filter does the opposite, retaining high frequencies and attenuating lower frequencies. A *band-pass* filter attenuates both low and high frequencies but retains frequencies within a limited *passband*. A *notch* filter does the opposite, retaining low and high frequencies but attenuating frequencies within a limited *stopband*.

Filters are used in a stereo system's *equalizer*, by attenuating or boosting specific frequencies based on user inputs. Filters can also be used to attenuate noise, if the spectra of the noise signal and the desirable signal occupy different regions of the frequency axis. For example, if a high-frequency noise signal is

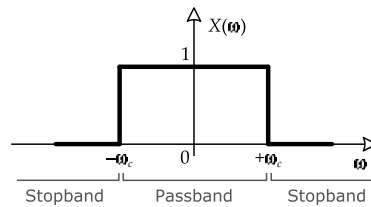


Figure 14.21. The frequency response $H(\omega)$ for an ideal filter has a value of one in the passband and zero in the stopband.

adversely affecting a lower-frequency voice or music signal, a low-pass filter could be used to eliminate the noise.

The frequency response $H(\omega)$ of an ideal filter looks like a rectangular box, with a value of one in the passband and zero in the stopband. When we multiply this by the Fourier transform of our input signal $X(\omega)$, the output $Y(\omega) = X(\omega)H(\omega)$ will have its passband frequencies preserved exactly, and its stopband frequencies all set to zero. The frequency response for an ideal filter is shown in Figure 14.21.

Of course, an ideal filter that completely passes certain frequencies and completely suppresses others may not be desirable. The frequency responses of most real-world filters have a gradual fall-off between the passband and stopband. This aids filtering in situations where there is no single, clear-cut line between the desirable frequencies and the unwanted frequencies. The frequency response of a low-pass filter with a gradual fall-off is shown in Figure 14.22.

The *equalizer* (EQ) found on most high-fidelity audio equipment permits the user to adjust the amount of bass, mid-range and treble that is output. An EQ is really just a collection of filters tuned to different frequency ranges and applied in series to an audio signal.

Filtering theory is an immense field of study, so we can't possibly do it justice here. For a great deal more information, see [41, Chapter 6].

14.3 The Technology of Sound

Before we can fully understand the software that comprises a game's audio engine, we need a firm grasp of audio hardware and technology and of the terminology used by industry professionals to describe it.

14.3.1 Analog Audio Technology

The earliest audio hardware was based on analog electronics. This was the easiest way to record, manipulate and play back audio compression waves, because sound is itself an analog physical phenomenon. In this section, we'll briefly explore some key analog audio technologies.

14.3.1.1 Microphones

A microphone (also known as a “mic” or “mike”) is a transducer that converts an audio compression wave into an electronic signal. Microphones make use of various technologies in order to convert the mechanical pressure variations of a sound wave into an equivalent signal based on variations in electric voltage. A *dynamic microphone* uses electromagnetic induction, while a *condenser microphone* utilizes changes in capacitance. Other types of mics use piezoelectric generation or light modulation to produce a voltage signal.

Different microphones have different sensitivity patterns, known as *polar patterns*. These patterns describe how sensitive the mic is to sound at various angles about its central axis. An omnidirectional mic is equally sensitive in all directions. A bidirectional mic has two sensitivity “lobes” in the shape of a figure eight. A cardioid mic has essentially a unidirectional sensitivity profile, so named because of its somewhat heart-shaped polar pattern. Some common

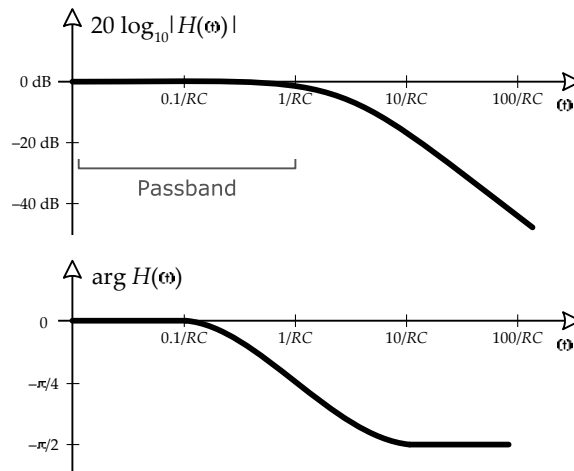


Figure 14.22. The frequency response $H(\omega)$ for an RC (resistor-capacitor) low-pass filter with a gradual fall-off. Both the horizontal and vertical axes of both plots are drawn using a logarithmic scale.

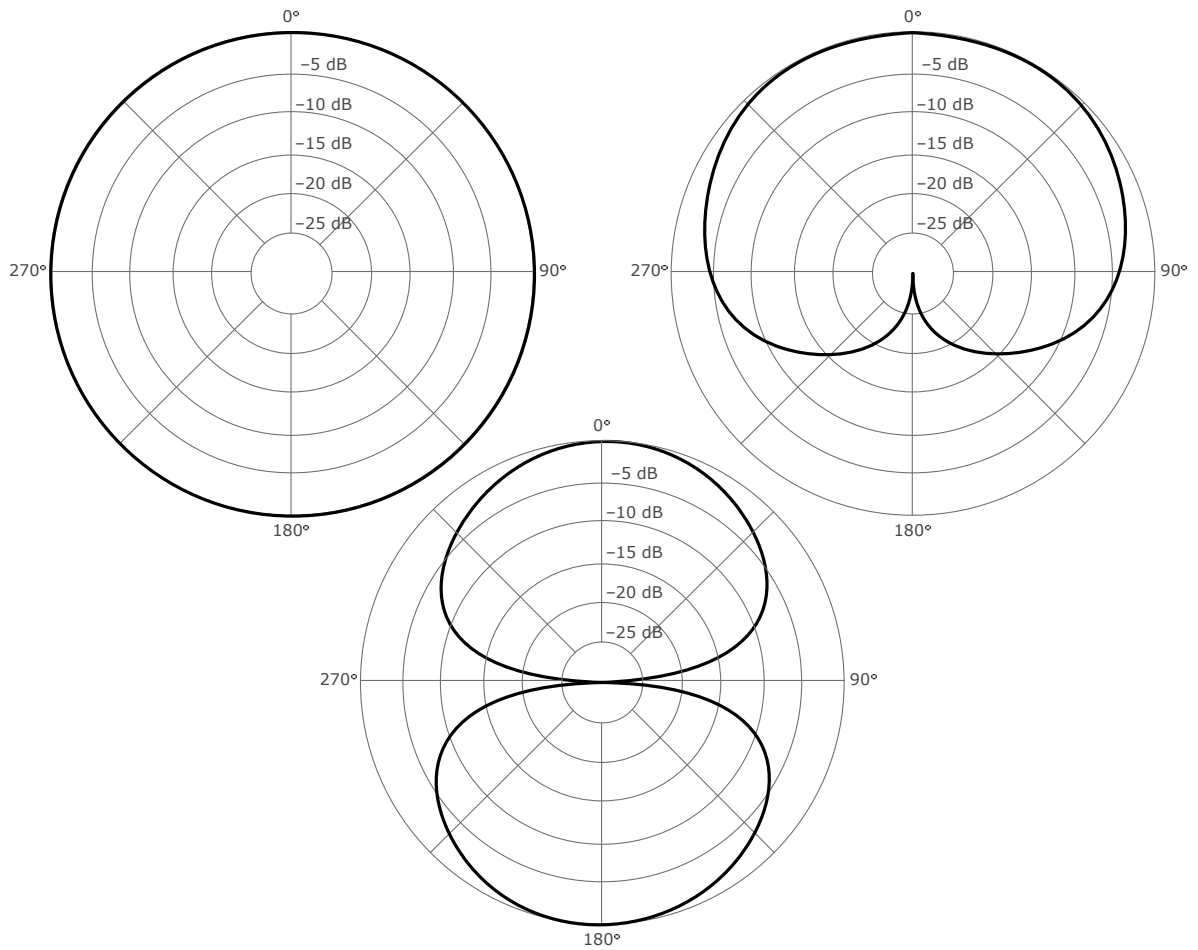


Figure 14.23. Three typical microphone polar patterns, clockwise from upper left: omnidirectional, cardioid and bidirectional.

microphone polar patterns are illustrated in Figure 14.23.

14.3.1.2 Speakers

A *speaker* is basically a microphone operated in reverse—it is a transducer that converts a varying input voltage signal into vibrations in a membrane, which in turn gives rise to air pressure variations that result in a sound pressure wave.

14.3.1.3 Speaker Layouts: Stereo

Sound systems usually support multiple speaker output channels. A stereo device such as an iPod, the sound system in your car or your grandpa's portable "boom box" supports at least two speakers for the left and right stereo channels. Some high-fidelity stereo systems also boast two additional "tweeters"—tiny speakers that are capable of reproducing the highest-frequency sounds within the left and right channels. This allows the two main speakers to be larger, and therefore better at covering the bass. Some stereo systems also support a subwoofer or LFE (low-frequency effects) speaker. Such systems are sometimes called 2.1 systems—two for the left and right, and "dot one" for the LFE speaker.

Headphones versus Speakers

It's important to distinguish between stereo speakers in an open room and stereo headphones. Stereo speakers in a room will typically be positioned in front of the listener and offset to either side. This means that the sound waves coming from the *left* speaker are actually received by the *right* ear as well, and vice versa. The waves from the more-distant speaker will arrive at the ear with a slight time delay (phase shift) and a slight attenuation. The phase-shifted sound waves from the more-distant speaker will tend to *interfere* with those coming from the closer speaker. The sound system should take this interference into account in order to produce the highest quality sound.

Headphones, on the other hand, come in direct contact with the ears, so the left and right channels are perfectly isolated and do not interfere with one another. Also, because headphones deliver sound almost directly to the ear canal, the head-related transfer effects (HRTF) of the shape of the ears themselves do not come into play (see Section 14.1.4), meaning that somewhat less spatial information is received by the listener.

14.3.1.4 Speaker Layouts: Surround Sound

Home theater *surround sound* systems typically come in two flavors: 5.1 and 7.1. As you undoubtedly guessed, these numbers refer to the five or seven "main" speakers, plus the one subwoofer. The goal of a surround sound system is to immerse the listener in a realistic soundscape, by providing *positional* information as well as high-fidelity sound reproduction (see Section 14.1.4). The main speaker channels in a 5.1 system are: center, front left, front right, rear left and rear right. A 7.1 system adds two additional speakers, surround left and surround right, which are intended to be placed directly to either side of the listener. Dolby Digital AC-3 and DTS are two popular surround sound

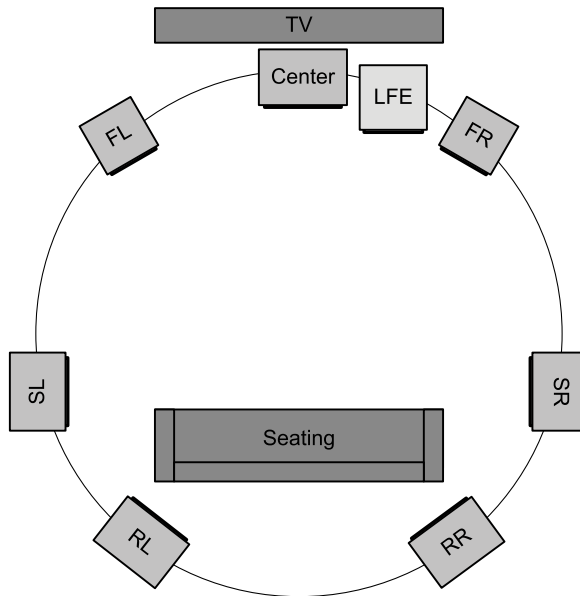


Figure 14.24. Speaker arrangement for a 7.1 surround sound home theater system.

technologies. The speaker layout of a typical 7.1 home theater is shown in Figure 14.24.

Dolby Surround, Dolby Pro Logic and Dolby Pro Logic II are technologies for expanding a stereo source signal into 5.1 surround sound. A stereo signal lacks the positional information necessary to drive the 5.1 speaker configuration directly. But using these Dolby technologies, an approximation of the missing positional information can be generated heuristically using various cues found within the original stereo source signal.

14.3.1.5 Analog Signal Levels

Audio voltage signals may be transmitted at various voltage levels. A microphone usually produces a low-amplitude voltage signal—these are called *mic-level* signals. For connections between components, higher-voltage *line-level* signals are used. There’s a big difference between professional audio equipment and consumer electronics when it comes to line-level voltages. Professional gear is usually designed to work with line levels ranging from 2.191 V (volts) peak-to-peak for a nominal signal up to a maximum voltage of 3.472 V peak-to-peak. The peak-to-peak voltage of a “line level” signal on consumer equipment varies quite a bit, but most consumer devices output up to 1.0 V peak-to-peak, and have inputs that can handle up to 2.0 V signals. It’s impor-

tant to match the levels of input and output signals when connecting audio equipment. Passing a voltage that is too high for the device to handle will cause clipping of the signal. And passing a voltage that is too low will result in audio that sounds quieter than it should.

14.3.1.6 Amplifiers

The small voltages produced by a microphone must be *amplified* in order to drive speakers with enough force to produce audible sound waves. An *amplifier* is an analog electronic circuit that produces at its output a nearly exact replica of its input signal, but with the *amplitude* of the signal increased significantly. An amp essentially increases the *power* content of a signal. It does this by drawing from some kind of power source, and driving the increased voltage produced by this power source in such a way as to mimic the behavior of the input signal over time. In other words, an amp *modulates* the output of its power source to match its much lower-voltage input signal.

The core technology behind an amplifier is the *transistor*—that well-known and utterly ingenious device that sits at the heart of many modern electronic devices, including its crowning achievement—the computer. A transistor makes use of a semiconducting material in order to link the voltages between two otherwise isolated, independent circuits. As such, a low-voltage signal can be used to drive a higher-voltage circuit. This is exactly what is required of an amplifier. We won't get into the details of how transistors and amplifiers work under the hood here. But if you're curious, you can whet your whistle with this great YouTube video on how the very first transistor worked: <https://www.youtube.com/watch?v=RdYHljZi7ys>. And you can read more about amplifier circuits here: <http://en.wikipedia.org/wiki/Amplifier>.

The *gain* A of an amplification system is defined as the ratio of output power P_{out} to input power P_{in} . Like sound pressure level, gain is typically measured in decibels:

$$A = 10 \log_{10} \left(\frac{P_{\text{out}}}{P_{\text{in}}} \right) \text{ dB.}$$

14.3.1.7 Volume/Gain Controls

A *volume control* is basically an inverse amplifier, also known as an *attenuator*. Rather than increasing the amplitude of an electrical signal, it *decreases* the amplitude, while keeping all other aspects of the waveform intact. In a home theater system, the D/A converter produces a voltage signal with a very small amplitude. The power amp boosts this signal up to the maximum “safe” output power, beyond which the sound produced by your speakers would begin to clip and distort (or even damage your hardware). The volume control then

attenuates this maximum output power to produce sound at the desired listening volume.

A volume control is much simpler to make than an amplifier. One can be constructed by introducing a variable resistor, also known as a *potentiometer*, into the circuit somewhere between the amplifier's output and the speakers. When the resistance is at its minimum (at or very close to zero), the amplitude of the input signal isn't changed, and a sound of maximum volume is produced. When the resistance is at its maximum setting, the input signal's amplitude is maximally attenuated, and a sound of minimum volume is produced.

If your stereo system at home reports the volume in decibels, you've probably noticed that the values are always negative. This is because the volume control is attenuating the output of the power amp. The volume meter is still measured like a gain, but the "input" power is the maximum power of the amp, and the "output" power is the volume selected by the user:

$$A = 10 \log_{10} \left(\frac{P_{\text{volume}}}{P_{\text{max}}} \right) \text{ dB},$$

which will be negative as long as $P_{\text{volume}} < P_{\text{max}}$.

14.3.1.8 Analog Wiring and Connectors

An analog monophonic audio voltage signal can be carried by a pair of wires; a stereo signal requires three wires (two channels plus a common ground). The wiring can be internal to a device, in which case it is usually called a *bus*. Wiring can also be external, for use in connecting different devices to one another.

External wiring is typically connected to audio hardware either via a direct "clip" or screw-post connector, of the kind found on high-end speakers, or via various standardized connectors. Examples include RCA jacks, large TRS (tip/ring/sleeve) jacks (the kind used by telephone operators in the early 1900s), TRS mini-jacks (found on your iPod, mobile phone and most PC sound cards), keyed jacks (found most often on high-quality microphones and power amps), and the list goes on.

Audio wiring is available in a wide range of quality levels. Thicker-gauge wiring offers less resistance and therefore can transmit signals over farther distances without unacceptable levels of attenuation. Optional shielding can help reduce noise. And of course the choice of which metal to use in the construction of the wires and connectors can make a difference in the quality of the wiring as well.

14.3.2 Digital Audio Technology

The introduction of the compact disc (CD) marked a turning point in the audio industry toward digital audio storage and processing. Digital technology opens up a great many new possibilities, from reducing the size and increasing the capacity of storage media, to using powerful computer hardware and software to synthesize and manipulate audio in previously unimagined ways. Today, analog audio storage devices are a thing of the past, and analog audio signals are typically employed only where necessary—at the microphone and the speaker.

As we saw in Section 14.2.1.1, the distinction between analog and digital audio technologies corresponds exactly to the distinction between *continuous-time* and *discrete-time* signals in the study of signal processing theory.

14.3.2.1 Analog-to-Digital Conversion: Pulse-Code Modulation

To record audio for use in a digital system, such as a computer or game console, the time-varying voltage of an analog audio signal must first be converted into digital form. Pulse-code modulation (PCM) is the standard method for encoding a sampled analog sound signal so that it can be stored in a computer's memory, transmitted over a digital telephony network or burned onto a compact disc.

In pulse-code modulation, voltage measurements are taken at regular time intervals. The voltage measurements may be stored in floating-point format, or they may be *quantized* so that each measurement can be stored in an integer with a fixed number of bits (typically 8, 16, 24 or 32). The sequence of measured voltage values is then stored into an array in memory, or written out to a long-term storage medium. The process of measuring a single analog voltage and converting it to quantized numeric form is called *analog-to-digital conversion* or A/D conversion. Specialized hardware is typically used to perform A/D conversions. When we repeat this process at regular time intervals, it is called *sampling*. A hardware or software component that performs A/D conversion and/or sampling is referred to as an A/D converter or ADC.

In math terms, given the continuous-time audio signal $p(t)$, we construct the *sampled* version $p[n]$ such that for each sample, $p[n] = p(nT_s)$, where n is a non-negative integer used to index the samples, and T_s is the amount of time between each sample, known as the *sampling period*. The basics of sampling are illustrated in Figure 14.25.

The digital signal that results from PCM sampling has two important properties:

- *Sampling rate.* This is the frequency at which the voltage measurements

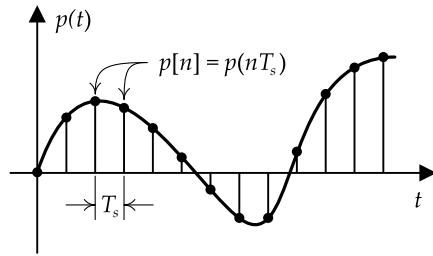


Figure 14.25. A discrete-time signal can be thought of as a sampled version of a continuous-time signal.

(samples) are taken. In principle an analog signal can be recorded digitally without *any* loss of fidelity, provided that it is sampled at a frequency *twice* that of the highest-frequency component present in the original signal. This somewhat astounding and incredibly useful fact is known as the *Shannon-Nyquist sampling theorem*. As we saw in Section 14.1.2.2, humans can only hear sounds within a limited band of frequencies (from 20 Hz to 20 kHz). So all audio signals of interest to human beings are *band-limited*, and can be faithfully recorded using a sampling rate of a little over 40 kHz. (Voice signals occupy a narrower band of frequencies, from 300 Hz to 3.4 kHz, so digital telephony systems can get away with a sampling frequency of only 8 kHz.)

- *Bit depth*. This describes the number of bits used to represent each quantized voltage measurement. *Quantization error* is the error introduced by rounding the measured voltage values to the nearest quantized value. All other things being equal, a greater bit depth results in lower quantization error, and therefore yields a higher-quality audio recording. A bit depth of 16 is typical among uncompressed audio data formats. Bit depth is sometimes referred to as *resolution*.

The Shannon-Nyquist Sampling Theorem

The *Shannon-Nyquist sampling theorem* states that if a *band-limited* continuous-time signal (i.e., a signal whose Fourier transform is zero everywhere outside a limited band of frequencies) is sampled to produce its discrete-time counterpart, the original continuous-time signal can be recovered *exactly* from the discrete signal, provided that the sampling rate is high enough. The minimum sampling frequency for which this relation holds is called the *Nyquist*

frequency.

$$\omega_s > 2\omega_{\max},$$

where

$$\omega_s = \frac{2\pi}{T_s}.$$

Clearly, it is the existence of this theorem that allows digital technology to be used in audio processing. Without it, digital audio would be doomed never to sound as good as analog audio, and computers would not be playing the significant role in the production of high-fidelity audio that they do today.

We won't get into all of the gory details of why the sampling theorem works. But we can gain some insight by realizing that the act of sampling a signal at regularly spaced intervals in time causes its frequency spectrum (Fourier transform) to be duplicated over and over along the frequency axis. The higher the sampling frequency, the more "spaced out" these copies of the signal's frequency spectrum will be. So if the original signal is band-limited,

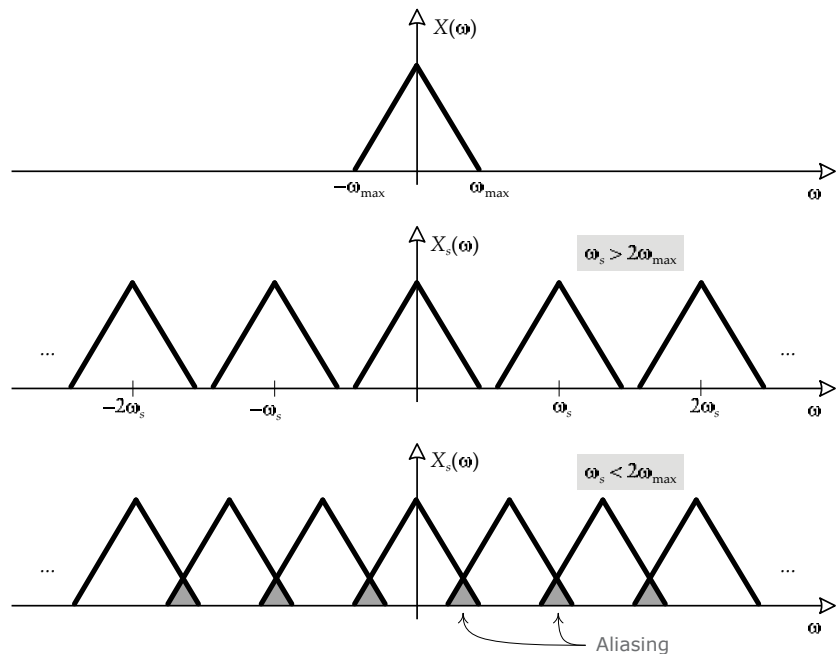


Figure 14.26. The frequency spectrum of a band-limited signal is zero everywhere except within a limited frequency band (top). If the sampling frequency exceeds the Nyquist frequency, the spectrum copies do not overlap and the original signal can be recovered exactly (middle). If the sampling frequency is too low, the spectrum copies overlap and aliasing results (bottom).

and if the sampling frequency is high enough, we can guarantee that the copies of the frequency spectrum will be spaced far enough apart so as not to overlap with one another. When this happens, we can recover the original frequency spectrum exactly via a low-pass filter that filters out all of the copies of the spectrum except the original. However, if the sampling frequency is too low, the spectrum copies will overlap with one another. This is called *aliasing*, and it prevents us from exactly recovering the original signal's spectrum. See Figure 14.26 for an illustration of aliased and unaliased sampling.

14.3.2.2 Digital-to-Analog Conversion: Demodulation

When a digital sound signal is to be played back, a process opposite to that of analog-to-digital conversion is required. We call this, sensibly enough, *digital-to-analog conversion* or D/A conversion for short. It is also termed *demodulation* because it undoes the effects of pulse-code modulation. A digital-to-analog conversion circuit is called a DAC.

D/A conversion hardware generates an analog voltage corresponding to each sampled voltage value in a digital signal, as represented by an array of quantized PCM values in memory. If we drive this hardware with new values periodically, at the rate at which the samples were measured during PCM, and presuming that the sample rate was high enough as per the Shannon-Nyquist sampling theorem, the analog voltage signal produced should exactly match the original voltage signal.

Practically speaking, when we drive an analog voltage circuit with a sequence of discrete voltage levels, unwanted high-frequency oscillations are often introduced as the hardware tries to rapidly change from one voltage level to another. D/A hardware typically includes a low-pass or band-pass filter to remove these unwanted oscillations, thereby ensuring an accurate reproduction of the original analog signal. For more information on filtering, see Section 14.2.5.8.

14.3.2.3 Digital Audio Formats and Codecs

Various data formats exist for storing PCM audio data on disc or transmitting it over the Internet. Each format has its history, and its pros and cons. Some formats such as AVI are actually “container” formats, which can encapsulate digital audio signals in more than one data format.

Some audio formats store the PCM data in an uncompressed form. Others utilize various forms of data compression to reduce the required file size or transmission bandwidth. Some compression schemes are *lossy*, meaning that some of the fidelity of the original signal is lost in the compression/decompression process. Other compression schemes are *lossless*, meaning that

the original PCM data can be recovered exactly after a round-trip compression/decompression cycle.

Let's take a look at a few of the most common audio data formats.

- *Raw header-less PCM* data is sometimes used in situations where the meta-information about the signal, such as the sample rate and bit depth, is known a priori.
- *Linear PCM* (LPCM) is an uncompressed audio format that can support up to eight channels of audio at a 48 kHz or 96 kHz sampling frequency, and 16, 20 or 24 bits per sample. The “linear” in LPCM refers to the fact that the amplitude measurements are taken on a linear scale (as opposed to, say, a logarithmic scale).
- WAV is an uncompressed file format created by Microsoft and IBM. Its use is commonplace on the Windows operating system. Its correct name is “waveform audio file format” although it is also rarely referred to as “audio for windows.” The WAV file format is actually one of a family of formats known as *resource interchange file format* (RIFF). The contents of a RIFF file are arranged in chunks, each with a four-character code (FOURCC) that defines the contents of the chunk and a chunk size field. The bitstream in a WAV file conforms to the linear pulse-code modulation (LPCM) format. WAV files can also contain compressed audio, but they are most commonly used for storing uncompressed audio data.
- WMA (Windows Media Audio) is proprietary audio compression technology designed by Microsoft as an alternative to MP3. See http://en.wikipedia.org/wiki/Windows_Media_Audio for details.
- AIFF (audio interchange file format) is a format developed by Apple Computer, Inc. and used widely on Macintosh computers. Like a WAV/RIFF file, an AIFF file typically contains uncompressed PCM data, and is comprised of chunks, each prefaced by a four-character code and a size field. AIFF-C is a compressed variant of the AIFF format.
- MP3 is a lossy compressed audio file format that has become the de facto standard on most digital audio players, and is also widely used by games and multimedia systems and services. The full name of this format is actually MPEG-1 or MPEG-2 audio layer III. MP3 compression can result in files that are one-tenth the size, but with very little perceptual difference from the original uncompressed audio. These results are achieved by making use of *perceptual coding*—a technique that eliminates portions of the audio signal that are beyond the perception of most people anyway.

- *ATRAC* stands for Adaptive Transform Acoustic Coding—a family of proprietary audio compression techniques developed by Sony. The format was originally developed to allow Sony’s MiniDisc media to contain audio with the same running time as a CD while occupying significantly less space and undergoing an imperceptible degradation in quality. See http://en.wikipedia.org/wiki/Adaptive_Transform_Acoustic_Coding for more details.
- *Ogg Vorbis* is an open source file format that offers lossy compression. Ogg refers to a “container” format that is commonly used in conjunction with the Vorbis data format.
- *Dolby Digital (AC-3)* is a lossy compression format supporting channel formats from mono to 5.1 surround sound.
- *DTS* is a collection of theater audio technologies developed by DTS, Inc. DTS Coherent Acoustics is a digital audio format transportable through S/PDIF interfaces (see Section 14.3.2.5) and used on DVDs and Laserdiscs.
- *VAG* is a proprietary audio file format available for use by all PlayStation 3 developers. It makes use of *adaptive differential PCM* (ADPCM), an analog-to-digital conversion scheme based on PCM. Differential PCM (DPCM) stores the deltas between samples rather than the absolute values of the samples themselves, in order to allow the signal to be compressed more effectively. Adaptive DPCM varies the sample rate dynamically in order to further improve the achievable compression ratio.
- *MPEG-4 SLS, MPEG-4 ALS and MPEG-4 DST* are formats that offer lossless compression.

This list is by no means comprehensive. In fact, there are a dizzying number of audio file formats, and an even longer list of compression/decompression algorithms. For an introduction to the fascinating world of audio data formats, check out our old friend Wikipedia: http://en.wikipedia.org/wiki/Digital_audio_format. The “PlayStation 3 Secrets” website also provides some excellent information on audio formats: <https://bit.ly/2HOVtvR>.

14.3.2.4 Parallel and Interleaved Audio Data

One way to organize multi-channel audio data is to store the samples for each monophonic channel into a separate buffer. In this case, you’d need six parallel buffers to describe a 5.1 audio signal. This arrangement is shown in Figure 14.27.

Multi-channel audio data can also be *interleaved* within a single buffer. In this case, all of the samples for each time index are grouped together in a pre-

Interleaved

C[n]
FL[n]
FR[n]
RL[n]
RR[n]
LFE[n]
C[n+1]
FL[n+1]
FR[n+1]

...

Figure 14.28. Six-channel (5.1) PCM bus data in interleaved format.

defined order. Figure 14.28 depicts an interleaved PCM buffer containing a six-channel (5.1) audio signal.

14.3.2.5 Digital Wiring and Connectors

S/PDIF (Sony/Philips Digital Interconnect Format) is an interconnect technology that transmits audio signals *digitally*, thereby eliminating the possibility of noise being introduced by analog wiring. The S/PDIF standard is physically realized either via a coaxial cable connection (also called S/PDIF) or a fiber-optic connection (known as TOSLINK).

Regardless of the physical interface (S/PDIF coaxial or TOSLINK optical), the S/PDIF transport protocol is limited to 2-channel 24-bit LPCM uncompressed audio at standard sampling rates ranging from 32 kHz to 192 kHz. However, not all equipment works at all sample rates. The same physical interfaces can be also be used to transport bitstream-encoded audio (e.g., Dolby Digital or DTS lossy compressed data) at bitrates ranging from 32 kbps to 640 kbps for Dolby Digital and 768 kbps to 1536 kbps for DTS, respectively.

Uncompressed multi-channel LPCM (i.e., greater than two stereo channels) can only be sent over an HDMI (high-definition multimedia interface) connection on consumer audio equipment. HDMI connectors are used for transmission of both uncompressed digital video and either compressed or uncompressed digital audio signals. HDMI supports up to a 36.86 Mbps bitrate for multi-channel or bitstream audio. However, HDMI bitrates for audio vary depending on the video mode—only 720 p/50 Hz modes or higher are capable of utilizing the full audio bandwidth. See the HDMI specification under the section “video dependency” for more information on this. Apple’s DisplayPort and Thunderbolt connectors are other high-bandwidth alternatives similar in many respects to HDMI.

USB connections are sometimes used to send audio signals. On most game consoles, the USB output is intended only to drive headphones.

Wireless audio connections are also possible. The Bluetooth standard is the

Parallel

C[n]	FL[n]	FR[n]	RL[n]	RR[n]	LFE[n]
C[n+1]	FL[n+1]	FR[n+1]	RL[n+1]	RR[n+1]	LFE[n+1]
C[n+2]	FL[n+2]	FR[n+2]	RL[n+2]	RR[n+2]	LFE[n+2]
...	...	...	...	...	...

Figure 14.27. Six-channel (5.1) PCM bus data in parallel format.

most commonly used method of transmitting audio signals wirelessly.

14.4 Rendering Audio in 3D

Thus far, we've learned about the physics of sound, the mathematics of signal processing and the various technologies that are used to record and play back sounds. In this section, we'll explore how all of this theory and technology can be put to use in a game engine, in order to produce realistic, immersive soundscapes for our games.

Any game that takes place in a virtual 3D world requires some sort of *3D audio rendering engine*. A high-quality 3D audio system should provide the player with a rich, immersive and believable soundscape that matches what's going on in this 3D world, while supporting the story and remaining true to the tonal design of the game.

- The inputs to this system are the myriad *3D sounds* that emanate from all over the game world: footsteps, speech, the sound of objects bumping into one another, gunfire, ambient sounds like wind or rainfall and so on.
- Its output is a handful of sound channels that, when played in the speakers, reproduce as believably as possible what the player would actually hear if he or she were really there in the virtual game world.

Ideally we'd like our audio engine to produce its output in full 7.1 or 5.1 surround sound, because this gives the ears the richest possible set of positional cues. However, audio engines must also support stereo output for players who don't have fancy home theater systems—or who just want to play their game using headphones so they don't wake their neighbors.

A game's audio engine is also responsible for playing sounds that do not originate in the virtual world. Examples include the music track, sounds made by the in-game menu system, a narrator's voice-over, the voice of the player character (especially in first-person shooters) and possibly certain ambient sounds. We call these *2D sounds*. Such sounds are designed to be played “directly” in the speakers, after having been mixed with the outputs of the 3D spatialization engine.

14.4.1 Overview of 3D Sound Rendering

The primary tasks performed by the 3D audio engine are as follows:

- *Sound synthesis* is the process of generating the sound signals that correspond to the events occurring in the game world. These might be produced by playing back pre-recorded sound *clips*, or they might be *procedurally* generated at runtime.
- *Spatialization* produces the illusion that each 3D sound is coming from the proper location in the game world, from the point of view of the listener. Spatialization is accomplished by controlling the *amplitude* of each sound wave (i.e., its gain or volume) in two ways:
 - *Distance-based attenuation* controls the *overall volume* of a sound in order to provide an indication of its *radial distance* from the listener.
 - *Pan* controls a sound's relative volume in each of the available speakers in order to provide an indication of *direction* from which the sound is arriving.
- *Acoustical modeling* heightens the realism of the rendered soundscape by mimicking the early reflections and late reverberations that characterize the listening space, and by accounting for the presence of obstacles that partially or completely block the path between the sound source and the listener. Some sound engines also model the frequency-dependent effects of *atmospheric absorption* (Section 14.1.3.2) and/or HRTF effects (Section 14.1.4).
- *Doppler shifting* may also be applied to account for any relative movement between a sound source and the listener.
- *Mixing* is the process of controlling the relative volumes of all the 2D and 3D sounds in our game. The mix is driven in part by physics and in part by aesthetic choices made by the game's sound designers.

14.4.2 Modeling the Audio World

In order to render the soundscape of a virtual world, we must first describe that world to the engine. The “audio world model” consists of the following elements:

- *3D sound sources*. Each 3D sound in the game world consists of a monophonic audio signal, emanating from a specific *position*. We must also provide the engine with its *velocity*, *radiation pattern* (omnidirectional, conical, planar) and *range* (beyond which the sound is inaudible).
- *Listener*. The listener is a “virtual microphone” located in the game world. It is defined by its *position*, *velocity* and *orientation*.

- *Environmental model.* This model either describes the *geometry* and *properties* of the surfaces and objects present in the virtual world, and/or it describes the *acoustic properties* of the listening spaces in which gameplay takes place.

The *positions* of the source and listener are used for *distance-based attenuation*; the *radiation pattern* of the sound source also factors into the distance-based attenuation calculation. The *orientation* of the listener defines a reference frame in which the *angular position* of the sound is calculated. This angle in turn determines the *pan*—the relative volumes of the sound in the five or seven main speakers of 5.1 or 7.1 surround sound, respectively. The *relative velocity* of source and listener is used when applying a *Doppler shift*. And last but not least, the *environmental model* is used for modeling the acoustics of the listening space and to account for partial or complete blockage of the sound path.

14.4.3 Distance-Based Attenuation

Distance-based attenuation reduces the volume of a 3D sound as the radial distance between it and the listener increases.

14.4.3.1 Fall-Off Min and Max

The number of sound sources in a typical game world is very large. Due to hardware and CPU bandwidth limitations, we couldn't possibly render them all. And we wouldn't want to, because thanks to distance-based fall-off all sounds beyond a certain distance from the listener can't be heard anyway. For this reason, each sound source is usually annotated with fall-off (FO) parameters.

The *fall-off min* ("FO min" for short) is a minimum radius, which we'll denote $r_{\min}$, within which the sound doesn't fall off at all and is heard at full volume. The *fall-off max* or "FO max" is a maximum radius, denoted $r_{\max}$, beyond which the sound source is considered to be silent and can therefore be ignored. Between the FO min and FO max, we need to blend smoothly from full volume down to zero.

14.4.3.2 Blending to Zero

One way to blend from maximum volume down to zero is to use a linear ramp between FO min and FO max. Depending on the type of sound, a linear fall-off might sound just fine.

In Section 14.1.3.1, we learned that sound intensity, which is closely related to our perception of "loudness," falls off with radial distance according to a

$1/r^2$ rule. Gain, which is proportional to the amplitude of the sound pressure, falls off as $1/r$. So really the right thing to do is to use a $1/r$ curve to blend the gain of a sound from full volume down to zero.

One problem with the function $1/r$ is that it is asymptotic—it never quite reaches zero, no matter how large r gets. We can fix this by shifting the curve slightly downward so that it crosses the r axis at $r_{\max}$. Or we can simply clamp the sound intensity to zero for all $r > r_{\max}$.

14.4.3.3 Bending the Rules

When making *The Last of Us*, Naughty Dog’s sound department discovered that attenuating character dialog using the $1/r^2$ rule caused speech to become unintelligible too quickly for characters that were only a modest distance away. This was a serious problem, especially during the stealth sections of gameplay, where hearing the enemies’ ambient conversations was important both as a tactical tool and as a means of advancing the storyline.

To solve this problem, the sound department at Naughty Dog utilized a sophisticated fall-off curve that causes dialog to roll off more slowly near the listener, more quickly in the mid-range, and then more slowly again as the distance to the listener grows very large. This allows speech to be audible over longer distances, while still retaining a natural-sounding fall-off.

The dialog fall-off curves were also adjusted dynamically at runtime, based on the current “tension level” of the game (i.e., whether the enemies are unaware of the player, are searching for him or are engaged in direct combat with him). This is what allows the voices in *The Last of Us* to project over longer distances during stealth gameplay, while not rising to overpowering levels when combat breaks out.

Finally, a “sweetener” reverb could be optionally enabled to allow character voices to bleed around corners, even when the direct path is 100% obstructed. This tool is incredibly helpful in situations where modeling realistic fall-off is less important than ensuring that the player can hear a conversation clearly.

There are all sorts of ways to “cheat” when designing your 3D audio model. But no matter what you do, always remember this simple lesson: Never be afraid to do whatever it takes to satisfy the *needs of your game*. Don’t worry—the laws of physics won’t be offended!

14.4.3.4 Atmospheric Attenuation

As we saw in Section 14.1.3.2, low-pitched sounds are attenuated by the atmosphere less than high-pitched sounds. Some games, including Naughty Dog’s *The Last of Us*, model this phenomenon by applying a low-pass filter to each

3D sound whose passband slides toward lower and lower frequencies as the distance between the sound source and the listener increases.

14.4.4 Pan

Panning is a technique used to provide the illusion that a 3D sound is coming from a particular direction. By controlling the volume (i.e., gain) of the sound in each of the available speakers, we can induce the perception of a *phantom image* of the sound in three-dimensional space. This method of panning is called *amplitude panning* because we are providing angular information to the listener by adjusting only the amplitudes of the sound waves produced at each speaker (as opposed to using phase offsets, reverb or filtering to provide positional cues). It is sometimes referred to as *IID panning* because it relies on the perceptual effects of interaural intensity difference (IID) to produce a sound's phantom image.

The term “pan” comes from early technology that used a “panoramic potentiometer” (variable resistor) or “pan pot” to control the relative volumes of the left and right speakers of a stereo system. Dialing the pan pot to one extreme would produce sound only in the left speaker; dialing it to the other extreme would drive the right speaker exclusively; and centering the pan pot dial would distribute the sound equally to both speakers.

To understand how pan works, we envision our listener located at the center of a circle. The speakers are positioned at various points on the circumference of this circle, so we'll call it the *speaker circle* in this book. The radius of the circle approximates the average distance between the listener and any one speaker.

For a stereo sound system, the front and right speakers are located roughly at ± 45 degrees to the left and right of center. For stereo headphones, they are positioned at ± 90 degrees (and the radius is much smaller). For 7.1 surround sound, we consider only the seven “main” speakers, as the LFE channel provides no positional cues. These speakers are located roughly as shown in Figure 14.29. When panning to a 5.1 system, we simply omit the surround left and surround right speakers.

For the time being, let's treat each 3D sound as a point source. To pan a sound, we first determine its *azimuthal* (horizontal) angle. The azimuthal angle must be measured in the local space of the listener, so that an angle of zero corresponds to the position directly in front of the listener. Next, we figure out which two speakers around the circumference of our circle are *adjacent* to this azimuthal angle. We convert the angle into a percentage of the arc between the two speakers. Finally, we use this percentage to determine the gains of the

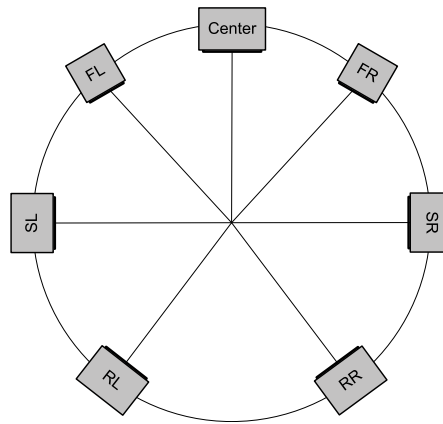


Figure 14.29. Speaker layout for 7.1 pan.

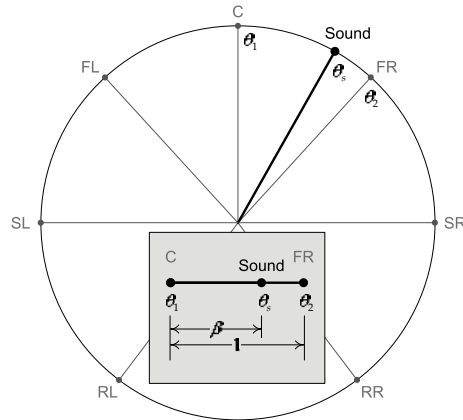


Figure 14.30. Treating the sound as a point source, the pan blend percentage β is calculated between the two speakers immediately adjacent to the source.

sound in each speaker.

To formulate this mathematically, let's use the symbol θ_s for the azimuthal angle of the sound. We'll call the angles of the two adjacent speakers θ_1 and θ_2 . The percentage β is then calculated as follows:

$$\beta = \frac{\theta_s - \theta_1}{\theta_2 - \theta_1}.$$

This calculation is illustrated in Figure 14.30.

14.4.4.1 Constant Gain Panning

Our first instinct might be to use the percentage β to perform a simple linear interpolation between the gains of the two speakers. Given the gain A of the unpanned sound, the gains of that sound as played in each speaker would be calculated as follows:

$$\begin{aligned} A_1 &= (1 - \beta)A; \\ A_2 &= \beta A. \end{aligned}$$

This is known as *constant gain* panning, because the net gain $A = A_1 + A_2$ is constant, independent of the values of θ_s and β .

The main problem with constant gain panning is that it does *not* produce the perception of constant *loudness* as the sound moves around the acoustic field. Gain controls the amplitude of the sound pressure wave, and therefore controls the *sound pressure level* (SPL). However, as we learned in Section 14.1.2, human perception of loudness is actually proportional to the *intensity* or *power* of a sound wave, both of which vary as the *square* of the SPL.

As an illustration of the problem, imagine that our sound is panned to the halfway point between our two speakers. Constant gain panning would have us set the gains A_1 and A_2 to $\frac{1}{2}A$ each. But this yields a total power of $A_1^2 + A_2^2 = (\frac{1}{2}A)^2 + (\frac{1}{2}A)^2 = \frac{1}{2}A^2$. In other words, the loudness of the sound will be *one-half* of what it would have been, had the sound been panned to only the left or the right speaker.

14.4.4.2 The Constant Power Pan Law

Clearly in order to keep the perception of loudness constant as a sound's image moves about the listener, we need to keep the *power* constant. This rule is known as the *constant power pan law*, or just the *pan law* for short.

There's a very easy way to implement the constant power pan law. Instead of linearly interpolating the gains, we use the sine and cosine of the blend percentage β to calculate them:

$$\begin{aligned} A_1 &= \sin\left(\frac{\pi}{2}\beta\right)A; \\ A_2 &= \cos\left(\frac{\pi}{2}\beta\right)A. \end{aligned}$$

Consider again a sound image that is panned to halfway between the two speakers ($\beta = \frac{1}{2}$). With constant power panning, the two speakers' gains will be set to $A_1 = A_2 = \frac{1}{\sqrt{2}}A$. This yields a total power of $A_1^2 + A_2^2 = (\frac{1}{\sqrt{2}}A)^2 + (\frac{1}{\sqrt{2}}A)^2 = A^2$. This works for any value of β , so the power A^2 is constant no matter where our sound image is placed around the circle.

Sound designers often apply a “3 dB rule” to account for the pan law: If a sound is to be mixed equally to two speakers, the gain in each speaker should be reduced by 3 dB relative to the gain that would be used if the sound were to be played in only one speaker. The value -3 dB arises because $\log_{10}\left(\frac{1}{\sqrt{2}}\right) \approx -0.15$. Voltage gain (or amplitude gain) is defined as $20 \log_{10}(A_{\text{out}}/A_{\text{in}})$, and $20 \times -0.15 = -3$ dB. (The 20 in front of the logarithm arises because a decibel is one-tenth of a bel, multiplied by two to account for the fact that we’re dealing with A^2 and not A .)

14.4.4.3 Headroom

Panning causes sounds to be rendered entirely by one speaker in some situations, and by two (or more, as we’ll see) speakers in others. Let’s say a sound is being played equally by two adjacent speakers, and its volume is so loud that each speaker is outputting its maximum power. What happens when that sound pans around to only one speaker? The answer is that we’d probably blow out the speaker, because our constant power pan law requires us to use more gain for one speaker than for two.

To prevent this problem, we need to artificially lower the maximum gains of our sounds across the board, such that the worst-case scenario of playing the sound in one speaker won’t overdrive that speaker. The practice of artificially reducing the maximum range of volume is known as “leaving oneself some *headroom*.”

The concept of headroom also applies to *mixing*. When two or more sounds are mixed, their amplitudes add up. By leaving some headroom in our mix, we can accommodate worst-case scenarios where a large number of high-volume sounds play simultaneously.

14.4.4.4 To Center or Not to Center?

In cinema, the center channel was historically used for speech; only the sound effects would be panned to the other speakers around the room. The idea behind this practice was that the characters in the movie are usually on-screen when they speak, so the audience expects to hear their voices front-and-center. This approach has the nice side-effect of separating out the speech from the rest of the sounds in the film, meaning that loud sound effects won’t use up all the available headroom and drown out the dialog.

In 3D games, the situation is quite different. The player generally wants to hear dialog coming from the “correct” location around him or her. If the player swings the camera by 180 degrees, the dialog should likewise swing about the

sound field by 180 degrees. As such, games usually do not assign all dialog to the center speaker; instead, it is included in the pan for both sound effects and dialog.

Of course, this brings us back to the headroom problem—loud gunfire can now completely drown out the speech. At Naughty Dog, we overcame this problem by “splitting the difference” and always playing *some* of the dialog in the center channel, as well as panning some of it to the rest of the speakers along with the sound effects.

14.4.4.5 Focus

When the source of a sound is far away from the listener, we can treat it as a point source. We simply calculate a single azimuthal angle and feed it into our constant power panning system. However, when a sound source approaches or actually enters into the circle that defines the radial distance of the speakers from the listener, it can no longer be accurately modeled as a point source represented by a single angle.

Consider the case of moving toward and past a sound source. At first, the sound source appears entirely in the front speakers. As it passes the listener, we somehow need to transfer the sound to the rear speakers. If we model the sound as a point source, our only option is to “pop” the sound from the fronts to the rears.

Ideally we’d like the sound’s image to gradually “spread out” around the speaker circle as it approaches. That way, as it nears the listener, we can start playing more of it in the side speakers. When the sound source is coincident with the listener, it can be played in all seven (or five) speakers. And once it passes, we can smoothly transition the sound to the rears, dropping the front gains to zero as it recedes behind the listener.

We can do this kind of thing and more if we model a sound source not as a point on the speaker circle but as an *arc*. Looking at it another way, we can think of each sound source as having an arbitrary shape in 3D space, and its *projection* onto the speaker circle *subtends* a certain angle, defining a “pie wedge” shape within the circle. This is analogous to the concept of *solid angle* often used in the calculation of ambient occlusion in 3D graphics—see http://en.wikipedia.org/wiki/Solid_angle for details.

We’ll call the angle subtended by an extended sound source the *focus angle*, and we’ll denote it α . A point source can be thought of as an “edge case” in which $\alpha = 0$. The focus angle is depicted in Figure 14.31.

To render a sound with a nonzero focus angle, we must first determine the subset of speakers that either intersect its projected arc on the speaker circle, or are immediately adjacent to the arc. Then we must divide the sound’s

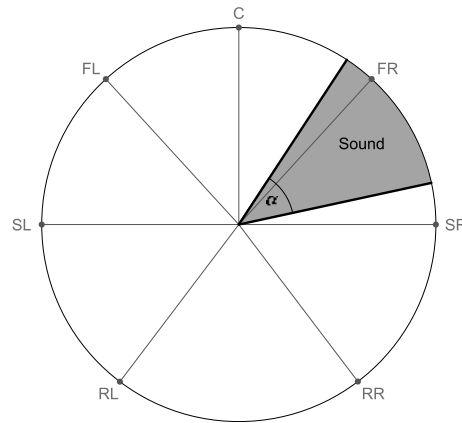


Figure 14.31. The focus angle α defines the projection of an extended sound source on the speaker circle.

intensity/power among these speakers in order to induce the perception of a phantom image that extends across the projected arc.

We can divide the sound amongst the relevant speakers in various ways. For example, we could arrange for all the speakers that lie *within* the focus “pie slice” to receive equal maximum power, and then apportion less of the sound to the two speakers immediately adjacent the arc to create a fall-off. But no matter how we do it, we must remember to always obey the constant power pan law. So, we must set the gains in such a way that the sum of their squares (i.e., the sum of their powers) equals the squared gain of the original unpanned sound source.

14.4.4.6 Dealing with Verticality

In both stereo and surround sound set-ups, the speakers all lie roughly in a horizontal plane. This arrangement makes it tricky to position sounds above or below the plane of the listener’s ears.

The ideal of course would be to model a true “periphonic” sound field by using a spherical speaker arrangement. A technology known as *Ambisonics* (<http://en.wikipedia.org/wiki/Ambisonics>) is capable of accommodating both planar and spherical speaker arrangements. However, it is not supported by any game console—at least not yet. Sony now offers a 3D audio technology in their Platinum Wireless Headset for PS4, and games are beginning to support it. But even in the presence of 3D audio technology, games still need to support a planar speaker arrangement for 5.1 and 7.1 sound systems.

It turns out that the concept of *focus* can be leveraged to simulate some de-

gree of verticality in our sound imagery. We simply *project* all sounds onto the horizontal plane, and then use a nonzero focus angle for any sounds whose projections fall too close to or within the speaker circle. An elevated sound that is far away will be rendered in virtually the same way as one that is not elevated. But as the elevated sound passes overhead, we blend it across multiple speakers, thereby producing a phantom image within the speaker circle. If we combine this with distance-based attenuation and frequency-dependent atmospheric absorptions, we can provide the listener with enough cues to make the sound seem to be located above or below the listener.

14.4.4.7 Further Reading on Pan

The basics of the constant power pan law can be found here: <http://www.rs-met.com/documents/tutorials/PanRules.pdf>. The following site is also a great resource on the topic: http://www.music.miami.edu/programs/mue/Research/jwest/Chap_3/Chap_3_IID_Based_Panning_Methods.html.

The paper entitled “Spatial Sound Generation and Perception by Amplitude Panning Techniques” by Ville Pukki of the Helsinki University of Technology, available at <https://aaltodoc.aalto.fi/bitstream/handle/123456789/2345/isbn9512255324.pdf?sequence=1>, provides a clear description of the spatialization problem and outlines the vector based amplitude panning (VBAP) method, as well as providing an extensive bibliography for further reading.

David Griesinger’s paper, “Stereo and Surround Panning in Practice,” also makes for a very interesting read; it is available at http://www.davidgriesinger.com/pan_laws.pdf. David’s website is chock full of research on sound perception and audio reproduction technologies.

14.4.5 Propagation, Reverb and Acoustics

Even if we were to implement distance-based attenuation, pan and Doppler, our 3D sound engine still wouldn’t be able to generate a realistic soundscape. This is because a lot of the auditory cues we humans use to sort out what kind of space we’re in come from the early reflections, late reverberations and head-related transfer function (HRTF) effects caused by sound waves taking multiple paths to reach our ears. The term “*sound propagation modeling*” can be applied to any technique that is designed to take into account the ways in which sound waves propagate through a space.

Many different approaches are used, both in research and in interactive media and games. These technologies fall into three basic categories:

- *geometric analysis* attempts to model the actual pathways taken by sound waves,

- *perceptually based models* focus on reproducing what the ear perceives using an LTI system model of the acoustics of a listening space, and
- *ad hoc methods* employ various kinds of approximations to produce reasonably accurate acoustics with minimal data and/or processing bandwidth.

The following paper does a good job of surveying many of the techniques that fall into the first two categories: <http://www-sop.inria.fr/reves/Nicolas.Tsingos/publis/presence03.pdf>. In this section, we'll briefly discuss LTI systems modeling, and then turn our attention to a few ad hoc methods, because they tend to be more practical for use in real games.

14.4.5.1 Modeling Propagation Effects with an LTI System

Imagine that I am standing in a room containing various objects made of various materials. A sound is made in the room. It reflects and diffracts and bounces around the room, and eventually reaches my ears. If you think about it, it doesn't really matter which specific paths those sound waves took. The only thing that affects my perception is the specific superposition of the dry direct sound waves and the various time-shifted and possibly muffled or otherwise altered wet indirect waves.

It turns out that all of these effects can be modeled with a linear time-invariant (LTI) system. Theoretically, if we could measure the *impulse response* of the room for a given pair of points that represent the source of the sound and the listener, we can determine exactly how *any* sound we might play at that source location should sound if heard at the listener position. All we need to do is convolve the dry sound with the impulse response!

$$p_{\text{wet}}(t) = p_{\text{dry}}(t) * h(t).$$

This technique seems like a silver bullet at first blush. However, it is actually more difficult and less practical than it may at first seem. It's pretty easy to determine the impulse response of a space in real life—you can record the sound of a short “click” that approximates the unit impulse $\delta(t)$, and the recorded signal will approximate $h(t)$. But in a virtual space, we'd need to perform a complex and expensive simulation of each play space in order to determine $h(t)$. Also, to model the room's acoustics accurately, we'd need to perform this calculation for a large number of source-listener point pairs throughout the game world, and once calculated the size of this data would be immense. Finally, the operation of convolution is itself not inexpensive, and game consoles and sound cards have in the past lacked the horsepower to do this for every sound in the game in real time.

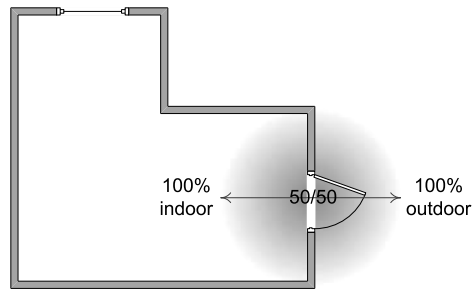


Figure 14.32. It's a good idea to cross-blend between reverb settings based on the position of the listener.

Modern gaming hardware is getting more powerful all the time, and a convolution-based approach to propagation modeling is becoming more feasible. For example, Micah Taylor et al. created a real-time demo of convolution reverb that produced promising results—see <https://intel.ly/2J8Gpsu>. That said, most games still don't use this approach, but instead they rely on various ad hoc methods and approximations to model environmental reverb.

14.4.5.2 Reverb Regions

One common approach to modeling the wet characteristics of a play space is to annotate the game world with manually placed regions, each of which is tagged with appropriate reverb settings such as pre-delay, decay, density and diffusion. See Section 14.1.3.4 for a discussion of these parameters. As the virtual listener moves through these regions, we can light up the appropriate reverb mode: If the player enters a large tiled room, we can bump up the echos; when the player enters a small closet, we can virtually eliminate the reverb to produce a very dry sound.

It's a good idea to smoothly cross-blend between reverb settings as the listener moves through the play space. We can use simple linear interpolation to perform this cross-blend for each parameter. The blend percentage is best calculated using a measure of how far "into" the region the listener is. For example, imagine moving between an outdoor space and an indoor space through a doorway. We could define a region around the doorway within which the blend occurs. If the listener is entirely outside the blend region, the blend percentage should yield 100% of the outdoor reverb settings and 0% of the indoor settings. If the listener is standing at the halfway point within the blend region, we'd want a 50/50 mix of the reverb settings. Once the listener passes out of the blend region inside the building, we'll have reached a 0% outdoor / 100% indoor blend. This idea is illustrated in Figure 14.32.

14.4.5.3 Obstruction, Occlusion and Exclusion

When using regions to define the acoustics of our play spaces, we typically assign a *single* impulse response function or a *single* collection of reverb settings to each region. This captures the essence of each play space (e.g., large tiled hall, small closet lined with coats, flat outdoor plain, etc.). But it results in a less-than-perfect reproduction of the acoustics that arise due to obstacles. For example, imagine a square room with a large pillar in the center. If a sound source is located in the corner of the room, a listener will perceive a very different timbre as he or she moves about the room, depending on whether the direct path is obstructed by the pillar or not. If we use a single set of reverb parameters for this room, we cannot capture these subtleties.

To address this problem, we can attempt to model the geometry and material properties of the environment in some way, determine how sound waves are affected by the obstacles in their path, and then use the results of this analysis to alter the “base” reverb settings associated with the room.

Figure 14.33 shows the three ways in which the objects and surfaces in the game world can affect the transmission of sound waves:

- *Occlusion*. This describes a situation in which there exists no unfettered path from the sound source to the listener. A listener might still be able to hear a fully occluded sound, if for example there is only a thin wall or door between it and the source of the sound. Either the dry and wet components of an occluded sound are both attenuated and/or muffled, or the sound is entirely silent from the point of view of the listener.
- *Obstruction*. This describes a case in which the direct path between the sound and the listener is blocked, but an indirect path is available. Obstruction can occur for example when a sound source passes behind a car, pillar or other obstacle. The dry component of an obstructed sound is either entirely absent or greatly muffled, and the wet component may be altered as well to account for the sound waves having to take a longer, more reflected path to the listener.
- *Exclusion*. This describes a case in which there is a free direct path between source and listener, but the indirect path is compromised in some way. This can happen if a sound is produced in one room and passes through a narrow opening such as a door or window to reach the listener. In an exclusion situation, the dry component of the sound remains unaltered but the wet component is attenuated, muffled or, for very narrow openings, entirely absent.

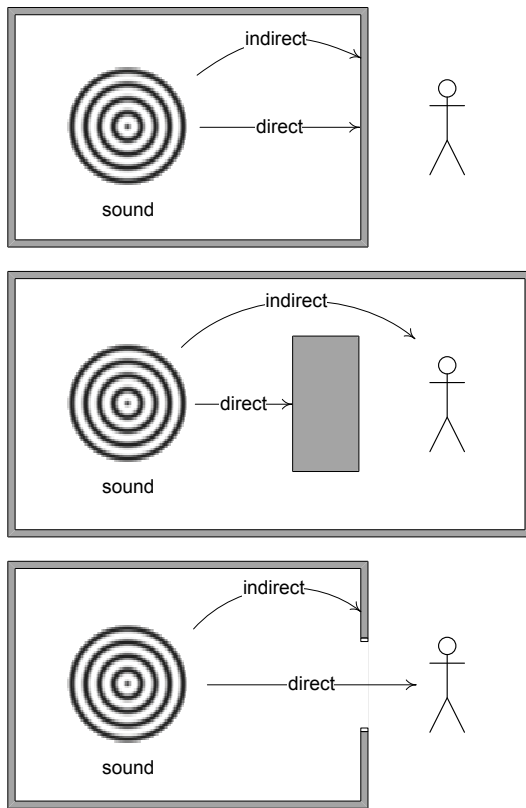


Figure 14.33. From top to bottom: occlusion, obstruction and exclusion.

Analyzing the Direct Path

Determining whether the direct path is blocked or not is not difficult. We simply cast a ray (see Section 13.3.7.1) from the listener to each sound source. If it is blocked, the direct path is occluded. If not, it is free.

If we wish to model sound transmission *through* walls and other obstacles, ray casting can still be used. We cast a ray from source to listener, and for each contact we query the material properties of the impacted surface to determine how much of the sound's energy it absorbs. If it allows some energy to pass through, we can cast another ray starting on the other side of the obstacle and continue tracing the path to the listener. Once all of the sound's energy has been absorbed, we can conclude that the sound cannot be heard. But if the ray makes it all the way to the listener without losing all sound energy, we can attenuate the gain of the dry sound component by the corresponding amount to simulate transmission of the sound.

Analyzing the Indirect Path

Determining whether the indirect path is occluded is a much more difficult problem. Ideally, we'd perform some kind of search (A* perhaps) to determine whether or not a path exists from the source to the listener, and also how much attenuation and reflection is introduced by each viable path. In practice, this *path tracing* method is rarely used because it is processor- and memory-intensive. And at the end of the day, we game programmers aren't really interested in creating physically accurate simulations that will win us Nobel prizes in physics. We merely want to produce a soundscape that is *immersive* and *believable*.

Never fear, all is not lost. There are all sorts of ways in which we can obtain an *approximate* model of the indirect path of a sound. For example, if we are using *reverb regions* to model the overall acoustics of the various spaces in our game (see Section 14.4.5.2), we could leverage these regions to determine whether an indirect path exists. For example, we could use some simple rules of thumb:

1. If the source and listener are in the same region, assume an indirect path exists.
2. If the source and listener are in different regions, assume the indirect path is occluded.

Using these assumptions combined with the results of our direct path ray cast, we can differentiate between the four cases: free, occluded, obstructed or excluded.

Accounting for Diffraction

When any wave passes through a narrow opening or interacts with a corner, it spreads out as shown in Figure 14.34. We call this phenomenon *diffraction*. Because of diffraction, sounds can be heard around corners as if a direct path existed, as long as the angular difference between the direct path and curved path is not too great.

One way to determine whether sound can diffract in order to reach the listener is to cast a few “curved” rays around the central “direct” ray. Most collision engines don't support curved path tracing, but we can emulate a curved path by using multiple straight-line ray casts. Figure 14.35 shows a simple example, in which five rays are cast from the sound source to the listener—one direct ray, plus two “curved” traces comprised of two straight-line ray casts each. Technically speaking we're employing a *piecewise-linear approximation* to each curved path we wish to trace.

If the direct ray is occluded but the curved traces can “see” the listener, this tells us that the listener is within the “diffraction region” around a nearby corner, and should hear the sound as if it is not occluded.

Applying the Model Using Reverb and Gain

Thus far, we’ve discussed how to determine whether the direct and indirect paths are blocked or not. This analysis can also tell us something about the acoustic impact of an occlusion or obstruction. (For example, sound passing through a wall can be muffled; sound taking a long “bouncy” path might introduce a lot of reverb.) The question now is: How do we apply this knowledge when rendering the sound?

One simple approach is to simply attenuate the dry and wet components of the sound individually, based on whether the direct or indirect paths are totally or partially blocked, respectively. To finesse the results, we can also apply more or less reverb to each component of the sound, based on whatever heuristic information we gathered when determining the path(s) taken by the sound. The needs of every game are different, so this is one of those times when trial and error is your best and only option!

Blending Obstructed Sounds

If you were to go off and implement everything we talked about in the sections above, you’d notice a glaring problem. As a sound source moves between the four states described above—for example, from being free to being obstructed—the timbre and loudness of the sound will seem to “pop.” There are a number of ways to smooth out such transitions. You could apply a little *hysteresis*, meaning that you delay the response of the sound system to changes in the obstructed state of each sound, and then use this short delay window to smoothly cross-blend between the two sets of reverb settings. But the delay

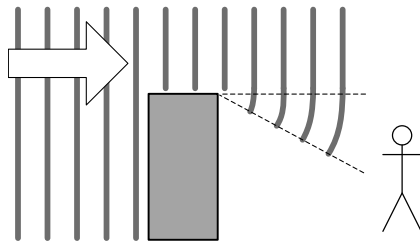


Figure 14.34. Diffraction causes the dry component of a sound to be clearly audible even when the direct path is blocked.

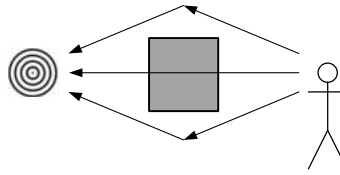


Figure 14.35. Curved ray casts can be approximated using multiple straight-line rays.

might be noticeable, so this isn't an ideal solution.

For the *Uncharted* and *The Last of Us* series, Naughty Dog's senior sound programmer Jonathan Lanier invented a proprietary system that he called *stochastic propagation modeling*. Without giving away any trade secrets, I can tell you that this system involves casting a bunch of rays to each sound source, some direct and some indirect, and accumulating these hit/miss results over many frames. From this data, we generate a probabilistic model of the degree of occlusion experienced by both the dry and wet components of each sound source. This allows us to smoothly transition a sound from being fully obstructed to fully free without noticeable "pops."

14.4.5.4 Sound Portals in *The Last of Us*

For *The Last of Us*, Naughty Dog needed a way to model the actual pathways that sounds take through the environment. If an enemy NPC is speaking while standing in a long hallway that connects to the room the player is in, we wanted to be able to hear the sound of his voice coming from the *doorway*, not "through the wall" along a straight-line path.

To do this, we used a network of interconnected regions. There were two kinds of regions: *rooms* and *portals*. For each sound source, we found a path from the listener to the sound by using connectivity information provided by the sound designer when laying out the regions. If both the sound source and listener were in the same room, we'd use the tried and true method of performing obstruction/occlusion/exclusion analysis that we used on the *Uncharted* series. But if the sound source was in a room directly connected to the listener's room via a portal, we would play the sound *as if* it were located in the portal region. We found that we only needed to go "one hop" in the room connectivity graph to make this work for all real situations that arose in the game. Obviously I'm leaving out a lot of important details here, but Figure 14.36 illustrates the basics of how this system worked.

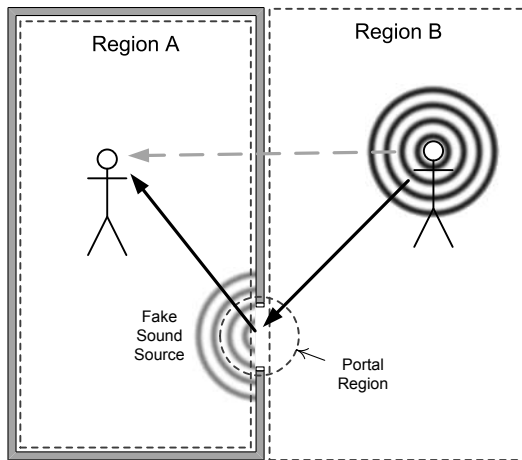


Figure 14.36. The portal-based audio propagation model used in *The Last of Us* by Naughty Dog, Inc.

14.4.5.5 Further Reading on Environmental Acoustics

Audio propagation modeling and acoustics analysis are areas of active research, and more and more advanced techniques are being applied in the game industry as hardware capabilities continue to improve. A few links are listed below to whet your appetite, but a Google search for “sound propagation” or “acoustics modeling” will provide many more hours of enjoyment!

- “Real-Time Sound Propagation in Video Games” by Jean-François Guay of Ubisoft Montreal (<https://bit.ly/2HdBiLc>);
- “Modern Audio Technologies in Games” presented at GDC 2003 by A. Menshikov (<https://bit.ly/2J7FYyD>);
- “3D Sound in Games” by Jake Simpson (<https://bit.ly/2HfVFTU>).

14.4.6 Doppler Shift

As we saw in Section 14.1.3.5, the Doppler effect is a change in frequency that’s dependent upon the *relative velocity* between source and listener: $\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{source}} - \mathbf{v}_{\text{listener}}$. This frequency change can be approximated by simply time-scaling the sound signal. This results in the “chipmunk effect” with which Alvin and the Chipmunks have made us all so familiar—by speeding up a sound, the pitch also rises. Because our sound signals are digital (i.e., sampled discrete-time signals), this kind of time scaling can be accomplished via sample rate conversion (see Section 14.5.4.4). However, this is not strictly the

correct thing to do, because the speeding up or slowing down of the sound can become noticeable.

The ideal solution is to apply a pitch shift without affecting the time axis. This can be done in a number of ways, including the *phase vocoder* and *time domain harmonic scaling* approaches. A complete description of these techniques is beyond our scope here, but you can read more about them at <http://www.dspdimension.com/admin/time-pitch-overview>.

Time-independent pitch shifting technology is an extremely powerful thing to have in your audio engine, in part because it also allows you to perform frequency-independent time scaling. So not only can you alter the pitch of sounds without changing timing for Doppler, you can also speed up or slow down sounds without altering their pitch for all sorts of other cool effects.

14.5 Audio Engine Architecture

To this point, we've discussed the concepts and methodologies behind 3D sound rendering, and the theory and technologies that underlie them. In this section, we'll turn our attention to the architecture of the software and hardware components used to implement a 3D audio rendering engine.

As with most computer systems, a game engine's audio rendering software is typically arranged into a "stack" of layered hardware and software components (see Figure 14.37).

- *Hardware* inevitably serves as the foundation of this structure, providing at minimum the necessary circuitry to drive the digital or analog speaker outputs that connect our PC or game console to a pair of headphones, a TV or a surround sound home theater system. Audio hardware may also provide "acceleration" to the software above it in the stack by supplying codecs, mixers, reverb tanks, effects units, waveform synthesizers and/or DSP chips in silicon. This hardware is often called the *sound card*

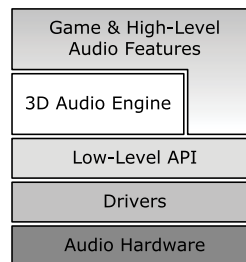


Figure 14.37. The audio hardware/software "stack."

because PCs sometimes provide their audio capabilities via a plug-in peripheral card.

- On a personal computer, the hardware is typically encapsulated in a *driver* layer, allowing the OS to support sound cards from a wide range of vendors.
- On both PCs and game consoles, the hardware and drivers are usually wrapped in a low-level *application programming interface* (API) designed to free the programmer from having to deal with the minutia of controlling the hardware and drivers directly.
- The *3D audio engine* itself is built on top of these foundations.

The feature set presented to the programmer by the audio hardware/software stack is usually modeled after the feature set of a *multi-channel mixer console* (http://en.wikipedia.org/wiki/Mixing_console) of the sort used in recording studios and at live concerts (see Figure 14.38). A mixer board can accept a relatively large number of audio inputs obtained from microphones and/or electronic instruments. The input sounds can be filtered and equalized, and reverb and other effects can be applied to them. The console is then used to mix all of the signals together, setting the relative volumes of the sounds as desired by the sound designer. The final mixed output is routed to the speakers (for a live performance) or to the individual channels of a multi-track recording.

In the same sense, the audio HW/SW stack must accept a large number of inputs (2D and 3D sounds), process them in various ways, mix them together so that their relative gains are set appropriately and finally pan these signals to the speaker output channels to produce the illusion of a three-dimensional soundscape for the human player.

14.5.1 The Audio Processing Pipeline

As we learned in Section 14.4.1, the process of rendering a 3D sound involves a number of discrete steps:

- For each 3D sound, a “dry” digital (PCM) signal must be *synthesized*.
- Distance-based attenuation is applied to provide a sense of distance from the listener, and *reverb* is applied to the signal to model the acoustics of the virtual listening space and to provide spatialization cues to the listener. This produces a new “wet” signal.
- The wet and dry signals are *panned* (independently) to one or more speakers in order to produce the final “image” of each signal in three-dimensional space.



Figure 14.38. A multi-channel mixer console by Focusrite with support for 72 inputs and 48 outputs.

- The panned multi-channel signals of all the 3D sounds are *mixed* together into a single multi-channel signal, which is either sent through a parallel bank of DACs and amps to drive the analog speaker outputs or sent directly to a digital output such as HDMI or S/PDIF.

Clearly, we think of the process of rendering 3D audio as a *pipeline*. And because a game world typically contains a large number of sound sources, multiple instances of this pipeline are in flight simultaneously. For this reason,

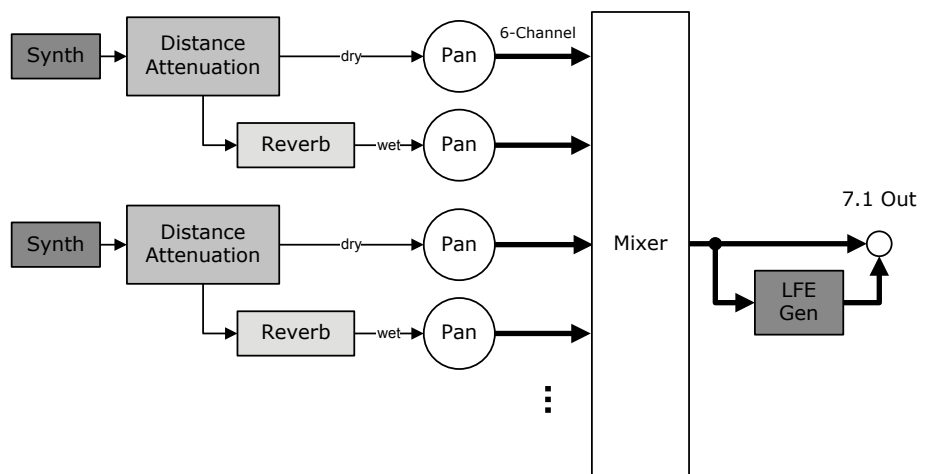


Figure 14.39. The audio processing graph (pipeline).

the audio processing pipeline is sometimes called the *audio processing graph*. It truly is a graph of interconnected components, ultimately culminating in the handful of speaker channels that comprise the final mixed, panned output. Figure 14.39 presents a high-level view of the audio graph.

14.5.2 Concepts and Terminology

Before we can explore the audio processing pipeline in any depth, we need to become familiar with a few concepts and the terminology used to describe them.

14.5.2.1 Voices

Each 2D or 3D sound passing through the audio rendering graph is called a *voice*. This term comes from the early days of electronic music: A synthesizer would produce musical notes via a set of waveform generators called “voices.”

A synthesizer contains a limited number of waveform generator circuits, so electronic musicians speak of how many simultaneous voices their synth can produce. In the same sense, a game’s audio rendering engine typically has a limited number of codecs, reverb units and so on. The maximum number of voices supported by a particular audio HW/SW stack is dictated by the number of independent parallel pathways through the audio graph. This number is generally bounded by limited memory resources, limited hardware resources and/or processing power limitations. This number is sometimes referred to as the degree of *polyphony* supported by the system.

2D Voices

A game’s audio rendering pipeline must also be capable of handling 2D sounds, such as music, menu sound effects, narrator voice-overs and so on. 2D voices are also processed by the audio rendering pipeline. The main things that differentiate 2D sound processing from 3D processing are:

- 2D sounds originate as multi-channel signals, one for each available speaker, whereas 3D sounds originate as dry monophonic signals. As such, 2D sounds do not pass through a pan pot.
- A 2D sound may contain “baked” reverb or other effects. If so, the sound may not make use of the reverb capabilities of the rendering engine.

As such, 2D sounds typically enter the pipeline just prior to the master mixer, where they are combined with the 3D sounds to produce the final “mix.”

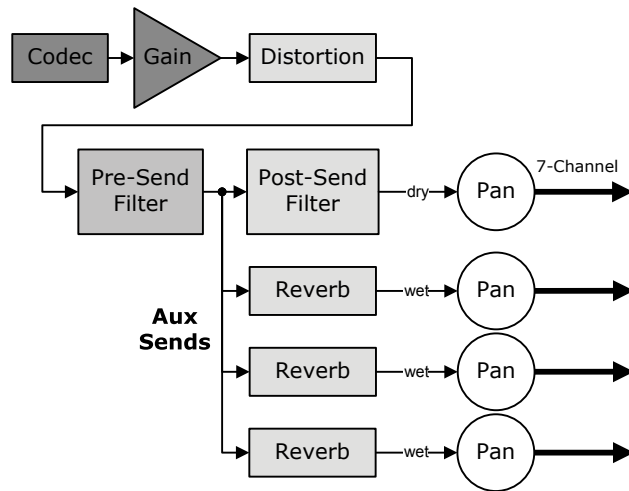


Figure 14.40. The pipeline through which an individual 3D voice passes on its way through the audio graph.

14.5.2.2 Buses

The interconnections between the components that make up the audio graph are called *buses*. In electronics, a bus is a circuit whose primary purpose is to connect other circuits to one another. In software, a bus is nothing more than a logical construct that describes the presence of an interconnection between components.

14.5.3 The Voice Bus

Figure 14.40 presents a more-detailed view of the pipeline of components through which a single 3D voice passes as it is rendered by the audio engine. In the following sections, we'll explore each of these components in detail and learn why they are interconnected in the way that they are.

14.5.3.1 Sound Synthesis: Codecs

An audio signal passes through the rendering graph in digital form. The term *synthesis* describes the process of generating these digital signals. Audio signals may be synthesized by simply “playing back” a pre-recorded audio clip. They might also be procedurally generated, perhaps by combining one or more fundamental waveforms (sinusoid, square wave, sawtooth, etc.), and/or by applying various filters to a harmonically rich noise signal. Since

most games use pre-recorded audio clips almost exclusively, we'll restrict our discussion to them here.

Pre-recorded audio clips can be provided to the game engine in any one of the myriad compressed and uncompressed audio file formats in use today (see Section 14.3.2.3). Raw PCM data is the “canonical” format accepted by the various components in the audio processing graph. Therefore, a device or software component known as a *codec* is used to convert each source audio clip into a raw PCM data stream. The codec interprets the source data format, decompresses the data if necessary, and then transmits it onto the voice bus for its journey through the audio processing graph.

14.5.3.2 Gain Control

The loudness of each source sound in the 3D world can be controlled in a number of ways: When recording the audio clip, we can set the recording levels to produce a sound at the desired loudness. We can process the clip in an offline tool to adjust its gain. At runtime, we can also dynamically adjust the volume of the clip using a *gain control* component within the audio graph. See Section 14.3.1.7 for a detailed discussion of gain control.

14.5.3.3 Aux Sends

When a sound engineer at a recording studio or live concert wants to apply effects to a sound, he or she can route the sound out of the multi-channel mixing console, through an effects “pedal,” and then back into the mixing board for further processing. These outputs are known as *auxiliary send* outputs, or *aux sends* for short.

Within the audio processing graph, the term “aux send” is used in an analogous manner: It describes a bifurcation point in the pipeline, splitting the signal into two parallel signals. One of these signals is for the dry component of the sound. The other is piped through a reverb/effects component to create the wet component of the sound.

14.5.3.4 Reverb

The wet signal path is typically routed through a component that adds early reflections and late reverberations. Reverb might be implemented using a convolution, as described in Section 14.4.5.1. If convolution is not practical in real time, either because the console or PC lacks DSP hardware or because the game's CPU and/or memory budgets are insufficient, reverb can be implemented using a *reverb tank*. This is essentially a buffering system that caches time-delayed copies of a sound that are then mixed with the original to mimic early reflections and/or late reverberations, combined with a *filter* to mimic the

interference effects and general attenuation of high-frequency components in the reflected sound waves.

14.5.3.5 Pre-Send Filter

The voice pipeline typically includes a filter that is applied before the aux send bifurcation, and therefore applies to both the dry and wet components of the sound. This is called a *pre-send filter*. It is generally used to model phenomena that occur at the source of the sound. For example, we could mimic the sound of someone wearing a gas mask with a pre-send filter.

14.5.3.6 Post-Send Filter

Another filter is typically provided after the aux send bifurcation. As such, this filter only applies to the dry component of the sound. This filter can be useful for modeling the muffling effect of an obstruction/occlusion on the direct sound path. At Naughty Dog, we also use a post-send filter to implement the frequency-specific fall-off that occurs due to atmospheric absorption (see Section 14.1.3.2).

14.5.3.7 Pan Pots

The dry and wet components of a 3D sound are monophonic signals throughout their journey along the voice bus. At the very end of the pipeline, each one of these two mono signals must be panned to the two stereo speakers/headphones or the five or seven surround sound speakers. For this reason, every 3D voice bus terminates in two or more pan pots, one for the dry signal and one or more for the wet. The components may be panned differently. The dry signal is panned according to the actual location of the source. The wet signal, however, may be panned with a wider focus to simulate the way in which reflected sound waves tend to impinge on the listener's head from various directions. If the sound is coming from a narrow doorway, the focus of the wet signal might be only a few degrees. But if the listener is standing in the center of a cavernous hall, the wet signal should probably be given a 360-degree focus (i.e., it should be rendered in all speakers equally).

14.5.4 Master Mixer

Each pan pot's output is a multi-channel bus, containing signals for each of the desired output channels (stereo or surround sound). A game typically has a large number of 3D sounds playing simultaneously. The *master mixer* takes all of these multi-channel inputs and mixes them together into a single multi-channel signal for output to the speakers.

Depending on the specifics of the implementation, the master mixer might be implemented in hardware, or it might live entirely in software. If the master mix is performed in hardware, the sound card designer has the option of performing an analog mix or a digital mix. (Software can only do digital mixing, for obvious reasons.)

14.5.4.1 Analog Mixing

An analog mixer is essentially just an summation circuit—the amplitudes of the individual input signals are added together, and the resultant wave’s amplitude is then attenuated to fall back within the desired signal voltage range.

14.5.4.2 Digital Mixing

Mixing can also be performed *digitally* by software running on a dedicated DSP chip or a general-purpose CPU. A digital mixer takes multiple PCM data streams as its inputs, and produces a single PCM data stream at its output.

A digital mixer’s job is a little more complicated than that of an analog mixer, because the collection of PCM channels it is combining may have been recorded at different sample rates and/or different bit depths. Two processes known as *sample depth conversion* and *sample rate conversion* must be executed on all of the mixer’s input signals to bring them into a common format. Once this has been done, mixing again becomes trivial. At each time index, the values of all the input samples are simply added together, and the final output amplitude is adjusted if necessary to bring the combined signal into the desired volume range.

14.5.4.3 Sample Depth Conversion

If the *bit depths* of the mixer’s input signals differ, *sample depth conversion* can be used to convert them to a common format. This operation is trivial. We simply de-quantize the input sample values into floating-point format, and then re-quantize each one at the desired output bit depth. See Section 12.8.2 for all the gory details on quantization.

14.5.4.4 Sample Rate Conversion

If the *sample rates* of the input signals differ, *sample rate conversion* must be used to convert them all into the desired output sample rate prior to mixing. In principle, this involves converting the signal into analog form, and then re-sampling it at the desired rate (which could be done using D/A and A/D hardware). In practice, analog sample rate conversion tends to introduce unwanted noise, so the conversion is almost always accomplished by running a direct digital-to-digital algorithm directly on the PCM data stream.

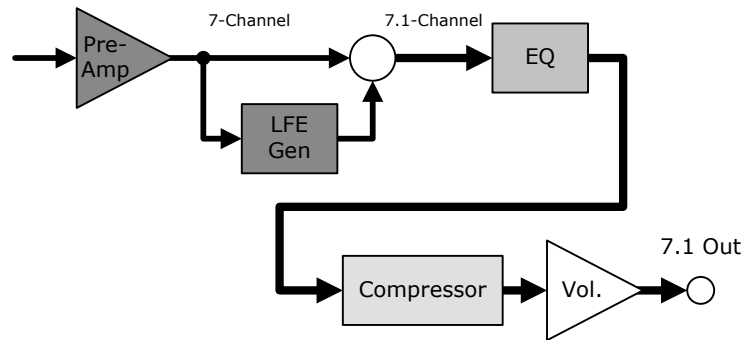


Figure 14.41. A typical master output bus.

An understanding of signal processing theory (see Section 14.2) is necessary to fully understand how these algorithms function, and a full discussion is beyond our scope here. But in certain simple cases, the concept is easy enough to grasp. For example, if we are *doubling* the sample rate, we can interpolate adjacent samples and insert these values as new samples, thereby doubling the number of samples. It's not quite as simple as this—one must take care to avoid introducing aliasing into the resulting signal, for example. See http://en.wikipedia.org/wiki/Sample_rate_conversion for a detailed discussion of sample rate conversion.

14.5.5 The Master Output Bus

Once the voices have been mixed, they are processed by the *master output bus*. This is a collection of components that process the output prior to sending it to the speakers. A typical master output bus is depicted in Figure 14.41, and its components are described briefly below. Every audio engine does things a bit differently, and not all engines support all of the components described below. Some engines may also introduce additional components not shown here.

- *Pre-amp.* The pre-amp allows the master signal's gain to be trimmed prior to passing through the remainder of the output bus.
- *LFE generator.* As we mentioned in Section 14.4.4, a pan pot only drives the two, five or seven “main” speakers of a stereo or surround sound system. The LFE (subwoofer) channel does not contribute to the positioning of a sound's 3D image. An *LFE generator* is a component that extracts the very lowest frequencies of the final mixed signal and uses this to drive the LFE channel.

- *Equalizer*. Most audio engines provide some kind of *equalizer* (EQ). As described in Section 14.2.5.8, an EQ allows specific frequency bands in the signal to be boosted or attenuated individually. A typical EQ divides the spectrum up into anywhere from four to tens of individually tunable bands.
- *Compressor*. A compressor performs *dynamic range compression* (DRC) on the audio signal. A compressor reduces the volume of the loudest portions of the signal and/or increases the volume of the quietest moments. It does this automatically by analyzing the input signal's volume characteristics and adjusting the compression dynamically. See http://en.wikipedia.org/wiki/Dynamic_range_compression for a detailed discussion of DRC.
- *Master gain control*. This component allows the overall volume of the entire game to be controlled.
- *Outputs*. The output of the master bus is a collection of line-level analog signals corresponding to the speaker channels and/or a digital HDMI or S/PDIF multi-channel signal, suitable for transmission to a TV or a home theater system.

14.5.6 Implementing a Bus

14.5.6.1 Analog Buses

An analog bus is implemented via a number of parallel electronic connections. To carry a monophonic audio signal, we need two parallel wires or “lines” on the circuit board: one to carry the voltage signal itself, and one to serve as ground.

An analog bus operates pretty much instantaneously. The output signal from an upstream component is immediately consumed by the next downstream component, because the signal is a continuous physical phenomenon. Such circuits are quite simple. The only real complication is ensuring that the voltage levels and impedances of the input and output signals match.

14.5.6.2 Digital Buses

One could imagine using simple digital circuitry to build instantaneous connections between our digital components. However, this would require the connected components to run in perfect lock-step: At the exact moment that the sender produces a byte of data, the receiver would have to consume it. Otherwise, the byte would be lost.

To overcome the synchronization problems inherent in connecting two digital components, *ring buffers* are typically used at the input and/or output of

each component. A ring buffer is a buffer that can be shared by two clients—one *reader* and one *writer*. To make this work, we maintain two pointers or indices within the buffer, called the *read head* and the *write head*. The reader consumes data at the read head, advancing it forward in the buffer as data is consumed, and wrapping when the end of the buffer is reached. The writer stores data into the buffer at the write head, advancing and wrapping as well. Neither head is permitted to “pass” the other, which guarantees that the two clients cannot conflict with one other (i.e., reading data that hasn’t been written yet, or writing over top of data that is currently being read).

The simplest way to connect, say, the digital output of a codec to the digital input of a DAC is to use a *shared* ring buffer. The codec writes to the exact same buffer read by the DAC.

While simple, the shared buffer approach only works when the two components have access to the same physical memory. This is trivial to do when the components are running in threads on a single CPU. To make a shared memory approach work between two separate operating system *processes*, each of which has its own private virtual memory space, the OS needs to provide a mechanism for mapping the same physical memory into each process’s virtual address space. This is usually only possible when the two processes are running on the same core, or on different cores within a multicore computer.

If the two components are running on different cores that cannot share memory (as would be the case if one were running on the PC and the other on a plug-in sound card, for example), then each component needs its own input or output buffer. Data must be *copied* from the output buffer of one component to the input buffer of the other. This might be accomplished via a *direct memory access controller* (DMAC), as is the case when transferring data between the PPU and the SPUs on the PS3. It might also be accomplished via a specialized bus, such as the ubiquitous PCI Express (PCIe) bus that is used to connect the main CPU core(s) to plug-in peripheral cards on a PC.

14.5.6.3 Bus Latency

In order to play sound, the game or application must feed audio data periodically into the codecs that ultimately drive the speaker outputs. We call this *servicing* the audio. The rate at which the game or app services its audio is crucial to proper sound production: If packets are sent too infrequently, the buffers will *underflow*, meaning that the device consumes all of the data before a new packet arrives. This causes the audio to drop out briefly while the software catches up. If packets are sent too often, the PCM buffers can *overflow*, causing packets to be lost. This causes the audio to seem to “skip.”

The size of the input and output buffers that comprise a digital bus dictates the *latency* of the sound system—in other words, how much delay is introduced by the bus. If the buffers are very small, latency is minimized, but this places a greater burden on the CPU because it must feed the buffers more frequently. Likewise, larger buffers translate into less CPU load but also higher latency. We usually measure the latency of a piece of audio hardware in milliseconds, rather than measuring the size of the buffers in bytes. This is done because buffer size depends upon the data format and the degree of compression supported by the codec, but the latency is really what we care about when trying to produce high-fidelity sound.

How much latency is acceptable? This depends on the application. Professional audio systems require very short latencies—on the order of 0.5 ms. This is because audio signals are often fed through a network of audio hardware before being synchronized with each other and often to a video signal as well. Every time latency is introduced by the hardware, accurate synchronization becomes more difficult.

Game consoles, on the other hand, can tolerate a longer latency. In a game, all we care about is synchronizing the audio and the graphics. If the game is rendering at 60 FPS, this translates into $1/60 = 16.6$ ms per frame. As long as the audio isn't delayed by more than 16 ms, we know it will be in sync with graphics rendered for that same frame. (In fact, many games use double or triple buffering for their rendering engines, which introduces one or two frames of delay between the time the game requests that a frame be drawn and when that frame will actually appear on the TV screen. The television may also introduce a delay. As such, a triple-buffered 60 Hz game can actually tolerate an audio latency of $3 \times 16 = 48$ ms or more.) The PlayStation 3's DMA controller runs every 5.5 ms, so PS3 audio systems are typically configured such that the audio buffers can hold an integer multiple of 5.5 ms worth of audio.

14.5.7 Asset Management

14.5.7.1 Audio Clips

The most atomic audio asset is a *clip*—a single digital sound asset with its own local timeline (analogous to an animation clip). A clip is sometimes called a *sound buffer*, because the digital sample data is stored in a buffer. A clip might encapsulate monophonic audio data (typical for 3D sound assets), or it might contain multi-channel audio (typically used for 2D assets or stereo sound sources in 3D). A clip may be stored in any of the audio file formats supported by your engine.

14.5.7.2 Sound Cues

A *sound cue* is a collection of audio clips plus metadata that describes how they should be processed and played. Cues are usually the primary means by which the game can request sounds to be played. (Playing individual clips may or may not be supported by the engine.) Cues also serve as a convenient division-of-labor mechanism: The sound designers can craft the cues using an offline tool, without having to worry about how or when they'll be played in-game. And the game programmers can play the cues conveniently in response to relevant events in the game, without having to worry about micromanaging the details of playback.

There are many ways in which the collection of clips in a cue could be interpreted and played back. A cue might contain clips representing the six channels of a pre-mixed 5.1 music recording. A cue might also collect up a set of raw sounds, from which a random selection can be made, for the sake of variety. A cue might also be set up to play its collection of raw sounds in a predefined sequence. A cue typically specifies whether the sound(s) it encapsulates represent a one-shot sound or a looping sound.

Some audio engines permit a cue to provide one or more optional sound clips that only play if the main sound is *interrupted* midway through playing. For example, a vocal cue might include a “glottal stop” sound that is played only if the person's line of dialog is interrupted. This feature can also be used to provide a distinct “tail” sound that is played when a looping cue is stopped. For example, a looping machine gun sound cue might use this “tail” clip feature to produce a suitable echoing fall-off sound when the firing ceases.

A cue's metadata might include whether it is intended to be played in 3D or 2D; the FO min, FO max and fall-off curve of the sound source; group membership (see Section 14.5.8.1); and possibly any special effects, filtering or equalization that should be applied when the sound is played. In Sony's *Scream* engine—the sound engine used by Naughty Dog in its *Uncharted* and *The Last of Us* series—a cue can contain arbitrary script code that allows a sound designer to completely control how the encapsulated sound asset(s) are played when the cue plays.

Playing a Cue

Every audio engine that supports the concept of cues provides an API for playing them. This API is usually the primary way—and sometimes the *only* way—for the game code to request that a sound be played.

The cue playing API generally allows the programmer to specify whether the cue should be played as a 2D or 3D sound, to provide 3D position and

velocity parameters, to specify whether the sound should loop or play only once, and to specify whether the source buffer is in-memory or streamed. The API usually also allows us to control the volume of the sound and possibly other aspects of playback.

Most APIs return a *sound handle* to the caller. This handle allows the program to keep track of a sound as it is playing, so that it can be modified or canceled before the sound ends. A sound handle is usually implemented under the hood as an index into a global handle table, rather than as a raw pointer to the data that describes the sound instance. That way, if the sound ends naturally, the handle can be “nulled out” automatically. A handle mechanism can also be used to make the system thread-safe—if one thread kills the sound, other threads that have handles to the sound will automatically see their handles become invalid.

14.5.7.3 Sound Banks

A 3D audio engine manages a *lot* of assets. The game world contains a large number of objects. Each object can generate a variety of sounds. And in addition to 3D sound effects, we have music, speech, menu sound effects and so on.

All of this audio data takes up an immense amount of space, so we can’t keep it all in memory at once. On the other hand, the individual audio clips are too fine-grained and too numerous for them to be managed on an individual basis. As such, most game engines package their sound clips and cues into coarse units called *sound banks*.

Some sound banks are loaded when the game starts up and left in memory forever. For example, the collection of sounds made by the player character are always needed, so we could keep them in memory indefinitely. Other banks might be loaded and unloaded dynamically as the needs of the game change. For example, the sounds in level A might not be used in level B, so we can load the “A” bank only when level A is being played. For example, in Naughty Dog’s *The Last of Us*, the sounds of rain, flowing water and the creaking of beams on the verge of collapse were only loaded when the player was in the tilted building in Boston.

Some sound engines permit banks to be *relocated* in memory. This feature can entirely eliminate the memory fragmentation problems that would otherwise arise as lots of differently sized banks are loaded and unloaded during gameplay. See Section 6.2.2.2 for more information on memory relocation.

14.5.7.4 Streaming Sounds

Some sounds are so long in duration that they cannot be conveniently stored in memory all at once. Music and speech are common examples. For these kinds of sounds, many game engines support *streaming audio*.

Streaming audio is possible because when playing a sound, the only information we actually need is the signal data at and around the current time index. To implement streaming, we maintain a relatively small ring buffer for each streaming sound. Prior to playing the sound, we pre-load a small chunk of it into the buffer, then play the sound normally. The audio pipeline consumes the data from the ring buffer as it plays, making room for us to load more data. As long as we keep filling the buffer with data before it is all consumed, our sound will play seamlessly.

14.5.8 Mixing Your Game

If we were to play every sound coming from every game object, properly attenuated and spatialized and acoustically modeled, using all the techniques and technology we've discussed thus far, what would be the result? We might expect the answer to this question to be "an incredibly immersive and believable soundscape that wins awards and makes us rich!" But what we'd actually get is cacophony.

What separates a good game from a great game is the *mix*—what you hear, how much of it you hear, and just as importantly what you don't hear. The goal of a game's sound designer is to produce a final mix that:

- sounds realistic and emersive;
- isn't too distracting, annoying or difficult to listen to;
- conveys all information relevant to gameplay and/or story effectively; and
- maintains a mood and tonal color that is always appropriate, given the events taking place in the game and the overall design of the game.

All sorts of different kinds of sounds must come together in the game's mix. These include music, speech, ambient sounds like rain, wind, insects or the creak of an old building, sound effects such as weapons fire, explosions and vehicles, and the bumps, slides and rolling sounds made by physically simulated objects.

Various techniques are employed to ensure the mix of the game meets these goals. We'll explore a few of them in the following sections.

14.5.8.1 Groups

The most obvious thing we can do to improve the mix of our game is to set the levels of all the source sounds in the 3D world appropriately. The important thing here is to ensure that each sound's gain is appropriate *relative* to the other sounds in the game. For example, footsteps should be quieter than gunfire.

In some games, the loudness of certain sounds needs to change dynamically. Often we want to control an entire category of sounds at once. For example, during a frenetic fight sequence, we might want to bring up the levels of the music and the weapons, and drop the volume of ancillary sound effects. Or during a quiet moment with characters talking to one another, we might want to boost the speech a little and tone down ambient sounds to ensure the dialog can be heard.

For this reason, many audio engines support the concept of *groups*—a concept “stolen” from our old friend the multi-channel mixing console. On a mixing board, a collection of sound inputs can be routed to an intermediate mixer circuit, combining them into a single “group signal.” The gain of this signal can then be controlled with a single knob on the board, thereby allowing the sound engineer to control the loudness of all input signals at once.

In the software world, groups are implemented by simply categorizing sound cues, rather than physically mixing their signals together. For example, we can classify a cue as being music, a sound effect, a weapon, a line of speech and so on. Then, the engine can provide a means of controlling the gains of all sounds in each category with a single control value. Groups usually also allow entire categories of sound to be paused, restarted and muted conveniently with a simple API call.

Some sound engines do also provide a mechanism for physically mixing groups of audio signals into a single signal, just as is done when working with groups on a mixing console. In Sony's *Scream* engine, this is called generating a *pre-master submix*. After the relative gains of the signals in the group have been locked down by the submix, the resulting signal can be routed through additional filters or other processing stages. This gives the sound designer even more control over the mix of the game.

14.5.8.2 Ducking

Ducking is a technique in which the volume/gain of certain sounds are temporarily reduced in order to make other sounds more audible. For example, when a character is speaking, the level of background noise could be reduced automatically to make the dialog more audible.

Ducking can be triggered in numerous ways. The presence of one particular type of sound might be used to duck another category of sounds. A game event might trigger a duck programmatically. Any triggering mechanism that is deemed appropriate can be used to initiate a duck.

The reduction of volume caused by a duck is typically accomplished via the group categorization system: When one category of sound is playing, it can automatically duck one or more other categories by various amounts. Or the game code can call a function to duck a group of sounds programmatically.

Ducking can also be performed by routing one sound signal into the *side-chain* input of the dynamic range compressor (DRC) on a different voice's bus. Recall from Section 14.5.5 that a DRC analyzes the volume characteristics of a signal and automatically compresses the loudness of the signal appropriately. When a side-chain input is connected to a DRC, it analyzes the *side-chain* signal when deciding how to adjust the volume. So, we can arrange for increased loudness in one signal to cause a decrease in the dynamic range of another signal.

14.5.8.3 Bus Presets and Mix Snapshots

Many sound engines allow the sound designer to set up configuration parameters, save them off and then recall and apply them conveniently at runtime. In Sony's *Scream* engine, these come in two basic flavors: *bus presets* and *mix snapshots*.

A bus preset is a set of configuration parameters that control aspects of the components on a single bus (voice bus or master output bus). For example, a bus preset might describe one particular reverb set-up that mimics, say, the acoustics of a large open hall, or the interior of a car, or a small broom closet. Or a bus preset might control the DRC settings on the master output bus. Many such presets can be created by the sound designer, and the appropriate ones activated at runtime as the game requires.

A mix snapshot is the same kind of idea applied to gain control. The gains of the various channels within a group can be established a priori and then applied at runtime as needed.

14.5.8.4 Instance Limiting

Instance limiting is a means of controlling the number of sounds that are permitted to play simultaneously. For example, even though 20 NPCs are all firing their guns at once, we might only play the three or four gun sounds that are nearest to the listener. Instance limiting is important for two reasons: First, it's a great way to prevent cacophony. Second, a sound engine typically supports only a fixed number of simultaneous voices, either because of hardware

limitations (e.g., the sound card only has N codecs) or because of memory or processor bandwidth limitations in the software, so we must use them wisely.

Per-Group Limits

Instance limiting is sometimes applied differently to different groups of sounds. For example, we might specify that we should play up to four guns simultaneously, hear up to three people speaking at a time and allow up to five other sound effects to play at once, plus up to two overlapping music tracks.

Prioritization and Voice Stealing

In a 3D game with lots of dynamic elements, there may be more sounds playing at any given time than the system has voices for. Some sound engines support a large number (or even an infinite number) of *virtual voices*. Each virtual voice represents a sound that is technically playing, but that can be muted or stopped temporarily so it ceases to occupy valuable hardware or software resources. The engine uses various criteria to dynamically determine which virtual voices should be mapped to “real” voices at any given moment.

One of the simplest ways to limit the number of sounds playing simultaneously is to assign a maximum radius to every 3D sound source. As we saw in Section 14.4.3.1, this is the FO max radius. If the listener is beyond this distance from the sound, it is considered inaudible and its virtual voice is temporarily muted or stopped, freeing its resources for use by other voices. The process of automatically silencing virtual voices is called *voice stealing*.

Another common approach is to assign each cue or group of cues a *priority*. When too many virtual voices are playing at once, those with lower priorities can be silenced (stolen) in favor of higher priority voices.

Sound engines may also provide various other mechanisms for controlling the details of the voice stealing algorithm. For example, a cue might be given a minimum play time, after which its voice is permitted to be stolen. Sounds might be faded out rather than cut off abruptly when their voice is stolen. And some cues might be temporarily marked as “unstealable” to ensure that they play, even despite their priority settings.

14.5.8.5 Mixing In-Game Cinematics

Under normal gameplay conditions, the listener or “virtual microphone” is typically positioned at or near the location of the camera, and the sound sources are modeled where they really are in the environment. Distance-based attenuation, direct and indirect sound path determination, voice limiting—all

are determined using these realistic positions.

However, during an *in-game cinematic*—a portion of the game in which player control is suspended so that a story moment can take place—the camera often pans out away from the player’s head. This kind of thing tends to wreak havoc with our 3D audio system. We could just keep the listener/mic locked to the location of the camera; but this is not always appropriate. For example, if there’s a long shot of two characters speaking, we probably still want to mix so that the characters’ voices can be heard, even though physically speaking they are too far away to be heard. In this case, we might want to detach the listener from the camera, and artificially position it nearer to the characters.

Mixing in-game cinematics is a lot closer to mixing a film. As such, a sound engine needs to be capable of “breaking the rules” and doing things that aren’t necessarily physically realistic.

14.5.9 Audio Engine Survey

It should be evident by now that creating a 3D audio engine is a massive undertaking. Luckily for us, lots of people have already put a great deal of effort into this task, and the result is a wide range of audio software that we can use pretty much out of the box. This ranges from low-level sound libraries all the way to fully featured 3D audio rendering engines.

In the following sections, we’ll survey a few of the most common audio libraries and engines. Some of these are specific to a particular target platform, while others are cross-platform.

14.5.9.1 Windows: The Universal Audio Architecture

In the early days of PC gaming, the feature set and architecture of PC sound cards varied a great deal from platform to platform and vendor to vendor. Microsoft attempted to encapsulate all of this diversity within its DirectSound API, supported by the Windows Driver Model (WDM) and the Kernel Audio Mixer (KMixer) driver. However, because vendors could not agree on a common feature set or set of standard interfaces, the same functionality would often be realized in very different ways on different sound cards. This required the operating system to manage a very large number of incompatible driver interfaces.

For Windows Vista and beyond, Microsoft introduced a new standard called the Universal Audio Architecture (UAA). Only a limited set of hardware features are supported by the standard UAA driver API—all remaining features are implemented in software (although hardware manufacturers

are still free to provide additional “hardware acceleration” features, as long as they provide custom drivers to expose them). Although the introduction of UAA limited the competitive advantage of prominent sound card vendors like Creative Labs, it did have the desired effect of creating a solid, feature-rich standard, which could be used by games and PC applications in a convenient way.

The UAA standard had another positive effect on the user’s aural experience. In the old DirectSound days, a game could take complete control of the sound card, meaning that sounds coming from the OS or other applications such as an email program would be “locked out” and their sounds would not play while the game was running. The new UAA architecture allowed the OS to claim ultimate control over the final mix heard through the PC’s speakers. Multiple applications could finally share the sound card in a reasonable and consistent way. Search online for “Universal Audio Architecture” to find more information on UAA.

The UAA is implemented on Windows by the so-called Windows Audio Session API, or WASAPI for short. This API is not really intended for use by games. It supports most advanced audio processing features in software only, with limited support for hardware acceleration. Instead, games usually make use of the XAudio2 API, which is described in the next section.

14.5.9.2 XAudio2

XAudio 2 is the high-powered low-level API that provides access to the audio hardware on Xbox 360, Xbox One and Windows. It replaces DirectAudio and provides access to a wide range of hardware-accelerated features including programmable DSP effects, submixing, support for a wide range of compressed and uncompressed audio formats, and multirate processing to lighten the load on the main CPU(s).

Atop the XAudio2 API sits a 3D audio rendering library called X3DAudio. These APIs are also available on the Windows platform for use by PC games. Microsoft used to offer a powerful audio authoring tool called the “cross-platform audio creation tool” or XACT for short, which was intended for use with XNA Game Studio, but neither XNA nor XACT are supported any longer.

14.5.9.3 Scream and BoomRangBuss

On the PS3 and PS4, Naughty Dog uses Sony’s 3D audio engine Scream and its synth library, BoomRangBuss.

The audio hardware on a PlayStation 3 is very much like a UAA-compliant audio device, supporting up to eight channels of audio for full 7.1 surround

sound support, plus a hardware mixer and HDMI, S/PDIF, analog and USB/Bluetooth outputs. This audio hardware is encapsulated by a collection of OS-level libraries including libaudio, libsynth and libmixer. On top of these libraries, game makers are free to implement their own audio software stacks. Sony also provides a powerful 3D-capable audio stack of its own called *Scream* which game studios can use “out of the box.” *Scream* is available on the PS3, PS4 and PSVita platforms. Its architecture mimics a fully featured multi-channel mixer console.

On top of *Scream*, Naughty Dog implemented a proprietary 3D environmental audio system for use on the *Uncharted* and *The Last of Us* series. This system provides stochastic obstruction/occlusion modeling and a portal-based audio rendering system that permits rendering a highly realistic soundscape.

Advanced Linux Sound Architecture

The Linux equivalent of the UAA driver model is called the Advanced Linux Sound Architecture (ALSA). This Linux kernel component replaced the original Open Sound System (OSSv3) as the standard way to expose audio functionality to applications and games. See http://www.alsa-project.org/main/index.php/Main_Page for more information on ALSA.

QNX Sound Architecture

QNX Sound Architecture (QSA) is a driver-level audio API for the QNX Neutrino real-time OS. As a game programmer, you’ll probably never use QNX. But its documentation does provide an excellent picture of the concepts and the typical feature set of audio hardware. See http://www.qnx.com/developers/docs/6.5.0/index.jsp?topic=%2Fcom.qnx.doc.neutrino_audio%2Fmixer.html for these docs.

14.5.9.4 Multiplatform 3D Audio Engines

A number of powerful, ready-to-use cross-platform 3D audio engines are available. We’ll outline the most well-known of these below.

- *OpenAL* is a cross-platform 3D audio rendering API that has been deliberately designed to mimic the design of the OpenGL graphics library. Early versions of the library were open source, but it is now licensed software. A number of vendors provide implementations of the OpenAL API spec, including OpenAL Soft (<http://kcat.strangesoft.net/openal.html>) and AeonWave-OpenAL (<http://www.adalin.com>).

- *AeonWave 4D* is a low-cost audio library for Windows and Linux by Adalin B.V.
- *FMOD Studio* is an audio authoring tool that features a “pro audio” look and feel (<http://www.fmod.org>). A full-featured runtime 3D audio API allows assets created in FMOD Studio to be rendered in real time on the Windows, Mac, iOS and Android platforms.
- *Miles Sound System* is a popular audio middleware solution by Rad Game Tools (<http://www.radgametools.com/miles.htm>). It provides a powerful audio processing graph and is available on virtually every gaming platform imaginable.
- *Wwise* is a 3D audio rendering engine by Audiokinetic (<https://www.audiokinetic.com>). It is notably not based around the concepts and features of a multi-channel mixing console, but rather presents the sound designer and programmer with a unique interface based on game objects and events.
- The *Unreal Engine* of course provides its own 3D audio engine and powerful integrated tool chain (<http://www.unrealengine.com>). For an in-depth look at Unreal’s audio feature set and tools, see [45].

14.6 Game-Specific Audio Features

On top of the 3D audio rendering pipeline, games typically implement all sorts of game-specific features and systems. Some examples include:

- *Split-screen support*. Multiplayer games that support split-screen play must provide some mechanism that allows *multiple listeners* in the 3D game world to *share* a single set of speakers in the living room.
- *Physics-driven audio*. Games that support dynamic, physically simulated objects like debris, destructible objects and rag dolls require a means of playing appropriate audio in response to impacts, sliding, rolling and breaking.
- *Dynamic music system*. Many story-driven games require the music to adapt in real time to the mood and tension of events in the game.
- *Character dialog system*. AI-driven characters seem a great deal more realistic when they speak to one another and to the player’s character.
- *Sound synthesis*. Some engines continue to provide the ability to synthesize sounds “from scratch” by combining various kinds of waveforms (sinusoid, square, sawtooth, etc.) at various volumes and frequencies. Advanced synthesis techniques are also becoming practical for use in real-time games. For example:

- *Musical instrument synthesizers* reproduce the natural sound of an analog musical instrument without the use of pre-recorded audio.
 - *Physically based sound synthesis* encompasses a broad range of techniques that attempt to accurately reproduce the sound that would be made by an object as it physically interacts with a virtual environment. Such systems make use of the contact, momentum, force, torque and deformation information available from a modern physics simulation engine, in concert with the properties of the material from which the object is made and its geometric shape, in order to synthesize suitable sounds for impacts, sliding, rolling, bending and so on. Here are just a few links to research on this fascinating topic: <http://gamma.cs.unc.edu/research/sound>, http://gamma.cs.unc.edu/AUDIO_MATERIAL, <http://www.cs.cornell.edu/projects/sound>, and <https://ccrma.stanford.edu/~bilbao/booktop/node14.html>.
 - *Vehicle engine synthesizers* aim to reproduce the sounds made by a vehicle, given inputs such as the acceleration, RPM and load placed on a virtual engine, and the mechanical movements of the vehicle. (The vehicle chase sequences in Naughty Dog's three *Uncharted* games all used various forms of dynamic engine modeling, although technically these systems were not synthesizers, because they produced their output by cross-fading between various pre-recorded sounds.)
 - *Articulatory speech synthesizers* produce human speech "from scratch" via a 3D model of the human vocal tract. VocalTractLab (<http://www.vocaltractlab.de>) is a free tool that allows students to learn about and experiment with vocal synthesis.
- *Crowd Modeling.* Games that feature crowds of people (audiences, city dwellers, etc.) require some means of rendering the sound of that crowd. This is not as simple as playing lots and lots of human voices over top of one another. Instead, it is usually necessary to model the crowd as multiple layers of sounds, including a background ambience plus individual vocalizations.

We can't possibly cover everything from the above list in one chapter. But let's spend a few more pages covering some of the most common game-specific features.

14.6.1 Supporting Split-Screen

Supporting split-screen multiplayer is a tricky problem, because you have *multiple listeners* in the virtual game world, but they must *share* a single set of speakers in the player's living room. If you simply pan the sounds multiple times, once for each listener, and then mix the results into the speakers evenly, the result won't always sound sensible. There is no perfect solution: For example, if player A is standing right next to an explosion and player B is standing far away, the person playing player B will still hear the explosion loud and clear. The best a game can do is cobble together a hybrid solution, in which some sounds are handled in a "physically correct" way and others are "fudged" in order to produce the most sensible-sounding experience for the players.

14.6.2 Character Dialog

Even if we've created characters for our game that look like photographs of real human beings, and even if they move in astoundingly realistic ways, they still won't seem real to the player until they can *speak* realistically. Speech communicates information crucial to gameplay. It's a central story-telling tool. And it cements the emotional bond between the human player and the characters in the game. Speech can also be the deciding factor in the player's perception of *intelligence* among the AI-controlled characters in a game.

At the Game Developer's Conference (GDC) in 2002, Chris Butcher and Jaime Griesemer of Bungie gave a talk entitled, "The Illusion of Intelligence: The Integration of AI and Level Design in Halo" (<http://bit.ly/1g7FbhD>). In their talk, they shared an anecdote about how important speech can be to communicating the motivations of an AI-driven character to the player. In *Halo*, when the Elite leader of a squad of Covenant was killed, the grunts would all run away in fear. In playtest after playtest, no one seemed to understand that it was the killing of the Elite that had triggered the grunts to flee. Finally, the grunts were given lines of dialog saying something to the effect of, "Leader dead—run away!" Only then did play testers start to really grok what was going on!

In this section, we'll explore some of the fundamental subsystems you'll find in the character dialog system of pretty much any character-based game. We'll also discuss some of the specific techniques and technologies used by Naughty Dog to create rich, realistic conversations in *The Last of Us*. For more information and in-game videos of Naughty Dog's character dialog system in action, check out the talk I gave at GDC 2014 entitled, "Context-Aware Character Dialog in *The Last of Us*," available in PDF and QuickTime formats at

<http://www.gameenginebook.com>.

14.6.2.1 Giving a Character a Voice

It's easy enough to give a game character a voice—simply play an appropriate pre-recorded sound whenever the character needs to speak. However, things are never quite that simple. The dialog system in a game engine is typically a reasonably complex beast. Here are just a few of the reasons why:

- We need a way to catalog all the possible lines of dialog that each character might be called upon to say, and give each of these lines some kind of unique id so that the game can trigger them when needed.
- We need to make sure that each uniquely identifiable character in the game speaks with a recognizable and consistent voice. For example, each of the hunters in the Pittsburgh section of *The Last of Us* was assigned to one of eight unique voices, so that no two hunters in a battle would sound the same.
- We may not know a priori which character is going to be called upon to say a specific line, so we often need to record the same line multiple times, spoken by various voice actors, so that the appropriate voice can be used to say the line when needed.
- We also usually want a lot of variety in the things that are said. So most dialog systems provide a means of selecting specific lines at random from a pool of possibilities.
- Speech audio assets tend to be of relatively long duration, which means they occupy a lot of memory. Many lines of dialog are part of cinematic sequences, and hence are only spoken once in the entire game. For these reasons, it's usually wasteful to store speech assets in memory. Instead, it's typical for speech audio assets to be *streamed* on demand (see Section 14.5.7.4).

Usually other vocal sounds, like the “efforts” made when lifting something heavy, jumping over an obstacle or getting punched in the gut, are handled by the same system that handles spoken dialog. This is done largely because a character's efforts need to match his or her spoken voice. So we may as well leverage the dialog system to produce effort sounds as well.

14.6.2.2 Defining a Line of Dialog

Most dialog systems introduce a level of indirection between a *request* to speak and the choice of the particular *audio clip* to play. The game programmer or designer requests *logical* lines of dialog, each of which is represented by a unique

identifier such as a string, or better yet a hashed string id (see Section 6.4.3.1). The sound designers can then “fill out” each logical line with one or more audio clips in order to provide the necessary variety both in voice quality and in terms of what exactly is said.

For example, let’s imagine a logical line in which the character says something to the effect of, “I’m out of ammo.” We’ll assign this logical dialog line the unique id `'line-out-of-ammo`, where the leading single quote indicates a hashed string id. Let’s assume also that there are ten different characters that might say this line: the player character (call him “drake”), the player’s sidekick (call her “elena”) and up to eight enemy characters (call them “pirate-a” through “pirate-h”). We’ll need some kind of data structure to define all the physical audio assets that make up this one logical line of dialog.

At Naughty Dog, sound designers use the Scheme programming language to define logical dialog lines using a custom syntax. We’ll use a similar syntax in our example below. However, the specifics of the implementation are not important here. All we’re interested in is the structure of the data itself:

```
(define-dialog-line 'line-out-of-ammo
  (character 'drake
    (lines
      drk-out-of-ammo-01    ;; "Dammit, I'm out!"
      drk-out-of-ammo-02    ;; "Crap, need more bullets."
      drk-out-of-ammo-03    ;; "Oh, now I'm REALLY mad."
    )
  )
  (character 'elena
    (lines
      eln-out-of-ammo-01    ;; "Help, I'm out!"
      eln-out-of-ammo-02    ;; "Got any more bullets?"
    )
  )
  (character 'pirate-a
    (lines
      pira-out-of-ammo-01    ;; "I'm out!"
      pira-out-of-ammo-02    ;; "Need more ammo!"
      ;; ...
    )
  )
  ;; ...
  (character 'pirate-h
    (lines
      pirh-out-of-ammo-01    ;; "I'm out!"
      pirh-out-of-ammo-02    ;; "Need more ammo!"
```

```

        )
    )
)

```

Rather than define your dialog lines in one monolithic data structure like the one shown above, it's usually better to break the lines out into separate files by character. For example, all of Drake's lines can be managed in one file, Elena's in another file and all the pirates' lines can be stored in a third file. This helps to prevent the sound designers from stepping on each others' toes. It also means that we can manage our memory more efficiently. For example, if there are no pirates in a given section of the game, there's no need to keep the data for the pirates' dialog lines in memory. It's also a good idea to split the dialog data up by level, for the same reason.

14.6.2.3 Playing a Line of Dialog

Given this data, the dialog system can easily convert a request for a logical line of dialog such as 'line-out-of-ammo' into a specific audio clip. It simply looks up the character's specific voice id in the table, and then makes a random choice amongst the various possible lines for that character.

It's usually a good idea to put in place some kind of mechanism to ensure that lines aren't repeated too often. One way to accomplish this is to store the indices of the various lines in an array and then randomly shuffle its contents. To select a line, we simply cycle through the shuffled array in order. Once all possible lines have been exhausted, we reshuffle the array, taking care that the most recently played line doesn't end up in the first slot. This prevents all repetition while keeping the line selections sounding random.

Dialog line requests are typically made by gameplay code in C++, Java, C# or whatever language your game is written in. Game designers may also request lines of dialog via script (Lua, Python, etc.). The dialog system's API is usually designed with simplicity of use in mind. If an AI programmer or game designer has to jump through a lot of hoops just to get a line of dialog to play, you may discover that your characters are uncannily silent! It's best to provide a simple, fire-and-forget interface. Leave all the hard work to the programmer who is crafting the dialog system.

For example, in *Uncharted 3: Drake's Deception*, C++ code could request a character to play a line of dialog by calling a simple `PlayDialog()` member function of the `Npc` class. These calls would be peppered throughout the AI decision-making code in order to trigger appropriate lines of dialog at key moments in the game. For example:

```

void Skill::OnEvent(const Event& evt)
{
    NPC* pNPC = GetSelf(); // grab a pointer to the NPC

    switch (evt.GetMessage())
    {
    case SID("player-seen"):
        // play a line of dialog...
        pNPC->PlayDialog(SID("line-player-seen"));
        // ... and move to closest cover
        pNPC->MoveTo(GetClosestCover());
        break;
    // ...
    }

    // ...
}

```

14.6.2.4 Priority and Interruption

What happens if a character is asked to speak while he's already speaking? What if he receives more than one speech command on the same frame? In both cases, a *priority system* is a good way to resolve ambiguities.

To implement such a system, we simply assign each line of dialog to a priority level. When a request to say a line of dialog comes in, the system looks at the priority of the currently playing line if any, and the priorities of the line or lines that have been requested this frame. It finds the highest priority line among these. If the currently playing line “wins,” it continues to play and the requested lines are ignored. If one of the requested lines is higher priority than the current line, or if the character isn't speaking yet, the new line plays, interrupting the current line if necessary.

Implementing the interruption of the speech itself is actually a bit tricky. We can't perform a cross-fade (i.e., fade the volume of the playing sound down and the new sound up) because this sounds strange and wrong when applied to the speech of a single character. Ideally, we'd want to play at least some kind of glottal stop sound just prior to starting the new line. It might even be appropriate to play a short phrase indicating that the character is surprised and/or annoyed by the interruption, and *then* play the new line of dialog. The dialog system in *The Last of Us* doesn't do any of these fancy things. It simply stops the current line and immediately plays the new one. This sounds pretty good most of the time. Of course, each game has its own unique speech patterns, and what works in one game may not work well in another. So as the saying goes, “Your mileage may vary.”

14.6.2.5 Conversations

In *The Last of Us*, Naughty Dog wanted the enemy NPCs to sound like they're having real conversations with one another. This meant that the characters would need to be capable of saying relatively long chains of lines, with back-and-forth banter between two or more characters. Likewise, in *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy*, we wanted the characters to have conversations while driving around Madagascar and India in their jeep. These conversations could even be interrupted (for example by the player deciding to exit the jeep to explore the area), and would continue where they left off when the player returned to the vehicle.

Conversations in *The Last of Us*, *Uncharted 4* and *The Lost Legacy* are constructed from logical *segments*. Each segment corresponds to one logical line, spoken by one particular actor in the conversation. Each segment is given a unique id, and the segments are *chained* together into a conversation via these ids. As an example, let's see how we would define the following conversation:

A: "Hey, did you find anything?"

B: "No, I've been looking for an hour and I ain't found nothin'."

A: "Well then shut up and keep looking!"

This conversation could be expressed in the Naughty Dog conversation system as follows:

```
(define-conversation-segment 'conv-searching-for-stuff-01
  :rule []
  :line 'line-did-you-find-anything
        ;; "Hey, did you find anything?"
  :next-seg 'conv-searching-for-stuff-02
)
(define-conversation-segment 'conv-searching-for-stuff-02
  :rule []
  :line 'line-nope-not-yet
        ;; "I've been looking for an hour..."
  :next-seg 'conv-searching-for-stuff-03
)
(define-conversation-segment 'conv-searching-for-stuff-03
  :rule []
  :line 'line-shut-up-keep-looking
        ;; "Well then shut up and keep looking!"
)
```

This syntax might seem a bit verbose at first glance. But as we'll see in Section 14.6.2.8, breaking the conversation out like this gives us a great deal of

flexibility. For example, it allows branching conversations to be defined in a natural and reasonably convenient way.

14.6.2.6 Interrupting Conversations

We saw in Section 14.6.2.4 that a simple priority system can be used to handle interruptions and to resolve contention when more than one logical line is requested simultaneously.

When *conversations* are in play, a priority system can still be used. But its implementation is a bit more complex in this case. For example, imagine a conversation between characters A and B. A says his line, then B says her line while A waits his turn. During the time that B is speaking, A is asked to play an entirely different line of dialog. He's not technically speaking, so by the rules for dialog prioritization, applied to each character individually, there would be no problem and the line would play. But this could sound very jarring, depending on what's being said.

A: "Hey, did you find anything?"

B: "No, I've been looking for an hour and..."

A: "Look, a shiny object!"
(*interruption by an unrelated line of dialog*)

B: "...I ain't found nothin'."

To overcome this issue on *The Last of Us*, we introduced the concept of conversations as "first-class entities." When a conversation is started, the system "knows" that each of the characters is involved in that conversation, even when he or she isn't speaking. Each conversation has a priority, and the prioritization rules are applied to entire conversations, not the individual lines on a per-character basis. So for example, when character A is asked to say, "Look, a shiny object!" the system knows that he is currently involved in the "Hey, did you find anything?" conversation. Presumably the line "Look, a shiny object!" is at the same or lower priority as the current conversation, so the interruption isn't allowed.

If the interrupting line is something higher priority like, "Holy cow, he's pointing a gun at us!" then the line *is* allowed to interrupt the existing conversation. In that case, all of the characters in the conversation are interrupted. The result is an interruption that sounds natural and intelligent.

A: "Hey, did you find anything?"

B: "No, I've been looking for an hour and..."

- A: “Holy cow, he’s pointing a gun at us!”
(*interruption by a higher-priority conversation*)
- B: “Get him!”
(*The original conversation is interrupted by the new one, and A and B go into combat mode.*)

14.6.2.7 Exclusivity

On *The Last of Us*, we also introduced the concept of *exclusivity*. Any line of dialog or conversation can be marked as either *non-exclusive*, *faction-exclusive* or *globally exclusive*. This mark-up controls how interruptions work for the given line or conversation.

- A *non-exclusive* line or conversation is permitted to play *over top* of other lines or conversations. For example, during a search for the player, it’s not a big problem if one hunter is mumbling to himself, “Huh, there’s nothing over here.” while another hunter is saying, “I’m getting tired of this.” The two hunters aren’t speaking to each other, so the overlap sounds perfectly natural.
- A *faction-exclusive* line or conversation interrupts all other lines or conversations within that character’s faction. For example, if the player (Joel) is spotted during a search, the hunter that saw him might say, “He’s over here!” The other hunters should immediately stop speaking, because we want to make it seem as if the hunters can hear one another, and also to communicate to the player that their collective focus has shifted. However, if Joel’s companion Ellie is whispering a warning to him at the time, we probably do not want to interrupt her. She is not part of the hunter gang, and what she has to say to Joel is relevant whether or not the hunters have spotted him.
- A *globally exclusive* line or conversation interrupts all other lines, across faction boundaries. This is useful in situations in which *every* character within earshot should react to hearing whatever is being said.

14.6.2.8 Choices and Branching Conversations

It’s often desirable to allow conversations to play out in different ways depending on what the player does, on what decisions the AI characters make and/or on other aspects of game world state. When authoring or editing conversations, the writers and sound designers would like to have control not only over which lines are said, but also over the logical conditions that control which branch of the conversation will be taken at any given moment during

gameplay. This puts the creative power in the hands of the people who need it, instead of forcing them to work through a programmer.

Naughty Dog implemented such a system for use on *The Last of Us*. It was inspired in part by an earlier system developed by Valve and described by Elan Ruskin in his talk, “Rule Databased for Contextual Dialog and Game Logic,” which he delivered at the Game Developer’s Conference in 2012. The talk is available here: <http://www.gdcvault.com/play/1015317/AI-driven-Dynamic-Dialog-through>. Naughty Dog’s conversation system differs from Valve’s in a number of significant ways, but the core idea behind both systems is similar. We’ll describe the Naughty Dog system here, since that’s the system with which the author has the most experience.

In Naughty Dog’s conversation system, each segment of a conversation can consist of one or more alternative lines of dialog. Each alternative within the segment carries with it a *selection rule*. If the rule evaluates to true, that alternative is selected; if the rule evaluates to false, the alternative is ignored.

A rule is comprised of one or more criteria. Each criterion is a simple logical expression that evaluates to a Boolean. The expressions (`'health > 5`) and (`'player-death-count == 1`) are examples of criteria. If more than one criterion is provided within a rule, they are logically combined using the Boolean AND operator. A rule only evaluates to true when *all* of its criteria evaluate to true.

Here’s an example of one segment of a conversation, with three alternatives that depend upon the health of the character doing the talking:

```
(define-conversation-segment 'conv-player-hit-by-bullet
  (
    :rule [ ('health < 25) ]
    :line 'line-i-need-a-doctor
          ;; "I'm bleeding bad... need a doctor!"
  )
  (
    :rule [ ('health < 75) ]
    :line 'line-im-in-trouble
          ;; "Now I'm in real trouble."
  )
  (
    :rule [ ] ;; no criteria acts as an "else" case
    :line 'line-that-was-close
          ;; "Ah! Can't let that happen again!"
  )
)
```

Branching Dialog

By breaking a conversation into segments, each of which contains one or more alternative lines, we open up the possibility of crafting branching conversations. For example, let's consider a conversation in which Ellie (the player's companion in *The Last of Us*) asks Joel (the player character) if he's all right when he's been shot at. If the player wasn't actually hit by the bullet, the conversation goes like this:

Ellie: "Are you OK?"

Joel: "Yeah, I'm fine."

Ellie: "Geez. Keep your head down!"

If Joel has been hit, the conversation plays out differently:

Ellie: "Are you OK?"

Joel: "(panting) Not exactly."

Ellie: "You're bleeding!"

We can express this branching conversation using the conversation syntax described above:

```
(define-conversation-segment 'conv-shot-at--start
  (
    :rule [ ]
    :line 'line-are-you-ok ;; "Are you OK?"
    :next-seg 'conv-shot-at--health-check
    :next-speaker 'listener ;; *** see comments below
  )
)

(define-conversation-segment 'conv-shot-at--health-check
  (
    :rule [ (('speaker 'shot-recently) == false) ]
    :line 'line-yeah-im-fine ;; "Yeah, I'm fine."
    :next-seg 'conv-shot-at--not-hit
    :next-speaker 'listener ;; *** see comments below
  )
  (
    :rule [ (('speaker 'shot-recently) == true) ]
    :line 'line-not-exactly ;; "(panting) Not exactly."
    :next-seg 'conv-shot-at--hit
    :next-speaker 'listener ;; *** see comments below
  )
)
```



```
(define-conversation-segment 'conv-shot-at--not-hit
  (
    :rule [ ]
    :line 'line-keep-head-down ;; "Geez. Keep your head down!"
  )
)

(define-conversation-segment 'conv-shot-at--hit
  (
    :rule [ ]
    :line 'line-youre-bleeding ;; "You're bleeding!"
  )
)
```

Speaker and Listener

There's a subtle aspect to what's going on in the branching conversation above. At any given moment in a two-person conversation, one person is the speaker and the other is the listener. The roles of speaker and listener ping-pong back and forth as the conversation progresses. In the first segment of the conversation, 'conv-shot-at--start, Ellie is the speaker and Joel is the listener. When we chain to the next segment, 'conv-shot-at--health-check, we specify the value 'listener for the field :next-speaker. This tells the system to use the current listener (Joel) as the next segment's speaker, thereby reversing the roles. In that segment, we check whether the speaker has been shot recently via the criteria (('speaker 'shot-recently) == false) and (('speaker 'shot-recently) == true). But now Joel is the speaker, so everything works out as we'd expect.

An abstract speaker/listener system doesn't seem all that useful for a conversation between two principal characters like Joel and Ellie. But by keeping the definition of the conversation abstract, we gain a significant amount of flexibility. For one thing, we could use the same conversation specification to define a conversation in which Joel asks Ellie if she's OK. This works because the entire conversation is defined in a way that is independent of which character is saying each line. Moreover, for enemy characters it's absolutely essential that conversations be defined in a generic manner, because we don't know which specific characters will be doing the speaking a priori. For enemy battle chatter, we typically select a pair of characters dynamically and fire off the conversation. It has to work, no matter which two characters are selected.

The speaker/listener system can be extended to two- or three-person conversations. The Naughty Dog conversation system supported up to three listeners, although the vast majority of our conversations were between only

two characters.

Fact Dictionaries

The criteria within a rule reference symbolic quantities like 'health and 'player-death-count. These symbolic quantities are implemented under the hood as entries in a *dictionary* data structure—basically a table containing key-value pairs. We call these *fact dictionaries*. An example of a fact dictionary is shown in Table 14.1.

You may have noticed in Table 14.1 that each value in the dictionary has an associated data type. In other words, the values in the dictionary are *variants*. A variant is a data object that is capable of holding values of various types, much like a union in C or C++. However, unlike a union, a variant also stores information about the type of data it currently contains. This allows us to validate the type of a value prior to using it. It also lets us convert data from one type to another. For example, if our variant holds the integer value 42, we could ask the variant to return it to us as the floating-point value 42.0f instead.

In *The Last of Us*, each character has its own fact dictionary containing facts about the character itself like health, weapon type, awareness level and so on. Each “faction” of characters also has a fact dictionary. This allows us to express facts about the faction as a whole, like how many characters remain alive within the group. Finally, there is a singleton “global” fact dictionary that contains information about the game as a whole, without respect to faction. Things like the amount of time spent playing, the name of the current level or how many times the player has retried a particular task are all things that can go into the global fact dictionary.

Criterion Syntax

When writing a criterion, the syntax allows for facts to be pulled from any dictionary by name. For example, (('self 'health) > 5) tells the sys-

Key	Value	Data Type
'name	'ellie	StringId
'faction	'buddy	StringId
'health	82	int
'is-joels-friend	true	bool
...	...	...

Table 14.1. An example of a fact dictionary.

tem to grab the fact dictionary of the character itself, look up the value of the 'health fact in that dictionary and then check if it is greater than 5. Likewise, (('global 'seconds-playing) <= 23.5) instructs the system to look up the 'seconds-playing fact from the global fact dictionary, and check that it is less than or equal to 23.5 seconds.

If the user doesn't specify a dictionary explicitly, as in ('health > 5), the system searches for the named fact by following a predefined search order. Check the character's fact dictionary first. If that fails, try to find it in the dictionary that matches the character's faction. Finally, if all else fails, look for the fact in the global dictionary. This "search path" feature allows sound designers to be as brief as possible when writing criteria (albeit with the loss of some specificity and clarity in the rules).

14.6.2.9 Context-Sensitive Dialog

In *The Last of Us*, we wanted to have enemy characters call out the location of the player in an intelligent way. If the player is hiding in a store, the enemies should shout, "He's in the store!" If he's hiding behind a car, we want the bad guys to say, "He's behind that car!" This makes the characters sound incredibly intelligent, yet it turns out to be a relatively simple thing to implement.

To make this work, the sound designers mark up our game worlds with regions. Each region is tagged with one of two kinds of *location tags*. A *specific* tag marks the region with a very specific location like "behind the counter" or "by the cash register." A *general* tag marks the region with a more general location like "in the store" or "in the street."

To determine which line of dialog to play, the system determines within which region the player is located, and within which region the enemy NPC is located. If they are both in the same *general* region, the player's *specific* tag is used to select dialog lines. When the NPC and player reside in *different* general regions, we fall back to using the player's general region tag to select the line. So if the enemy and the player are both in the store, we might select a line like, "He's by the window!" But if the NPC is in the store and the player is out in the street, we might hear the NPC say, "He's out in the street! Get him!" See Figure 14.42 for an illustration of how this system works.

This very simple system proved incredibly powerful. It was difficult to set up due to the sheer number of combinations of lines that had to be recorded and configured, but the final result in-game was worth the effort.

14.6.2.10 Dialog Actions

Lines of dialog delivered without body language usually look uncanny and unrealistic. Some dialog lines are delivered as part of a full-body animation—an in-game cinematic for example. But some lines must be delivered while the

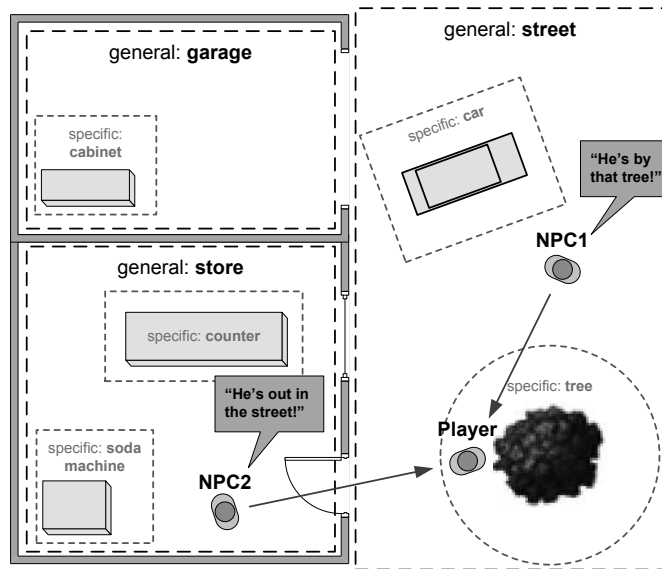


Figure 14.42. General and specific regions for context-sensitive dialog line selection.

character is busy doing something else, like walking, running or firing their weapon. Ideally we'd like to spice up such lines of dialog with some gestures to breathe life into them.

On *The Last of Us*, we implemented a gesture system using additive animation technology (see Section 12.6.5). These gestures could be explicitly called out by C++ code or script. In addition, each line of dialog could have a script associated with it whose timeline was synchronized with the audio. This allowed us to trigger gestures at precise moments during key lines of dialog.

14.6.3 Music

Music is an incredibly important aspect of pretty much any good game. It sets the tone, drives the player's sense of tension, and can make (or break) an emotional scene. A game engine's music system is typically charged with the following duties:

- Provide the ability to play back music tracks as streaming audio clips (because music clips are almost always too large to fit in memory).
- Provide musical variety.
- Match the music to the events occurring in the game.
- Seamlessly transition from one piece of music to the next.

- Mix the audio with the other sounds in the game in a suitable and pleasing manner.
- Allow music to be temporarily ducked to enhance the audibility of specific sounds or conversations in-game.
- Permit brief pieces of music or sound effects known as *stingers* to temporarily interrupt the currently playing music track.
- Allow music to be paused and restarted. (You don't need a full orchestra playing a grandiose theme during every single second of gameplay, you know!)

We generally expect the music to change to match the changing levels of tension and/or emotional moods of the events happening in the game. One way to accomplish this is to create multiple *playlists*, each of which is intended for a different level of tension or emotional mood in the game. Each playlist contains one or more pieces of music, from which selections may be made randomly or sequentially. As the tension and mood change in the game—as battles begin and end, touching cutscenes come and go and so on—the music system detects these changes and selects new music playlists as appropriate. Some games implement a “stack” of music selections at increasing tension levels—calm music for when no enemies are around, tense music when the player is approaching an unsuspecting group of enemies, startling music at first contact and fast-paced music during battle.

Stingers are another way to match the music to the events in the game. A stinger is a short musical clip or sound effect that can temporarily interrupt the currently playing music track, or play over top of it while the main track's volume is ducked down. For example, the first time the player makes line-of-sight with a new enemy, we might want to play an ominous “rumbling” sound to give the player a cue that danger is near. Or when the player dies, we may want to quickly switch to a snippet of “death music.” Both of these are situations in which a stinger might be used.

Transitioning smoothly between different music streams is somewhat of a challenge. We cannot blindly cross-blend between totally unrelated pieces of music and expect it to always sound good. The tempos of the two pieces may not match, and the “beat” of one piece of music might not line up with that of the next. The key is to time each transition properly. A rapid cross-fade can be useful if the tempos don't match; a longer cross-fade might work well if the tempos are nearly identical. This takes some trial and error to get right. Even getting a piece of music to loop properly requires some tweaking by the sound engineer.

The topic of game music is a broad one, and we can't really do it justice here. If you are interested in learning more, [45] is a great book to start with.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>